



US 20250284664A1

(19) **United States**

(12) **Patent Application Publication**
FREILICH et al.

(10) **Pub. No.: US 2025/0284664 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **PROVIDING BLOCK-BASED DATASET STORAGE USING LAYERED VIRTUAL AND BACKEND DEVICES**

(60) Provisional application No. 62/875,947, filed on Jul. 18, 2019, provisional application No. 62/878,877, filed on Jul. 26, 2019.

(71) Applicant: **PURE STORAGE, INC.**, SANTA CLARA, CA (US)

Publication Classification

(72) Inventors: **JOSHUA FREILICH**, KALAHEO, HI (US); **RONALD KARR**, PALO ALTO, CA (US); **ANDREW KLEINERMAN**, SAN CARLOS, CA (US); **KIRON VIJAYASANKAR**, LOS ALTOS, CA (US)

(51) **Int. Cl.**
G06F 16/182 (2019.01)
G06F 3/06 (2006.01)

(52) **U.S. Cl.**
CPC **G06F 16/1844** (2019.01); **G06F 3/0617** (2013.01); **G06F 3/064** (2013.01); **G06F 3/0655** (2013.01); **G06F 3/067** (2013.01)

(21) Appl. No.: **19/220,631**

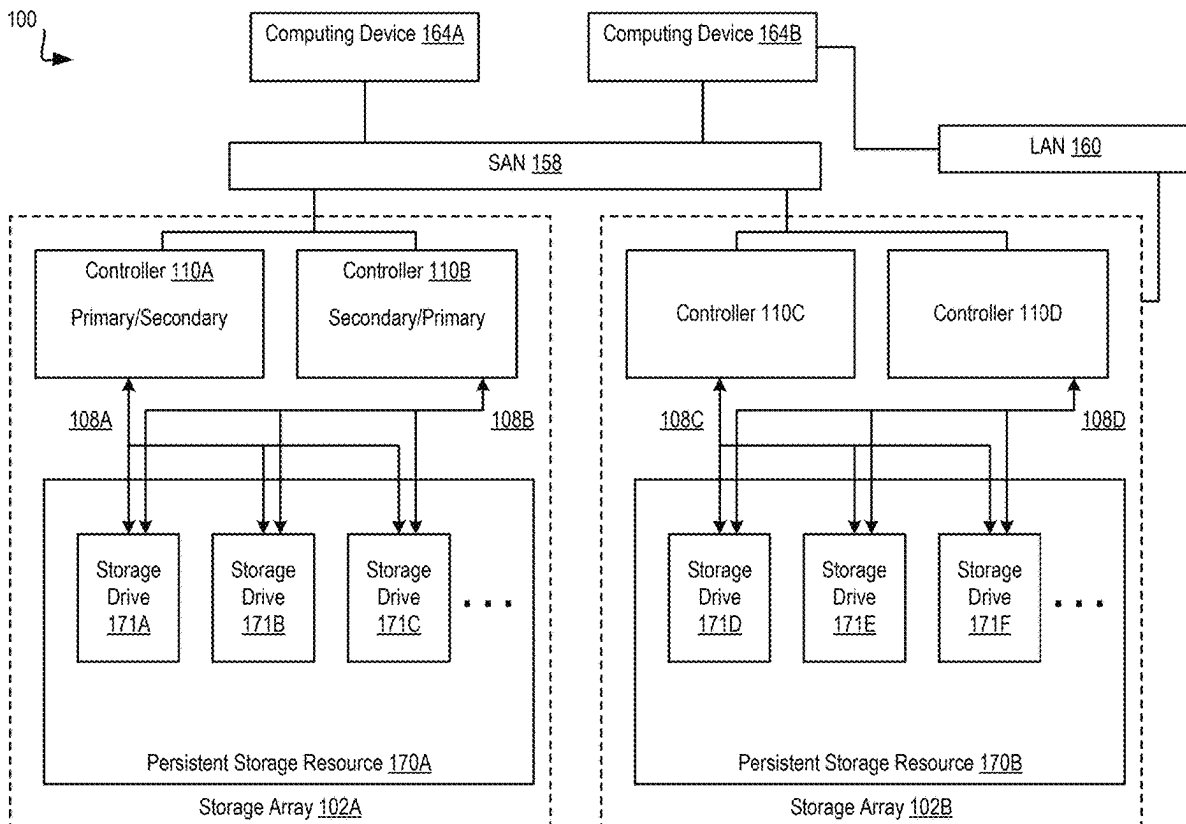
(57) **ABSTRACT**

(22) Filed: **May 28, 2025**

A method of providing block-based storage is described. The method comprises: creating a virtual storage system for storing a dataset, the virtual storage system including a virtual drive layer that includes a plurality of virtual drives for storing at least a portion of the dataset as block data; and a block storage backend layer for storing at least the portion of the dataset as block data, the block storage backend layer including one or more block-based storage devices; receiving an I/O operation with respect to the dataset; and based on the received I/O operation, storing, in the block storage backend layer, an update to the dataset.

Related U.S. Application Data

(63) Continuation of application No. 18/344,656, filed on Jun. 29, 2023, now Pat. No. 12,353,364, which is a continuation-in-part of application No. 17/955,428, filed on Sep. 28, 2022, now Pat. No. 12,032,530, which is a continuation of application No. 16/524,542, filed on Jul. 29, 2019, now Pat. No. 11,487,715.



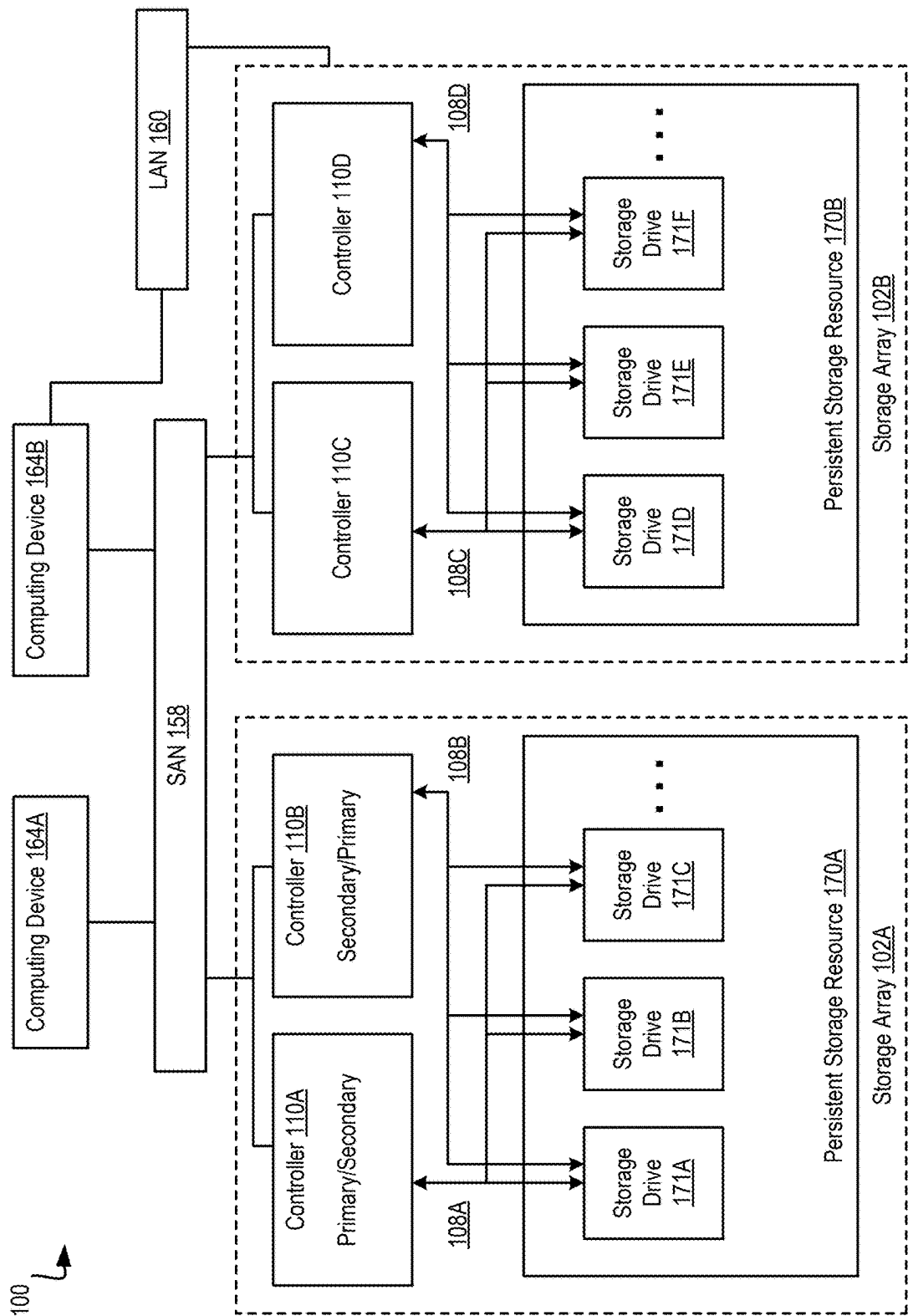


FIG. 1A

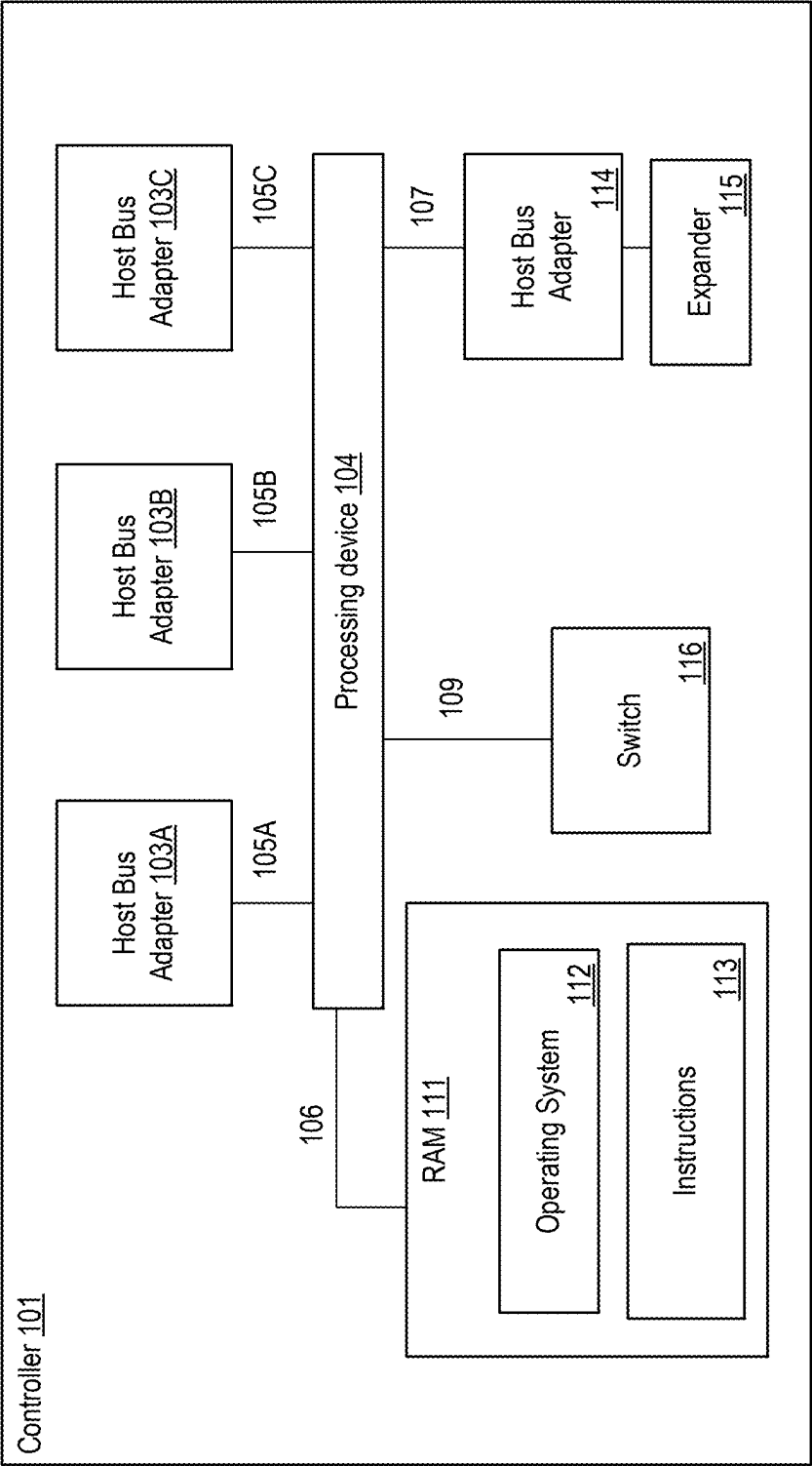


FIG. 1B

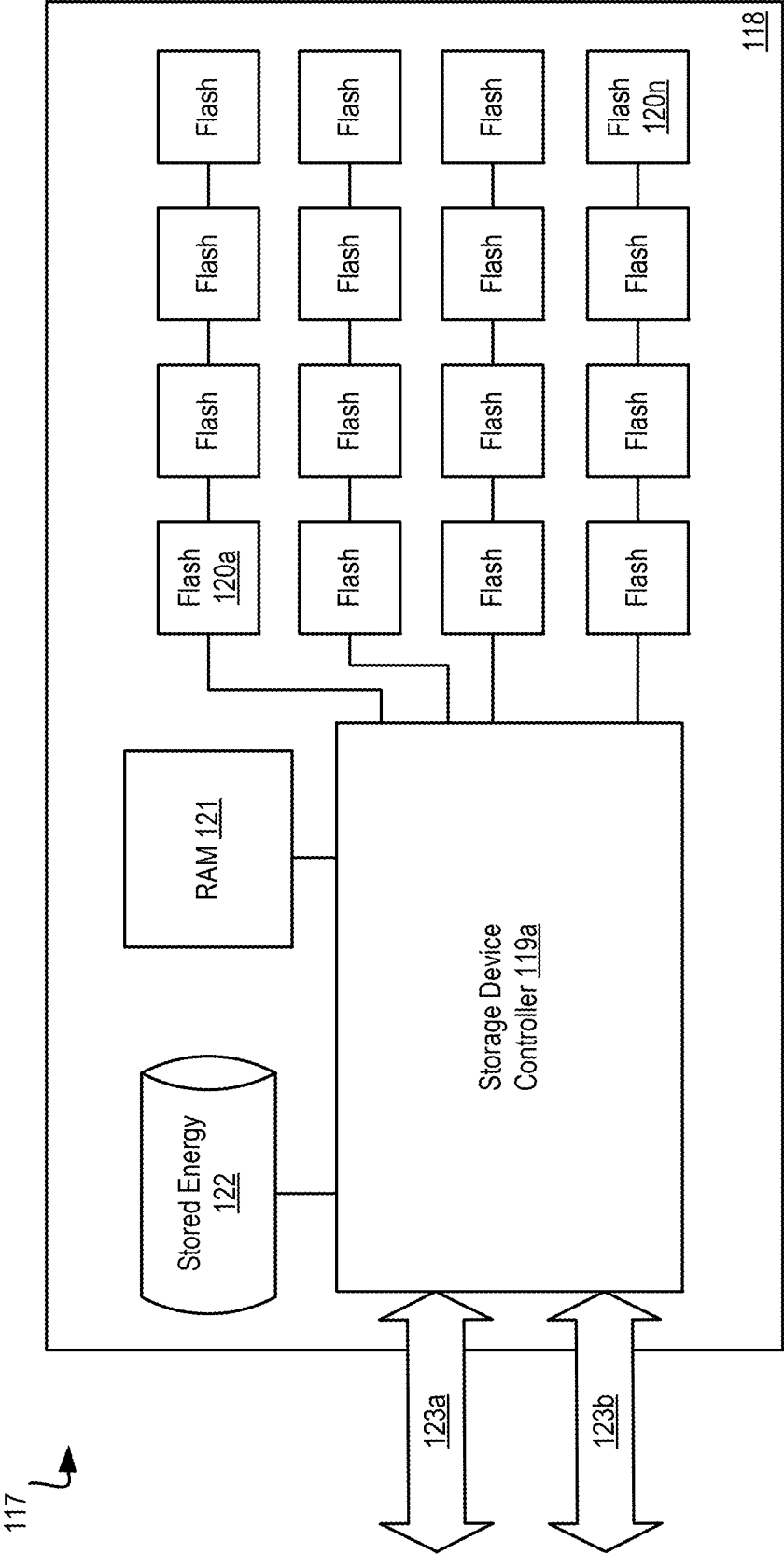


FIG. 1C

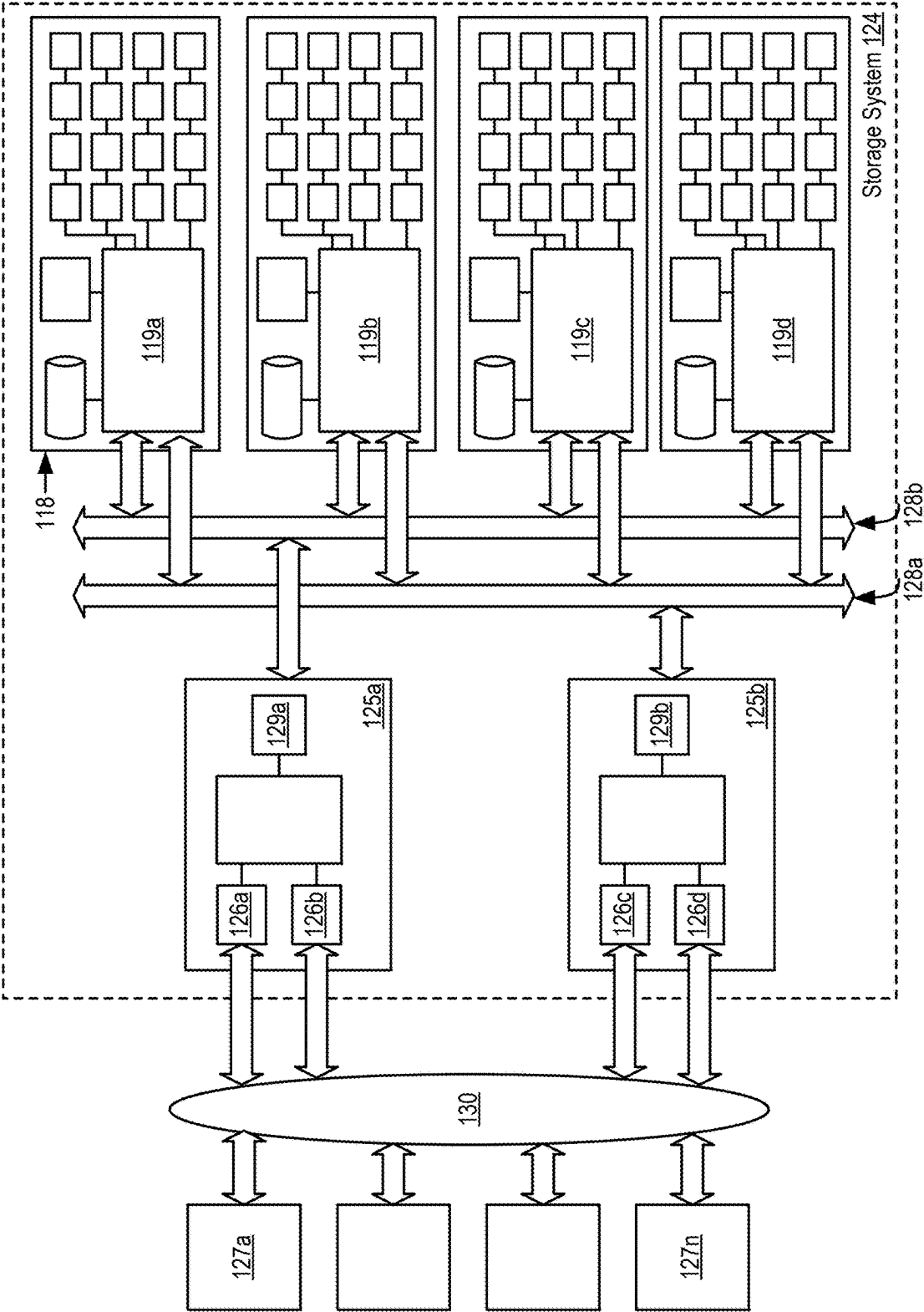


FIG. 1D

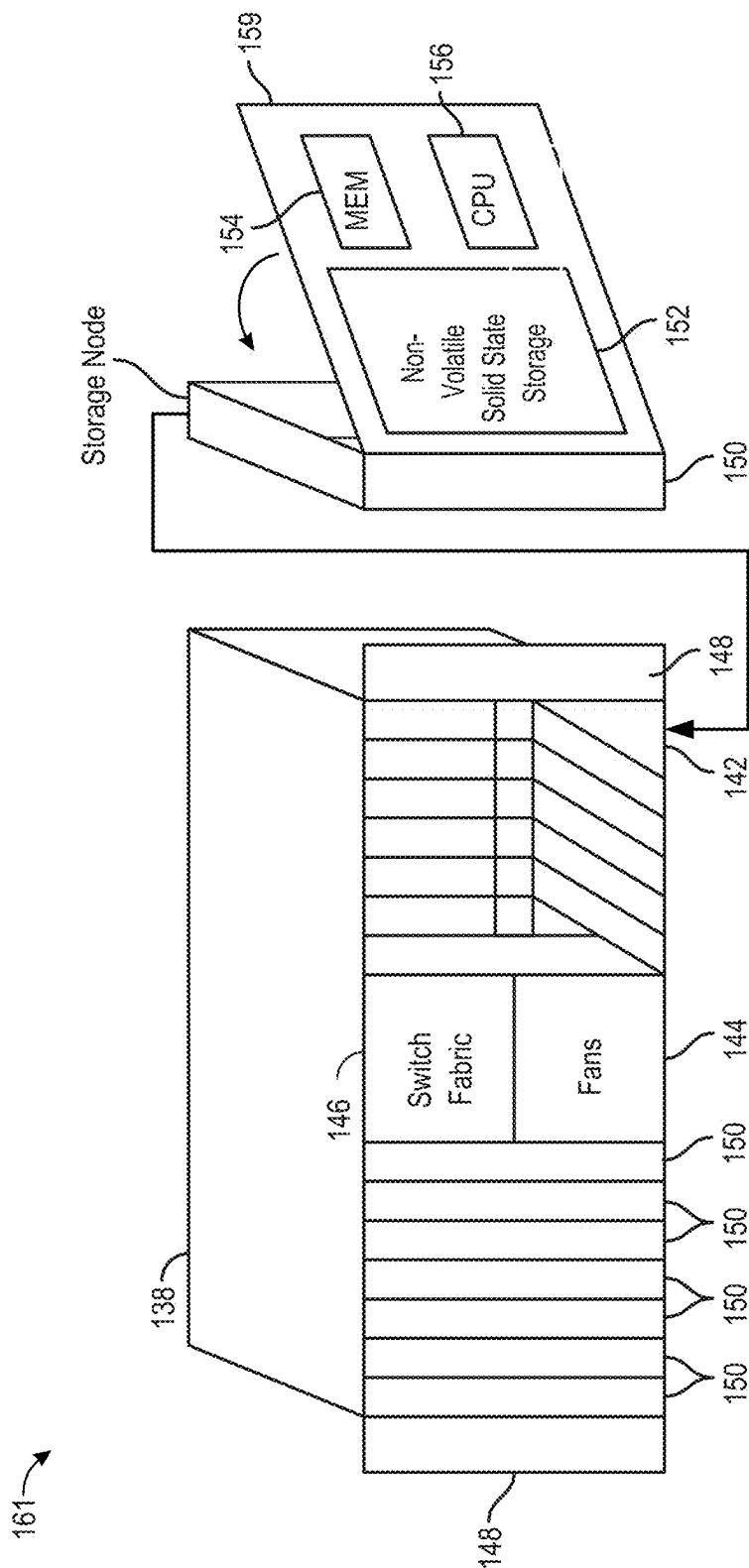


FIG. 2A

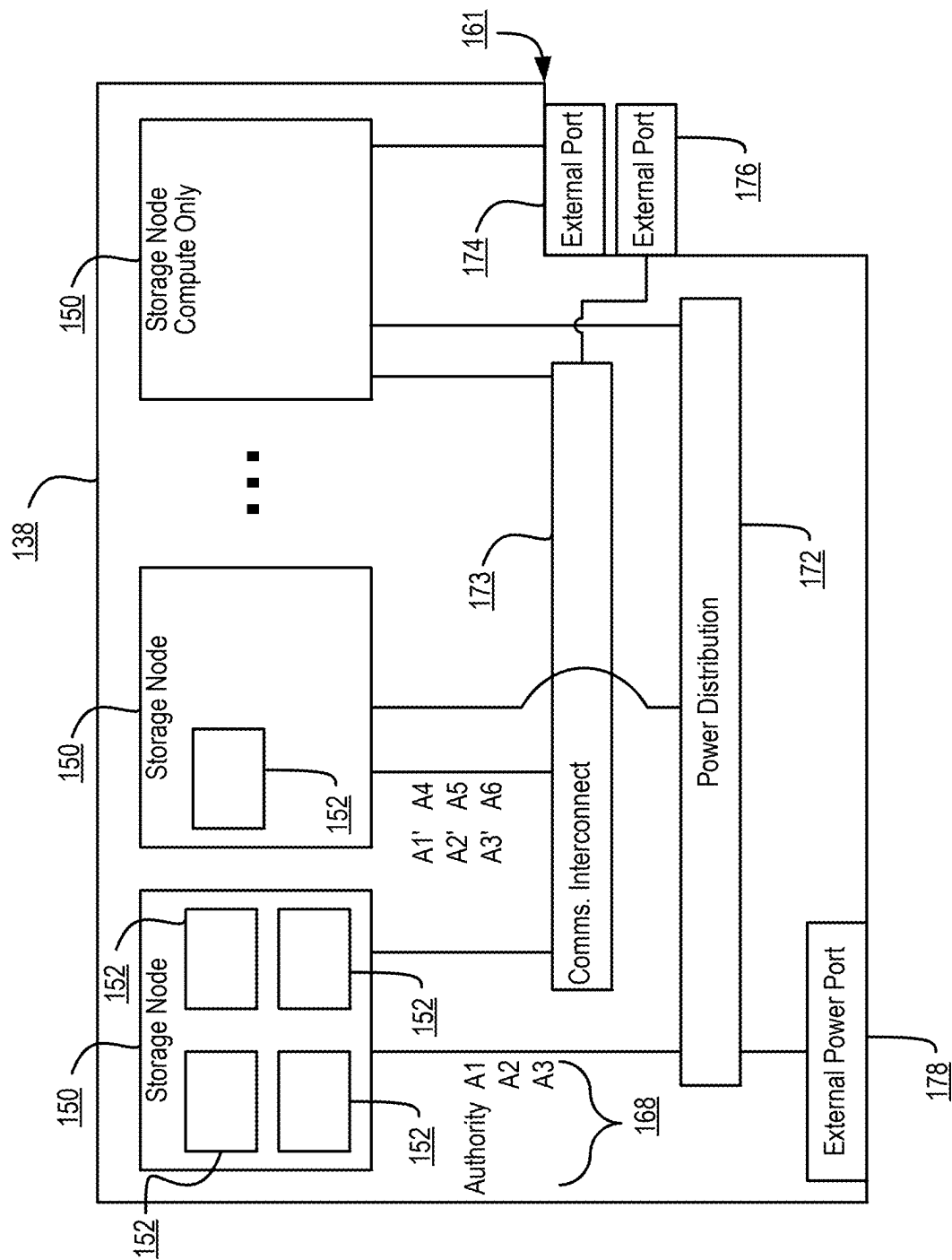


FIG. 2B

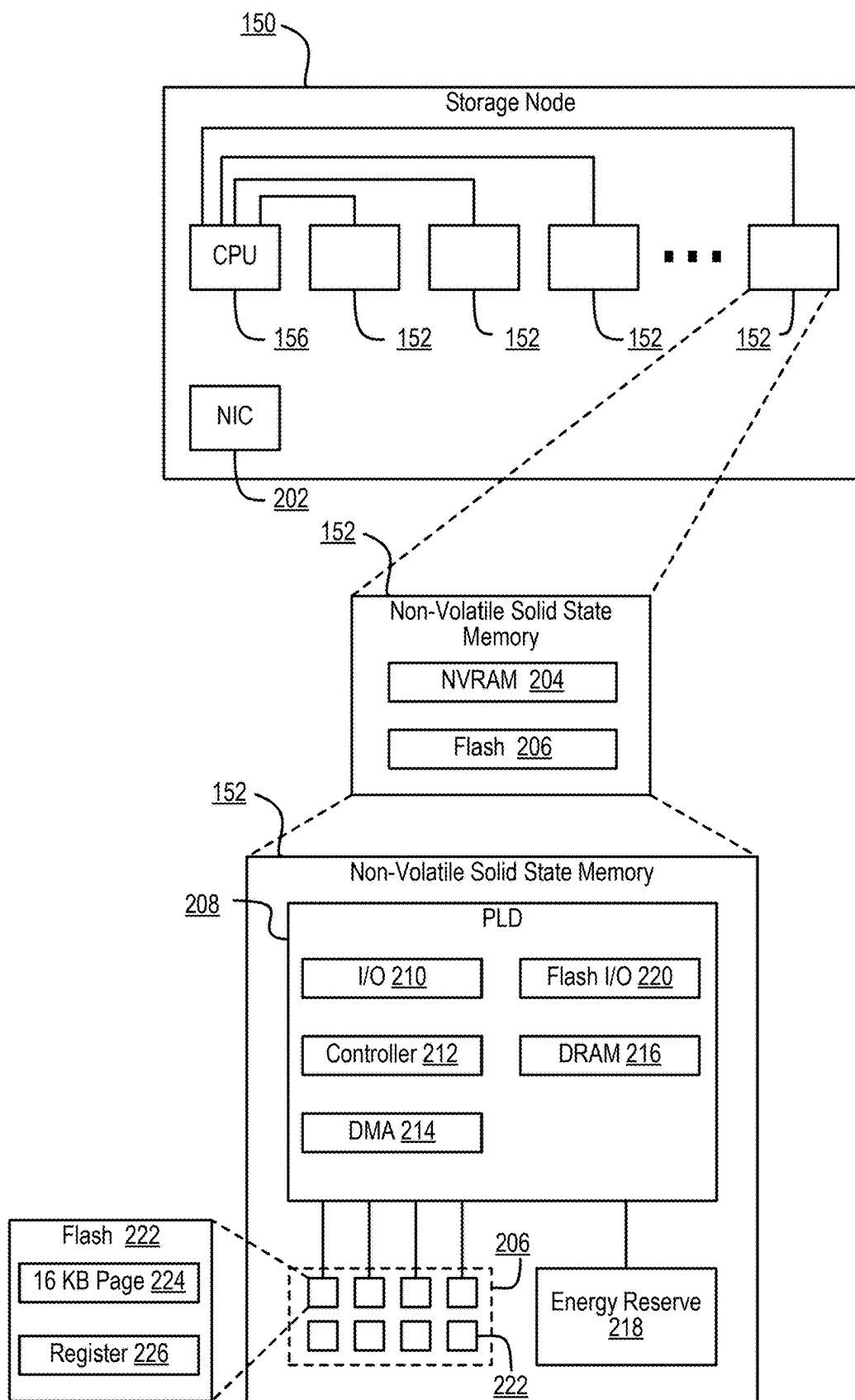


FIG. 2C

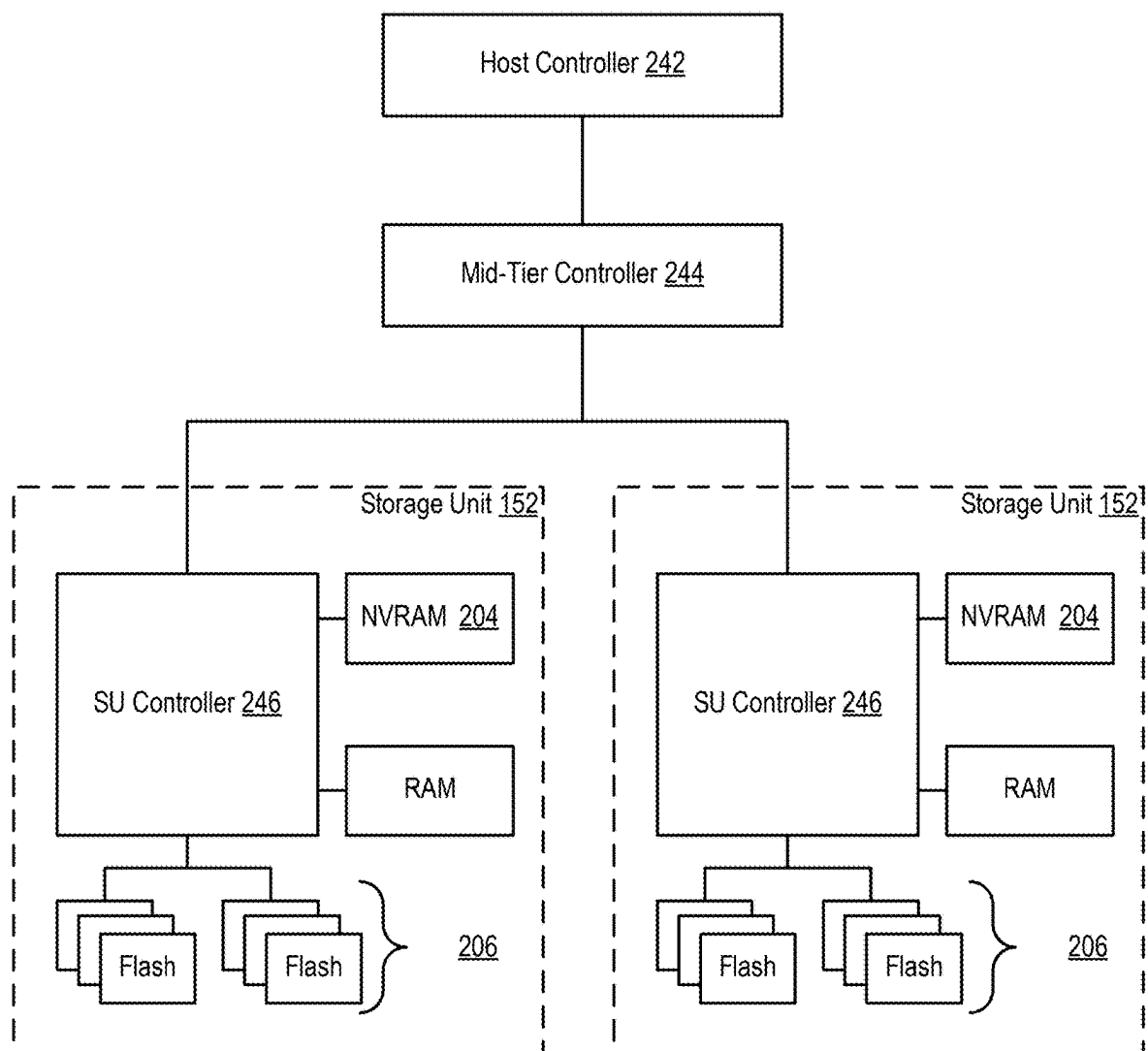


FIG. 2D

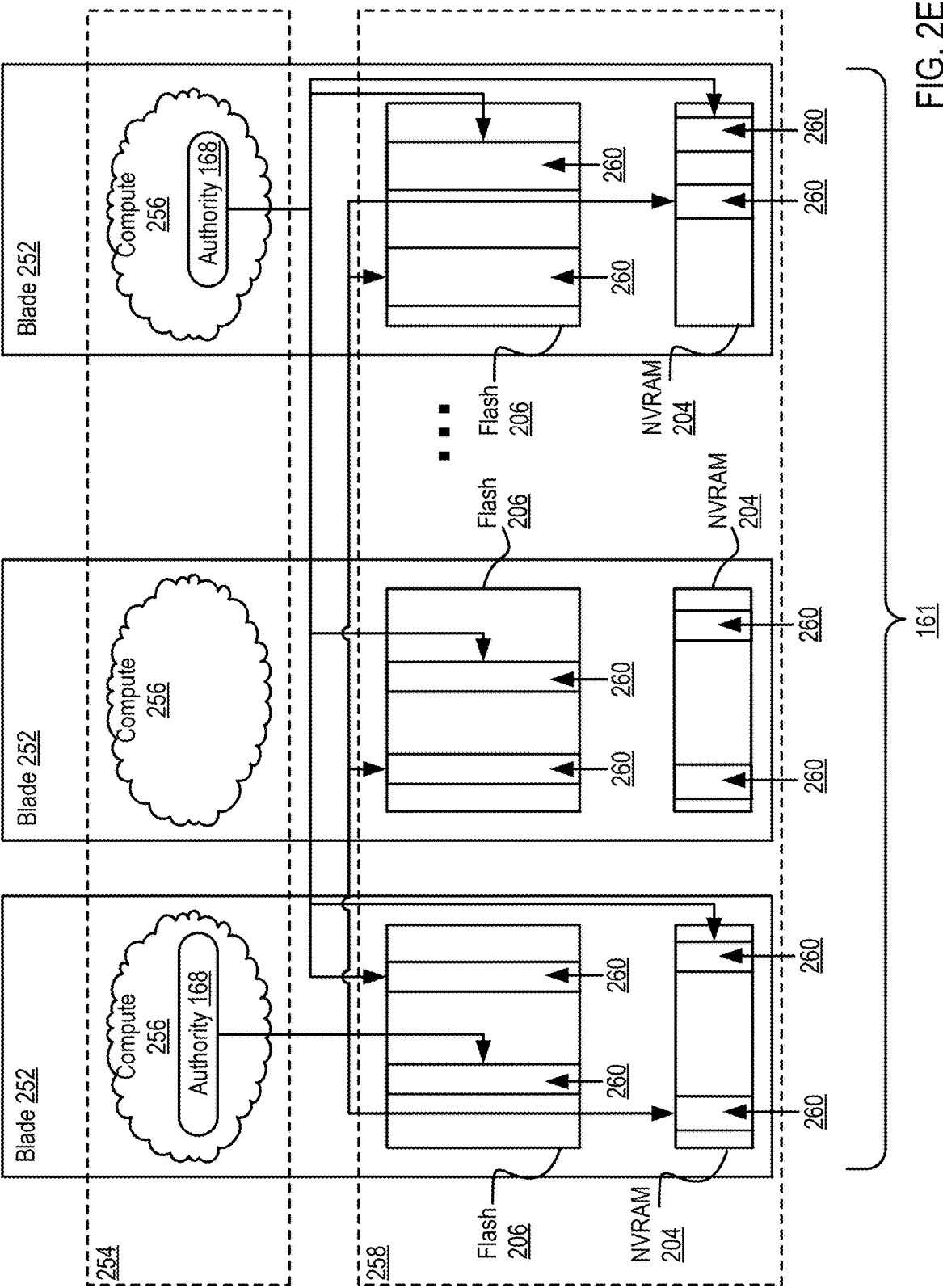


FIG. 2E

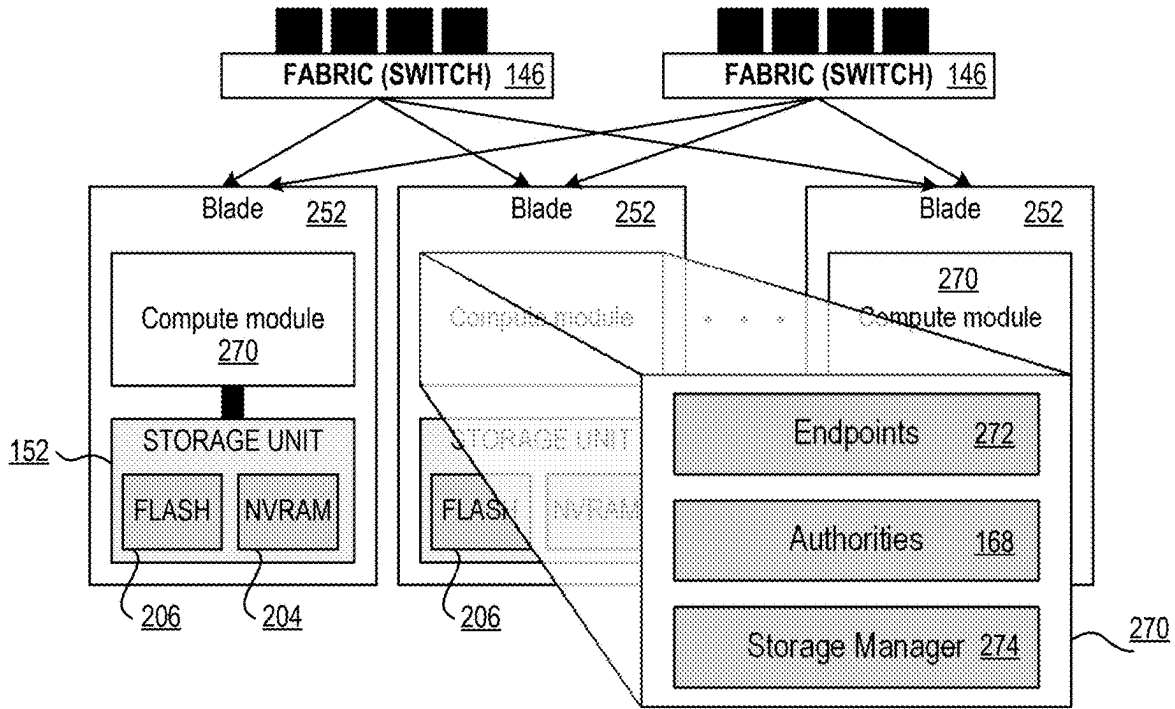


FIG. 2F

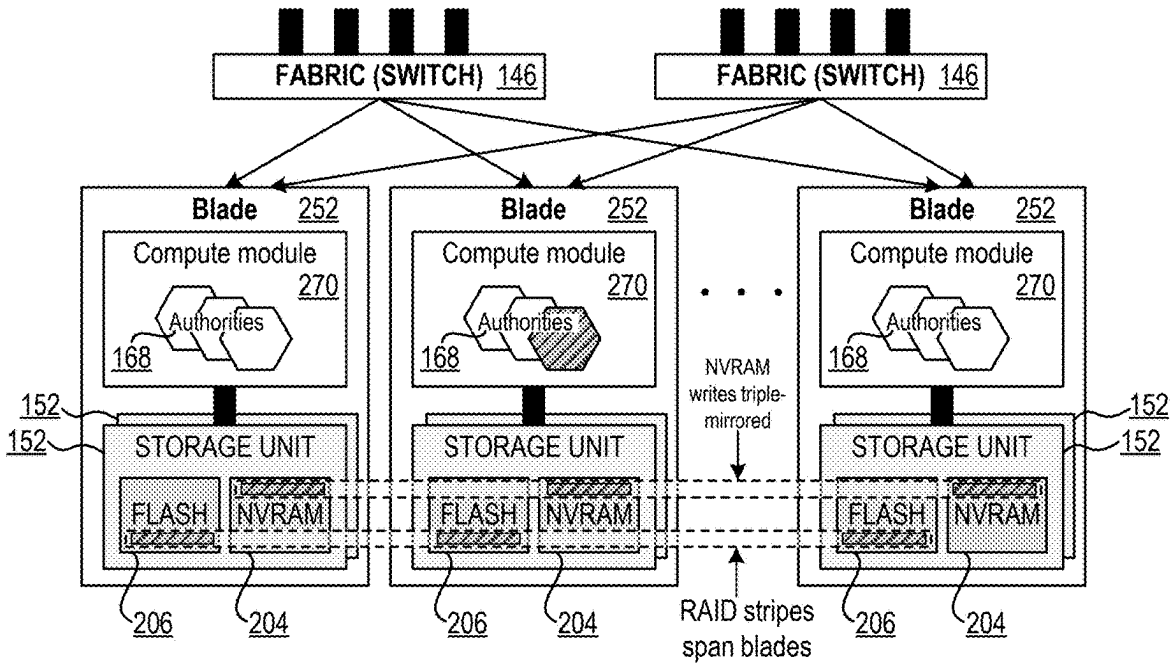


FIG. 2G

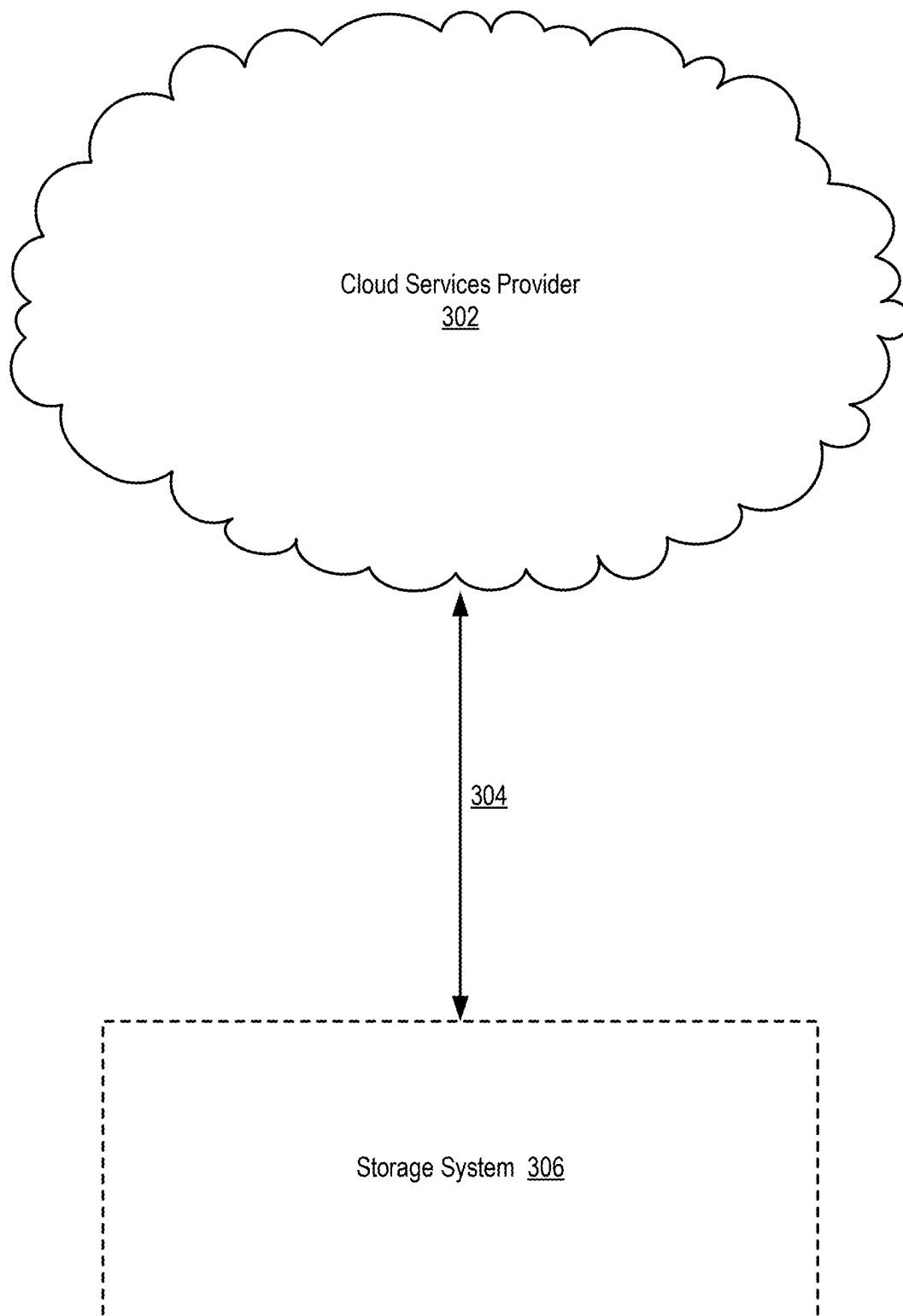


FIG. 3A

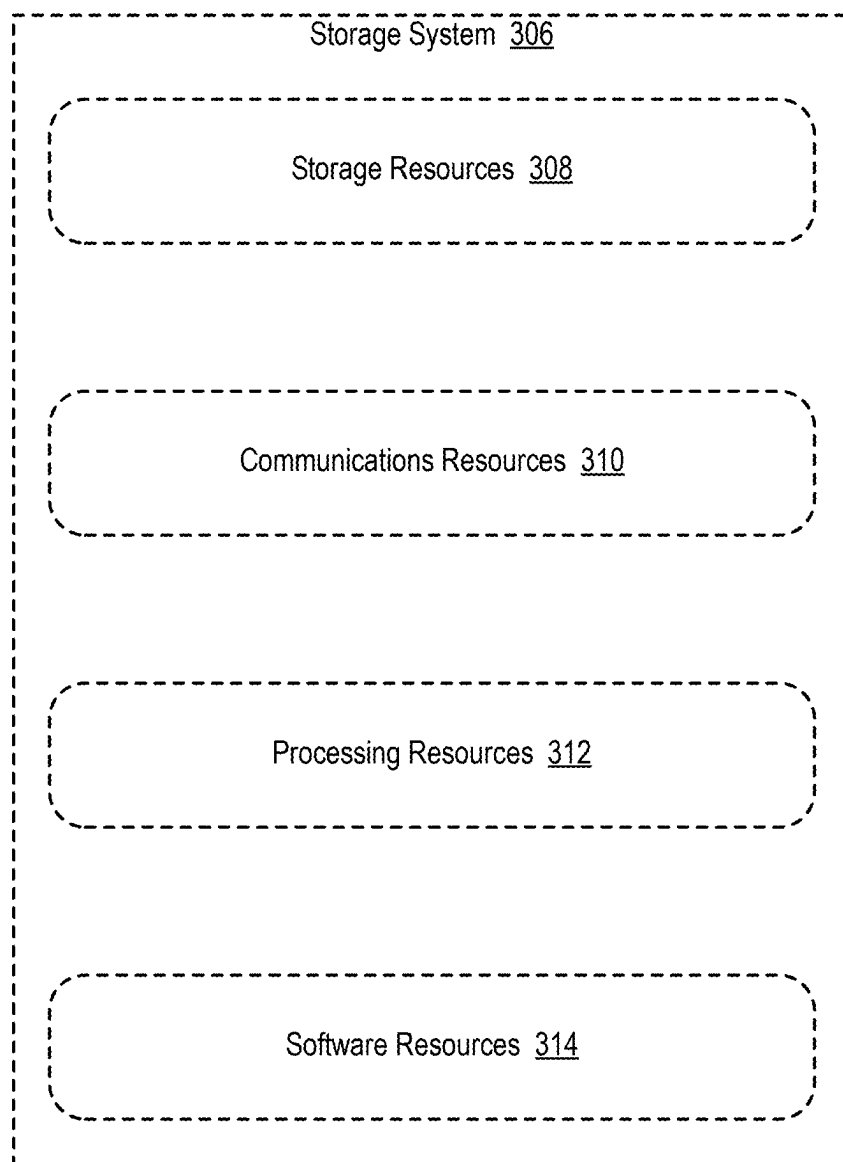


FIG. 3B

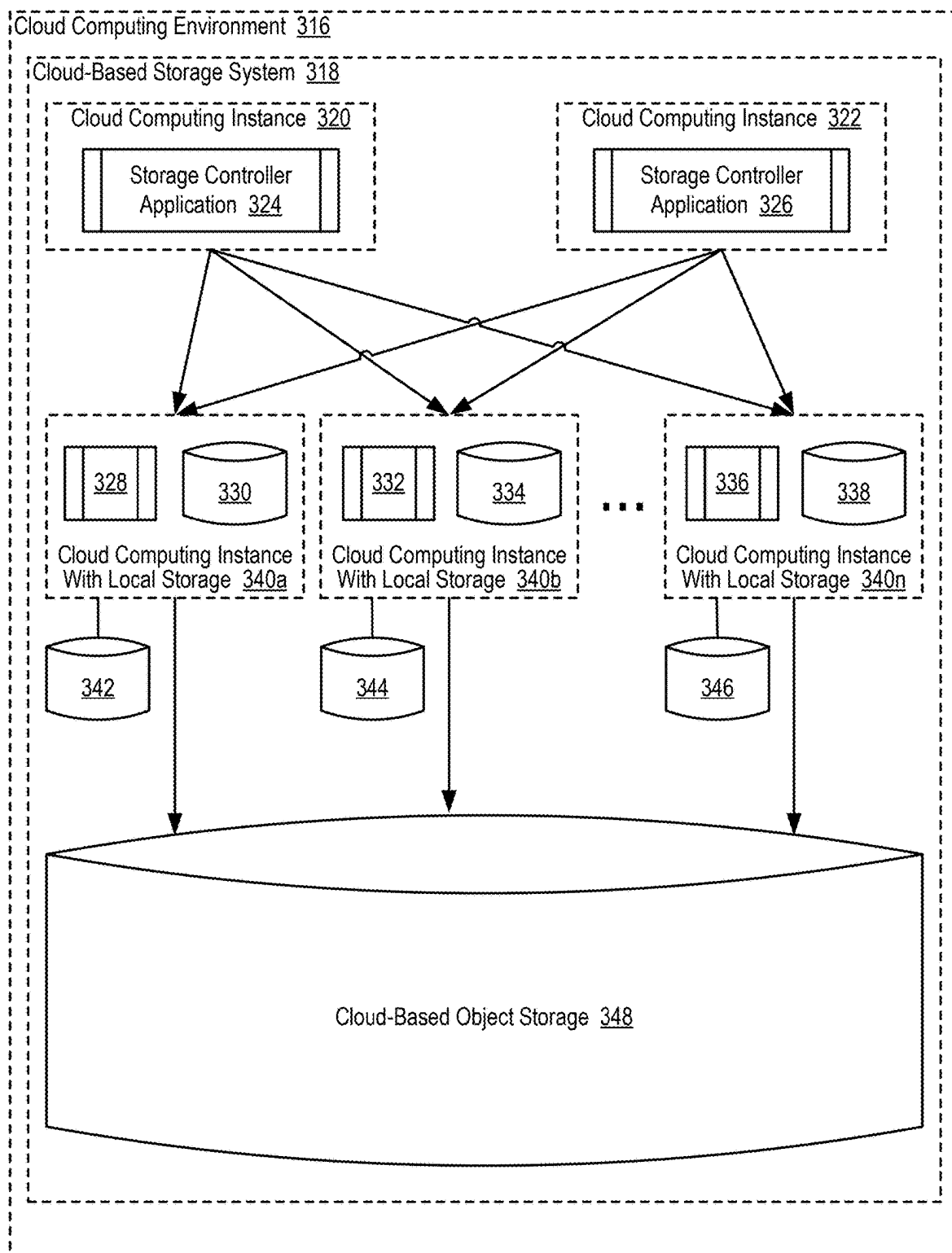


FIG. 3C

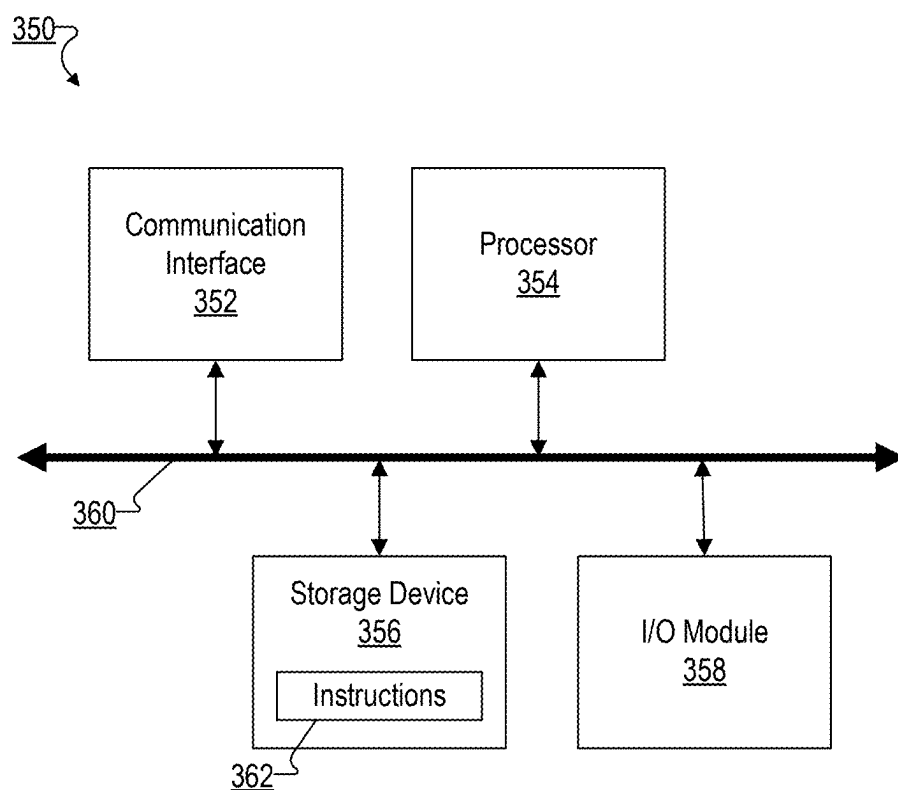


FIG. 3D

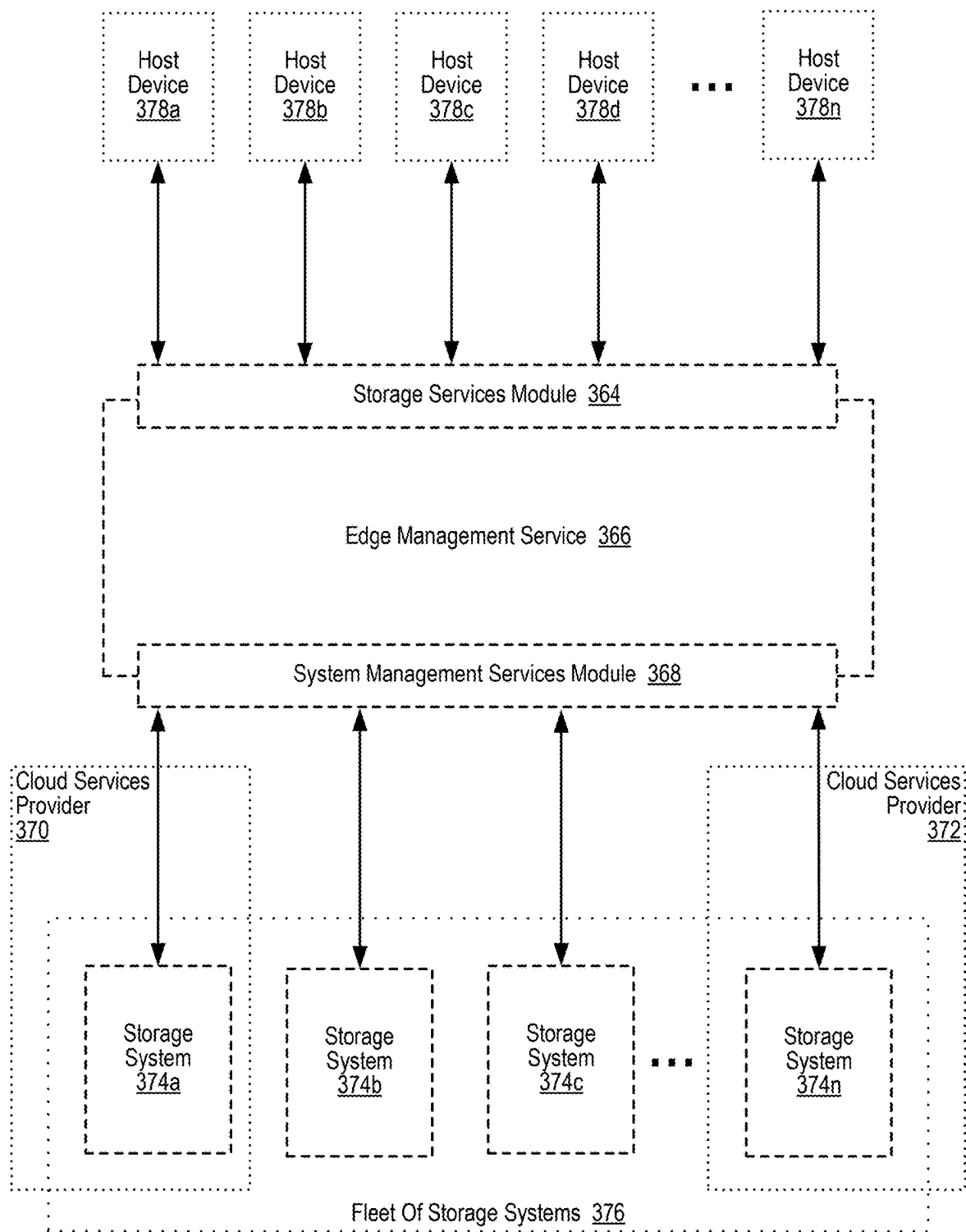


FIG. 3E

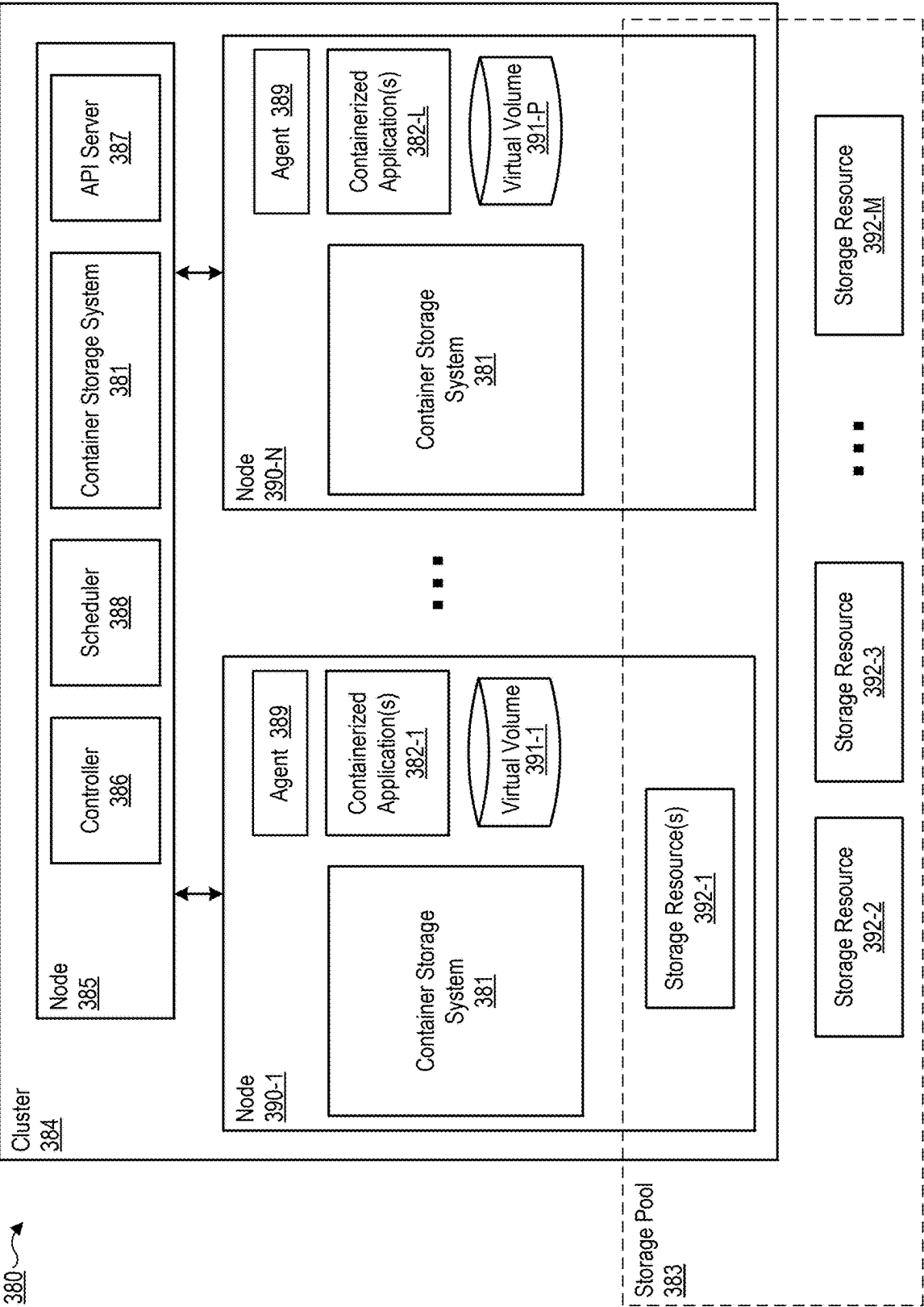


FIG. 3F

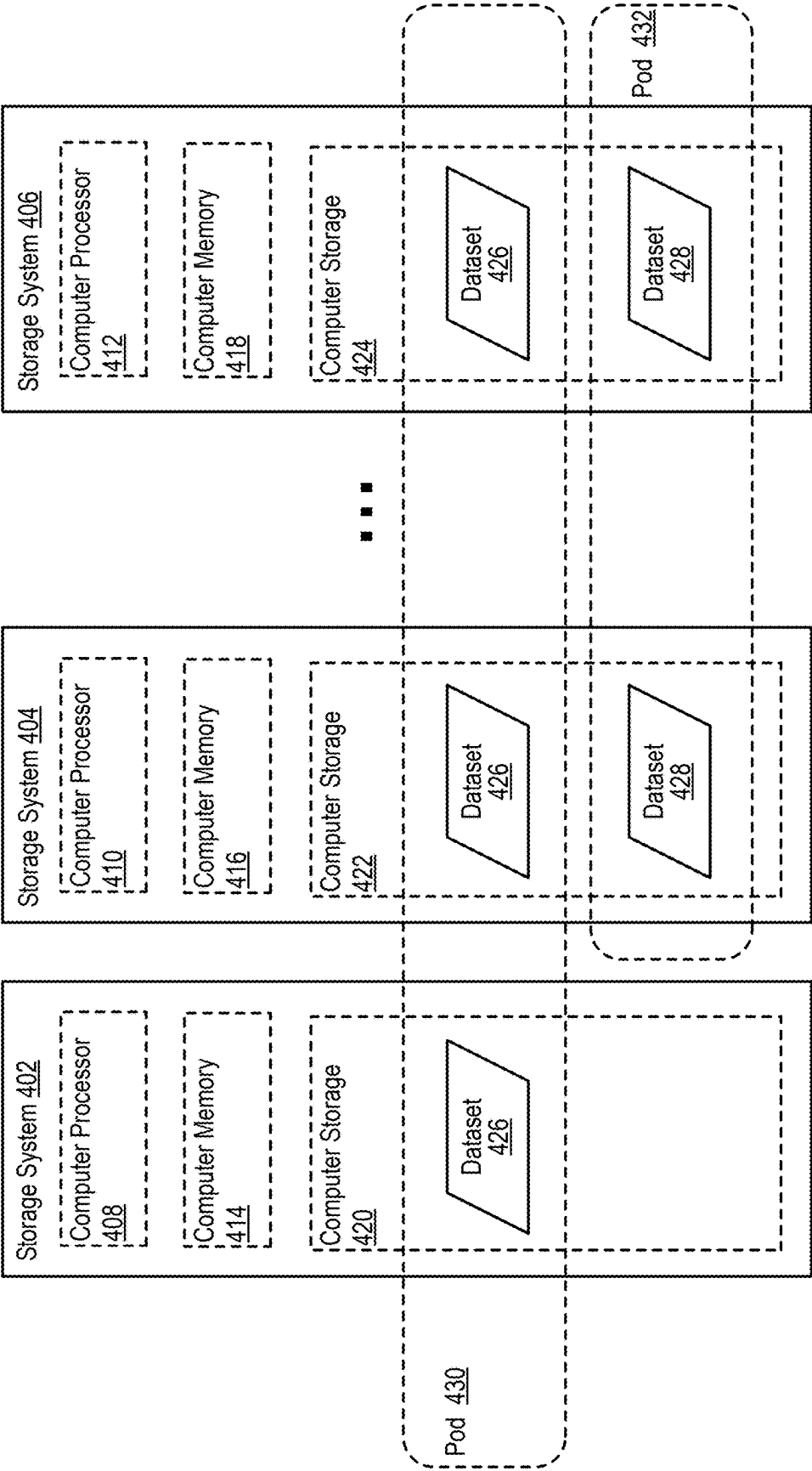


FIG. 4

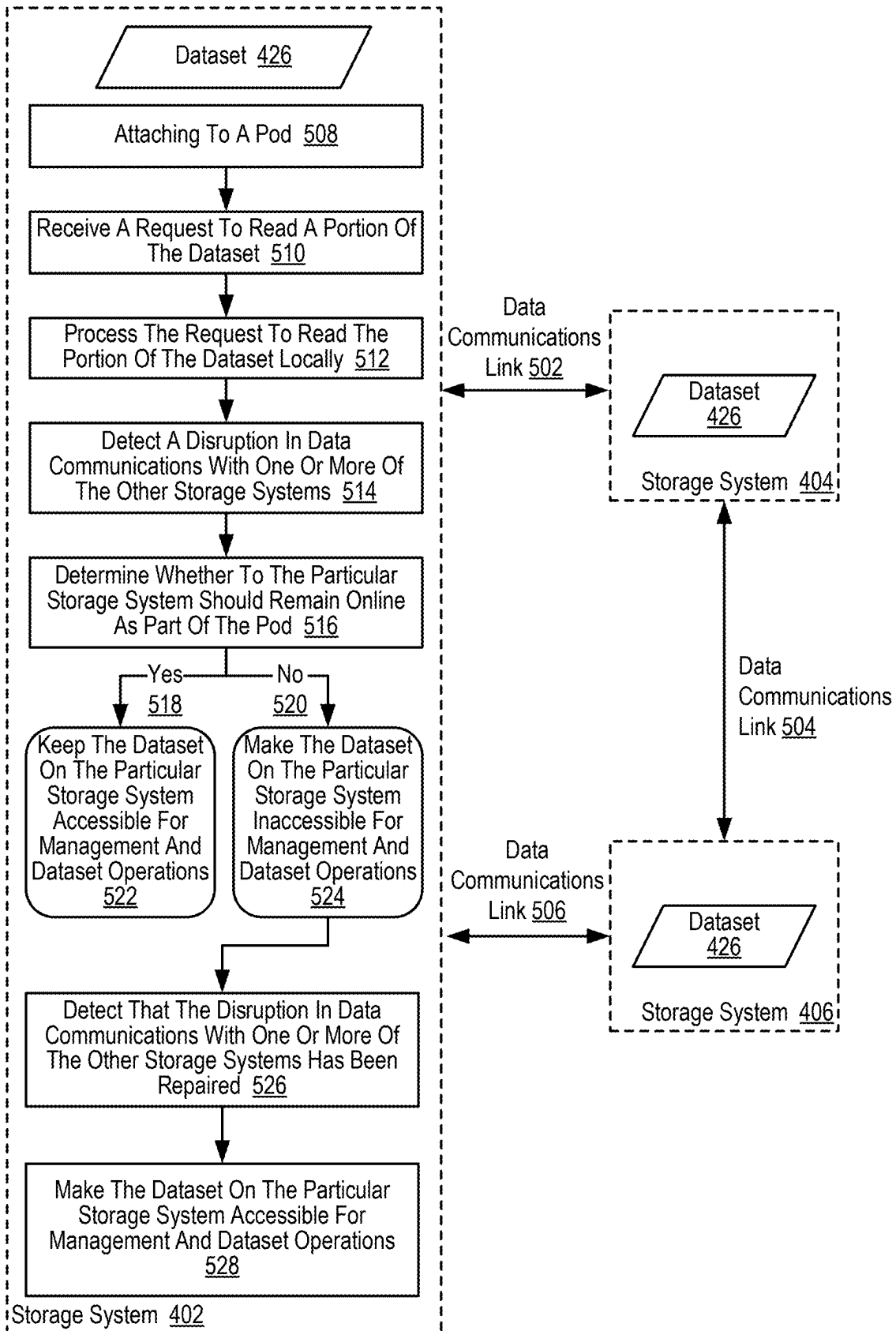


FIG. 5

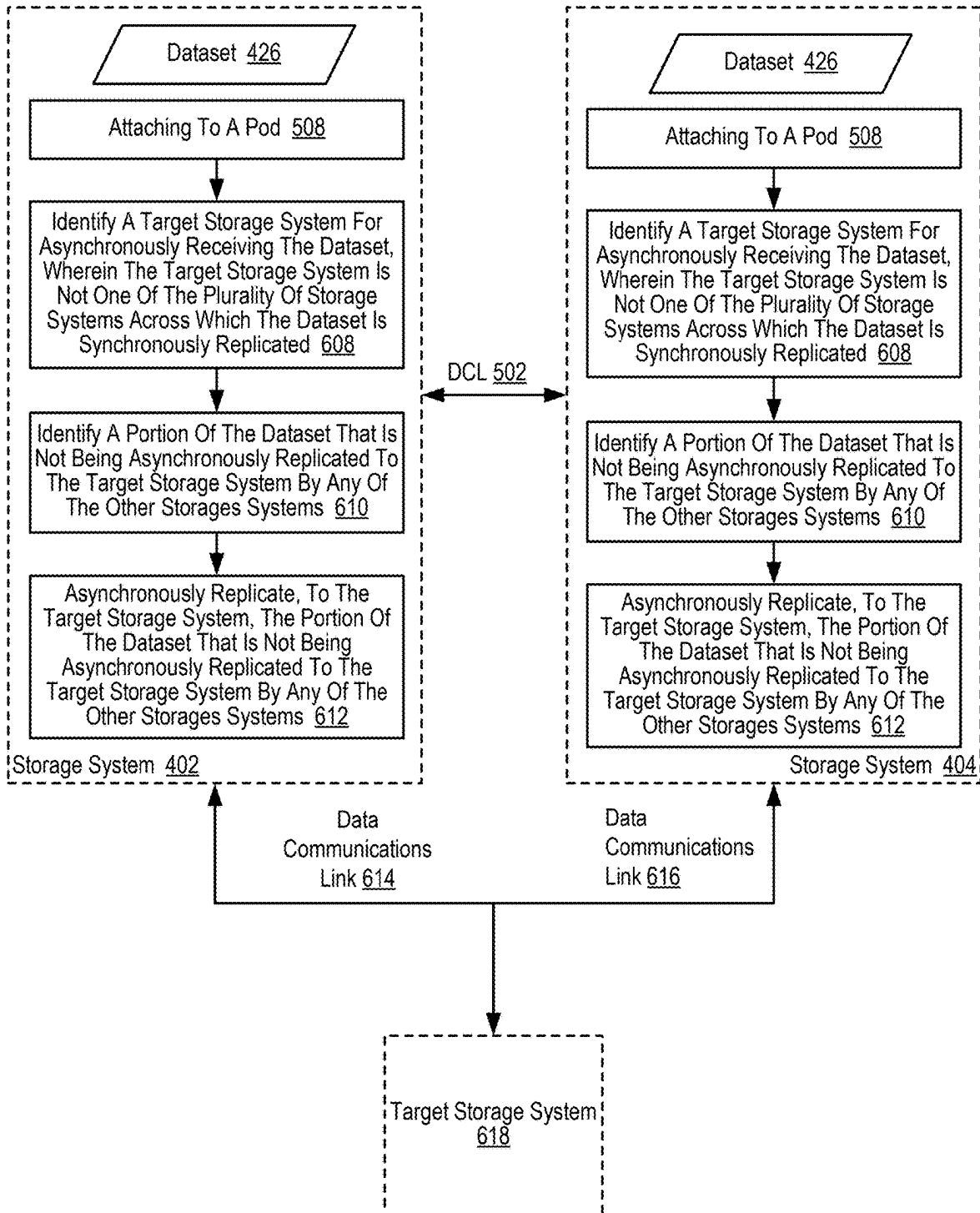


FIG. 6

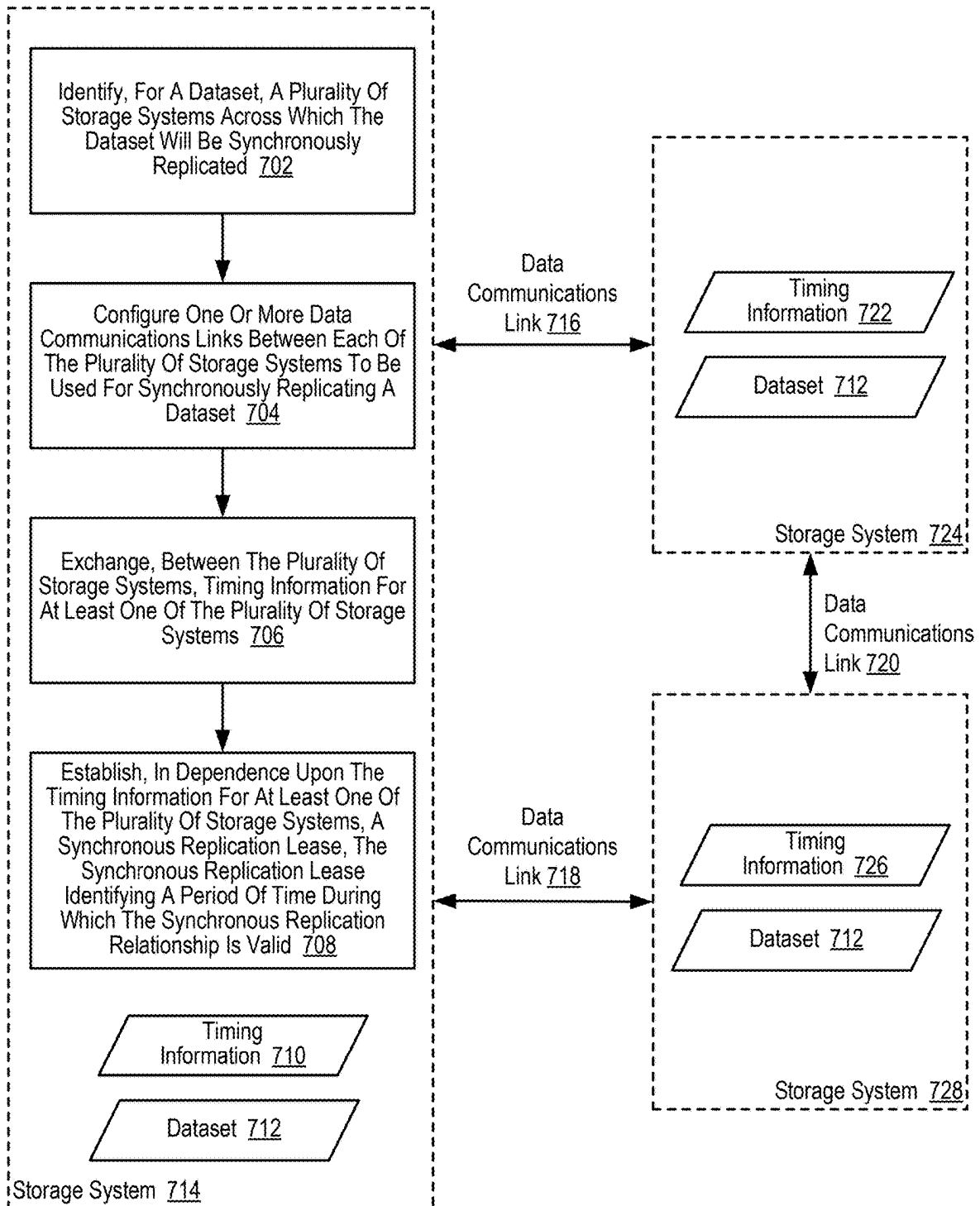


FIG. 7

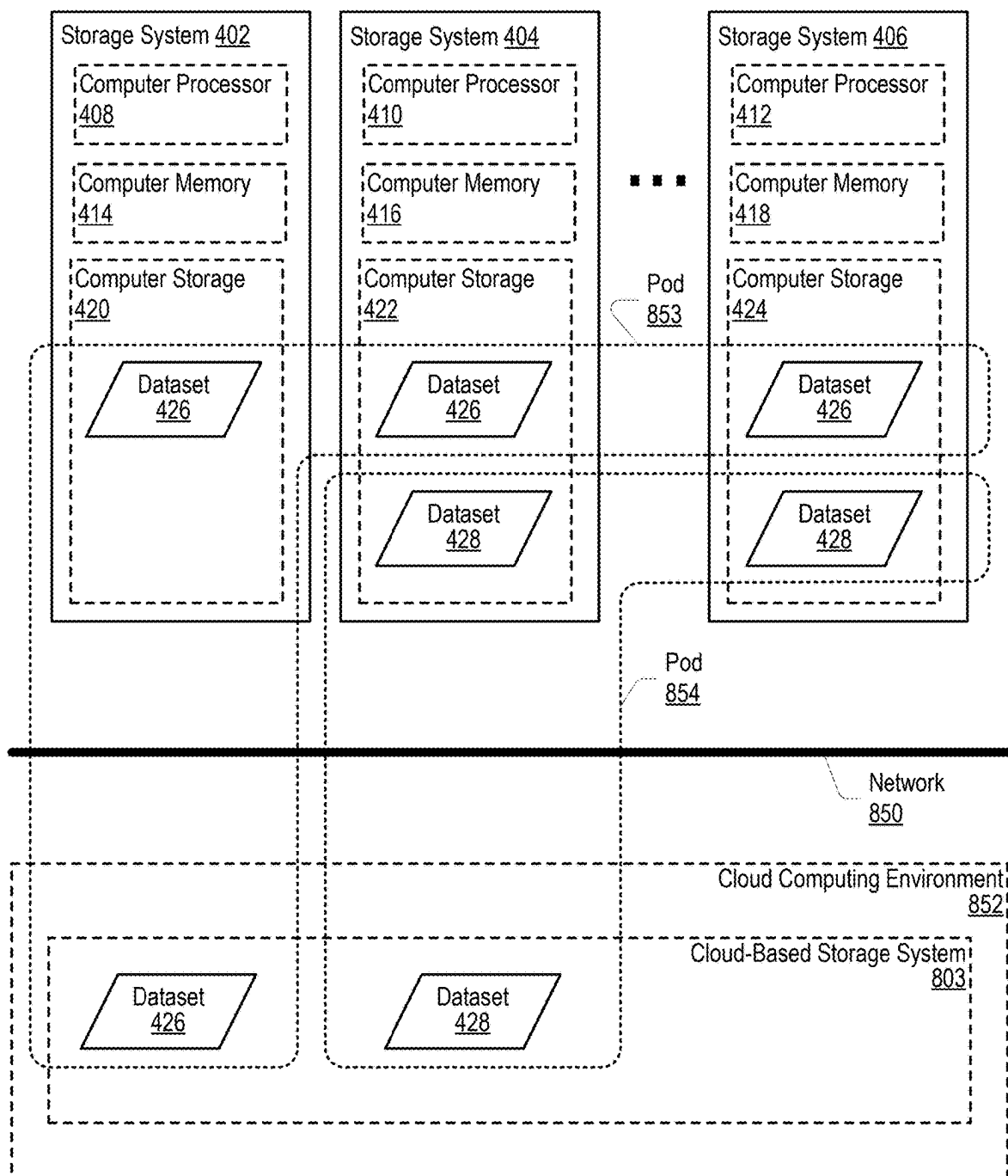


FIG. 8

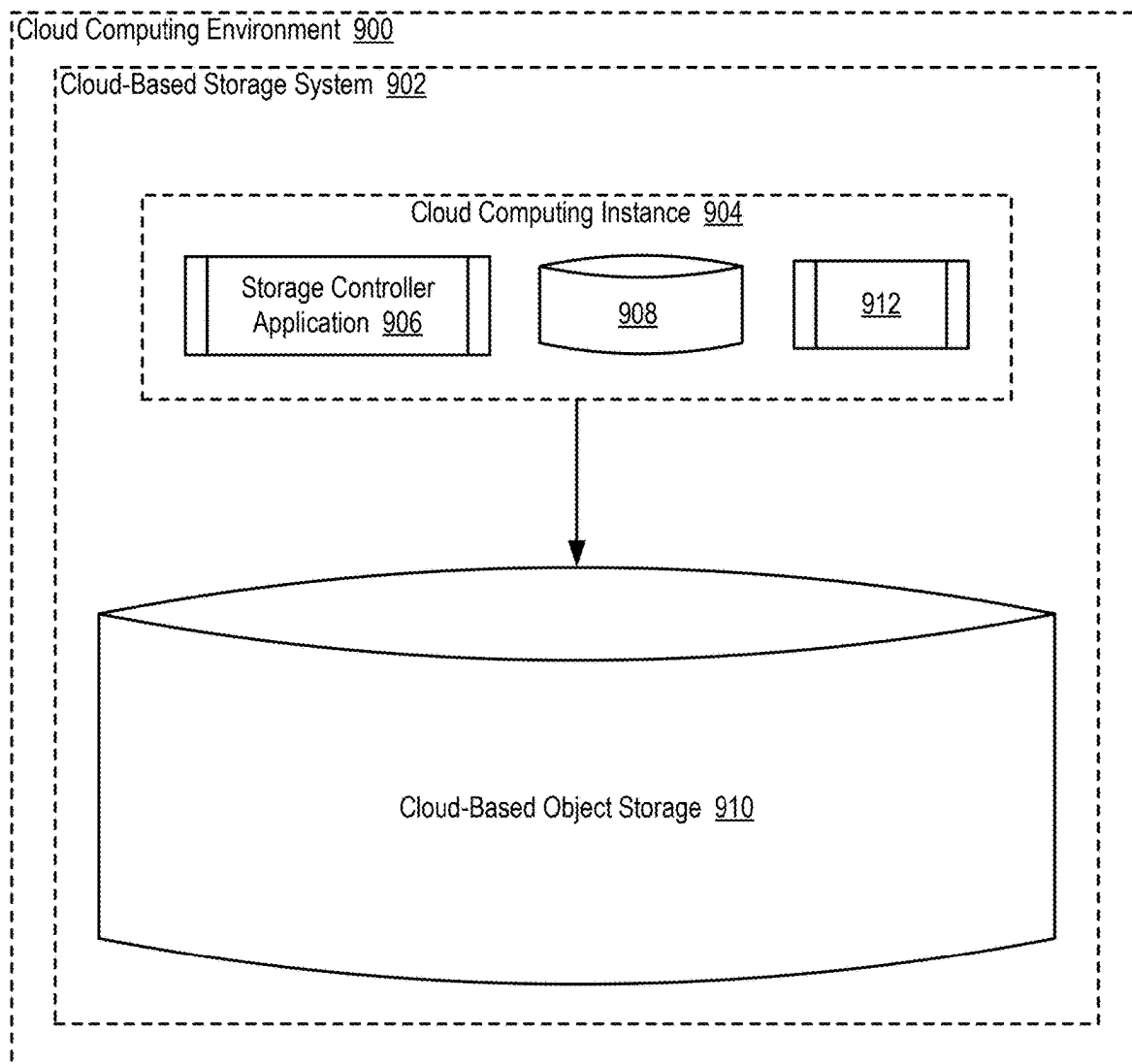


FIG. 9

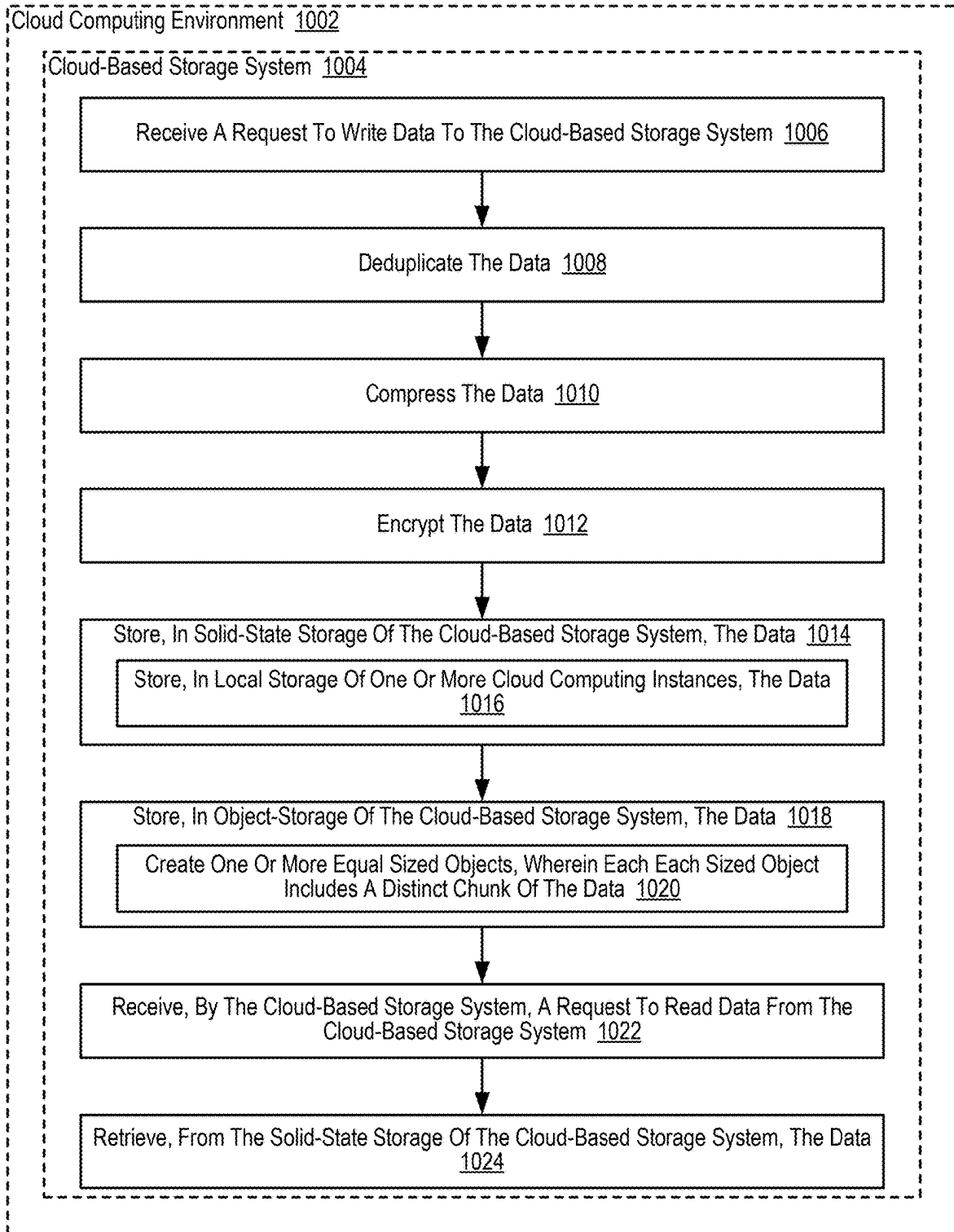


FIG. 10

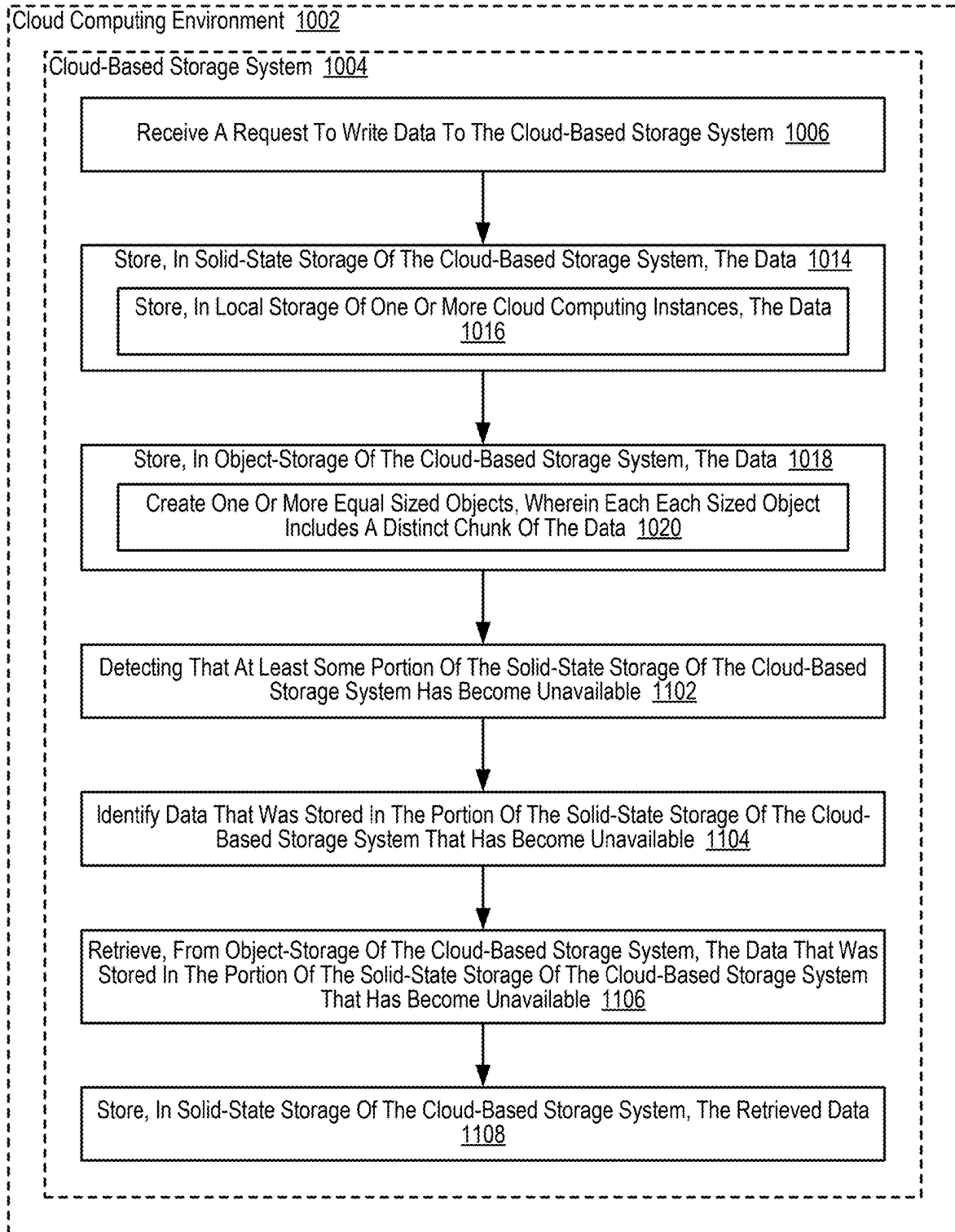


FIG. 11

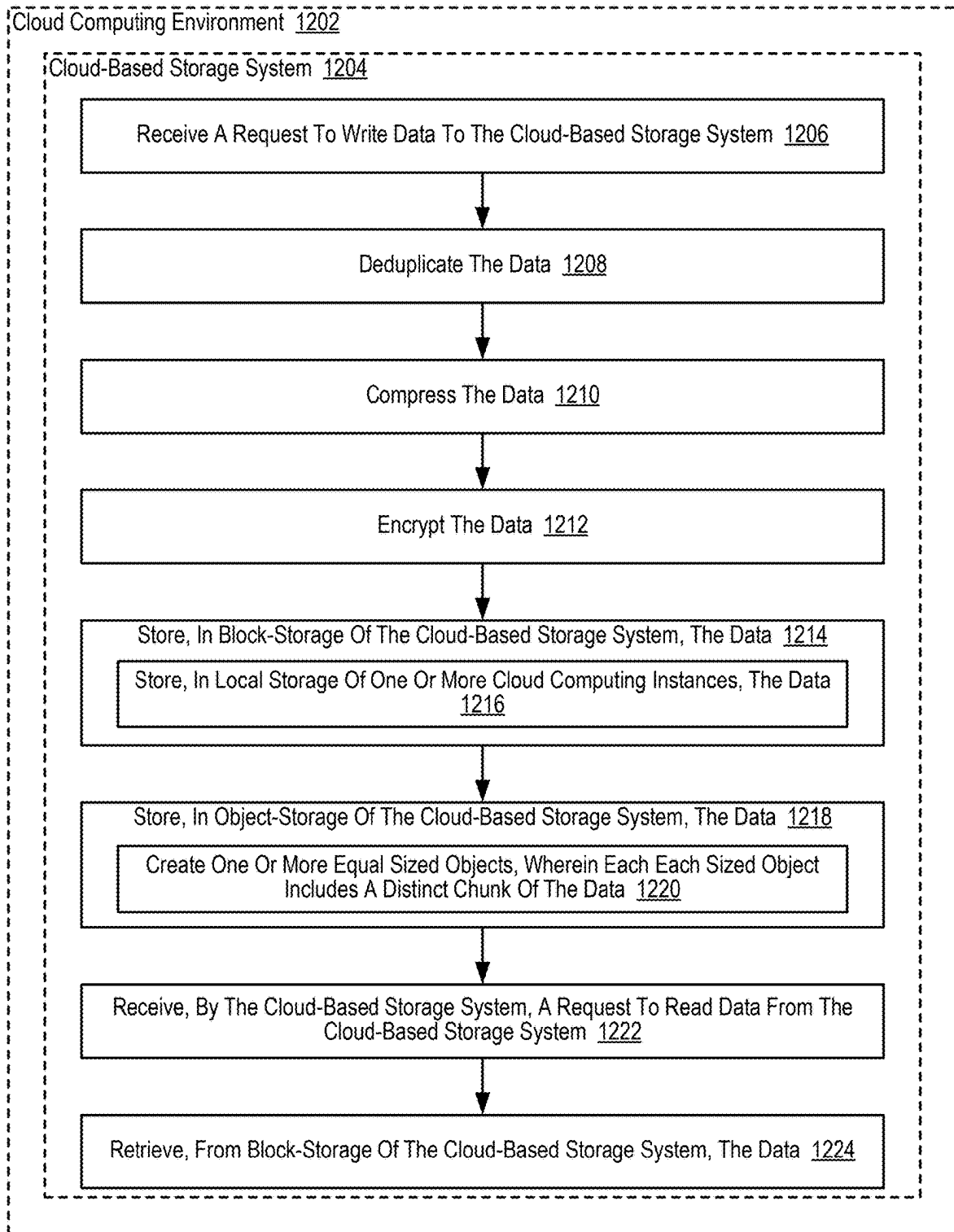


FIG. 12

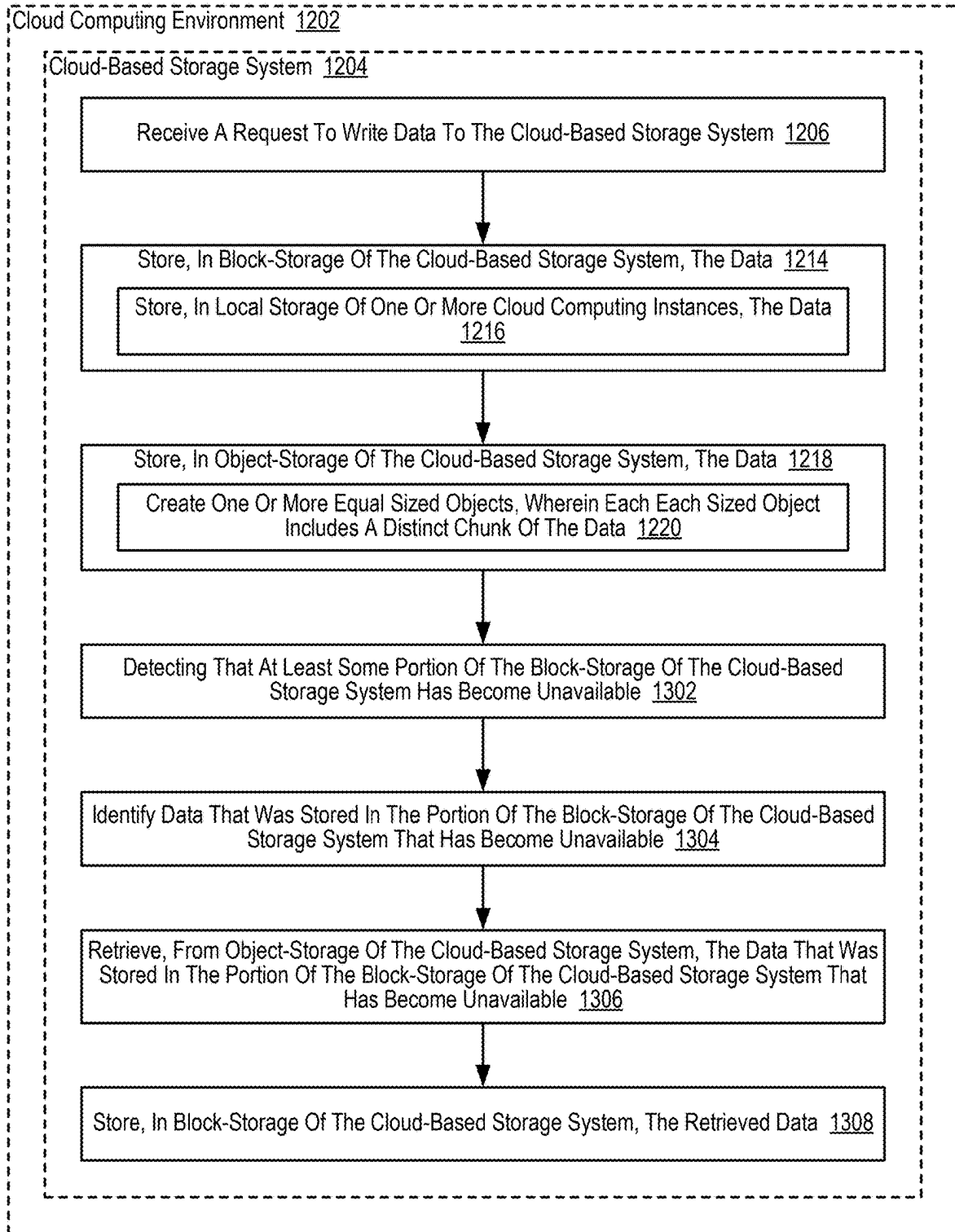


FIG. 13

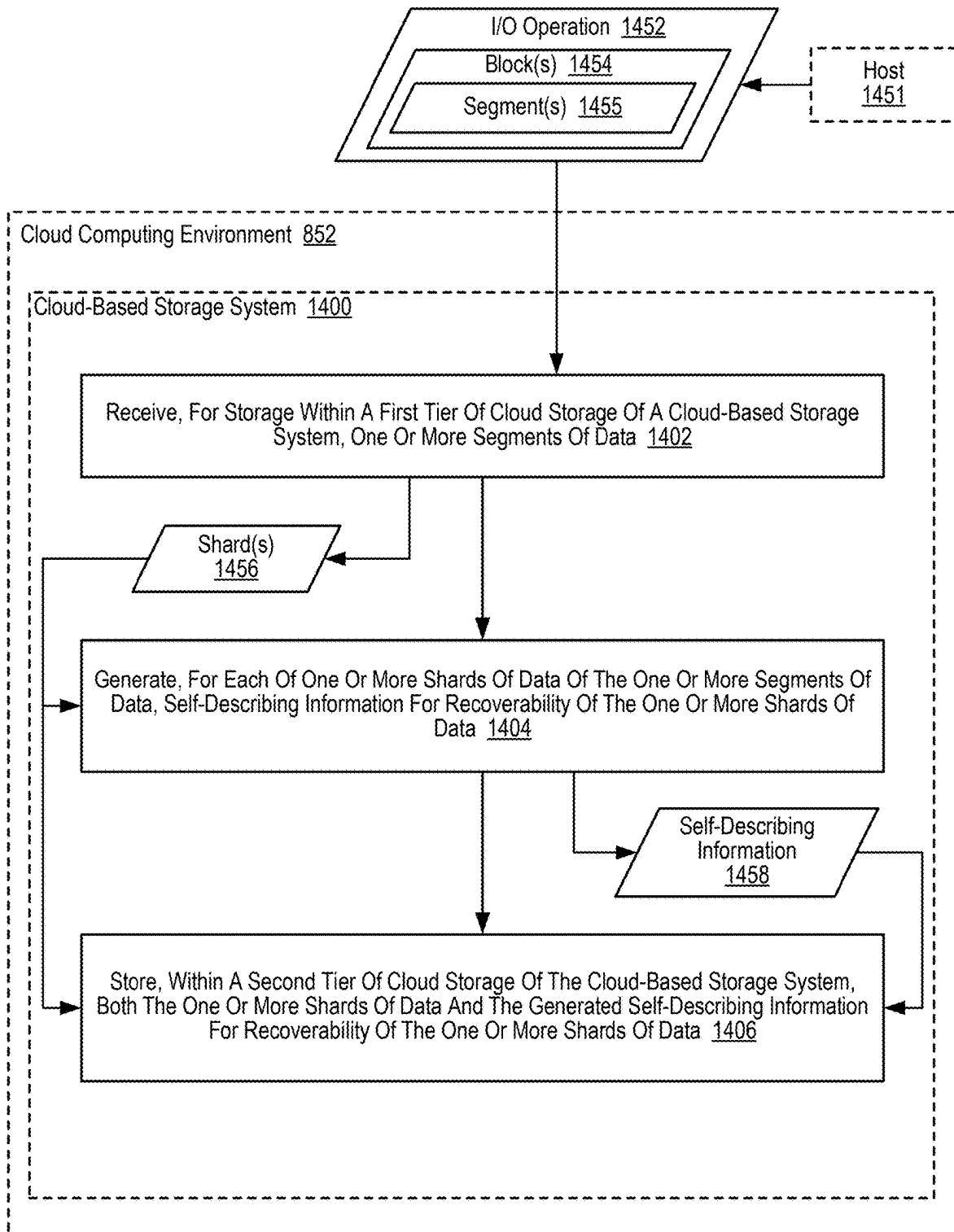


FIG. 14

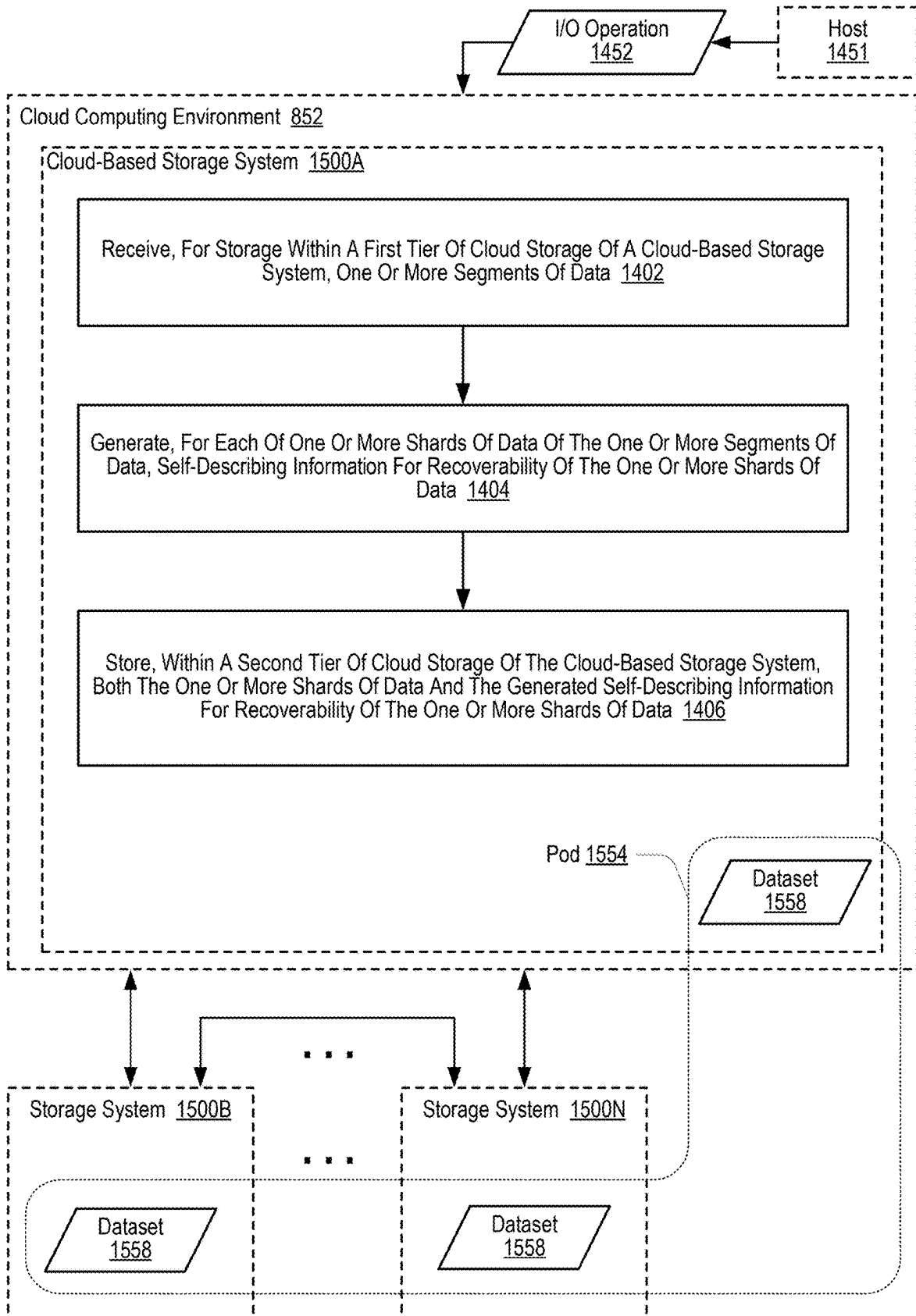


FIG. 15

FIG. 16

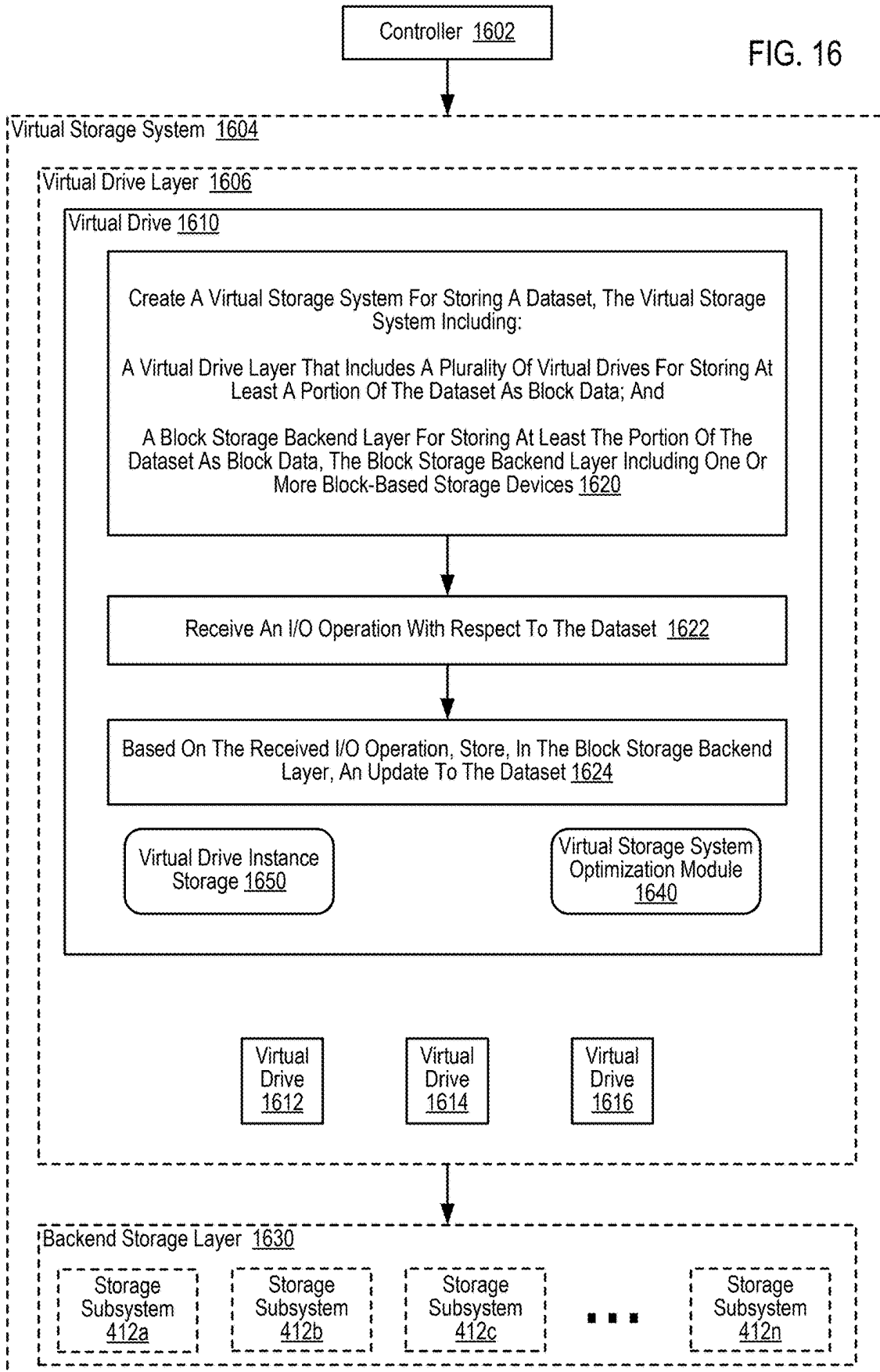


FIG. 17

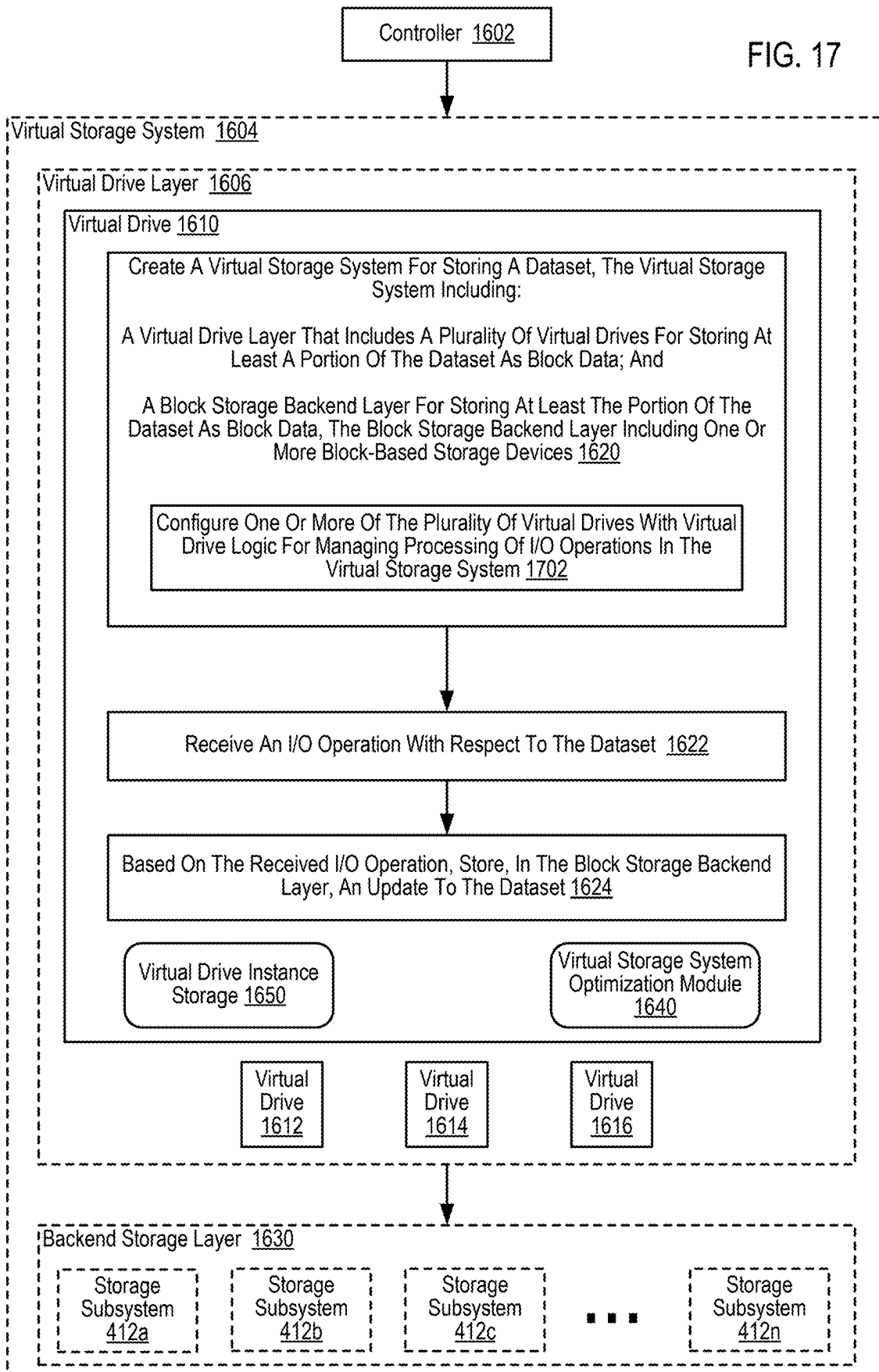


FIG. 18

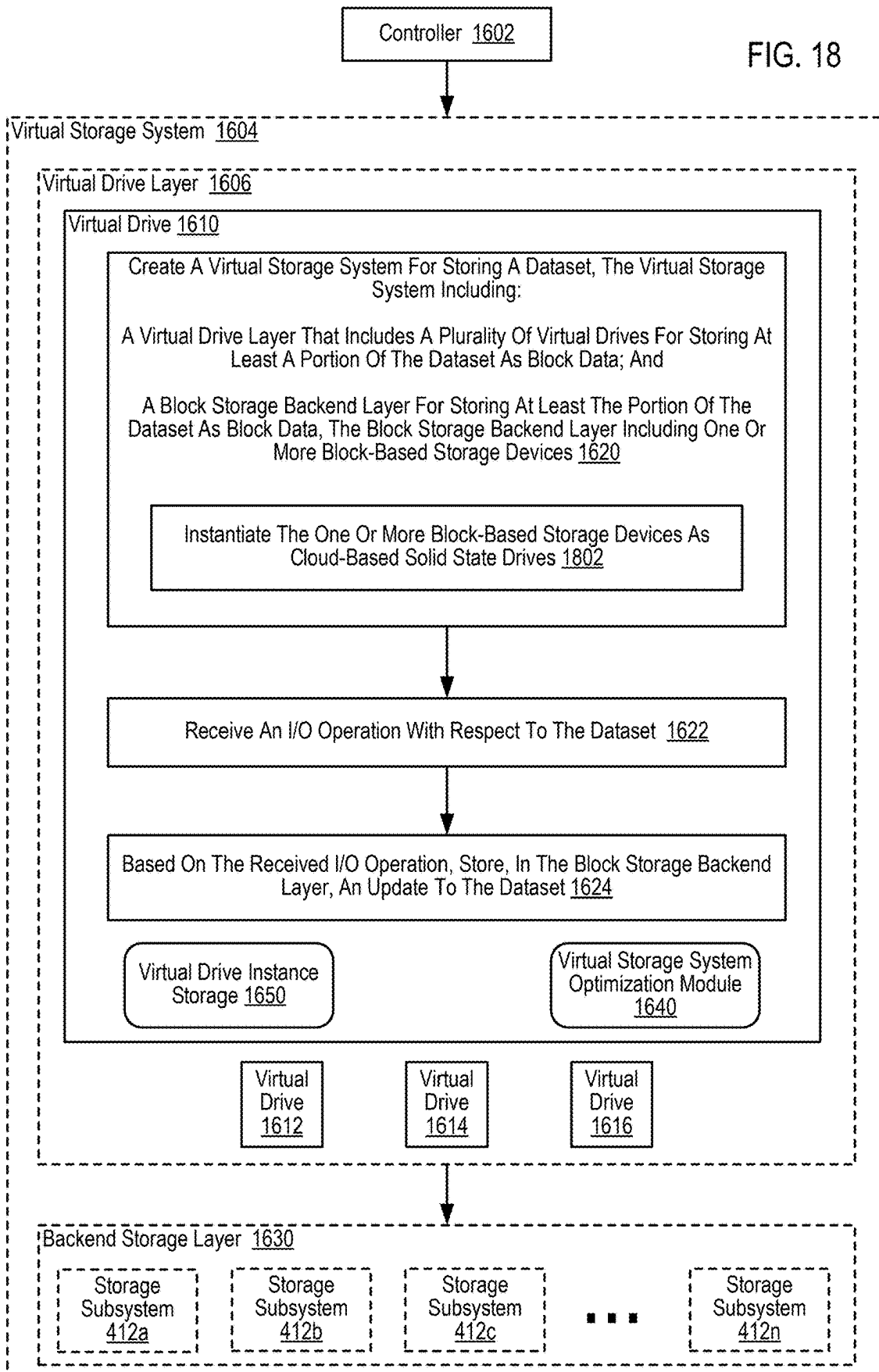


FIG. 19

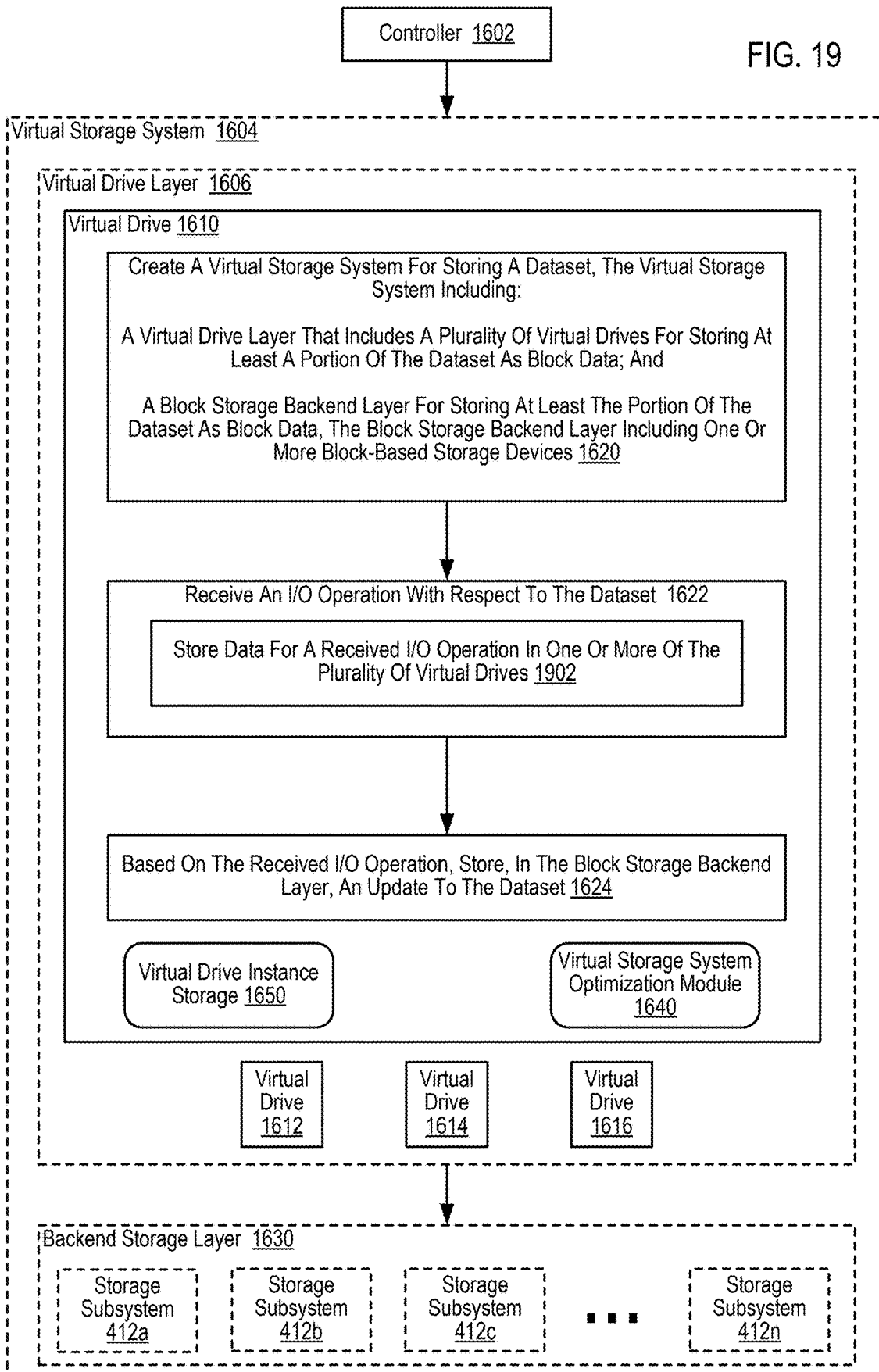
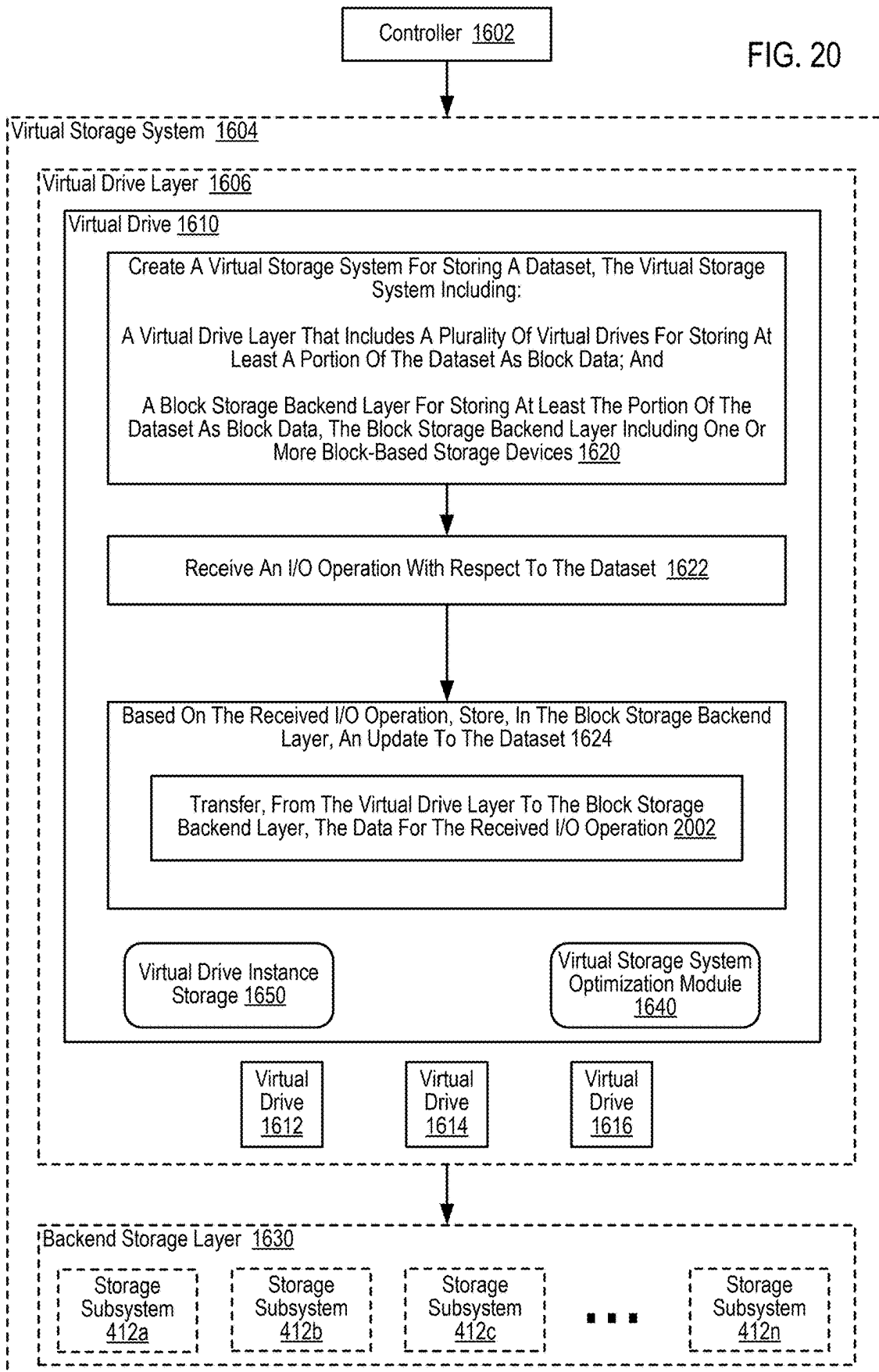


FIG. 20



PROVIDING BLOCK-BASED DATASET STORAGE USING LAYERED VIRTUAL AND BACKEND DEVICES

BRIEF DESCRIPTION OF DRAWINGS

[0001] FIG. 1A illustrates a first example system for data storage.

[0002] FIG. 1B illustrates a second example system for data storage.

[0003] FIG. 1C illustrates a third example system for data storage.

[0004] FIG. 1D illustrates a fourth example system for data storage.

[0005] FIG. 2A is a perspective view of a storage cluster with multiple storage nodes and internal storage coupled to each storage node to provide network attached storage.

[0006] FIG. 2B is a block diagram showing an interconnect switch coupling multiple storage nodes in accordance with some embodiments.

[0007] FIG. 2C is a multiple level block diagram, showing contents of a storage node and contents of one of the non-volatile solid state storage units.

[0008] FIG. 2D shows a storage server environment, which uses embodiments of the storage nodes and storage units of some previous figures in accordance with some embodiments.

[0009] FIG. 2E is a blade hardware block diagram, showing a control plane, compute and storage planes, and authorities interacting with underlying physical resources.

[0010] FIG. 2F depicts elasticity software layers in blades of a storage cluster.

[0011] FIG. 2G depicts authorities and storage resources in blades of a storage cluster.

[0012] FIG. 3A sets forth a diagram of a storage system that is coupled for data communications with a cloud services provider in accordance with some embodiments of the present disclosure.

[0013] FIG. 3B sets forth a diagram of a storage system.

[0014] FIG. 3C sets forth a diagram of a storage system.

[0015] FIG. 3D sets forth a diagram of a storage system.

[0016] FIG. 3E illustrates an example of a fleet of storage systems for providing storage services (also referred to herein as ‘data services’).

[0017] FIG. 3F sets forth a diagram of multiple storage systems in accordance with some embodiments of the present disclosure.

[0018] FIG. 4 sets forth a block diagram illustrating a plurality of storage systems that support a pod according to some embodiments of the present disclosure.

[0019] FIG. 5 sets forth a flow chart illustrating an example method of establishing a synchronous replication relationship between two or more storage systems.

[0020] FIG. 6 sets forth a flow chart illustrating an example method of establishing a synchronous replication relationship between two or more storage systems.

[0021] FIG. 7 sets forth a flow chart illustrating an example method of establishing a synchronous replication relationship between two or more storage systems.

[0022] FIG. 8 sets forth a block diagram illustrating a plurality of storage systems that support a pod according to some embodiments of the present disclosure.

[0023] FIG. 9 sets forth an example of a cloud-based storage system.

[0024] FIG. 10 sets forth a flow chart illustrating an example method of servicing I/O operations in a cloud-based storage system.

[0025] FIG. 11 sets forth a flow chart illustrating an example method of servicing I/O operations in a cloud-based storage system.

[0026] FIG. 12 sets forth a flow chart illustrating an example method of servicing I/O operations in a cloud-based storage system.

[0027] FIG. 13 sets forth a flow chart illustrating an additional example method of servicing I/O operations in a cloud-based storage system.

[0028] FIG. 14 sets forth a flow chart illustrating an example method of data resiliency in a cloud-based storage system in accordance with some embodiments of the present disclosure.

[0029] FIG. 15 sets forth a flow chart illustrating an example method of data resiliency in a cloud-based storage system in accordance with some embodiments of the present disclosure.

[0030] FIG. 16 sets forth a flow chart illustrating an example method of providing block-based storage in accordance with some embodiments of the present disclosure.

[0031] FIG. 17 sets forth a flow chart illustrating an example method of providing block-based storage in accordance with some embodiments of the present disclosure.

[0032] FIG. 18 sets forth a flow chart illustrating an example method of providing block-based storage in accordance with some embodiments of the present disclosure.

[0033] FIG. 19 sets forth a flow chart illustrating an example method of providing block-based storage in accordance with some embodiments of the present disclosure.

[0034] FIG. 20 sets forth a flow chart illustrating an example method of providing block-based storage in accordance with some embodiments of the present disclosure.

DESCRIPTION OF EMBODIMENTS

[0035] Example methods, apparatus, and products for resiliency in a cloud-based storage system in accordance with embodiments of the present disclosure are described with reference to the accompanying drawings, beginning with FIG. 1A. FIG. 1A illustrates an example system for data storage, in accordance with some implementations. System 100 (also referred to as “storage system” herein) includes numerous elements for purposes of illustration rather than limitation. It may be noted that system 100 may include the same, more, or fewer elements configured in the same or different manner in other implementations.

[0036] System 100 includes a number of computing devices 164A-B. Computing devices (also referred to as “client devices” herein) may be embodied, for example, a server in a data center, a workstation, a personal computer, a notebook, or the like. Computing devices 164A-B may be coupled for data communications to one or more storage arrays 102A-B through a storage area network (‘SAN’) 158 or a local area network (‘LAN’) 160.

[0037] The SAN 158 may be implemented with a variety of data communications fabrics, devices, and protocols. For example, the fabrics for SAN 158 may include Fibre Channel, Ethernet, Infiniband, Serial Attached Small Computer System Interface (‘SAS’), or the like. Data communications protocols for use with SAN 158 may include Advanced Technology Attachment (‘ATA’), Fibre Channel Protocol, Small Computer System Interface (‘SCSI’), Internet Small

Computer System Interface ('iSCSI'), HyperSCSI, Non-Volatile Memory Express ('NVMe') over Fabrics, or the like. It may be noted that SAN 158 is provided for illustration, rather than limitation. Other data communication couplings may be implemented between computing devices 164A-B and storage arrays 102A-B.

[0038] The LAN 160 may also be implemented with a variety of fabrics, devices, and protocols. For example, the fabrics for LAN 160 may include Ethernet (802.3), wireless (802.11), or the like. Data communication protocols for use in LAN 160 may include Transmission Control Protocol ('TCP'), User Datagram Protocol ('UDP'), Internet Protocol ('IP'), HyperText Transfer Protocol ('HTTP'), Wireless Access Protocol ('WAP'), Handheld Device Transport Protocol ('HDTIP'), Session Initiation Protocol ('SIP'), Real Time Protocol ('RTP'), or the like.

[0039] Storage arrays 102A-B may provide persistent data storage for the computing devices 164A-B. Storage array 102A may be contained in a chassis (not shown), and storage array 102B may be contained in another chassis (not shown), in some implementations. Storage array 102A and 102B may include one or more storage array controllers 110A-D (also referred to as "controller" herein). A storage array controller 110A-D may be embodied as a module of automated computing machinery comprising computer hardware, computer software, or a combination of computer hardware and software. In some implementations, the storage array controllers 110A-D may be configured to carry out various storage tasks. Storage tasks may include writing data received from the computing devices 164A-B to storage array 102A-B, erasing data from storage array 102A-B, retrieving data from storage array 102A-B and providing data to computing devices 164A-B, monitoring and reporting of storage device utilization and performance, performing redundancy operations, such as Redundant Array of Independent Drives ('RAID') or RAID-like data redundancy operations, compressing data, encrypting data, and so forth.

[0040] Storage array controller 110A-D may be implemented in a variety of ways, including as a Field Programmable Gate Array ('FPGA'), a Programmable Logic Chip ('PLC'), an Application Specific Integrated Circuit ('ASIC'), System-on-Chip ('SOC'), or any computing device that includes discrete components such as a processing device, central processing unit, computer memory, or various adapters. Storage array controller 110A-D may include, for example, a data communications adapter configured to support communications via the SAN 158 or LAN 160. In some implementations, storage array controller 110A-D may be independently coupled to the LAN 160. In some implementations, storage array controller 110A-D may include an I/O controller or the like that couples the storage array controller 110A-D for data communications, through a midplane (not shown), to a persistent storage resource 170A-B (also referred to as a "storage resource" herein). The persistent storage resource 170A-B may include any number of storage drives 171A-F (also referred to as "storage devices" herein) and any number of non-volatile Random Access Memory ('NVRAM') devices (not shown).

[0041] In some implementations, the NVRAM devices of a persistent storage resource 170A-B may be configured to receive, from the storage array controller 110A-D, data to be stored in the storage drives 171A-F. In some examples, the data may originate from computing devices 164A-B. In

some examples, writing data to the NVRAM device may be carried out more quickly than directly writing data to the storage drive 171A-F. In some implementations, the storage array controller 110A-D may be configured to utilize the NVRAM devices as a quickly accessible buffer for data destined to be written to the storage drives 171A-F. Latency for write requests using NVRAM devices as a buffer may be improved relative to a system in which a storage array controller 110A-D writes data directly to the storage drives 171A-F. In some implementations, the NVRAM devices may be implemented with computer memory in the form of high bandwidth, low latency RAM. The NVRAM device is referred to as "non-volatile" because the NVRAM device may receive or include a unique power source that maintains the state of the RAM after main power loss to the NVRAM device. Such a power source may be a battery, one or more capacitors, or the like. In response to a power loss, the NVRAM device may be configured to write the contents of the RAM to a persistent storage, such as the storage drives 171A-F.

[0042] In some implementations, storage drive 171A-F may refer to any device configured to record data persistently, where "persistently" or "persistent" refers as to a device's ability to maintain recorded data after loss of power. In some implementations, storage drive 171A-F may correspond to non-disk storage media. For example, the storage drive 171A-F may be one or more solid-state drives ('SSDs'), flash memory based storage, any type of solid-state non-volatile memory, or any other type of non-mechanical storage device. In other implementations, storage drive 171A-F may include mechanical or spinning hard disk, such as hard-disk drives ('HDD').

[0043] In some implementations, the storage array controllers 110A-D may be configured for offloading device management responsibilities from storage drive 171A-F in storage array 102A-B. For example, storage array controllers 110A-D may manage control information that may describe the state of one or more memory blocks in the storage drives 171A-F. The control information may indicate, for example, that a particular memory block has failed and should no longer be written to, that a particular memory block contains boot code for a storage array controller 110A-D, the number of program-erase ('P/E') cycles that have been performed on a particular memory block, the age of data stored in a particular memory block, the type of data that is stored in a particular memory block, and so forth. In some implementations, the control information may be stored with an associated memory block as metadata. In other implementations, the control information for the storage drives 171A-F may be stored in one or more particular memory blocks of the storage drives 171A-F that are selected by the storage array controller 110A-D. The selected memory blocks may be tagged with an identifier indicating that the selected memory block contains control information. The identifier may be utilized by the storage array controllers 110A-D in conjunction with storage drives 171A-F to quickly identify the memory blocks that contain control information. For example, the storage controllers 110A-D may issue a command to locate memory blocks that contain control information. It may be noted that control information may be so large that parts of the control information may be stored in multiple locations, that the control information may be stored in multiple locations for purposes

of redundancy, for example, or that the control information may otherwise be distributed across multiple memory blocks in the storage drives 171A-F.

[0044] In some implementations, storage array controllers 110A-D may offload device management responsibilities from storage drives 171A-F of storage array 102A-B by retrieving, from the storage drives 171A-F, control information describing the state of one or more memory blocks in the storage drives 171A-F. Retrieving the control information from the storage drives 171A-F may be carried out, for example, by the storage array controller 110A-D querying the storage drives 171A-F for the location of control information for a particular storage drive 171A-F. The storage drives 171A-F may be configured to execute instructions that enable the storage drives 171A-F to identify the location of the control information. The instructions may be executed by a controller (not shown) associated with or otherwise located on the storage drive 171A-F and may cause the storage drive 171A-F to scan a portion of each memory block to identify the memory blocks that store control information for the storage drives 171A-F. The storage drives 171A-F may respond by sending a response message to the storage array controller 110A-D that includes the location of control information for the storage drive 171A-F. Responsive to receiving the response message, storage array controllers 110A-D may issue a request to read data stored at the address associated with the location of control information for the storage drives 171A-F.

[0045] In other implementations, the storage array controllers 110A-D may further offload device management responsibilities from storage drives 171A-F by performing, in response to receiving the control information, a storage drive management operation. A storage drive management operation may include, for example, an operation that is typically performed by the storage drive 171A-F (e.g., the controller (not shown) associated with a particular storage drive 171A-F). A storage drive management operation may include, for example, ensuring that data is not written to failed memory blocks within the storage drive 171A-F, ensuring that data is written to memory blocks within the storage drive 171A-F in such a way that adequate wear leveling is achieved, and so forth.

[0046] In some implementations, storage array 102A-B may implement two or more storage array controllers 110A-D. For example, storage array 102A may include storage array controllers 110A and storage array controllers 110B. At a given instant, a single storage array controller 110A-D (e.g., storage array controller 110A) of a storage system 100 may be designated with primary status (also referred to as “primary controller” herein), and other storage array controllers 110A-D (e.g., storage array controller 110B) may be designated with secondary status (also referred to as “secondary controller” herein). The primary controller may have particular rights, such as permission to alter data in persistent storage resource 170A-B (e.g., writing data to persistent storage resource 170A-B). At least some of the rights of the primary controller may supersede the rights of the secondary controller. For instance, the secondary controller may not have permission to alter data in persistent storage resource 170A-B when the primary controller has the right. The status of storage array controllers 110A-D may change. For example, storage array controller 110A may be designated with secondary status, and storage array controller 110B may be designated with primary status.

[0047] In some implementations, a primary controller, such as storage array controller 110A, may serve as the primary controller for one or more storage arrays 102A-B, and a second controller, such as storage array controller 110B, may serve as the secondary controller for the one or more storage arrays 102A-B. For example, storage array controller 110A may be the primary controller for storage array 102A and storage array 102B, and storage array controller 110B may be the secondary controller for storage array 102A and 102B. In some implementations, storage array controllers 110C and 110D (also referred to as “storage processing modules”) may neither have primary or secondary status. Storage array controllers 110C and 110D, implemented as storage processing modules, may act as a communication interface between the primary and secondary controllers (e.g., storage array controllers 110A and 110B, respectively) and storage array 102B. For example, storage array controller 110A of storage array 102A may send a write request, via SAN 158, to storage array 102B. The write request may be received by both storage array controllers 110C and 110D of storage array 102B. Storage array controllers 110C and 110D facilitate the communication, e.g., send the write request to the appropriate storage drive 171A-F. It may be noted that in some implementations storage processing modules may be used to increase the number of storage drives controlled by the primary and secondary controllers.

[0048] In some implementations, storage array controllers 110A-D are communicatively coupled, via a midplane (not shown), to one or more storage drives 171A-F and to one or more NVRAM devices (not shown) that are included as part of a storage array 102A-B. The storage array controllers 110A-D may be coupled to the midplane via one or more data communication links and the midplane may be coupled to the storage drives 171A-F and the NVRAM devices via one or more data communications links. The data communications links described herein are collectively illustrated by data communications links 108A-D and may include a Peripheral Component Interconnect Express (‘PCIe’) bus, for example.

[0049] FIG. 1B illustrates an example system for data storage, in accordance with some implementations. Storage array controller 101 illustrated in FIG. 1B may be similar to the storage array controllers 110A-D described with respect to FIG. 1A. In one example, storage array controller 101 may be similar to storage array controller 110A or storage array controller 110B. Storage array controller 101 includes numerous elements for purposes of illustration rather than limitation. It may be noted that storage array controller 101 may include the same, more, or fewer elements configured in the same or different manner in other implementations. It may be noted that elements of FIG. 1A may be included below to help illustrate features of storage array controller 101.

[0050] Storage array controller 101 may include one or more processing devices 104 and random access memory (‘RAM’) 111. Processing device 104 (or controller 101) represents one or more general-purpose processing devices such as a microprocessor, central processing unit, or the like. More particularly, the processing device 104 (or controller 101) may be a complex instruction set computing (‘CISC’) microprocessor, reduced instruction set computing (‘RISC’) microprocessor, very long instruction word (‘VLIW’) microprocessor, or a processor implementing other instruc-

tion sets or processors implementing a combination of instruction sets. The processing device **104** (or controller **101**) may also be one or more special-purpose processing devices such as an ASIC, an FPGA, a digital signal processor ('DSP'), network processor, or the like.

[0051] The processing device **104** may be connected to the RAM **111** via a data communications link **106**, which may be embodied as a high speed memory bus such as a Double-Data Rate **4** ('DDR4') bus. Stored in RAM **111** is an operating system **112**. In some implementations, instructions **113** are stored in RAM **111**. Instructions **113** may include computer program instructions for performing operations in a direct-mapped flash storage system. In one embodiment, a direct-mapped flash storage system is one that addresses data blocks within flash drives directly and without an address translation performed by the storage controllers of the flash drives.

[0052] In some implementations, storage array controller **101** includes one or more host bus adapters **103A-C** that are coupled to the processing device **104** via a data communications link **105A-C**. In some implementations, host bus adapters **103A-C** may be computer hardware that connects a host system (e.g., the storage array controller) to other network and storage arrays. In some examples, host bus adapters **103A-C** may be a Fibre Channel adapter that enables the storage array controller **101** to connect to a SAN, an Ethernet adapter that enables the storage array controller **101** to connect to a LAN, or the like. Host bus adapters **103A-C** may be coupled to the processing device **104** via a data communications link **105A-C** such as, for example, a PCIe bus.

[0053] In some implementations, storage array controller **101** may include a host bus adapter **114** that is coupled to an expander **115**. The expander **115** may be used to attach a host system to a larger number of storage drives. The expander **115** may, for example, be a SAS expander utilized to enable the host bus adapter **114** to attach to storage drives in an implementation where the host bus adapter **114** is embodied as a SAS controller.

[0054] In some implementations, storage array controller **101** may include a switch **116** coupled to the processing device **104** via a data communications link **109**. The switch **116** may be a computer hardware device that can create multiple endpoints out of a single endpoint, thereby enabling multiple devices to share a single endpoint. The switch **116** may, for example, be a PCIe switch that is coupled to a PCIe bus (e.g., data communications link **109**) and presents multiple PCIe connection points to the midplane.

[0055] In some implementations, storage array controller **101** includes a data communications link **107** for coupling the storage array controller **101** to other storage array controllers. In some examples, data communications link **107** may be a QuickPath Interconnect (QPI) interconnect.

[0056] A traditional storage system that uses traditional flash drives may implement a process across the flash drives that are part of the traditional storage system. For example, a higher level process of the storage system may initiate and control a process across the flash drives. However, a flash drive of the traditional storage system may include its own storage controller that also performs the process. Thus, for the traditional storage system, a higher level process (e.g., initiated by the storage system) and a lower level process (e.g., initiated by a storage controller of the storage system) may both be performed.

[0057] To resolve various deficiencies of a traditional storage system, operations may be performed by higher level processes and not by the lower level processes. For example, the flash storage system may include flash drives that do not include storage controllers that provide the process. Thus, the operating system of the flash storage system itself may initiate and control the process. This may be accomplished by a direct-mapped flash storage system that addresses data blocks within the flash drives directly and without an address translation performed by the storage controllers of the flash drives.

[0058] In some implementations, storage drive **171A-F** may be one or more zoned storage devices. In some implementations, the one or more zoned storage devices may be a shingled HDD. In some implementations, the one or more storage devices may be a flash-based SSD. In a zoned storage device, a zoned namespace on the zoned storage device can be addressed by groups of blocks that are grouped and aligned by a natural size, forming a number of addressable zones. In some implementations utilizing an SSD, the natural size may be based on the erase block size of the SSD. In some implementations, the zones of the zoned storage device may be defined during initialization of the zoned storage device. In some implementations, the zones may be defined dynamically as data is written to the zoned storage device.

[0059] In some implementations, zones may be heterogeneous, with some zones each being a page group and other zones being multiple page groups. In some implementations, some zones may correspond to an erase block and other zones may correspond to multiple erase blocks. In an implementation, zones may be any combination of differing numbers of pages in page groups and/or erase blocks, for heterogeneous mixes of programming modes, manufacturers, product types and/or product generations of storage devices, as applied to heterogeneous assemblies, upgrades, distributed storages, etc. In some implementations, zones may be defined as having usage characteristics, such as a property of supporting data with particular kinds of longevity (very short lived or very long lived, for example). These properties could be used by a zoned storage device to determine how the zone will be managed over the zone's expected lifetime.

[0060] It should be appreciated that a zone is a virtual construct. Any particular zone may not have a fixed location at a storage device. Until allocated, a zone may not have any location at a storage device. A zone may correspond to a number representing a chunk of virtually allocatable space that is the size of an erase block or other block size in various implementations. When the system allocates or opens a zone, zones get allocated to flash or other solid-state storage memory and, as the system writes to the zone, pages are written to that mapped flash or other solid-state storage memory of the zoned storage device. When the system closes the zone, the associated erase block(s) or other sized block(s) are completed. At some point in the future, the system may delete a zone which will free up the zone's allocated space. During its lifetime, a zone may be moved around to different locations of the zoned storage device, e.g., as the zoned storage device does internal maintenance.

[0061] In some implementations, the zones of the zoned storage device may be in different states. A zone may be in an empty state in which data has not been stored at the zone. An empty zone may be opened explicitly, or implicitly by

writing data to the zone. This is the initial state for zones on a fresh zoned storage device, but may also be the result of a zone reset. In some implementations, an empty zone may have a designated location within the flash memory of the zoned storage device. In an implementation, the location of the empty zone may be chosen when the zone is first opened or first written to (or later if writes are buffered into memory). A zone may be in an open state either implicitly or explicitly, where a zone that is in an open state may be written to store data with write or append commands. In an implementation, a zone that is in an open state may also be written to using a copy command that copies data from a different zone. In some implementations, a zoned storage device may have a limit on the number of open zones at a particular time.

[0062] A zone in a closed state is a zone that has been partially written to, but has entered a closed state after issuing an explicit close operation. A zone in a closed state may be left available for future writes, but may reduce some of the run-time overhead consumed by keeping the zone in an open state. In some implementations, a zoned storage device may have a limit on the number of closed zones at a particular time. A zone in a full state is a zone that is storing data and can no longer be written to. A zone may be in a full state either after writes have written data to the entirety of the zone or as a result of a zone finish operation. Prior to a finish operation, a zone may or may not have been completely written. After a finish operation, however, the zone may not be opened a written to further without first performing a zone reset operation.

[0063] The mapping from a zone to an erase block (or to a shingled track in an HDD) may be arbitrary, dynamic, and hidden from view. The process of opening a zone may be an operation that allows a new zone to be dynamically mapped to underlying storage of the zoned storage device, and then allows data to be written through appending writes into the zone until the zone reaches capacity. The zone can be finished at any point, after which further data may not be written into the zone. When the data stored at the zone is no longer needed, the zone can be reset which effectively deletes the zone's content from the zoned storage device, making the physical storage held by that zone available for the subsequent storage of data. Once a zone has been written and finished, the zoned storage device ensures that the data stored at the zone is not lost until the zone is reset. In the time between writing the data to the zone and the resetting of the zone, the zone may be moved around between shingle tracks or erase blocks as part of maintenance operations within the zoned storage device, such as by copying data to keep the data refreshed or to handle memory cell aging in an SSD.

[0064] In some implementations utilizing an HDD, the resetting of the zone may allow the shingle tracks to be allocated to a new, opened zone that may be opened at some point in the future. In some implementations utilizing an SSD, the resetting of the zone may cause the associated physical erase block(s) of the zone to be erased and subsequently reused for the storage of data. In some implementations, the zoned storage device may have a limit on the number of open zones at a point in time to reduce the amount of overhead dedicated to keeping zones open.

[0065] The operating system of the flash storage system may identify and maintain a list of allocation units across multiple flash drives of the flash storage system. The allo-

cation units may be entire erase blocks or multiple erase blocks. The operating system may maintain a map or address range that directly maps addresses to erase blocks of the flash drives of the flash storage system.

[0066] Direct mapping to the erase blocks of the flash drives may be used to rewrite data and erase data. For example, the operations may be performed on one or more allocation units that include a first data and a second data where the first data is to be retained and the second data is no longer being used by the flash storage system. The operating system may initiate the process to write the first data to new locations within other allocation units and erasing the second data and marking the allocation units as being available for use for subsequent data. Thus, the process may only be performed by the higher level operating system of the flash storage system without an additional lower level process being performed by controllers of the flash drives.

[0067] Advantages of the process being performed only by the operating system of the flash storage system include increased reliability of the flash drives of the flash storage system as unnecessary or redundant write operations are not being performed during the process. One possible point of novelty here is the concept of initiating and controlling the process at the operating system of the flash storage system. In addition, the process can be controlled by the operating system across multiple flash drives. This is contrast to the process being performed by a storage controller of a flash drive.

[0068] A storage system can consist of two storage array controllers that share a set of drives for failover purposes, or it could consist of a single storage array controller that provides a storage service that utilizes multiple drives, or it could consist of a distributed network of storage array controllers each with some number of drives or some amount of Flash storage where the storage array controllers in the network collaborate to provide a complete storage service and collaborate on various aspects of a storage service including storage allocation and garbage collection.

[0069] FIG. 1C illustrates a third example system 117 for data storage in accordance with some implementations. System 117 (also referred to as "storage system" herein) includes numerous elements for purposes of illustration rather than limitation. It may be noted that system 117 may include the same, more, or fewer elements configured in the same or different manner in other implementations.

[0070] In one embodiment, system 117 includes a dual Peripheral Component Interconnect ('PCI') flash storage device 118 with separately addressable fast write storage. System 117 may include a storage device controller 119. In one embodiment, storage device controller 119A-D may be a CPU, ASIC, FPGA, or any other circuitry that may implement control structures necessary according to the present disclosure. In one embodiment, system 117 includes flash memory devices (e.g., including flash memory devices 120a-n), operatively coupled to various channels of the storage device controller 119. Flash memory devices 120a-n, may be presented to the controller 119A-D as an addressable collection of Flash pages, erase blocks, and/or control elements sufficient to allow the storage device controller 119A-D to program and retrieve various aspects of the Flash. In one embodiment, storage device controller 119A-D may perform operations on flash memory devices 120a-n including storing and retrieving data content of pages, arranging

and erasing any blocks, tracking statistics related to the use and reuse of Flash memory pages, erase blocks, and cells, tracking and predicting error codes and faults within the Flash memory, controlling voltage levels associated with programming and retrieving contents of Flash cells, etc.

[0071] In one embodiment, system 117 may include RAM 121 to store separately addressable fast-write data. In one embodiment, RAM 121 may be one or more separate discrete devices. In another embodiment, RAM 121 may be integrated into storage device controller 119A-D or multiple storage device controllers. The RAM 121 may be utilized for other purposes as well, such as temporary program memory for a processing device (e.g., a CPU) in the storage device controller 119.

[0072] In one embodiment, system 117 may include a stored energy device 122, such as a rechargeable battery or a capacitor. Stored energy device 122 may store energy sufficient to power the storage device controller 119, some amount of the RAM (e.g., RAM 121), and some amount of Flash memory (e.g., Flash memory 120a-120n) for sufficient time to write the contents of RAM to Flash memory. In one embodiment, storage device controller 119A-D may write the contents of RAM to Flash Memory if the storage device controller detects loss of external power.

[0073] In one embodiment, system 117 includes two data communications links 123a, 123b. In one embodiment, data communications links 123a, 123b may be PCI interfaces. In another embodiment, data communications links 123a, 123b may be based on other communications standards (e.g., HyperTransport, InfiniBand, etc.). Data communications links 123a, 123b may be based on non-volatile memory express ('NVMe') or NVMe over fabrics ('NVMe-f') specifications that allow external connection to the storage device controller 119A-D from other components in the storage system 117. It should be noted that data communications links may be interchangeably referred to herein as PCI buses for convenience.

[0074] System 117 may also include an external power source (not shown), which may be provided over one or both data communications links 123a, 123b, or which may be provided separately. An alternative embodiment includes a separate Flash memory (not shown) dedicated for use in storing the content of RAM 121. The storage device controller 119A-D may present a logical device over a PCI bus which may include an addressable fast-write logical device, or a distinct part of the logical address space of the storage device 118, which may be presented as PCI memory or as persistent storage. In one embodiment, operations to store into the device are directed into the RAM 121. On power failure, the storage device controller 119A-D may write stored content associated with the addressable fast-write logical storage to Flash memory (e.g., Flash memory 120a-n) for long-term persistent storage.

[0075] In one embodiment, the logical device may include some presentation of some or all of the content of the Flash memory devices 120a-n, where that presentation allows a storage system including a storage device 118 (e.g., storage system 117) to directly address Flash memory pages and directly reprogram erase blocks from storage system components that are external to the storage device through the PCI bus. The presentation may also allow one or more of the external components to control and retrieve other aspects of the Flash memory including some or all of: tracking statistics related to use and reuse of Flash memory pages, erase

blocks, and cells across all the Flash memory devices; tracking and predicting error codes and faults within and across the Flash memory devices; controlling voltage levels associated with programming and retrieving contents of Flash cells; etc.

[0076] In one embodiment, the stored energy device 122 may be sufficient to ensure completion of in-progress operations to the Flash memory devices 120a-120n. The stored energy device 122 may power storage device controller 119A-D and associated Flash memory devices (e.g., 120a-n) for those operations, as well as for the storing of fast-write RAM to Flash memory. Stored energy device 122 may be used to store accumulated statistics and other parameters kept and tracked by the Flash memory devices 120a-n and/or the storage device controller 119. Separate capacitors or stored energy devices (such as smaller capacitors near or embedded within the Flash memory devices themselves) may be used for some or all of the operations described herein.

[0077] Various schemes may be used to track and optimize the life span of the stored energy component, such as adjusting voltage levels over time, partially discharging the stored energy device 122 to measure corresponding discharge characteristics, etc. If the available energy decreases over time, the effective available capacity of the addressable fast-write storage may be decreased to ensure that it can be written safely based on the currently available stored energy.

[0078] FIG. 1D illustrates a third example storage system 124 for data storage in accordance with some implementations. In one embodiment, storage system 124 includes storage controllers 125a, 125b. In one embodiment, storage controllers 125a, 125b are operatively coupled to Dual PCI storage devices. Storage controllers 125a, 125b may be operatively coupled (e.g., via a storage network 130) to some number of host computers 127a-n.

[0079] In one embodiment, two storage controllers (e.g., 125a and 125b) provide storage services, such as a SCS) block storage array, a file server, an object server, a database or data analytics service, etc. The storage controllers 125a, 125b may provide services through some number of network interfaces (e.g., 126a-d) to host computers 127a-n outside of the storage system 124. Storage controllers 125a, 125b may provide integrated services or an application entirely within the storage system 124, forming a converged storage and compute system. The storage controllers 125a, 125b may utilize the fast write memory within or across storage devices 119a-d to journal in progress operations to ensure the operations are not lost on a power failure, storage controller removal, storage controller or storage system shutdown, or some fault of one or more software or hardware components within the storage system 124.

[0080] In one embodiment, storage controllers 125a, 125b operate as PCI masters to one or the other PCI buses 128a, 128b. In another embodiment, 128a and 128b may be based on other communications standards (e.g., HyperTransport, InfiniBand, etc.). Other storage system embodiments may operate storage controllers 125a, 125b as multi-masters for both PCI buses 128a, 128b. Alternately, a PCI/NVMe/NVMe-f switching infrastructure or fabric may connect multiple storage controllers. Some storage system embodiments may allow storage devices to communicate with each other directly rather than communicating only with storage controllers. In one embodiment, a storage device controller 119a may be operable under direction from a storage controller

125a to synthesize and transfer data to be stored into Flash memory devices from data that has been stored in RAM (e.g., RAM **121** of FIG. **1C**). For example, a recalculated version of RAM content may be transferred after a storage controller has determined that an operation has fully committed across the storage system, or when fast-write memory on the device has reached a certain used capacity, or after a certain amount of time, to ensure improved safety of the data or to release addressable fast-write capacity for reuse. This mechanism may be used, for example, to avoid a second transfer over a bus (e.g., **128a**, **128b**) from the storage controllers **125a**, **125b**. In one embodiment, a recalculation may include compressing data, attaching indexing or other metadata, combining multiple data segments together, performing erasure code calculations, etc.

[0081] In one embodiment, under direction from a storage controller **125a**, **125b**, a storage device controller **119a**, **119b** may be operable to calculate and transfer data to other storage devices from data stored in RAM (e.g., RAM **121** of FIG. **1C**) without involvement of the storage controllers **125a**, **125b**. This operation may be used to mirror data stored in one storage controller **125a** to another storage controller **125b**, or it could be used to offload compression, data aggregation, and/or erasure coding calculations and transfers to storage devices to reduce load on storage controllers or the storage controller interface **129a**, **129b** to the PCI bus **128a**, **128b**.

[0082] A storage device controller **119A-D** may include mechanisms for implementing high availability primitives for use by other parts of a storage system external to the Dual PCI storage device **118**. For example, reservation or exclusion primitives may be provided so that, in a storage system with two storage controllers providing a highly available storage service, one storage controller may prevent the other storage controller from accessing or continuing to access the storage device. This could be used, for example, in cases where one controller detects that the other controller is not functioning properly or where the interconnect between the two storage controllers may itself not be functioning properly.

[0083] In one embodiment, a storage system for use with Dual PCI direct mapped storage devices with separately addressable fast write storage includes systems that manage erase blocks or groups of erase blocks as allocation units for storing data on behalf of the storage service, or for storing metadata (e.g., indexes, logs, etc.) associated with the storage service, or for proper management of the storage system itself. Flash pages, which may be a few kilobytes in size, may be written as data arrives or as the storage system is to persist data for long intervals of time (e.g., above a defined threshold of time). To commit data more quickly, or to reduce the number of writes to the Flash memory devices, the storage controllers may first write data into the separately addressable fast write storage on one or more storage devices.

[0084] In one embodiment, the storage controllers **125a**, **125b** may initiate the use of erase blocks within and across storage devices (e.g., **118**) in accordance with an age and expected remaining lifespan of the storage devices, or based on other statistics. The storage controllers **125a**, **125b** may initiate garbage collection and data migration between storage devices in accordance with pages that are no longer needed as well as to manage Flash page and erase block lifespans and to manage overall system performance.

[0085] In one embodiment, the storage system **124** may utilize mirroring and/or erasure coding schemes as part of storing data into addressable fast write storage and/or as part of writing data into allocation units associated with erase blocks. Erasure codes may be used across storage devices, as well as within erase blocks or allocation units, or within and across Flash memory devices on a single storage device, to provide redundancy against single or multiple storage device failures or to protect against internal corruptions of Flash memory pages resulting from Flash memory operations or from degradation of Flash memory cells. Mirroring and erasure coding at various levels may be used to recover from multiple types of failures that occur separately or in combination.

[0086] The embodiments depicted with reference to FIGS. **2A-G** illustrate a storage cluster that stores user data, such as user data originating from one or more user or client systems or other sources external to the storage cluster. The storage cluster distributes user data across storage nodes housed within a chassis, or across multiple chassis, using erasure coding and redundant copies of metadata. Erasure coding refers to a method of data protection or reconstruction in which data is stored across a set of different locations, such as disks, storage nodes or geographic locations. Flash memory is one type of solid-state memory that may be integrated with the embodiments, although the embodiments may be extended to other types of solid-state memory or other storage medium, including non-solid state memory. Control of storage locations and workloads are distributed across the storage locations in a clustered peer-to-peer system. Tasks such as mediating communications between the various storage nodes, detecting when a storage node has become unavailable, and balancing I/Os (inputs and outputs) across the various storage nodes, are all handled on a distributed basis. Data is laid out or distributed across multiple storage nodes in data fragments or stripes that support data recovery in some embodiments. Ownership of data can be reassigned within a cluster, independent of input and output patterns. This architecture described in more detail below allows a storage node in the cluster to fail, with the system remaining operational, since the data can be reconstructed from other storage nodes and thus remain available for input and output operations. In various embodiments, a storage node may be referred to as a cluster node, a blade, or a server.

[0087] The storage cluster may be contained within a chassis, i.e., an enclosure housing one or more storage nodes. A mechanism to provide power to each storage node, such as a power distribution bus, and a communication mechanism, such as a communication bus that enables communication between the storage nodes are included within the chassis. The storage cluster can run as an independent system in one location according to some embodiments. In one embodiment, a chassis contains at least two instances of both the power distribution and the communication bus which may be enabled or disabled independently. The internal communication bus may be an Ethernet bus, however, other technologies such as PCIe, InfiniBand, and others, are equally suitable. The chassis provides a port for an external communication bus for enabling communication between multiple chassis, directly or through a switch, and with client systems. The external communication may use a technology such as Ethernet, InfiniBand, Fibre Channel, etc. In some embodiments, the external communication bus uses

different communication bus technologies for inter-chassis and client communication. If a switch is deployed within or between chassis, the switch may act as a translation between multiple protocols or technologies. When multiple chassis are connected to define a storage cluster, the storage cluster may be accessed by a client using either proprietary interfaces or standard interfaces such as network file system ('NFS'), common internet file system ('CIFS'), small computer system interface ('SCSI') or hypertext transfer protocol ('HTTP'). Translation from the client protocol may occur at the switch, chassis external communication bus or within each storage node. In some embodiments, multiple chassis may be coupled or connected to each other through an aggregator switch. A portion and/or all of the coupled or connected chassis may be designated as a storage cluster. As discussed above, each chassis can have multiple blades, each blade has a media access control ('MAC') address, but the storage cluster is presented to an external network as having a single cluster IP address and a single MAC address in some embodiments.

[0088] Each storage node may be one or more storage servers and each storage server is connected to one or more non-volatile solid state memory units, which may be referred to as storage units or storage devices. One embodiment includes a single storage server in each storage node and between one to eight non-volatile solid state memory units, however this one example is not meant to be limiting. The storage server may include a processor, DRAM and interfaces for the internal communication bus and power distribution for each of the power buses. Inside the storage node, the interfaces and storage unit share a communication bus, e.g., PCI Express, in some embodiments. The non-volatile solid state memory units may directly access the internal communication bus interface through a storage node communication bus, or request the storage node to access the bus interface. The non-volatile solid state memory unit contains an embedded CPU, solid state storage controller, and a quantity of solid state mass storage, e.g., between 2-32 terabytes ('TB') in some embodiments. An embedded volatile storage medium, such as DRAM, and an energy reserve apparatus are included in the non-volatile solid state memory unit. In some embodiments, the energy reserve apparatus is a capacitor, super-capacitor, or battery that enables transferring a subset of DRAM contents to a stable storage medium in the case of power loss. In some embodiments, the non-volatile solid state memory unit is constructed with a storage class memory, such as phase change or magnetoresistive random access memory ('MRAM') that substitutes for DRAM and enables a reduced power hold-up apparatus.

[0089] One of many features of the storage nodes and non-volatile solid state storage is the ability to proactively rebuild data in a storage cluster. The storage nodes and non-volatile solid state storage can determine when a storage node or non-volatile solid state storage in the storage cluster is unreachable, independent of whether there is an attempt to read data involving that storage node or non-volatile solid state storage. The storage nodes and non-volatile solid state storage then cooperate to recover and rebuild the data in at least partially new locations. This constitutes a proactive rebuild, in that the system rebuilds data without waiting until the data is needed for a read access initiated from a client

system employing the storage cluster. These and further details of the storage memory and operation thereof are discussed below.

[0090] FIG. 2A is a perspective view of a storage cluster **161**, with multiple storage nodes **150** and internal solid-state memory coupled to each storage node to provide network attached storage or storage area network, in accordance with some embodiments. A network attached storage, storage area network, or a storage cluster, or other storage memory, could include one or more storage clusters **161**, each having one or more storage nodes **150**, in a flexible and reconfigurable arrangement of both the physical components and the amount of storage memory provided thereby. The storage cluster **161** is designed to fit in a rack, and one or more racks can be set up and populated as desired for the storage memory. The storage cluster **161** has a chassis **138** having multiple slots **142**. It should be appreciated that chassis **138** may be referred to as a housing, enclosure, or rack unit. In one embodiment, the chassis **138** has fourteen slots **142**, although other numbers of slots are readily devised. For example, some embodiments have four slots, eight slots, sixteen slots, thirty-two slots, or other suitable number of slots. Each slot **142** can accommodate one storage node **150** in some embodiments. Chassis **138** includes flaps **148** that can be utilized to mount the chassis **138** on a rack. Fans **144** provide air circulation for cooling of the storage nodes **150** and components thereof, although other cooling components could be used, or an embodiment could be devised without cooling components. A switch fabric **146** couples storage nodes **150** within chassis **138** together and to a network for communication to the memory. In an embodiment depicted in herein, the slots **142** to the left of the switch fabric **146** and fans **144** are shown occupied by storage nodes **150**, while the slots **142** to the right of the switch fabric **146** and fans **144** are empty and available for insertion of storage node **150** for illustrative purposes. This configuration is one example, and one or more storage nodes **150** could occupy the slots **142** in various further arrangements. The storage node arrangements need not be sequential or adjacent in some embodiments. Storage nodes **150** are hot pluggable, meaning that a storage node **150** can be inserted into a slot **142** in the chassis **138**, or removed from a slot **142**, without stopping or powering down the system. Upon insertion or removal of storage node **150** from slot **142**, the system automatically reconfigures in order to recognize and adapt to the change. Reconfiguration, in some embodiments, includes restoring redundancy and/or rebalancing data or load.

[0091] Each storage node **150** can have multiple components. In the embodiment shown here, the storage node **150** includes a printed circuit board **159** populated by a CPU **156**, i.e., processor, a memory **154** coupled to the CPU **156**, and a non-volatile solid state storage **152** coupled to the CPU **156**, although other mountings and/or components could be used in further embodiments. The memory **154** has instructions which are executed by the CPU **156** and/or data operated on by the CPU **156**. As further explained below, the non-volatile solid state storage **152** includes flash or, in further embodiments, other types of solid-state memory.

[0092] Referring to FIG. 2A, storage cluster **161** is scalable, meaning that storage capacity with non-uniform storage sizes is readily added, as described above. One or more storage nodes **150** can be plugged into or removed from each chassis and the storage cluster self-configures in some

embodiments. Plug-in storage nodes **150**, whether installed in a chassis as delivered or later added, can have different sizes. For example, in one embodiment a storage node **150** can have any multiple of 4 TB, e.g., 8 TB, 12 TB, 16 TB, 32 TB, etc. In further embodiments, a storage node **150** could have any multiple of other storage amounts or capacities. Storage capacity of each storage node **150** is broadcast, and influences decisions of how to stripe the data. For maximum storage efficiency, an embodiment can self-configure as wide as possible in the stripe, subject to a predetermined requirement of continued operation with loss of up to one, or up to two, non-volatile solid state storage **152** units or storage nodes **150** within the chassis.

[0093] FIG. 2B is a block diagram showing a communications interconnect **173** and power distribution bus **172** coupling multiple storage nodes **150**. Referring back to FIG. 2A, the communications interconnect **173** can be included in or implemented with the switch fabric **146** in some embodiments. Where multiple storage clusters **161** occupy a rack, the communications interconnect **173** can be included in or implemented with a top of rack switch, in some embodiments. As illustrated in FIG. 2B, storage cluster **161** is enclosed within a single chassis **138**. External port **176** is coupled to storage nodes **150** through communications interconnect **173**, while external port **174** is coupled directly to a storage node. External power port **178** is coupled to power distribution bus **172**. Storage nodes **150** may include varying amounts and differing capacities of non-volatile solid state storage **152** as described with reference to FIG. 2A. In addition, one or more storage nodes **150** may be a compute only storage node as illustrated in FIG. 2B. Authorities **168** are implemented on the non-volatile solid state storage **152**, for example as lists or other data structures stored in memory. In some embodiments the authorities are stored within the non-volatile solid state storage **152** and supported by software executing on a controller or other processor of the non-volatile solid state storage **152**. In a further embodiment, authorities **168** are implemented on the storage nodes **150**, for example as lists or other data structures stored in the memory **154** and supported by software executing on the CPU **156** of the storage node **150**. Authorities **168** control how and where data is stored in the non-volatile solid state storage **152** in some embodiments. This control assists in determining which type of erasure coding scheme is applied to the data, and which storage nodes **150** have which portions of the data. Each authority **168** may be assigned to a non-volatile solid state storage **152**. Each authority may control a range of inode numbers, segment numbers, or other data identifiers which are assigned to data by a file system, by the storage nodes **150**, or by the non-volatile solid state storage **152**, in various embodiments.

[0094] Every piece of data, and every piece of metadata, has redundancy in the system in some embodiments. In addition, every piece of data and every piece of metadata has an owner, which may be referred to as an authority. If that authority is unreachable, for example through failure of a storage node, there is a plan of succession for how to find that data or that metadata. In various embodiments, there are redundant copies of authorities **168**. Authorities **168** have a relationship to storage nodes **150** and non-volatile solid state storage **152** in some embodiments. Each authority **168**, covering a range of data segment numbers or other identifiers of the data, may be assigned to a specific non-volatile solid state storage **152**. In some embodiments the authorities

168 for all of such ranges are distributed over the non-volatile solid state storage **152** of a storage cluster. Each storage node **150** has a network port that provides access to the non-volatile solid state storage(s) **152** of that storage node **150**. Data can be stored in a segment, which is associated with a segment number and that segment number is an indirection for a configuration of a RAID (redundant array of independent disks) stripe in some embodiments. The assignment and use of the authorities **168** thus establishes an indirection to data. Indirection may be referred to as the ability to reference data indirectly, in this case via an authority **168**, in accordance with some embodiments. A segment identifies a set of non-volatile solid state storage **152** and a local identifier into the set of non-volatile solid state storage **152** that may contain data. In some embodiments, the local identifier is an offset into the device and may be reused sequentially by multiple segments. In other embodiments the local identifier is unique for a specific segment and never reused. The offsets in the non-volatile solid state storage **152** are applied to locating data for writing to or reading from the non-volatile solid state storage **152** (in the form of a RAID stripe). Data is striped across multiple units of non-volatile solid state storage **152**, which may include or be different from the non-volatile solid state storage **152** having the authority **168** for a particular data segment.

[0095] If there is a change in where a particular segment of data is located, e.g., during a data move or a data reconstruction, the authority **168** for that data segment should be consulted, at that non-volatile solid state storage **152** or storage node **150** having that authority **168**. In order to locate a particular piece of data, embodiments calculate a hash value for a data segment or apply an inode number or a data segment number. The output of this operation points to a non-volatile solid state storage **152** having the authority **168** for that particular piece of data. In some embodiments there are two stages to this operation. The first stage maps an entity identifier (ID), e.g., a segment number, inode number, or directory number to an authority identifier. This mapping may include a calculation such as a hash or a bit mask. The second stage is mapping the authority identifier to a particular non-volatile solid state storage **152**, which may be done through an explicit mapping. The operation is repeatable, so that when the calculation is performed, the result of the calculation repeatably and reliably points to a particular non-volatile solid state storage **152** having that authority **168**. The operation may include the set of reachable storage nodes as input. If the set of reachable non-volatile solid state storage units changes the optimal set changes. In some embodiments, the persisted value is the current assignment (which is always true) and the calculated value is the target assignment the cluster will attempt to reconfigure towards. This calculation may be used to determine the optimal non-volatile solid state storage **152** for an authority in the presence of a set of non-volatile solid state storage **152** that are reachable and constitute the same cluster. The calculation also determines an ordered set of peer non-volatile solid state storage **152** that will also record the authority to non-volatile solid state storage mapping so that the authority may be determined even if the assigned non-volatile solid state storage is unreachable. A duplicate or substitute authority **168** may be consulted if a specific authority **168** is unavailable in some embodiments.

[0096] With reference to FIGS. 2A and 2B, two of the many tasks of the CPU 156 on a storage node 150 are to break up write data, and reassemble read data. When the system has determined that data is to be written, the authority 168 for that data is located as above. When the segment ID for data is already determined the request to write is forwarded to the non-volatile solid state storage 152 currently determined to be the host of the authority 168 determined from the segment. The host CPU 156 of the storage node 150, on which the non-volatile solid state storage 152 and corresponding authority 168 reside, then breaks up or shards the data and transmits the data out to various non-volatile solid state storage 152. The transmitted data is written as a data stripe in accordance with an erasure coding scheme. In some embodiments, data is requested to be pulled, and in other embodiments, data is pushed. In reverse, when data is read, the authority 168 for the segment ID containing the data is located as described above. The host CPU 156 of the storage node 150 on which the non-volatile solid state storage 152 and corresponding authority 168 reside requests the data from the non-volatile solid state storage and corresponding storage nodes pointed to by the authority. In some embodiments the data is read from flash storage as a data stripe. The host CPU 156 of storage node 150 then reassembles the read data, correcting any errors (if present) according to the appropriate erasure coding scheme, and forwards the reassembled data to the network. In further embodiments, some or all of these tasks can be handled in the non-volatile solid state storage 152. In some embodiments, the segment host requests the data be sent to storage node 150 by requesting pages from storage and then sending the data to the storage node making the original request.

[0097] In embodiments, authorities 168 operate to determine how operations will proceed against particular logical elements. Each of the logical elements may be operated on through a particular authority across a plurality of storage controllers of a storage system. The authorities 168 may communicate with the plurality of storage controllers so that the plurality of storage controllers collectively perform operations against those particular logical elements.

[0098] In embodiments, logical elements could be, for example, files, directories, object buckets, individual objects, delineated parts of files or objects, other forms of key-value pair databases, or tables. In embodiments, performing an operation can involve, for example, ensuring consistency, structural integrity, and/or recoverability with other operations against the same logical element, reading metadata and data associated with that logical element, determining what data should be written durably into the storage system to persist any changes for the operation, or where metadata and data can be determined to be stored across modular storage devices attached to a plurality of the storage controllers in the storage system.

[0099] In some embodiments the operations are token based transactions to efficiently communicate within a distributed system. Each transaction may be accompanied by or associated with a token, which gives permission to execute the transaction. The authorities 168 are able to maintain a pre-transaction state of the system until completion of the operation in some embodiments. The token based communication may be accomplished without a global lock across the system, and also enables restart of an operation in case of a disruption or other failure.

[0100] In some systems, for example in UNIX-style file systems, data is handled with an index node or inode, which specifies a data structure that represents an object in a file system. The object could be a file or a directory, for example. Metadata may accompany the object, as attributes such as permission data and a creation timestamp, among other attributes. A segment number could be assigned to all or a portion of such an object in a file system. In other systems, data segments are handled with a segment number assigned elsewhere. For purposes of discussion, the unit of distribution is an entity, and an entity can be a file, a directory or a segment. That is, entities are units of data or metadata stored by a storage system. Entities are grouped into sets called authorities. Each authority has an authority owner, which is a storage node that has the exclusive right to update the entities in the authority. In other words, a storage node contains the authority, and that the authority, in turn, contains entities.

[0101] A segment is a logical container of data in accordance with some embodiments. A segment is an address space between medium address space and physical flash locations, i.e., the data segment number, are in this address space. Segments may also contain meta-data, which enable data redundancy to be restored (rewritten to different flash locations or devices) without the involvement of higher level software. In one embodiment, an internal format of a segment contains client data and medium mappings to determine the position of that data. Each data segment is protected, e.g., from memory and other failures, by breaking the segment into a number of data and parity shards, where applicable. The data and parity shards are distributed, i.e., striped, across non-volatile solid state storage 152 coupled to the host CPUs 156 (See FIGS. 2E and 2G) in accordance with an erasure coding scheme. Usage of the term segments refers to the container and its place in the address space of segments in some embodiments. Usage of the term stripe refers to the same set of shards as a segment and includes how the shards are distributed along with redundancy or parity information in accordance with some embodiments.

[0102] A series of address-space transformations takes place across an entire storage system. At the top are the directory entries (file names) which link to an inode. Inodes point into medium address space, where data is logically stored. Medium addresses may be mapped through a series of indirect mediums to spread the load of large files, or implement data services like deduplication or snapshots. Segment addresses are then translated into physical flash locations. Physical flash locations have an address range bounded by the amount of flash in the system in accordance with some embodiments. Medium addresses and segment addresses are logical containers, and in some embodiments use a 128 bit or larger identifier so as to be practically infinite, with a likelihood of reuse calculated as longer than the expected life of the system. Addresses from logical containers are allocated in a hierarchical fashion in some embodiments. Initially, each non-volatile solid state storage 152 unit may be assigned a range of address space. Within this assigned range, the non-volatile solid state storage 152 is able to allocate addresses without synchronization with other non-volatile solid state storage 152.

[0103] Data and metadata is stored by a set of underlying storage layouts that are optimized for varying workload patterns and storage devices. These layouts incorporate multiple redundancy schemes, compression formats and

index algorithms. Some of these layouts store information about authorities and authority masters, while others store file metadata and file data. The redundancy schemes include error correction codes that tolerate corrupted bits within a single storage device (such as a NAND flash chip), erasure codes that tolerate the failure of multiple storage nodes, and replication schemes that tolerate data center or regional failures. In some embodiments, low density parity check ('LDPC') code is used within a single storage unit. Reed-Solomon encoding is used within a storage cluster, and mirroring is used within a storage grid in some embodiments. Metadata may be stored using an ordered log structured index (such as a Log Structured Merge Tree), and large data may not be stored in a log structured layout.

[0104] In order to maintain consistency across multiple copies of an entity, the storage nodes agree implicitly on two things through calculations: (1) the authority that contains the entity, and (2) the storage node that contains the authority. The assignment of entities to authorities can be done by pseudo randomly assigning entities to authorities, by splitting entities into ranges based upon an externally produced key, or by placing a single entity into each authority. Examples of pseudorandom schemes are linear hashing and the Replication Under Scalable Hashing ('RUSH') family of hashes, including Controlled Replication Under Scalable Hashing ('CRUSH'). In some embodiments, pseudo-random assignment is utilized only for assigning authorities to nodes because the set of nodes can change. The set of authorities cannot change so any subjective function may be applied in these embodiments. Some placement schemes automatically place authorities on storage nodes, while other placement schemes rely on an explicit mapping of authorities to storage nodes. In some embodiments, a pseudorandom scheme is utilized to map from each authority to a set of candidate authority owners. A pseudorandom data distribution function related to CRUSH may assign authorities to storage nodes and create a list of where the authorities are assigned. Each storage node has a copy of the pseudorandom data distribution function, and can arrive at the same calculation for distributing, and later finding or locating an authority. Each of the pseudorandom schemes requires the reachable set of storage nodes as input in some embodiments in order to conclude the same target nodes. Once an entity has been placed in an authority, the entity may be stored on physical devices so that no expected failure will lead to unexpected data loss. In some embodiments, rebalancing algorithms attempt to store the copies of all entities within an authority in the same layout and on the same set of machines.

[0105] Examples of expected failures include device failures, stolen machines, datacenter fires, and regional disasters, such as nuclear or geological events. Different failures lead to different levels of acceptable data loss. In some embodiments, a stolen storage node impacts neither the security nor the reliability of the system, while depending on system configuration, a regional event could lead to no loss of data, a few seconds or minutes of lost updates, or even complete data loss.

[0106] In the embodiments, the placement of data for storage redundancy is independent of the placement of authorities for data consistency. In some embodiments, storage nodes that contain authorities do not contain any persistent storage. Instead, the storage nodes are connected to non-volatile solid state storage units that do not contain

authorities. The communications interconnect between storage nodes and non-volatile solid state storage units consists of multiple communication technologies and has non-uniform performance and fault tolerance characteristics. In some embodiments, as mentioned above, non-volatile solid state storage units are connected to storage nodes via PCI express, storage nodes are connected together within a single chassis using Ethernet backplane, and chassis are connected together to form a storage cluster. Storage clusters are connected to clients using Ethernet or fiber channel in some embodiments. If multiple storage clusters are configured into a storage grid, the multiple storage clusters are connected using the Internet or other long-distance networking links, such as a "metro scale" link or private link that does not traverse the internet.

[0107] Authority owners have the exclusive right to modify entities, to migrate entities from one non-volatile solid state storage unit to another non-volatile solid state storage unit, and to add and remove copies of entities. This allows for maintaining the redundancy of the underlying data. When an authority owner fails, is going to be decommissioned, or is overloaded, the authority is transferred to a new storage node. Transient failures make it non-trivial to ensure that all non-faulty machines agree upon the new authority location. The ambiguity that arises due to transient failures can be achieved automatically by a consensus protocol such as Paxos, hot-warm failover schemes, via manual intervention by a remote system administrator, or by a local hardware administrator (such as by physically removing the failed machine from the cluster, or pressing a button on the failed machine). In some embodiments, a consensus protocol is used, and failover is automatic. If too many failures or replication events occur in too short a time period, the system goes into a self-preservation mode and halts replication and data movement activities until an administrator intervenes in accordance with some embodiments.

[0108] As authorities are transferred between storage nodes and authority owners update entities in their authorities, the system transfers messages between the storage nodes and non-volatile solid state storage units. With regard to persistent messages, messages that have different purposes are of different types. Depending on the type of the message, the system maintains different ordering and durability guarantees. As the persistent messages are being processed, the messages are temporarily stored in multiple durable and non-durable storage hardware technologies. In some embodiments, messages are stored in RAM, NVRAM and on NAND flash devices, and a variety of protocols are used in order to make efficient use of each storage medium. Latency-sensitive client requests may be persisted in replicated NVRAM, and then later NAND, while background rebalancing operations are persisted directly to NAND.

[0109] Persistent messages are persistently stored prior to being transmitted. This allows the system to continue to serve client requests despite failures and component replacement. Although many hardware components contain unique identifiers that are visible to system administrators, manufacturer, hardware supply chain and ongoing monitoring quality control infrastructure, applications running on top of the infrastructure address virtualize addresses. These virtualized addresses do not change over the lifetime of the storage system, regardless of component failures and replacements. This allows each component of the storage

system to be replaced over time without reconfiguration or disruptions of client request processing, i.e., the system supports non-disruptive upgrades.

[0110] In some embodiments, the virtualized addresses are stored with sufficient redundancy. A continuous monitoring system correlates hardware and software status and the hardware identifiers. This allows detection and prediction of failures due to faulty components and manufacturing details. The monitoring system also enables the proactive transfer of authorities and entities away from impacted devices before failure occurs by removing the component from the critical path in some embodiments.

[0111] FIG. 2C is a multiple level block diagram, showing contents of a storage node 150 and contents of a non-volatile solid state storage 152 of the storage node 150. Data is communicated to and from the storage node 150 by a network interface controller ('NIC') 202 in some embodiments. Each storage node 150 has a CPU 156, and one or more non-volatile solid state storage 152, as discussed above. Moving down one level in FIG. 2C, each non-volatile solid state storage 152 has a relatively fast non-volatile solid state memory, such as nonvolatile random access memory ('NVRAM') 204, and flash memory 206. In some embodiments, NVRAM 204 may be a component that does not require program/erase cycles (DRAM, MRAM, PCM), and can be a memory that can support being written vastly more often than the memory is read from. Moving down another level in FIG. 2C, the NVRAM 204 is implemented in one embodiment as high speed volatile memory, such as dynamic random access memory (DRAM) 216, backed up by energy reserve 218. Energy reserve 218 provides sufficient electrical power to keep the DRAM 216 powered long enough for contents to be transferred to the flash memory 206 in the event of power failure. In some embodiments, energy reserve 218 is a capacitor, super-capacitor, battery, or other device, that supplies a suitable supply of energy sufficient to enable the transfer of the contents of DRAM 216 to a stable storage medium in the case of power loss. The flash memory 206 is implemented as multiple flash dies 222, which may be referred to as packages of flash dies 222 or an array of flash dies 222. It should be appreciated that the flash dies 222 could be packaged in any number of ways, with a single die per package, multiple dies per package (i.e., multichip packages), in hybrid packages, as bare dies on a printed circuit board or other substrate, as encapsulated dies, etc. In the embodiment shown, the non-volatile solid state storage 152 has a controller 212 or other processor, and an input output (I/O) port 210 coupled to the controller 212. I/O port 210 is coupled to the CPU 156 and/or the network interface controller 202 of the flash storage node 150. Flash input output (I/O) port 220 is coupled to the flash dies 222, and a direct memory access unit (DMA) 214 is coupled to the controller 212, the DRAM 216 and the flash dies 222. In the embodiment shown, the I/O port 210, controller 212, DMA unit 214 and flash I/O port 220 are implemented on a programmable logic device ('PLD') 208, e.g., an FPGA. In this embodiment, each flash die 222 has pages, organized as sixteen kB (kilobyte) pages 224, and a register 226 through which data can be written to or read from the flash die 222. In further embodiments, other types of solid-state memory are used in place of, or in addition to flash memory illustrated within flash die 222.

[0112] Storage clusters 161, in various embodiments as disclosed herein, can be contrasted with storage arrays in

general. The storage nodes 150 are part of a collection that creates the storage cluster 161. Each storage node 150 owns a slice of data and computing required to provide the data. Multiple storage nodes 150 cooperate to store and retrieve the data. Storage memory or storage devices, as used in storage arrays in general, are less involved with processing and manipulating the data. Storage memory or storage devices in a storage array receive commands to read, write, or erase data. The storage memory or storage devices in a storage array are not aware of a larger system in which they are embedded, or what the data means. Storage memory or storage devices in storage arrays can include various types of storage memory, such as RAM, solid state drives, hard disk drives, etc. The non-volatile solid state storage 152 units described herein have multiple interfaces active simultaneously and serving multiple purposes. In some embodiments, some of the functionality of a storage node 150 is shifted into a storage unit 152, transforming the storage unit 152 into a combination of storage unit 152 and storage node 150. Placing computing (relative to storage data) into the storage unit 152 places this computing closer to the data itself. The various system embodiments have a hierarchy of storage node layers with different capabilities. By contrast, in a storage array, a controller owns and knows everything about all of the data that the controller manages in a shelf or storage devices. In a storage cluster 161, as described herein, multiple controllers in multiple non-volatile solid state storage 152 units and/or storage nodes 150 cooperate in various ways (e.g., for erasure coding, data sharding, metadata communication and redundancy, storage capacity expansion or contraction, data recovery, and so on).

[0113] FIG. 2D shows a storage server environment, which uses embodiments of the storage nodes 150 and storage 152 units of FIGS. 2A-C. In this version, each non-volatile solid state storage 152 unit has a processor such as controller 212 (see FIG. 2C), an FPGA, flash memory 206, and NVRAM 204 (which is super-capacitor backed DRAM 216, see FIGS. 2B and 2C) on a PCIe (peripheral component interconnect express) board in a chassis 138 (see FIG. 2A). The non-volatile solid state storage 152 unit may be implemented as a single board containing storage, and may be the largest tolerable failure domain inside the chassis. In some embodiments, up to two non-volatile solid state storage 152 units may fail and the device will continue with no data loss.

[0114] The physical storage is divided into named regions based on application usage in some embodiments. The NVRAM 204 is a contiguous block of reserved memory in the non-volatile solid state storage 152 DRAM 216, and is backed by NAND flash. NVRAM 204 is logically divided into multiple memory regions written for two as spool (e.g., spool_region). Space within the NVRAM 204 spools is managed by each authority 168 independently. Each device provides an amount of storage space to each authority 168. That authority 168 further manages lifetimes and allocations within that space. Examples of a spool include distributed transactions or notions. When the primary power to a non-volatile solid state storage 152 unit fails, onboard super-capacitors provide a short duration of power hold up. During this holdup interval, the contents of the NVRAM 204 are flushed to flash memory 206. On the next power-on, the contents of the NVRAM 204 are recovered from the flash memory 206.

[0115] As for the storage unit controller, the responsibility of the logical “controller” is distributed across each of the blades containing authorities 168. This distribution of logical control is shown in FIG. 2D as a host controller 242, mid-tier controller 244 and storage unit controller(s) 246. Management of the control plane and the storage plane are treated independently, although parts may be physically co-located on the same blade. Each authority 168 effectively serves as an independent controller. Each authority 168 provides its own data and metadata structures, its own background workers, and maintains its own lifecycle.

[0116] FIG. 2E is a blade 252 hardware block diagram, showing a control plane 254, compute and storage planes 256, 258, and authorities 168 interacting with underlying physical resources, using embodiments of the storage nodes 150 and storage units 152 of FIGS. 2A-C in the storage server environment of FIG. 2D. The control plane 254 is partitioned into a number of authorities 168 which can use the compute resources in the compute plane 256 to run on any of the blades 252. The storage plane 258 is partitioned into a set of devices, each of which provides access to flash 206 and NVRAM 204 resources. In one embodiment, the compute plane 256 may perform the operations of a storage array controller, as described herein, on one or more devices of the storage plane 258 (e.g., a storage array).

[0117] In the compute and storage planes 256, 258 of FIG. 2E, the authorities 168 interact with the underlying physical resources (i.e., devices). From the point of view of an authority 168, its resources are striped over all of the physical devices. From the point of view of a device, it provides resources to all authorities 168, irrespective of where the authorities happen to run. Each authority 168 has allocated or has been allocated one or more partitions 260 of storage memory in the storage units 152, e.g., partitions 260 in flash memory 206 and NVRAM 204. Each authority 168 uses those allocated partitions 260 that belong to it, for writing or reading user data. Authorities can be associated with differing amounts of physical storage of the system. For example, one authority 168 could have a larger number of partitions 260 or larger sized partitions 260 in one or more storage units 152 than one or more other authorities 168.

[0118] FIG. 2F depicts elasticity software layers in blades 252 of a storage cluster, in accordance with some embodiments. In the elasticity structure, elasticity software is symmetric, i.e., each blade’s compute module 270 runs the three identical layers of processes depicted in FIG. 2F. Storage managers 274 execute read and write requests from other blades 252 for data and metadata stored in local storage unit 152 NVRAM 204 and flash 206. Authorities 168 fulfill client requests by issuing the necessary reads and writes to the blades 252 on whose storage units 152 the corresponding data or metadata resides. Endpoints 272 parse client connection requests received from switch fabric 146 supervisory software, relay the client connection requests to the authorities 168 responsible for fulfillment, and relay the authorities’ 168 responses to clients. The symmetric three-layer structure enables the storage system’s high degree of concurrency. Elasticity scales out efficiently and reliably in these embodiments. In addition, elasticity implements a unique scale-out technique that balances work evenly across all resources regardless of client access pattern, and maximizes concurrency by eliminating much of the need for inter-blade coordination that typically occurs with conventional distributed locking.

[0119] Still referring to FIG. 2F, authorities 168 running in the compute modules 270 of a blade 252 perform the internal operations required to fulfill client requests. One feature of elasticity is that authorities 168 are stateless, i.e., they cache active data and metadata in their own blades’ 252 DRAMs for fast access, but the authorities store every update in their NVRAM 204 partitions on three separate blades 252 until the update has been written to flash 206. All the storage system writes to NVRAM 204 are in triplicate to partitions on three separate blades 252 in some embodiments. With triple-mirrored NVRAM 204 and persistent storage protected by parity and Reed-Solomon RAID checksums, the storage system can survive concurrent failure of two blades 252 with no loss of data, metadata, or access to either.

[0120] Because authorities 168 are stateless, they can migrate between blades 252. Each authority 168 has a unique identifier. NVRAM 204 and flash 206 partitions are associated with authorities’ 168 identifiers, not with the blades 252 on which they are running in some embodiments. Thus, when an authority 168 migrates, the authority 168 continues to manage the same storage partitions from its new location. When a new blade 252 is installed in an embodiment of the storage cluster, the system automatically rebalances load by: partitioning the new blade’s 252 storage for use by the system’s authorities 168, migrating selected authorities 168 to the new blade 252, starting endpoints 272 on the new blade 252 and including them in the switch fabric’s 146 client connection distribution algorithm.

[0121] From their new locations, migrated authorities 168 persist the contents of their NVRAM 204 partitions on flash 206, process read and write requests from other authorities 168, and fulfill the client requests that endpoints 272 direct to them. Similarly, if a blade 252 fails or is removed, the system redistributes its authorities 168 among the system’s remaining blades 252. The redistributed authorities 168 continue to perform their original functions from their new locations.

[0122] FIG. 2G depicts authorities 168 and storage resources in blades 252 of a storage cluster, in accordance with some embodiments. Each authority 168 is exclusively responsible for a partition of the flash 206 and NVRAM 204 on each blade 252. The authority 168 manages the content and integrity of its partitions independently of other authorities 168. Authorities 168 compress incoming data and preserve it temporarily in their NVRAM 204 partitions, and then consolidate, RAID-protect, and persist the data in segments of the storage in their flash 206 partitions. As the authorities 168 write data to flash 206, storage managers 274 perform the necessary flash translation to optimize write performance and maximize media longevity. In the background, authorities 168 “garbage collect,” or reclaim space occupied by data that clients have made obsolete by overwriting the data. It should be appreciated that since authorities’ 168 partitions are disjoint, there is no need for distributed locking to execute client and writes or to perform background functions.

[0123] The embodiments described herein may utilize various software, communication and/or networking protocols. In addition, the configuration of the hardware and/or software may be adjusted to accommodate various protocols. For example, the embodiments may utilize Active Directory, which is a database based system that provides authentication, directory, policy, and other services in a WINDOWS™ environment. In these embodiments, LDAP

(Lightweight Directory Access Protocol) is one example application protocol for querying and modifying items in directory service providers such as Active Directory. In some embodiments, a network lock manager ('NLM') is utilized as a facility that works in cooperation with the Network File System ('NFS') to provide a System V style of advisory file and record locking over a network. The Server Message Block ('SMB') protocol, one version of which is also known as Common Internet File System ('CIFS'), may be integrated with the storage systems discussed herein. SMB operates as an application-layer network protocol typically used for providing shared access to files, printers, and serial ports and miscellaneous communications between nodes on a network. SMB also provides an authenticated inter-process communication mechanism. AMAZON™ S3 (Simple Storage Service) is a web service offered by Amazon Web Services, and the systems described herein may interface with Amazon S3 through web services interfaces (REST (representational state transfer), SOAP (simple object access protocol), and BitTorrent). A RESTful API (application programming interface) breaks down a transaction to create a series of small modules. Each module addresses a particular underlying part of the transaction. The control or permissions provided with these embodiments, especially for object data, may include utilization of an access control list ('ACL'). The ACL is a list of permissions attached to an object and the ACL specifies which users or system processes are granted access to objects, as well as what operations are allowed on given objects. The systems may utilize Internet Protocol version 6 ('IPv6'), as well as IPv4, for the communications protocol that provides an identification and location system for computers on networks and routes traffic across the Internet. The routing of packets between networked systems may include Equal-cost multi-path routing ('ECMP'), which is a routing strategy where next-hop packet forwarding to a single destination can occur over multiple "best paths" which tie for top place in routing metric calculations. Multi-path routing can be used in conjunction with most routing protocols because it is a per-hop decision limited to a single router. The software may support Multi-tenancy, which is an architecture in which a single instance of a software application serves multiple customers. Each customer may be referred to as a tenant. Tenants may be given the ability to customize some parts of the application, but may not customize the application's code, in some embodiments. The embodiments may maintain audit logs. An audit log is a document that records an event in a computing system. In addition to documenting what resources were accessed, audit log entries typically include destination and source addresses, a timestamp, and user login information for compliance with various regulations. The embodiments may support various key management policies, such as encryption key rotation. In addition, the system may support dynamic root passwords or some variation dynamically changing passwords.

[0124] FIG. 3A sets forth a diagram of a storage system 306 that is coupled for data communications with a cloud services provider 302 in accordance with some embodiments of the present disclosure. Although depicted in less detail, the storage system 306 depicted in FIG. 3A may be similar to the storage systems described above with reference to FIGS. 1A-1D and FIGS. 2A-2G. In some embodiments, the storage system 306 depicted in FIG. 3A may be embodied as a storage system that includes imbalanced

active/active controllers, as a storage system that includes balanced active/active controllers, as a storage system that includes active/active controllers where less than all of each controller's resources are utilized such that each controller has reserve resources that may be used to support failover, as a storage system that includes fully active/active controllers, as a storage system that includes dataset-segregated controllers, as a storage system that includes dual-layer architectures with front-end controllers and back-end integrated storage controllers, as a storage system that includes scale-out clusters of dual-controller arrays, as well as combinations of such embodiments.

[0125] In the example depicted in FIG. 3A, the storage system 306 is coupled to the cloud services provider 302 via a data communications link 304. Such a data communications link 304 may be fully wired, fully wireless, or some aggregation of wired and wireless data communications pathways. In such an example, digital information may be exchanged between the storage system 306 and the cloud services provider 302 via the data communications link 304 using one or more data communications protocols. For example, digital information may be exchanged between the storage system 306 and the cloud services provider 302 via the data communications link 304 using the handheld device transfer protocol ('HDTP'), hypertext transfer protocol ('HTTP'), internet protocol ('IP'), real-time transfer protocol ('RTP'), transmission control protocol ('TCP'), user datagram protocol ('UDP'), wireless application protocol ('WAP'), or other protocol.

[0126] The cloud services provider 302 depicted in FIG. 3A may be embodied, for example, as a system and computing environment that provides a vast array of services to users of the cloud services provider 302 through the sharing of computing resources via the data communications link 304. The cloud services provider 302 may provide on-demand access to a shared pool of configurable computing resources such as computer networks, servers, storage, applications and services, and so on.

[0127] In the example depicted in FIG. 3A, the cloud services provider 302 may be configured to provide a variety of services to the storage system 306 and users of the storage system 306 through the implementation of various service models. For example, the cloud services provider 302 may be configured to provide services through the implementation of an infrastructure as a service ('IaaS') service model, through the implementation of a platform as a service ('PaaS') service model, through the implementation of a software as a service ('SaaS') service model, through the implementation of an authentication as a service ('AaaS') service model, through the implementation of a storage as a service model where the cloud services provider 302 offers access to its storage infrastructure for use by the storage system 306 and users of the storage system 306, and so on.

[0128] In the example depicted in FIG. 3A, the cloud services provider 302 may be embodied, for example, as a private cloud, as a public cloud, or as a combination of a private cloud and public cloud. In an embodiment in which the cloud services provider 302 is embodied as a private cloud, the cloud services provider 302 may be dedicated to providing services to a single organization rather than providing services to multiple organizations. In an embodiment where the cloud services provider 302 is embodied as a public cloud, the cloud services provider 302 may provide services to multiple organizations. In still alternative

embodiments, the cloud services provider 302 may be embodied as a mix of a private and public cloud services with a hybrid cloud deployment.

[0129] Although not explicitly depicted in FIG. 3A, readers will appreciate that a vast amount of additional hardware components and additional software components may be necessary to facilitate the delivery of cloud services to the storage system 306 and users of the storage system 306. For example, the storage system 306 may be coupled to (or even include) a cloud storage gateway. Such a cloud storage gateway may be embodied, for example, as hardware-based or software-based appliance that is located on-premises with the storage system 306. Such a cloud storage gateway may operate as a bridge between local applications that are executing on the storage system 306 and remote, cloud-based storage that is utilized by the storage system 306. Through the use of a cloud storage gateway, organizations may move primary iSCSI or NAS to the cloud services provider 302, thereby enabling the organization to save space on their on-premises storage systems. Such a cloud storage gateway may be configured to emulate a disk array, a block-based device, a file server, or other storage system that can translate the SCSI commands, file server commands, or other appropriate command into REST-space protocols that facilitate communications with the cloud services provider 302.

[0130] In order to enable the storage system 306 and users of the storage system 306 to make use of the services provided by the cloud services provider 302, a cloud migration process may take place during which data, applications, or other elements from an organization's local systems (or even from another cloud environment) are moved to the cloud services provider 302. In order to successfully migrate data, applications, or other elements to the cloud services provider's 302 environment, middleware such as a cloud migration tool may be utilized to bridge gaps between the cloud services provider's 302 environment and an organization's environment. In order to further enable the storage system 306 and users of the storage system 306 to make use of the services provided by the cloud services provider 302, a cloud orchestrator may also be used to arrange and coordinate automated tasks in pursuit of creating a consolidated process or workflow. Such a cloud orchestrator may perform tasks such as configuring various components, whether those components are cloud components or on-premises components, as well as managing the interconnections between such components.

[0131] In the example depicted in FIG. 3A, and as described briefly above, the cloud services provider 302 may be configured to provide services to the storage system 306 and users of the storage system 306 through the usage of a SaaS service model. For example, the cloud services provider 302 may be configured to provide access to data analytics applications to the storage system 306 and users of the storage system 306. Such data analytics applications may be configured, for example, to receive vast amounts of telemetry data phoned home by the storage system 306. Such telemetry data may describe various operating characteristics of the storage system 306 and may be analyzed for a vast array of purposes including, for example, to determine the health of the storage system 306, to identify workloads that are executing on the storage system 306, to predict when the storage system 306 will run out of various resources, to recommend configuration changes, hardware

or software upgrades, workflow migrations, or other actions that may improve the operation of the storage system 306.

[0132] The cloud services provider 302 may also be configured to provide access to virtualized computing environments to the storage system 306 and users of the storage system 306. Examples of such virtualized environments can include virtual machines that are created to emulate an actual computer, virtualized desktop environments that separate a logical desktop from a physical machine, virtualized file systems that allow uniform access to different types of concrete file systems, and many others.

[0133] Although the example depicted in FIG. 3A illustrates the storage system 306 being coupled for data communications with the cloud services provider 302, in other embodiments the storage system 306 may be part of a hybrid cloud deployment in which private cloud elements (e.g., private cloud services, on-premises infrastructure, and so on) and public cloud elements (e.g., public cloud services, infrastructure, and so on that may be provided by one or more cloud services providers) are combined to form a single solution, with orchestration among the various platforms. Such a hybrid cloud deployment may leverage hybrid cloud management software such as, for example, Azure™ Arc from Microsoft™, that centralize the management of the hybrid cloud deployment to any infrastructure and enable the deployment of services anywhere. In such an example, the hybrid cloud management software may be configured to create, update, and delete resources (both physical and virtual) that form the hybrid cloud deployment, to allocate compute and storage to specific workloads, to monitor workloads and resources for performance, policy compliance, updates and patches, security status, or to perform a variety of other tasks.

[0134] Readers will appreciate that by pairing the storage systems described herein with one or more cloud services providers, various offerings may be enabled. For example, disaster recovery as a service ('DRaaS') may be provided where cloud resources are utilized to protect applications and data from disruption caused by disaster, including in embodiments where the storage systems may serve as the primary data store. In such embodiments, a total system backup may be taken that allows for business continuity in the event of system failure. In such embodiments, cloud data backup techniques (by themselves or as part of a larger DRaaS solution) may also be integrated into an overall solution that includes the storage systems and cloud services providers described herein.

[0135] The storage systems described herein, as well as the cloud services providers, may be utilized to provide a wide array of security features. For example, the storage systems may encrypt data at rest (and data may be sent to and from the storage systems encrypted) and may make use of Key Management-as-a-Service ('KMaaS') to manage encryption keys, keys for locking and unlocking storage devices, and so on. Likewise, cloud data security gateways or similar mechanisms may be utilized to ensure that data stored within the storage systems does not improperly end up being stored in the cloud as part of a cloud data backup operation. Furthermore, microsegmentation or identity-based-segmentation may be utilized in a data center that includes the storage systems or within the cloud services provider, to create secure zones in data centers and cloud deployments that enables the isolation of workloads from one another.

[0136] For further explanation, FIG. 3B sets forth a diagram of a storage system 306 in accordance with some embodiments of the present disclosure. Although depicted in less detail, the storage system 306 depicted in FIG. 3B may be similar to the storage systems described above with reference to FIGS. 1A-1D and FIGS. 2A-2G as the storage system may include many of the components described above.

[0137] The storage system 306 depicted in FIG. 3B may include a vast amount of storage resources 308, which may be embodied in many forms. For example, the storage resources 308 can include nano-RAM or another form of nonvolatile random access memory that utilizes carbon nanotubes deposited on a substrate, 3D crosspoint non-volatile memory, flash memory including single-level cell ('SLC') NAND flash, multi-level cell ('MLC') NAND flash, triple-level cell ('TLC') NAND flash, quad-level cell ('QLC') NAND flash, or others. Likewise, the storage resources 308 may include non-volatile magnetoresistive random-access memory ('MRAM'), including spin transfer torque ('STT') MRAM. The example storage resources 308 may alternatively include non-volatile phase-change memory ('PCM'), quantum memory that allows for the storage and retrieval of photonic quantum information, resistive random-access memory ('ReRAM'), storage class memory ('SCM'), or other form of storage resources, including any combination of resources described herein. Readers will appreciate that other forms of computer memories and storage devices may be utilized by the storage systems described above, including DRAM, SRAM, EEPROM, universal memory, and many others. The storage resources 308 depicted in FIG. 3A may be embodied in a variety of form factors, including but not limited to, dual in-line memory modules ('DIMMs'), non-volatile dual in-line memory modules ('NVDIMMs'), M.2, U.2, and others.

[0138] The storage resources 308 depicted in FIG. 3B may include various forms of SCM. SCM may effectively treat fast, non-volatile memory (e.g., NAND flash) as an extension of DRAM such that an entire dataset may be treated as an in-memory dataset that resides entirely in DRAM. SCM may include non-volatile media such as, for example, NAND flash. Such NAND flash may be accessed utilizing NVMe that can use the PCIe bus as its transport, providing for relatively low access latencies compared to older protocols. In fact, the network protocols used for SSDs in all-flash arrays can include NVMe using Ethernet (ROCE, NVME TCP), Fibre Channel (NVMe FC), InfiniBand (iWARP), and others that make it possible to treat fast, non-volatile memory as an extension of DRAM. In view of the fact that DRAM is often byte-addressable and fast, non-volatile memory such as NAND flash is block-addressable, a controller software/hardware stack may be needed to convert the block data to the bytes that are stored in the media. Examples of media and software that may be used as SCM can include, for example, 3D XPoint, Intel Memory Drive Technology, Samsung's Z-SSD, and others.

[0139] The storage resources 308 depicted in FIG. 3B may also include racetrack memory (also referred to as domain-wall memory). Such racetrack memory may be embodied as a form of non-volatile, solid-state memory that relies on the intrinsic strength and orientation of the magnetic field created by an electron as it spins in addition to its electronic charge, in solid-state devices. Through the use of spin-coherent electric current to move magnetic domains along a

nanoscopic permalloy wire, the domains may pass by magnetic read/write heads positioned near the wire as current is passed through the wire, which alter the domains to record patterns of bits. In order to create a racetrack memory device, many such wires and read/write elements may be packaged together.

[0140] The example storage system 306 depicted in FIG. 3B may implement a variety of storage architectures. For example, storage systems in accordance with some embodiments of the present disclosure may utilize block storage where data is stored in blocks, and each block essentially acts as an individual hard drive. Storage systems in accordance with some embodiments of the present disclosure may utilize object storage, where data is managed as objects. Each object may include the data itself, a variable amount of metadata, and a globally unique identifier, where object storage can be implemented at multiple levels (e.g., device level, system level, interface level). Storage systems in accordance with some embodiments of the present disclosure utilize file storage in which data is stored in a hierarchical structure. Such data may be saved in files and folders, and presented to both the system storing it and the system retrieving it in the same format.

[0141] The example storage system 306 depicted in FIG. 3B may be embodied as a storage system in which additional storage resources can be added through the use of a scale-up model, additional storage resources can be added through the use of a scale-out model, or through some combination thereof. In a scale-up model, additional storage may be added by adding additional storage devices. In a scale-out model, however, additional storage nodes may be added to a cluster of storage nodes, where such storage nodes can include additional processing resources, additional networking resources, and so on.

[0142] The example storage system 306 depicted in FIG. 3B may leverage the storage resources described above in a variety of different ways. For example, some portion of the storage resources may be utilized to serve as a write cache, storage resources within the storage system may be utilized as a read cache, or tiering may be achieved within the storage systems by placing data within the storage system in accordance with one or more tiering policies.

[0143] The storage system 306 depicted in FIG. 3B also includes communications resources 310 that may be useful in facilitating data communications between components within the storage system 306, as well as data communications between the storage system 306 and computing devices that are outside of the storage system 306, including embodiments where those resources are separated by a relatively vast expanse. The communications resources 310 may be configured to utilize a variety of different protocols and data communication fabrics to facilitate data communications between components within the storage systems as well as computing devices that are outside of the storage system. For example, the communications resources 310 can include fibre channel ('FC') technologies such as FC fabrics and FC protocols that can transport SCSI commands over FC network, FC over ethernet ('FCOE') technologies through which FC frames are encapsulated and transmitted over Ethernet networks, InfiniBand ('IB') technologies in which a switched fabric topology is utilized to facilitate transmissions between channel adapters, NVMe Express ('NVMe') technologies and NVMe over fabrics ('NVMeoF') technologies through which non-volatile storage media attached via

a PCI express ('PCIe') bus may be accessed, and others. In fact, the storage systems described above may, directly or indirectly, make use of neutrino communication technologies and devices through which information (including binary information) is transmitted using a beam of neutrinos.

[0144] The communications resources 310 can also include mechanisms for accessing storage resources 308 within the storage system 306 utilizing serial attached SCSI ('SAS'), serial ATA ('SATA') bus interfaces for connecting storage resources 308 within the storage system 306 to host bus adapters within the storage system 306, internet small computer systems interface ('iSCSI') technologies to provide block-level access to storage resources 308 within the storage system 306, and other communications resources that may be useful in facilitating data communications between components within the storage system 306, as well as data communications between the storage system 306 and computing devices that are outside of the storage system 306.

[0145] The storage system 306 depicted in FIG. 3B also includes processing resources 312 that may be useful in executing computer program instructions and performing other computational tasks within the storage system 306. The processing resources 312 may include one or more ASICs that are customized for some particular purpose as well as one or more CPUs. The processing resources 312 may also include one or more DSPs, one or more FPGAs, one or more systems on a chip ('SoCs'), or other form of processing resources 312. The storage system 306 may utilize the storage resources 312 to perform a variety of tasks including, but not limited to, supporting the execution of software resources 314 that will be described in greater detail below.

[0146] The storage system 306 depicted in FIG. 3B also includes software resources 314 that, when executed by processing resources 312 within the storage system 306, may perform a vast array of tasks. The software resources 314 may include, for example, one or more modules of computer program instructions that when executed by processing resources 312 within the storage system 306 are useful in carrying out various data protection techniques. Such data protection techniques may be carried out, for example, by system software executing on computer hardware within the storage system, by a cloud services provider, or in other ways. Such data protection techniques can include data archiving, data backup, data replication, data snapshotting, data and database cloning, and other data protection techniques.

[0147] The software resources 314 may also include software that is useful in implementing software-defined storage ('SDS'). In such an example, the software resources 314 may include one or more modules of computer program instructions that, when executed, are useful in policy-based provisioning and management of data storage that is independent of the underlying hardware. Such software resources 314 may be useful in implementing storage virtualization to separate the storage hardware from the software that manages the storage hardware.

[0148] The software resources 314 may also include software that is useful in facilitating and optimizing I/O operations that are directed to the storage system 306. For example, the software resources 314 may include software modules that perform various data reduction techniques such as, for example, data compression, data deduplication, and

others. The software resources 314 may include software modules that intelligently group together I/O operations to facilitate better usage of the underlying storage resource 308, software modules that perform data migration operations to migrate from within a storage system, as well as software modules that perform other functions. Such software resources 314 may be embodied as one or more software containers or in many other ways.

[0149] For further explanation, FIG. 3C sets forth an example of a cloud-based storage system 318 in accordance with some embodiments of the present disclosure. In the example depicted in FIG. 3C, the cloud-based storage system 318 is created entirely in a cloud computing environment 316 such as, for example, Amazon Web Services ('AWS')™, Microsoft Azure™, Google Cloud Platform™, IBM Cloud™, Oracle Cloud™, and others. The cloud-based storage system 318 may be used to provide services similar to the services that may be provided by the storage systems described above.

[0150] The cloud-based storage system 318 depicted in FIG. 3C includes two cloud computing instances 320, 322 that each are used to support the execution of a storage controller application 324, 326. The cloud computing instances 320, 322 may be embodied, for example, as instances of cloud computing resources (e.g., virtual machines) that may be provided by the cloud computing environment 316 to support the execution of software applications such as the storage controller application 324, 326. For example, each of the cloud computing instances 320, 322 may execute on an Azure VM, where each Azure VM may include high speed temporary storage that may be leveraged as a cache (e.g., as a read cache). In one embodiment, the cloud computing instances 320, 322 may be embodied as Amazon Elastic Compute Cloud ('EC2') instances. In such an example, an Amazon Machine Image ('AMI') that includes the storage controller application 324, 326 may be booted to create and configure a virtual machine that may execute the storage controller application 324, 326.

[0151] In the example method depicted in FIG. 3C, the storage controller application 324, 326 may be embodied as a module of computer program instructions that, when executed, carries out various storage tasks. For example, the storage controller application 324, 326 may be embodied as a module of computer program instructions that, when executed, carries out the same tasks as the controllers 110A, 110B in FIG. 1A described above such as writing data to the cloud-based storage system 318, erasing data from the cloud-based storage system 318, retrieving data from the cloud-based storage system 318, monitoring and reporting of storage device utilization and performance, performing redundancy operations, such as RAID or RAID-like data redundancy operations, compressing data, encrypting data, deduplicating data, and so forth. Readers will appreciate that because there are two cloud computing instances 320, 322 that each include the storage controller application 324, 326, in some embodiments one cloud computing instance 320 may operate as the primary controller as described above while the other cloud computing instance 322 may operate as the secondary controller as described above. Readers will appreciate that the storage controller application 324, 326 depicted in FIG. 3C may include identical source code that is executed within different cloud computing instances 320, 322 such as distinct EC2 instances.

[0152] Readers will appreciate that other embodiments that do not include a primary and secondary controller are within the scope of the present disclosure. For example, each cloud computing instance 320, 322 may operate as a primary controller for some portion of the address space supported by the cloud-based storage system 318, each cloud computing instance 320, 322 may operate as a primary controller where the servicing of I/O operations directed to the cloud-based storage system 318 are divided in some other way, and so on. In fact, in other embodiments where costs savings may be prioritized over performance demands, only a single cloud computing instance may exist that contains the storage controller application.

[0153] The cloud-based storage system 318 depicted in FIG. 3C includes cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338. The cloud computing instances 340a, 340b, 340n may be embodied, for example, as instances of cloud computing resources that may be provided by the cloud computing environment 316 to support the execution of software applications. The cloud computing instances 340a, 340b, 340n of FIG. 3C may differ from the cloud computing instances 320, 322 described above as the cloud computing instances 340a, 340b, 340n of FIG. 3C have local storage 330, 334, 338 resources whereas the cloud computing instances 320, 322 that support the execution of the storage controller application 324, 326 need not have local storage resources. The cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338 may be embodied, for example, as EC2 M5 instances that include one or more SSDs, as EC2 R5 instances that include one or more SSDs, as EC2 I3 instances that include one or more SSDs, and so on. In some embodiments, the local storage 330, 334, 338 must be embodied as solid-state storage (e.g., SSDs) rather than storage that makes use of hard disk drives.

[0154] In the example depicted in FIG. 3C, each of the cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338 can include a software daemon 328, 332, 336 that, when executed by a cloud computing instance 340a, 340b, 340n can present itself to the storage controller applications 324, 326 as if the cloud computing instance 340a, 340b, 340n were a physical storage device (e.g., one or more SSDs). In such an example, the software daemon 328, 332, 336 may include computer program instructions similar to those that would normally be contained on a storage device such that the storage controller applications 324, 326 can send and receive the same commands that a storage controller would send to storage devices. In such a way, the storage controller applications 324, 326 may include code that is identical to (or substantially identical to) the code that would be executed by the controllers in the storage systems described above. In these and similar embodiments, communications between the storage controller applications 324, 326 and the cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338 may utilize iSCSI, NVMe over TCP, messaging, a custom protocol, or in some other mechanism.

[0155] In the example depicted in FIG. 3C, each of the cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338 may also be coupled to block storage 342, 344, 346 that is offered by the cloud computing environment 316 such as, for example, Amazon Elastic Block Store ('EBS') volumes. In such an example, the block storage 342, 344, 346 that is offered by the cloud computing

environment 316 may be utilized in a manner that is similar to how the NVRAM devices described above are utilized, as the software daemon 328, 332, 336 (or some other module) that is executing within a particular cloud computing instance 340a, 340b, 340n may, upon receiving a request to write data, initiate a write of the data to its attached EBS volume as well as a write of the data to its local storage 330, 334, 338 resources. In some alternative embodiments, data may only be written to the local storage 330, 334, 338 resources within a particular cloud computing instance 340a, 340b, 340n. In an alternative embodiment, rather than using the block storage 342, 344, 346 that is offered by the cloud computing environment 316 as NVRAM, actual RAM on each of the cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338 may be used as NVRAM, thereby decreasing network utilization costs that would be associated with using an EBS volume as the NVRAM. In yet another embodiment, high performance block storage resources such as one or more Azure Ultra Disks may be utilized as the NVRAM.

[0156] When a request to write data is received by a particular cloud computing instance 340a, 340b, 340n with local storage 330, 334, 338, the software daemon 328, 332, 336 may be configured to not only write the data to its own local storage 330, 334, 338 resources and any appropriate block storage 342, 344, 346 resources, but the software daemon 328, 332, 336 may also be configured to write the data to cloud-based object storage 348 that is attached to the particular cloud computing instance 340a, 340b, 340n. The cloud-based object storage 348 that is attached to the particular cloud computing instance 340a, 340b, 340n may be embodied, for example, as Amazon Simple Storage Service ('S3'). In other embodiments, the cloud computing instances 320, 322 that each include the storage controller application 324, 326 may initiate the storage of the data in the local storage 330, 334, 338 of the cloud computing instances 340a, 340b, 340n and the cloud-based object storage 348. In other embodiments, rather than using both the cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338 (also referred to herein as 'virtual drives') and the cloud-based object storage 348 to store data, a persistent storage layer may be implemented in other ways. For example, one or more Azure Ultra disks may be used to persistently store data (e.g., after the data has been written to the NVRAM layer). In an embodiment where one or more Azure Ultra disks may be used to persistently store data, the usage of a cloud-based object storage 348 may be eliminated such that data is only stored persistently in the Azure Ultra disks without also writing the data to an object storage layer.

[0157] While the local storage 330, 334, 338 resources and the block storage 342, 344, 346 resources that are utilized by the cloud computing instances 340a, 340b, 340n may support block-level access, the cloud-based object storage 348 that is attached to the particular cloud computing instance 340a, 340b, 340n supports only object-based access. The software daemon 328, 332, 336 may therefore be configured to take blocks of data, package those blocks into objects, and write the objects to the cloud-based object storage 348 that is attached to the particular cloud computing instance 340a, 340b, 340n.

[0158] In some embodiments, all data that is stored by the cloud-based storage system 318 may be stored in both: 1) the cloud-based object storage 348, and 2) at least one of the local storage 330, 334, 338 resources or block storage 342,

344, 346 resources that are utilized by the cloud computing instances **340a, 340b, 340n**. In such embodiments, the local storage **330, 334, 338** resources and block storage **342, 344, 346** resources that are utilized by the cloud computing instances **340a, 340b, 340n** may effectively operate as cache that generally includes all data that is also stored in S3, such that all reads of data may be serviced by the cloud computing instances **340a, 340b, 340n** without requiring the cloud computing instances **340a, 340b, 340n** to access the cloud-based object storage **348**. Readers will appreciate that in other embodiments, however, all data that is stored by the cloud-based storage system **318** may be stored in the cloud-based object storage **348**, but less than all data that is stored by the cloud-based storage system **318** may be stored in at least one of the local storage **330, 334, 338** resources or block storage **342, 344, 346** resources that are utilized by the cloud computing instances **340a, 340b, 340n**. In such an example, various policies may be utilized to determine which subset of the data that is stored by the cloud-based storage system **318** should reside in both: 1) the cloud-based object storage **348**, and 2) at least one of the local storage **330, 334, 338** resources or block storage **342, 344, 346** resources that are utilized by the cloud computing instances **340a, 340b, 340n**.

[0159] One or more modules of computer program instructions that are executing within the cloud-based storage system **318** (e.g., a monitoring module that is executing on its own EC2 instance) may be designed to handle the failure of one or more of the cloud computing instances **340a, 340b, 340n** with local storage **330, 334, 338**. In such an example, the monitoring module may handle the failure of one or more of the cloud computing instances **340a, 340b, 340n** with local storage **330, 334, 338** by creating one or more new cloud computing instances with local storage, retrieving data that was stored on the failed cloud computing instances **340a, 340b, 340n** from the cloud-based object storage **348**, and storing the data retrieved from the cloud-based object storage **348** in local storage on the newly created cloud computing instances. Readers will appreciate that many variants of this process may be implemented.

[0160] Readers will appreciate that various performance aspects of the cloud-based storage system **318** may be monitored (e.g., by a monitoring module that is executing in an EC2 instance) such that the cloud-based storage system **318** can be scaled-up or scaled-out as needed. For example, if the cloud computing instances **320, 322** that are used to support the execution of a storage controller application **324, 326** are undersized and not sufficiently servicing the I/O requests that are issued by users of the cloud-based storage system **318**, a monitoring module may create a new, more powerful cloud computing instance (e.g., a cloud computing instance of a type that includes more processing power, more memory, etc . . .) that includes the storage controller application such that the new, more powerful cloud computing instance can begin operating as the primary controller. Likewise, if the monitoring module determines that the cloud computing instances **320, 322** that are used to support the execution of a storage controller application **324, 326** are oversized and that cost savings could be gained by switching to a smaller, less powerful cloud computing instance, the monitoring module may create a new, less powerful (and less expensive) cloud computing instance that includes the

storage controller application such that the new, less powerful cloud computing instance can begin operating as the primary controller.

[0161] The storage systems described above may carry out intelligent data backup techniques through which data stored in the storage system may be copied and stored in a distinct location to avoid data loss in the event of equipment failure or some other form of catastrophe. For example, the storage systems described above may be configured to examine each backup to avoid restoring the storage system to an undesirable state. Consider an example in which malware infects the storage system. In such an example, the storage system may include software resources **314** that can scan each backup to identify backups that were captured before the malware infected the storage system and those backups that were captured after the malware infected the storage system. In such an example, the storage system may restore itself from a backup that does not include the malware—or at least not restore the portions of a backup that contained the malware. In such an example, the storage system may include software resources **314** that can scan each backup to identify the presences of malware (or a virus, or some other undesirable), for example, by identifying write operations that were serviced by the storage system and originated from a network subnet that is suspected to have delivered the malware, by identifying write operations that were serviced by the storage system and originated from a user that is suspected to have delivered the malware, by identifying write operations that were serviced by the storage system and examining the content of the write operation against fingerprints of the malware, and in many other ways.

[0162] Readers will further appreciate that the backups (often in the form of one or more snapshots) may also be utilized to perform rapid recovery of the storage system. Consider an example in which the storage system is infected with ransomware that locks users out of the storage system. In such an example, software resources **314** within the storage system may be configured to detect the presence of ransomware and may be further configured to restore the storage system to a point-in-time, using the retained backups, prior to the point-in-time at which the ransomware infected the storage system. In such an example, the presence of ransomware may be explicitly detected through the use of software tools utilized by the system, through the use of a key (e.g., a USB drive) that is inserted into the storage system, or in a similar way. Likewise, the presence of ransomware may be inferred in response to system activity meeting a predetermined fingerprint such as, for example, no reads or writes coming into the system for a predetermined period of time.

[0163] Readers will appreciate that the various components described above may be grouped into one or more optimized computing packages as converged infrastructures. Such converged infrastructures may include pools of computers, storage and networking resources that can be shared by multiple applications and managed in a collective manner using policy-driven processes. Such converged infrastructures may be implemented with a converged infrastructure reference architecture, with standalone appliances, with a software driven hyper-converged approach (e.g., hyper-converged infrastructures), or in other ways.

[0164] Readers will appreciate that the storage systems described in this disclosure may be useful for supporting various types of software applications. In fact, the storage

systems may be ‘application aware’ in the sense that the storage systems may obtain, maintain, or otherwise have access to information describing connected applications (e.g., applications that utilize the storage systems) to optimize the operation of the storage system based on intelligence about the applications and their utilization patterns. For example, the storage system may optimize data layouts, optimize caching behaviors, optimize ‘QoS’ levels, or perform some other optimization that is designed to improve the storage performance that is experienced by the application.

[0165] As an example of one type of application that may be supported by the storage systems described herein, the storage system 306 may be useful in supporting artificial intelligence (‘AI’) applications, database applications, XOps projects (e.g., DevOps projects, DataOps projects, MLOps projects, ModelOps projects, PlatformOps projects), electronic design automation tools, event-driven software applications, high performance computing applications, simulation applications, high-speed data capture and analysis applications, machine learning applications, media production applications, media serving applications, picture archiving and communication systems (‘PACS’) applications, software development applications, virtual reality applications, augmented reality applications, and many other types of applications by providing storage resources to such applications.

[0166] In view of the fact that the storage systems include compute resources, storage resources, and a wide variety of other resources, the storage systems may be well suited to support applications that are resource intensive such as, for example, AI applications. AI applications may be deployed in a variety of fields, including: predictive maintenance in manufacturing and related fields, healthcare applications such as patient data & risk analytics, retail and marketing deployments (e.g., search advertising, social media advertising), supply chains solutions, fintech solutions such as business analytics & reporting tools, operational deployments such as real-time analytics tools, application performance management tools, IT infrastructure management tools, and many others.

[0167] Such AI applications may enable devices to perceive their environment and take actions that maximize their chance of success at some goal. Examples of such AI applications can include IBM Watson™, Microsoft Oxford™, Google DeepMind™, Baidu Minwa™, and others.

[0168] The storage systems described above may also be well suited to support other types of applications that are resource intensive such as, for example, machine learning applications. Machine learning applications may perform various types of data analysis to automate analytical model building. Using algorithms that iteratively learn from data, machine learning applications can enable computers to learn without being explicitly programmed. One particular area of machine learning is referred to as reinforcement learning, which involves taking suitable actions to maximize reward in a particular situation.

[0169] In addition to the resources already described, the storage systems described above may also include graphics processing units (‘GPUs’), occasionally referred to as visual processing unit (‘VPUs’). Such GPUs may be embodied as specialized electronic circuits that rapidly manipulate and alter memory to accelerate the creation of images in a frame

buffer intended for output to a display device. Such GPUs may be included within any of the computing devices that are part of the storage systems described above, including as one of many individually scalable components of a storage system, where other examples of individually scalable components of such storage system can include storage components, memory components, compute components (e.g., CPUs, FPGAs, ASICs), networking components, software components, and others. In addition to GPUs, the storage systems described above may also include neural network processors (‘NNPs’) for use in various aspects of neural network processing. Such NNPs may be used in place of (or in addition to) GPUs and may also be independently scalable.

[0170] As described above, the storage systems described herein may be configured to support artificial intelligence applications, machine learning applications, big data analytics applications, and many other types of applications. The rapid growth in these sort of applications is being driven by three technologies: deep learning (DL), GPU processors, and Big Data. Deep learning is a computing model that makes use of massively parallel neural networks inspired by the human brain. Instead of experts handcrafting software, a deep learning model writes its own software by learning from lots of examples. Such GPUs may include thousands of cores that are well-suited to run algorithms that loosely represent the parallel nature of the human brain.

[0171] Advances in deep neural networks, including the development of multi-layer neural networks, have ignited a new wave of algorithms and tools for data scientists to tap into their data with artificial intelligence (AI). With improved algorithms, larger data sets, and various frameworks (including open-source software libraries for machine learning across a range of tasks), data scientists are tackling new use cases like autonomous driving vehicles, natural language processing and understanding, computer vision, machine reasoning, strong AI, and many others. Applications of AI techniques have materialized in a wide array of products include, for example, Amazon Echo’s speech recognition technology that allows users to talk to their machines, Google Translate™ which allows for machine-based language translation, Spotify’s Discover Weekly that provides recommendations on new songs and artists that a user may like based on the user’s usage and traffic analysis, Quill’s text generation offering that takes structured data and turns it into narrative stories, Chatbots that provide real-time, contextually specific answers to questions in a dialog format, and many others.

[0172] Data is the heart of modern AI and deep learning algorithms. Before training can begin, one problem that must be addressed revolves around collecting the labeled data that is crucial for training an accurate AI model. A full scale AI deployment may be required to continuously collect, clean, transform, label, and store large amounts of data. Adding additional high quality data points directly translates to more accurate models and better insights. Data samples may undergo a series of processing steps including, but not limited to: 1) ingesting the data from an external source into the training system and storing the data in raw form, 2) cleaning and transforming the data in a format convenient for training, including linking data samples to the appropriate label, 3) exploring parameters and models, quickly testing with a smaller dataset, and iterating to converge on the most promising models to push into the production

cluster, 4) executing training phases to select random batches of input data, including both new and older samples, and feeding those into production GPU servers for computation to update model parameters, and 5) evaluating including using a holdback portion of the data not used in training in order to evaluate model accuracy on the holdout data. This lifecycle may apply for any type of parallelized machine learning, not just neural networks or deep learning. For example, standard machine learning frameworks may rely on CPUs instead of GPUs but the data ingest and training workflows may be the same. Readers will appreciate that a single shared storage data hub creates a coordination point throughout the lifecycle without the need for extra data copies among the ingest, preprocessing, and training stages. Rarely is the ingested data used for only one purpose, and shared storage gives the flexibility to train multiple different models or apply traditional analytics to the data.

[0173] Readers will appreciate that each stage in the AI data pipeline may have varying requirements from the data hub (e.g., the storage system or collection of storage systems). Scale-out storage systems must deliver uncompromising performance for all manner of access types and patterns—from small, metadata-heavy to large files, from random to sequential access patterns, and from low to high concurrency. The storage systems described above may serve as an ideal AI data hub as the systems may service unstructured workloads. In the first stage, data is ideally ingested and stored on to the same data hub that following stages will use, in order to avoid excess data copying. The next two steps can be done on a standard compute server that optionally includes a GPU, and then in the fourth and last stage, full training production jobs are run on powerful GPU-accelerated servers. Often, there is a production pipeline alongside an experimental pipeline operating on the same dataset. Further, the GPU-accelerated servers can be used independently for different models or joined together to train on one larger model, even spanning multiple systems for distributed training. If the shared storage tier is slow, then data must be copied to local storage for each phase, resulting in wasted time staging data onto different servers. The ideal data hub for the AI training pipeline delivers performance similar to data stored locally on the server node while also having the simplicity and performance to enable all pipeline stages to operate concurrently.

[0174] In order for the storage systems described above to serve as a data hub or as part of an AI deployment, in some embodiments the storage systems may be configured to provide DMA between storage devices that are included in the storage systems and one or more GPUs that are used in an AI or big data analytics pipeline. The one or more GPUs may be coupled to the storage system, for example, via NVMe-over-Fabrics (‘NVMe-oF’) such that bottlenecks such as the host CPU can be bypassed and the storage system (or one of the components contained therein) can directly access GPU memory. In such an example, the storage systems may leverage API hooks to the GPUs to transfer data directly to the GPUs. For example, the GPUs may be embodied as Nvidia™ GPUs and the storage systems may support GPUDirect Storage (‘GDS’) software, or have similar proprietary software, that enables the storage system to transfer data to the GPUs via RDMA or similar mechanism.

[0175] Although the preceding paragraphs discuss deep learning applications, readers will appreciate that the storage

systems described herein may also be part of a distributed deep learning (‘DDL’) platform to support the execution of DDL algorithms. The storage systems described above may also be paired with other technologies such as TensorFlow, an open-source software library for dataflow programming across a range of tasks that may be used for machine learning applications such as neural networks, to facilitate the development of such machine learning models, applications, and so on.

[0176] The storage systems described above may also be used in a neuromorphic computing environment. Neuromorphic computing is a form of computing that mimics brain cells. To support neuromorphic computing, an architecture of interconnected “neurons” replace traditional computing models with low-powered signals that go directly between neurons for more efficient computation. Neuromorphic computing may make use of very-large-scale integration (VLSI) systems containing electronic analog circuits to mimic neuro-biological architectures present in the nervous system, as well as analog, digital, mixed-mode analog/digital VLSI, and software systems that implement models of neural systems for perception, motor control, or multisensory integration.

[0177] Readers will appreciate that the storage systems described above may be configured to support the storage or use of (among other types of data) blockchains and derivative items such as, for example, open source blockchains and related tools that are part of the IBM™ Hyperledger project, permissioned blockchains in which a certain number of trusted parties are allowed to access the block chain, blockchain products that enable developers to build their own distributed ledger projects, and others. Blockchains and the storage systems described herein may be leveraged to support on-chain storage of data as well as off-chain storage of data.

[0178] Off-chain storage of data can be implemented in a variety of ways and can occur when the data itself is not stored within the blockchain. For example, in one embodiment, a hash function may be utilized and the data itself may be fed into the hash function to generate a hash value. In such an example, the hashes of large pieces of data may be embedded within transactions, instead of the data itself. Readers will appreciate that, in other embodiments, alternatives to blockchains may be used to facilitate the decentralized storage of information. For example, one alternative to a blockchain that may be used is a blockweave. While conventional blockchains store every transaction to achieve validation, a blockweave permits secure decentralization without the usage of the entire chain, thereby enabling low cost on-chain storage of data. Such blockweaves may utilize a consensus mechanism that is based on proof of access (PoA) and proof of work (PoW).

[0179] The storage systems described above may, either alone or in combination with other computing devices, be used to support in-memory computing applications. In-memory computing involves the storage of information in RAM that is distributed across a cluster of computers. Readers will appreciate that the storage systems described above, especially those that are configurable with customizable amounts of processing resources, storage resources, and memory resources (e.g., those systems in which blades that contain configurable amounts of each type of resource), may be configured in a way so as to provide an infrastructure that can support in-memory computing. Likewise, the stor-

age systems described above may include component parts (e.g., NVDIMMs, 3D crosspoint storage that provide fast random access memory that is persistent) that can actually provide for an improved in-memory computing environment as compared to in-memory computing environments that rely on RAM distributed across dedicated servers.

[0180] In some embodiments, the storage systems described above may be configured to operate as a hybrid in-memory computing environment that includes a universal interface to all storage media (e.g., RAM, flash storage, 3D crosspoint storage). In such embodiments, users may have no knowledge regarding the details of where their data is stored but they can still use the same full, unified API to address data. In such embodiments, the storage system may (in the background) move data to the fastest layer available—including intelligently placing the data in dependence upon various characteristics of the data or in dependence upon some other heuristic. In such an example, the storage systems may even make use of existing products such as Apache Ignite and GridGain to move data between the various storage layers, or the storage systems may make use of custom software to move data between the various storage layers. The storage systems described herein may implement various optimizations to improve the performance of in-memory computing such as, for example, having computations occur as close to the data as possible.

[0181] Readers will further appreciate that in some embodiments, the storage systems described above may be paired with other resources to support the applications described above. For example, one infrastructure could include primary compute in the form of servers and workstations which specialize in using General-purpose computing on graphics processing units (“GPGPU”) to accelerate deep learning applications that are interconnected into a computation engine to train parameters for deep neural networks. Each system may have Ethernet external connectivity, InfiniBand external connectivity, some other form of external connectivity, or some combination thereof. In such an example, the GPUs can be grouped for a single large training or used independently to train multiple models. The infrastructure could also include a storage system such as those described above to provide, for example, a scale-out all-flash file or object store through which data can be accessed via high-performance protocols such as NFS, S3, and so on. The infrastructure can also include, for example, redundant top-of-rack Ethernet switches connected to storage and compute via ports in MLAG port channels for redundancy. The infrastructure could also include additional compute in the form of whitebox servers, optionally with GPUs, for data ingestion, pre-processing, and model debugging. Readers will appreciate that additional infrastructures are also possible.

[0182] Readers will appreciate that the storage systems described above, either alone or in coordination with other computing machinery may be configured to support other AI related tools. For example, the storage systems may make use of tools like ONNX or other open neural network exchange formats that make it easier to transfer models written in different AI frameworks. Likewise, the storage systems may be configured to support tools like Amazon’s Glue that allow developers to prototype, build, and train deep learning models. In fact, the storage systems described above may be part of a larger platform, such as IBM™

Cloud Private for Data, that includes integrated data science, data engineering and application building services.

[0183] Readers will further appreciate that the storage systems described above may also be deployed as an edge solution. Such an edge solution may be in place to optimize cloud computing systems by performing data processing at the edge of the network, near the source of the data. Edge computing can push applications, data and computing power (i.e., services) away from centralized points to the logical extremes of a network. Through the use of edge solutions such as the storage systems described above, computational tasks may be performed using the compute resources provided by such storage systems, data may be stored using the storage resources of the storage system, and cloud-based services may be accessed through the use of various resources of the storage system (including networking resources). By performing computational tasks on the edge solution, storing data on the edge solution, and generally making use of the edge solution, the consumption of expensive cloud-based resources may be avoided and, in fact, performance improvements may be experienced relative to a heavier reliance on cloud-based resources.

[0184] While many tasks may benefit from the utilization of an edge solution, some particular uses may be especially suited for deployment in such an environment. For example, devices like drones, autonomous cars, robots, and others may require extremely rapid processing—so fast, in fact, that sending data up to a cloud environment and back to receive data processing support may simply be too slow. As an additional example, some IoT devices such as connected video cameras may not be well-suited for the utilization of cloud-based resources as it may be impractical (not only from a privacy perspective, security perspective, or a financial perspective) to send the data to the cloud simply because of the pure volume of data that is involved. As such, many tasks that really rely on data processing, storage, or communications may be better suited by platforms that include edge solutions such as the storage systems described above.

[0185] The storage systems described above may alone, or in combination with other computing resources, serve as a network edge platform that combines compute resources, storage resources, networking resources, cloud technologies and network virtualization technologies, and so on. As part of the network, the edge may take on characteristics similar to other network facilities, from the customer premise and backhaul aggregation facilities to Points of Presence (PoPs) and regional data centers. Readers will appreciate that network workloads, such as Virtual Network Functions (VNFs) and others, will reside on the network edge platform. Enabled by a combination of containers and virtual machines, the network edge platform may rely on controllers and schedulers that are no longer geographically co-located with the data processing resources. The functions, as microservices, may split into control planes, user and data planes, or even state machines, allowing for independent optimization and scaling techniques to be applied. Such user and data planes may be enabled through increased accelerators, both those residing in server platforms, such as FPGAs and Smart NICs, and through SDN-enabled merchant silicon and programmable ASICs.

[0186] The storage systems described above may also be optimized for use in big data analytics, including being leveraged as part of a composable data analytics pipeline where containerized analytics architectures, for example,

make analytics capabilities more composable. Big data analytics may be generally described as the process of examining large and varied data sets to uncover hidden patterns, unknown correlations, market trends, customer preferences and other useful information that can help organizations make more-informed business decisions. As part of that process, semi-structured and unstructured data such as, for example, internet clickstream data, web server logs, social media content, text from customer emails and survey responses, mobile-phone call-detail records, IoT sensor data, and other data may be converted to a structured form.

[0187] The storage systems described above may also support (including implementing as a system interface) applications that perform tasks in response to human speech. For example, the storage systems may support the execution intelligent personal assistant applications such as, for example, Amazon's Alexa™, Apple Siri™, Google Voice™, Samsung Bixby™, Microsoft Cortana™, and others. While the examples described in the previous sentence make use of voice as input, the storage systems described above may also support chatbots, talkbots, chatterbots, or artificial conversational entities or other applications that are configured to conduct a conversation via auditory or textual methods. Likewise, the storage system may actually execute such an application to enable a user such as a system administrator to interact with the storage system via speech. Such applications are generally capable of voice interaction, music playback, making to-do lists, setting alarms, streaming podcasts, playing audiobooks, and providing weather, traffic, and other real time information, such as news, although in embodiments in accordance with the present disclosure, such applications may be utilized as interfaces to various system management operations.

[0188] The storage systems described above may also implement AI platforms for delivering on the vision of self-driving storage. Such AI platforms may be configured to deliver global predictive intelligence by collecting and analyzing large amounts of storage system telemetry data points to enable effortless management, analytics and support. In fact, such storage systems may be capable of predicting both capacity and performance, as well as generating intelligent advice on workload deployment, interaction and optimization. Such AI platforms may be configured to scan all incoming storage system telemetry data against a library of issue fingerprints to predict and resolve incidents in real-time, before they impact customer environments, and captures hundreds of variables related to performance that are used to forecast performance load.

[0189] The storage systems described above may support the serialized or simultaneous execution of artificial intelligence applications, machine learning applications, data analytics applications, data transformations, and other tasks that collectively may form an AI ladder. Such an AI ladder may effectively be formed by combining such elements to form a complete data science pipeline, where exist dependencies between elements of the AI ladder. For example, AI may require that some form of machine learning has taken place, machine learning may require that some form of analytics has taken place, analytics may require that some form of data and information architecting has taken place, and so on. As such, each element may be viewed as a rung in an AI ladder that collectively can form a complete and sophisticated AI solution.

[0190] The storage systems described above may also, either alone or in combination with other computing environments, be used to deliver an AI everywhere experience where AI permeates wide and expansive aspects of business and life. For example, AI may play an important role in the delivery of deep learning solutions, deep reinforcement learning solutions, artificial general intelligence solutions, autonomous vehicles, cognitive computing solutions, commercial UAVs or drones, conversational user interfaces, enterprise taxonomies, ontology management solutions, machine learning solutions, smart dust, smart robots, smart workplaces, and many others.

[0191] The storage systems described above may also, either alone or in combination with other computing environments, be used to deliver a wide range of transparently immersive experiences (including those that use digital twins of various "things" such as people, places, processes, systems, and so on) where technology can introduce transparency between people, businesses, and things. Such transparently immersive experiences may be delivered as augmented reality technologies, connected homes, virtual reality technologies, brain-computer interfaces, human augmentation technologies, nanotube electronics, volumetric displays, 4D printing technologies, or others.

[0192] The storage systems described above may also, either alone or in combination with other computing environments, be used to support a wide variety of digital platforms. Such digital platforms can include, for example, 5G wireless systems and platforms, digital twin platforms, edge computing platforms, IoT platforms, quantum computing platforms, serverless PaaS, software-defined security, neuromorphic computing platforms, and so on.

[0193] The storage systems described above may also be part of a multi-cloud environment in which multiple cloud computing and storage services are deployed in a single heterogeneous architecture. In order to facilitate the operation of such a multi-cloud environment, DevOps tools may be deployed to enable orchestration across clouds. Likewise, continuous development and continuous integration tools may be deployed to standardize processes around continuous integration and delivery, new feature rollout and provisioning cloud workloads. By standardizing these processes, a multi-cloud strategy may be implemented that enables the utilization of the best provider for each workload.

[0194] The storage systems described above may be used as a part of a platform to enable the use of crypto-anchors that may be used to authenticate a product's origins and contents to ensure that it matches a blockchain record associated with the product. Similarly, as part of a suite of tools to secure data stored on the storage system, the storage systems described above may implement various encryption technologies and schemes, including lattice cryptography. Lattice cryptography can involve constructions of cryptographic primitives that involve lattices, either in the construction itself or in the security proof. Unlike public-key schemes such as the RSA, Diffie-Hellman or Elliptic-Curve cryptosystems, which are easily attacked by a quantum computer, some lattice-based constructions appear to be resistant to attack by both classical and quantum computers.

[0195] A quantum computer is a device that performs quantum computing. Quantum computing is computing using quantum-mechanical phenomena, such as superposition and entanglement. Quantum computers differ from traditional computers that are based on transistors, as such

traditional computers require that data be encoded into binary digits (bits), each of which is always in one of two definite states (0 or 1). In contrast to traditional computers, quantum computers use quantum bits, which can be in superpositions of states. A quantum computer maintains a sequence of qubits, where a single qubit can represent a one, a zero, or any quantum superposition of those two qubit states. A pair of qubits can be in any quantum superposition of 4 states, and three qubits in any superposition of 8 states. A quantum computer with n qubits can generally be in an arbitrary superposition of up to 2^n different states simultaneously, whereas a traditional computer can only be in one of these states at any one time. A quantum Turing machine is a theoretical model of such a computer.

[0196] The storage systems described above may also be paired with FPGA-accelerated servers as part of a larger AI or ML infrastructure. Such FPGA-accelerated servers may reside near (e.g., in the same data center) the storage systems described above or even incorporated into an appliance that includes one or more storage systems, one or more FPGA-accelerated servers, networking infrastructure that supports communications between the one or more storage systems and the one or more FPGA-accelerated servers, as well as other hardware and software components. Alternatively, FPGA-accelerated servers may reside within a cloud computing environment that may be used to perform compute-related tasks for AI and ML jobs. Any of the embodiments described above may be used to collectively serve as a FPGA-based AI or ML platform. Readers will appreciate that, in some embodiments of the FPGA-based AI or ML platform, the FPGAs that are contained within the FPGA-accelerated servers may be reconfigured for different types of ML models (e.g., LSTMs, CNNs, GRUs). The ability to reconfigure the FPGAs that are contained within the FPGA-accelerated servers may enable the acceleration of a ML or AI application based on the most optimal numerical precision and memory model being used. Readers will appreciate that by treating the collection of FPGA-accelerated servers as a pool of FPGAs, any CPU in the data center may utilize the pool of FPGAs as a shared hardware microservice, rather than limiting a server to dedicated accelerators plugged into it.

[0197] The FPGA-accelerated servers and the GPU-accelerated servers described above may implement a model of computing where, rather than keeping a small amount of data in a CPU and running a long stream of instructions over it as occurred in more traditional computing models, the machine learning model and parameters are pinned into the high-bandwidth on-chip memory with lots of data streaming through the high-bandwidth on-chip memory. FPGAs may even be more efficient than GPUs for this computing model, as the FPGAs can be programmed with only the instructions needed to run this kind of computing model.

[0198] The storage systems described above may be configured to provide parallel storage, for example, through the use of a parallel file system such as BeeGFS. Such parallel file systems may include a distributed metadata architecture. For example, the parallel file system may include a plurality of metadata servers across which metadata is distributed, as well as components that include services for clients and storage servers.

[0199] The systems described above can support the execution of a wide array of software applications. Such software applications can be deployed in a variety of ways,

including container-based deployment models. Containerized applications may be managed using a variety of tools. For example, containerized applications may be managed using Docker Swarm, Kubernetes, and others. Containerized applications may be used to facilitate a serverless, cloud native computing deployment and management model for software applications. In support of a serverless, cloud native computing deployment and management model for software applications, containers may be used as part of an event handling mechanisms (e.g., AWS Lambdas) such that various events cause a containerized application to be spun up to operate as an event handler.

[0200] The systems described above may be deployed in a variety of ways, including being deployed in ways that support fifth generation ('5G') networks. 5G networks may support substantially faster data communications than previous generations of mobile communications networks and, as a consequence may lead to the disaggregation of data and computing resources as modern massive data centers may become less prominent and may be replaced, for example, by more-local, micro data centers that are close to the mobile-network towers. The systems described above may be included in such local, micro data centers and may be part of or paired to multi-access edge computing ('MEC') systems. Such MEC systems may enable cloud computing capabilities and an IT service environment at the edge of the cellular network. By running applications and performing related processing tasks closer to the cellular customer, network congestion may be reduced and applications may perform better.

[0201] The storage systems described above may also be configured to implement NVMe Zoned Namespaces. Through the use of NVMe Zoned Namespaces, the logical address space of a namespace is divided into zones. Each zone provides a logical block address range that must be written sequentially and explicitly reset before rewriting, thereby enabling the creation of namespaces that expose the natural boundaries of the device and offload management of internal mapping tables to the host. In order to implement NVMe Zoned Name Spaces ('ZNS'), ZNS SSDs or some other form of zoned block devices may be utilized that expose a namespace logical address space using zones. With the zones aligned to the internal physical properties of the device, several inefficiencies in the placement of data can be eliminated. In such embodiments, each zone may be mapped, for example, to a separate application such that functions like wear levelling and garbage collection could be performed on a per-zone or per-application basis rather than across the entire device. In order to support ZNS, the storage controllers described herein may be configured with to interact with zoned block devices through the usage of, for example, the Linux™ kernel zoned block device interface or other tools.

[0202] The storage systems described above may also be configured to implement zoned storage in other ways such as, for example, through the usage of shingled magnetic recording (SMR) storage devices. In examples where zoned storage is used, device-managed embodiments may be deployed where the storage devices hide this complexity by managing it in the firmware, presenting an interface like any other storage device. Alternatively, zoned storage may be implemented via a host-managed embodiment that depends on the operating system to know how to handle the drive, and only write sequentially to certain regions of the drive.

Zoned storage may similarly be implemented using a host-aware embodiment in which a combination of a drive managed and host managed implementation is deployed.

[0203] The storage systems described herein may be used to form a data lake. A data lake may operate as the first place that an organization's data flows to, where such data may be in a raw format. Metadata tagging may be implemented to facilitate searches of data elements in the data lake, especially in embodiments where the data lake contains multiple stores of data, in formats not easily accessible or readable (e.g., unstructured data, semi-structured data, structured data). From the data lake, data may go downstream to a data warehouse where data may be stored in a more processed, packaged, and consumable format. The storage systems described above may also be used to implement such a data warehouse. In addition, a data mart or data hub may allow for data that is even more easily consumed, where the storage systems described above may also be used to provide the underlying storage resources necessary for a data mart or data hub. In embodiments, queries the data lake may require a schema-on-read approach, where data is applied to a plan or schema as it is pulled out of a stored location, rather than as it goes into the stored location.

[0204] The storage systems described herein may also be configured to implement a recovery point objective ('RPO'), which may be established by a user, established by an administrator, established as a system default, established as part of a storage class or service that the storage system is participating in the delivery of, or in some other way. A "recovery point objective" is a goal for the maximum time difference between the last update to a source dataset and the last recoverable replicated dataset update that would be correctly recoverable, given a reason to do so, from a continuously or frequently updated copy of the source dataset. An update is correctly recoverable if it properly takes into account all updates that were processed on the source dataset prior to the last recoverable replicated dataset update.

[0205] In synchronous replication, the RPO would be zero, meaning that under normal operation, all completed updates on the source dataset should be present and correctly recoverable on the copy dataset. In best effort nearly synchronous replication, the RPO can be as low as a few seconds. In snapshot-based replication, the RPO can be roughly calculated as the interval between snapshots plus the time to transfer the modifications between a previous already transferred snapshot and the most recent to-be-replicated snapshot.

[0206] If updates accumulate faster than they are replicated, then an RPO can be missed. If more data to be replicated accumulates between two snapshots, for snapshot-based replication, than can be replicated between taking the snapshot and replicating that snapshot's cumulative updates to the copy, then the RPO can be missed. If, again in snapshot-based replication, data to be replicated accumulates at a faster rate than could be transferred in the time between subsequent snapshots, then replication can start to fall further behind which can extend the miss between the expected recovery point objective and the actual recovery point that is represented by the last correctly replicated update.

[0207] The storage systems described above may also be part of a shared nothing storage cluster. In a shared nothing storage cluster, each node of the cluster has local storage and

communicates with other nodes in the cluster through networks, where the storage used by the cluster is (in general) provided only by the storage connected to each individual node. A collection of nodes that are synchronously replicating a dataset may be one example of a shared nothing storage cluster, as each storage system has local storage and communicates to other storage systems through a network, where those storage systems do not (in general) use storage from somewhere else that they share access to through some kind of interconnect. In contrast, some of the storage systems described above are themselves built as a shared-storage cluster, since there are drive shelves that are shared by the paired controllers. Other storage systems described above, however, are built as a shared nothing storage cluster, as all storage is local to a particular node (e.g., a blade) and all communication is through networks that link the compute nodes together.

[0208] In other embodiments, other forms of a shared nothing storage cluster can include embodiments where any node in the cluster has a local copy of all storage they need, and where data is mirrored through a synchronous style of replication to other nodes in the cluster either to ensure that the data isn't lost or because other nodes are also using that storage. In such an embodiment, if a new cluster node needs some data, that data can be copied to the new node from other nodes that have copies of the data.

[0209] In some embodiments, mirror-copy-based shared storage clusters may store multiple copies of all the cluster's stored data, with each subset of data replicated to a particular set of nodes, and different subsets of data replicated to different sets of nodes. In some variations, embodiments may store all of the cluster's stored data in all nodes, whereas in other variations nodes may be divided up such that a first set of nodes will all store the same set of data and a second, different set of nodes will all store a different set of data.

[0210] Readers will appreciate that RAFT-based databases (e.g., etcd) may operate like shared-nothing storage clusters where all RAFT nodes store all data. The amount of data stored in a RAFT cluster, however, may be limited so that extra copies don't consume too much storage. A container server cluster might also be able to replicate all data to all cluster nodes, presuming the containers don't tend to be too large and their bulk data (the data manipulated by the applications that run in the containers) is stored elsewhere such as in an S3 cluster or an external file server. In such an example, the container storage may be provided by the cluster directly through its shared-nothing storage model, with those containers providing the images that form the execution environment for parts of an application or service.

[0211] For further explanation, FIG. 3D illustrates an exemplary computing device 350 that may be specifically configured to perform one or more of the processes described herein. As shown in FIG. 3D, computing device 350 may include a communication interface 352, a processor 354, a storage device 356, and an input/output ("I/O") module 358 communicatively connected one to another via a communication infrastructure 360. While an exemplary computing device 350 is shown in FIG. 3D, the components illustrated in FIG. 3D are not intended to be limiting. Additional or alternative components may be used in other embodiments. Components of computing device 350 shown in FIG. 3D will now be described in additional detail.

[0212] Communication interface 352 may be configured to communicate with one or more computing devices. Examples of communication interface 352 include, without limitation, a wired network interface (such as a network interface card), a wireless network interface (such as a wireless network interface card), a modem, an audio/video connection, and any other suitable interface.

[0213] Processor 354 generally represents any type or form of processing unit capable of processing data and/or interpreting, executing, and/or directing execution of one or more of the instructions, processes, and/or operations described herein. Processor 354 may perform operations by executing computer-executable instructions 362 (e.g., an application, software, code, and/or other executable data instance) stored in storage device 356.

[0214] Storage device 356 may include one or more data storage media, devices, or configurations and may employ any type, form, and combination of data storage media and/or device. For example, storage device 356 may include, but is not limited to, any combination of the non-volatile media and/or volatile media described herein. Electronic data, including data described herein, may be temporarily and/or permanently stored in storage device 356. For example, data representative of computer-executable instructions 362 configured to direct processor 354 to perform any of the operations described herein may be stored within storage device 356. In some examples, data may be arranged in one or more databases residing within storage device 356.

[0215] I/O module 358 may include one or more I/O modules configured to receive user input and provide user output. I/O module 358 may include any hardware, firmware, software, or combination thereof supportive of input and output capabilities. For example, I/O module 358 may include hardware and/or software for capturing user input, including, but not limited to, a keyboard or keypad, a touchscreen component (e.g., touchscreen display), a receiver (e.g., an RF or infrared receiver), motion sensors, and/or one or more input buttons.

[0216] I/O module 358 may include one or more devices for presenting output to a user, including, but not limited to, a graphics engine, a display (e.g., a display screen), one or more output drivers (e.g., display drivers), one or more audio speakers, and one or more audio drivers. In certain embodiments, I/O module 358 is configured to provide graphical data to a display for presentation to a user. The graphical data may be representative of one or more graphical user interfaces and/or any other graphical content as may serve a particular implementation. In some examples, any of the systems, computing devices, and/or other components described herein may be implemented by computing device 350.

[0217] For further explanation, FIG. 3E illustrates an example of a fleet of storage systems 376 for providing storage services (also referred to herein as ‘data services’). The fleet of storage systems 376 depicted in FIG. 3E includes a plurality of storage systems 374a, 374b, 374c, through 374n that may each be similar to the storage systems described herein. The storage systems 374a, 374b, 374c, through 374n in the fleet of storage systems 376 may be embodied as identical storage systems or as different types of storage systems. For example, two of the storage systems 374a, 374n depicted in FIG. 3E are depicted as being cloud-based storage systems, as the resources that collec-

tively form each of the storage systems 374a, 374n are provided by distinct cloud services providers 370, 372. For example, the first cloud services provider 370 may be Amazon AWS™ whereas the second cloud services provider 372 is Microsoft Azure™, although in other embodiments one or more public clouds, private clouds, or combinations thereof may be used to provide the underlying resources that are used to form a particular storage system in the fleet of storage systems 376.

[0218] The example depicted in FIG. 3E includes an edge management service 366 for delivering storage services in accordance with some embodiments of the present disclosure. The storage services (also referred to herein as ‘data services’) that are delivered may include, for example, services to provide a certain amount of storage to a consumer, services to provide storage to a consumer in accordance with a predetermined service level agreement, services to provide storage to a consumer in accordance with predetermined regulatory requirements, and many others.

[0219] The edge management service 366 depicted in FIG. 3E may be embodied, for example, as one or more modules of computer program instructions executing on computer hardware such as one or more computer processors. Alternatively, the edge management service 366 may be embodied as one or more modules of computer program instructions executing on a virtualized execution environment such as one or more virtual machines, in one or more containers, or in some other way. In other embodiments, the edge management service 366 may be embodied as a combination of the embodiments described above, including embodiments where the one or more modules of computer program instructions that are included in the edge management service 366 are distributed across multiple physical or virtual execution environments.

[0220] The edge management service 366 may operate as a gateway for providing storage services to storage consumers, where the storage services leverage storage offered by one or more storage systems 374a, 374b, 374c, through 374n. For example, the edge management service 366 may be configured to provide storage services to host devices 378a, 378b, 378c, 378d, 378n that are executing one or more applications that consume the storage services. In such an example, the edge management service 366 may operate as a gateway between the host devices 378a, 378b, 378c, 378d, 378n and the storage systems 374a, 374b, 374c, through 374n, rather than requiring that the host devices 378a, 378b, 378c, 378d, 378n directly access the storage systems 374a, 374b, 374c, through 374n.

[0221] The edge management service 366 of FIG. 3E exposes a storage services module 364 to the host devices 378a, 378b, 378c, 378d, 378n of FIG. 3E, although in other embodiments the edge management service 366 may expose the storage services module 364 to other consumers of the various storage services. The various storage services may be presented to consumers via one or more user interfaces, via one or more APIs, or through some other mechanism provided by the storage services module 364. As such, the storage services module 364 depicted in FIG. 3E may be embodied as one or more modules of computer program instructions executing on physical hardware, on a virtualized execution environment, or combinations thereof, where executing such modules causes enables a consumer of storage services to be offered, select, and access the various storage services.

[0222] The edge management service 366 of FIG. 3E also includes a system management services module 368. The system management services module 368 of FIG. 3E includes one or more modules of computer program instructions that, when executed, perform various operations in coordination with the storage systems 374a, 374b, 374c, through 374n to provide storage services to the host devices 378a, 378b, 378c, 378d, 378n. The system management services module 368 may be configured, for example, to perform tasks such as provisioning storage resources from the storage systems 374a, 374b, 374c, through 374n via one or more APIs exposed by the storage systems 374a, 374b, 374c, through 374n, migrating datasets or workloads amongst the storage systems 374a, 374b, 374c, through 374n via one or more APIs exposed by the storage systems 374a, 374b, 374c, through 374n, setting one or more tunable parameters (i.e., one or more configurable settings) on the storage systems 374a, 374b, 374c, through 374n via one or more APIs exposed by the storage systems 374a, 374b, 374c, through 374n, and so on. For example, many of the services described below relate to embodiments where the storage systems 374a, 374b, 374c, through 374n are configured to operate in some way. In such examples, the system management services module 368 may be responsible for using APIs (or some other mechanism) provided by the storage systems 374a, 374b, 374c, through 374n to configure the storage systems 374a, 374b, 374c, through 374n to operate in the ways described below.

[0223] In addition to configuring the storage systems 374a, 374b, 374c, through 374n, the edge management service 366 itself may be configured to perform various tasks required to provide the various storage services. Consider an example in which the storage service includes a service that, when selected and applied, causes personally identifiable information (“PII”) contained in a dataset to be obfuscated when the dataset is accessed. In such an example, the storage systems 374a, 374b, 374c, through 374n may be configured to obfuscate PII when servicing read requests directed to the dataset. Alternatively, the storage systems 374a, 374b, 374c, through 374n may service reads by returning data that includes the PII, but the edge management service 366 itself may obfuscate the PII as the data is passed through the edge management service 366 on its way from the storage systems 374a, 374b, 374c, through 374n to the host devices 378a, 378b, 378c, 378d, 378n.

[0224] The storage systems 374a, 374b, 374c, through 374n depicted in FIG. 3E may be embodied as one or more of the storage systems described above with reference to FIGS. 1A-3D, including variations thereof. In fact, the storage systems 374a, 374b, 374c, through 374n may serve as a pool of storage resources where the individual components in that pool have different performance characteristics, different storage characteristics, and so on. For example, one of the storage systems 374a may be a cloud-based storage system, another storage system 374b may be a storage system that provides block storage, another storage system 374c may be a storage system that provides file storage, another storage system 374d may be a relatively high-performance storage system while another storage system 374n may be a relatively low-performance storage system, and so on. In alternative embodiments, only a single storage system may be present.

[0225] The storage systems 374a, 374b, 374c, through 374n depicted in FIG. 3E may also be organized into

different failure domains so that the failure of one storage system 374a should be totally unrelated to the failure of another storage system 374b. For example, each of the storage systems may receive power from independent power systems, each of the storage systems may be coupled for data communications over independent data communications networks, and so on. Furthermore, the storage systems in a first failure domain may be accessed via a first gateway whereas storage systems in a second failure domain may be accessed via a second gateway. For example, the first gateway may be a first instance of the edge management service 366 and the second gateway may be a second instance of the edge management service 366, including embodiments where each instance is distinct, or each instance is part of a distributed edge management service 366.

[0226] As an illustrative example of available storage services, storage services may be presented to a user that are associated with different levels of data protection. For example, storage services may be presented to the user that, when selected and enforced, guarantee the user that data associated with that user will be protected such that various recovery point objectives (“RPO”) can be guaranteed. A first available storage service may ensure, for example, that some dataset associated with the user will be protected such that any data that is more than 5 seconds old can be recovered in the event of a failure of the primary data store whereas a second available storage service may ensure that the dataset that is associated with the user will be protected such that any data that is more than 5 minutes old can be recovered in the event of a failure of the primary data store.

[0227] An additional example of storage services that may be presented to a user, selected by a user, and ultimately applied to a dataset associated with the user can include one or more data compliance services. Such data compliance services may be embodied, for example, as services that may be provided to consumers (i.e., a user) the data compliance services to ensure that the user’s datasets are managed in a way to adhere to various regulatory requirements. For example, one or more data compliance services may be offered to a user to ensure that the user’s datasets are managed in a way so as to adhere to the General Data Protection Regulation (“GDPR”), one or data compliance services may be offered to a user to ensure that the user’s datasets are managed in a way so as to adhere to the Sarbanes-Oxley Act of 2002 (“SOX”), or one or more data compliance services may be offered to a user to ensure that the user’s datasets are managed in a way so as to adhere to some other regulatory act. In addition, the one or more data compliance services may be offered to a user to ensure that the user’s datasets are managed in a way so as to adhere to some non-governmental guidance (e.g., to adhere to best practices for auditing purposes), the one or more data compliance services may be offered to a user to ensure that the user’s datasets are managed in a way so as to adhere to a particular clients or organizations requirements, and so on.

[0228] In order to provide this particular data compliance service, the data compliance service may be presented to a user (e.g., via a GUI) and selected by the user. In response to receiving the selection of the particular data compliance service, one or more storage services policies may be applied to a dataset associated with the user to carry out the particular data compliance service. For example, a storage services policy may be applied requiring that the dataset be

encrypted prior to being stored in a storage system, prior to being stored in a cloud environment, or prior to being stored elsewhere. In order to enforce this policy, a requirement may be enforced not only requiring that the dataset be encrypted when stored, but a requirement may be put in place requiring that the dataset be encrypted prior to transmitting the dataset (e.g., sending the dataset to another party). In such an example, a storage services policy may also be put in place requiring that any encryption keys used to encrypt the dataset are not stored on the same system that stores the dataset itself. Readers will appreciate that many other forms of data compliance services may be offered and implemented in accordance with embodiments of the present disclosure.

[0229] The storage systems 374a, 374b, 374c, through 374n in the fleet of storage systems 376 may be managed collectively, for example, by one or more fleet management modules. The fleet management modules may be part of or separate from the system management services module 368 depicted in FIG. 3E. The fleet management modules may perform tasks such as monitoring the health of each storage system in the fleet, initiating updates or upgrades on one or more storage systems in the fleet, migrating workloads for loading balancing or other performance purposes, and many other tasks. As such, and for many other reasons, the storage systems 374a, 374b, 374c, through 374n may be coupled to each other via one or more data communications links in order to exchange data between the storage systems 374a, 374b, 374c, through 374n.

[0230] In some embodiments, one or more storage systems or one or more elements of storage systems (e.g., features, services, operations, components, etc. of storage systems), such as any of the illustrative storage systems or storage system elements described herein may be implemented in one or more container systems. A container system may include any system that supports execution of one or more containerized applications or services. Such a service may be software deployed as infrastructure for building applications, for operating a run-time environment, and/or as infrastructure for other services. In the discussion that follows, descriptions of containerized applications generally apply to containerized services as well.

[0231] A container may combine one or more elements of a containerized software application together with a runtime environment for operating those elements of the software application bundled into a single image. For example, each such container of a containerized application may include executable code of the software application and various dependencies, libraries, and/or other components, together with network configurations and configured access to additional resources, used by the elements of the software application within the particular container in order to enable operation of those elements. A containerized application can be represented as a collection of such containers that together represent all the elements of the application combined with the various run-time environments needed for all those elements to run. As a result, the containerized application may be abstracted away from host operating systems as a combined collection of lightweight and portable packages and configurations, where the containerized application may be uniformly deployed and consistently executed in different computing environments that use different container-compatible operating systems or different infrastructures. In some embodiments, a containerized application

shares a kernel with a host computer system and executes as an isolated environment (an isolated collection of files and directories, processes, system and network resources, and configured access to additional resources and capabilities) that is isolated by an operating system of a host system in conjunction with a container management framework. When executed, a containerized application may provide one or more containerized workloads and/or services.

[0232] The container system may include and/or utilize a cluster of nodes. For example, the container system may be configured to manage deployment and execution of containerized applications on one or more nodes in a cluster. The containerized applications may utilize resources of the nodes, such as memory, processing and/or storage resources provided and/or accessed by the nodes. The storage resources may include any of the illustrative storage resources described herein and may include on-node resources such as a local tree of files and directories, off-node resources such as external networked file systems, databases or object stores, or both on-node and off-node resources. Access to additional resources and capabilities that could be configured for containers of a containerized application could include specialized computation capabilities such as GPUs and AI/ML engines, or specialized hardware such as sensors and cameras.

[0233] In some embodiments, the container system may include a container orchestration system (which may also be referred to as a container orchestrator, a container orchestration platform, etc.) designed to make it reasonably simple and for many use cases automated to deploy, scale, and manage containerized applications. In some embodiments, the container system may include a storage management system configured to provision and manage storage resources (e.g., virtual volumes) for private or shared use by cluster nodes and/or containers of containerized applications.

[0234] FIG. 3F illustrates an example container system 380. In this example, the container system 380 includes a container storage system 381 that may be configured to perform one or more storage management operations to organize, provision, and manage storage resources for use by one or more containerized applications 382-1 through 382-L of container system 380. In particular, the container storage system 381 may organize storage resources into one or more storage pools 383 of storage resources for use by containerized applications 382-1 through 382-L. The container storage system may itself be implemented as a containerized service.

[0235] The container system 380 may include or be implemented by one or more container orchestration systems, including Kubernetes™, Mesos™, Docker Swarm™, among others. The container orchestration system may manage the container system 380 running on a cluster 384 through services implemented by a control node, depicted as 385, and may further manage the container storage system or the relationship between individual containers and their storage, memory and CPU limits, networking, and their access to additional resources or services.

[0236] A control plane of the container system 380 may implement services that include: deploying applications via a controller 386, monitoring applications via the controller 386, providing an interface via an API server 387, and scheduling deployments via scheduler 388. In this example, controller 386, scheduler 388, API server 387, and container

storage system **381** are implemented on a single node, node **385**. In other examples, for resiliency, the control plane may be implemented by multiple, redundant nodes, where if a node that is providing management services for the container system **380** fails, then another, redundant node may provide management services for the cluster **384**.

[0237] A data plane of the container system **380** may include a set of nodes that provides container runtimes for executing containerized applications. An individual node within the cluster **384** may execute a container runtime, such as Docker™, and execute a container manager, or node agent, such as a kubelet in Kubernetes (not depicted) that communicates with the control plane via a local network-connected agent (sometimes called a proxy), such as an agent **389**. The agent **389** may route network traffic to and from containers using, for example, Internet Protocol (IP) port numbers. For example, a containerized application may request a storage class from the control plane, where the request is handled by the container manager, and the container manager communicates the request to the control plane using the agent **389**.

[0238] Cluster **384** may include a set of nodes that run containers for managed containerized applications. A node may be a virtual or physical machine. A node may be a host system.

[0239] The container storage system **381** may orchestrate storage resources to provide storage to the container system **380**. For example, the container storage system **381** may provide persistent storage to containerized applications **382-1-382-L** using the storage pool **383**. The container storage system **381** may itself be deployed as a containerized application by a container orchestration system.

[0240] For example, the container storage system **381** application may be deployed within cluster **384** and perform management functions for providing storage to the containerized applications **382**. Management functions may include determining one or more storage pools from available storage resources, provisioning virtual volumes on one or more nodes, replicating data, responding to and recovering from host and network faults, or handling storage operations. The storage pool **383** may include storage resources from one or more local or remote sources, where the storage resources may be different types of storage, including, as examples, block storage, file storage, and object storage.

[0241] The container storage system **381** may also be deployed on a set of nodes for which persistent storage may be provided by the container orchestration system. In some examples, the container storage system **381** may be deployed on all nodes in a cluster **384** using, for example, a Kubernetes DaemonSet. In this example, nodes **390-1** through **390-N** provide a container runtime where container storage system **381** executes. In other examples, some, but not all nodes in a cluster may execute the container storage system **381**.

[0242] The container storage system **381** may handle storage on a node and communicate with the control plane of container system **380**, to provide dynamic volumes, including persistent volumes. A persistent volume may be mounted on a node as a virtual volume, such as virtual volumes **391-1** and **391-P**. After a virtual volume **391** is mounted, containerized applications may request and use, or be otherwise configured to use, storage provided by the virtual volume **391**. In this example, the container storage system **381** may install a driver on a kernel of a node, where

the driver handles storage operations directed to the virtual volume. In this example, the driver may receive a storage operation directed to a virtual volume, and in response, the driver may perform the storage operation on one or more storage resources within the storage pool **383**, possibly under direction from or using additional logic within containers that implement the container storage system **381** as a containerized service.

[0243] The container storage system **381** may, in response to being deployed as a containerized service, determine available storage resources. For example, storage resources **392-1** through **392-M** may include local storage, remote storage (storage on a separate node in a cluster), or both local and remote storage. Storage resources may also include storage from external sources such as various combinations of block storage systems, file storage systems, and object storage systems. The storage resources **392-1** through **392-M** may include any type(s) and/or configuration(s) of storage resources (e.g., any of the illustrative storage resources described above), and the container storage system **381** may be configured to determine the available storage resources in any suitable way, including based on a configuration file. For example, a configuration file may specify account and authentication information for cloud-based object storage **348** or for a cloud-based storage system **318**. The container storage system **381** may also determine availability of one or more storage devices **356** or one or more storage systems. An aggregate amount of storage from one or more of storage device(s) **356**, storage system(s), cloud-based storage system(s) **318**, edge management services **366**, cloud-based object storage **348**, or any other storage resources, or any combination or sub-combination of such storage resources may be used to provide the storage pool **383**. The storage pool **383** is used to provision storage for the one or more virtual volumes mounted on one or more of the nodes **390** within cluster **384**. In some implementations, the container storage system **381** may create multiple storage pools. For example, the container storage system **381** may aggregate storage resources of a same type into an individual storage pool. In this example, a storage type may be one of: a storage device **356**, a storage array **102**, a cloud-based storage system **318**, storage via an edge management service **366**, or a cloud-based object storage **348**. Or it could be storage configured with a certain level or type of redundancy or distribution, such as a particular combination of striping, mirroring, or erasure coding.

[0244] The container storage system **381** may execute within the cluster **384** as a containerized container storage system service, where instances of containers that implement elements of the containerized container storage system service may operate on different nodes within the cluster **384**. In this example, the containerized container storage system service may operate in conjunction with the container orchestration system of the container system **380** to handle storage operations, mount virtual volumes to provide storage to a node, aggregate available storage into a storage pool **383**, provision storage for a virtual volume from a storage pool **383**, generate backup data, replicate data between nodes, clusters, environments, among other storage system operations. In some examples, the containerized container storage system service may provide storage services across multiple clusters operating in distinct computing environments. For example, other storage system operations may include storage system operations described

herein. Persistent storage provided by the containerized container storage system service may be used to implement stateful and/or resilient containerized applications.

[0245] The container storage system **381** may be configured to perform any suitable storage operations of a storage system. For example, the container storage system **381** may be configured to perform one or more of the illustrative storage management operations described herein to manage storage resources used by the container system.

[0246] In some embodiments, one or more storage operations, including one or more of the illustrative storage management operations described herein, may be containerized. For example, one or more storage operations may be implemented as one or more containerized applications configured to be executed to perform the storage operation(s). Such containerized storage operations may be executed in any suitable runtime environment to manage any storage system(s), including any of the illustrative storage systems described herein.

[0247] The storage systems described herein may support various forms of data replication. For example, two or more of the storage systems may synchronously replicate a dataset between each other. In synchronous replication, distinct copies of a particular dataset may be maintained by multiple storage systems, but all accesses (e.g., a read) of the dataset should yield consistent results regardless of which storage system the access was directed to. For example, a read directed to any of the storage systems that are synchronously replicating the dataset should return identical results. As such, while updates to the version of the dataset need not occur at exactly the same time, precautions must be taken to ensure consistent accesses to the dataset. For example, if an update (e.g., a write) that is directed to the dataset is received by a first storage system, the update may only be acknowledged as being completed if all storage systems that are synchronously replicating the dataset have applied the update to their copies of the dataset. In such an example, synchronous replication may be carried out through the use of I/O forwarding (e.g., a write received at a first storage system is forwarded to a second storage system), communications between the storage systems (e.g., each storage system indicating that it has completed the update), or in other ways.

[0248] In other embodiments, a dataset may be replicated through the use of checkpoints. In checkpoint-based replication (also referred to as ‘nearly synchronous replication’), a set of updates to a dataset (e.g., one or more write operations directed to the dataset) may occur between different checkpoints, such that a dataset has been updated to a specific checkpoint only if all updates to the dataset prior to the specific checkpoint have been completed. Consider an example in which a first storage system stores a live copy of a dataset that is being accessed by users of the dataset. In this example, assume that the dataset is being replicated from the first storage system to a second storage system using checkpoint-based replication. For example, the first storage system may send a first checkpoint (at time $t=0$) to the second storage system, followed by a first set of updates to the dataset, followed by a second checkpoint (at time $t=1$), followed by a second set of updates to the dataset, followed by a third checkpoint (at time $t=2$). In such an example, if the second storage system has performed all updates in the first set of updates but has not yet performed all updates in the second set of updates, the copy of the dataset that is stored

on the second storage system may be up-to-date until the second checkpoint. Alternatively, if the second storage system has performed all updates in both the first set of updates and the second set of updates, the copy of the dataset that is stored on the second storage system may be up-to-date until the third checkpoint. Readers will appreciate that various types of checkpoints may be used (e.g., metadata only checkpoints), checkpoints may be spread out based on a variety of factors (e.g., time, number of operations, an RPO setting), and so on.

[0249] In other embodiments, a dataset may be replicated through snapshot-based replication (also referred to as ‘asynchronous replication’). In snapshot-based replication, snapshots of a dataset may be sent from a replication source such as a first storage system to a replication target such as a second storage system. In such an embodiment, each snapshot may include the entire dataset or a subset of the dataset such as, for example, only the portions of the dataset that have changed since the last snapshot was sent from the replication source to the replication target. Readers will appreciate that snapshots may be sent on-demand, based on a policy that takes a variety of factors into consideration (e.g., time, number of operations, an RPO setting), or in some other way.

[0250] The storage systems described above may, either alone or in combination, be configured to serve as a continuous data protection store. A continuous data protection store is a feature of a storage system that records updates to a dataset in such a way that consistent images of prior contents of the dataset can be accessed with a low time granularity (often on the order of seconds, or even less), and stretching back for a reasonable period of time (often hours or days). These allow access to very recent consistent points in time for the dataset, and also allow access to points in time for a dataset that might have just preceded some event that, for example, caused parts of the dataset to be corrupted or otherwise lost, while retaining close to the maximum number of updates that preceded that event. Conceptually, they are like a sequence of snapshots of a dataset taken very frequently and kept for a long period of time, though continuous data protection stores are often implemented quite differently from snapshots. A storage system implementing a data continuous data protection store may further provide a means of accessing these points in time, accessing one or more of these points in time as snapshots or as cloned copies, or reverting the dataset back to one of those recorded points in time.

[0251] Over time, to reduce overhead, some points in the time held in a continuous data protection store can be merged with other nearby points in time, essentially deleting some of these points in time from the store. This can reduce the capacity needed to store updates. It may also be possible to convert a limited number of these points in time into longer duration snapshots. For example, such a store might keep a low granularity sequence of points in time stretching back a few hours from the present, with some points in time merged or deleted to reduce overhead for up to an additional day. Stretching back in the past further than that, some of these points in time could be converted to snapshots representing consistent point-in-time images from only every few hours.

[0252] Although some embodiments are described largely in the context of a storage system, readers of skill in the art will recognize that embodiments of the present disclosure

may also take the form of a computer program product disposed upon computer readable storage media for use with any suitable processing system. Such computer readable storage media may be any storage medium for machine-readable information, including magnetic media, optical media, solid-state media, or other suitable media. Examples of such media include magnetic disks in hard drives or diskettes, compact disks for optical drives, magnetic tape, and others as will occur to those of skill in the art. Persons skilled in the art will immediately recognize that any computer system having suitable programming means will be capable of executing the steps described herein as embodied in a computer program product. Persons skilled in the art will recognize also that, although some of the embodiments described in this specification are oriented to software installed and executing on computer hardware, nevertheless, alternative embodiments implemented as firmware or as hardware are well within the scope of the present disclosure.

[0253] In some examples, a non-transitory computer-readable medium storing computer-readable instructions may be provided in accordance with the principles described herein. The instructions, when executed by a processor of a computing device, may direct the processor and/or computing device to perform one or more operations, including one or more of the operations described herein. Such instructions may be stored and/or transmitted using any of a variety of known computer-readable media.

[0254] A non-transitory computer-readable medium as referred to herein may include any non-transitory storage medium that participates in providing data (e.g., instructions) that may be read and/or executed by a computing device (e.g., by a processor of a computing device). For example, a non-transitory computer-readable medium may include, but is not limited to, any combination of non-volatile storage media and/or volatile storage media. Exemplary non-volatile storage media include, but are not limited to, read-only memory, flash memory, a solid-state drive, a magnetic storage device (e.g., a hard disk, a floppy disk, magnetic tape, etc.), ferroelectric random-access memory ("RAM"), and an optical disc (e.g., a compact disc, a digital video disc, a Blu-ray disc, etc.). Exemplary volatile storage media include, but are not limited to, RAM (e.g., dynamic RAM).

[0255] For further explanation, FIG. 4 sets forth a block diagram illustrating a plurality of storage systems (402, 404, 406) that support a pod according to some embodiments of the present disclosure. Although depicted in less detail, the storage systems (402, 404, 406) depicted in FIG. 4 may be similar to the storage systems described above with reference to FIGS. 1A-1D, FIGS. 2A-2G, FIGS. 3A-3B, or any combination thereof. In fact, the storage systems (402, 404, 406) depicted in FIG. 4 may include the same, fewer, or additional components as the storage systems described above.

[0256] In the example depicted in FIG. 4, each of the storage systems (402, 404, 406) is depicted as having at least one computer processor (408, 410, 412), computer memory (414, 416, 418), and computer storage (420, 422, 424). Although in some embodiments the computer memory (414, 416, 418) and the computer storage (420, 422, 424) may be part of the same hardware devices, in other embodiments the computer memory (414, 416, 418) and the computer storage (420, 422, 424) may be part of different hardware devices. The distinction between the computer memory (414, 416,

418) and the computer storage (420, 422, 424) in this particular example may be that the computer memory (414, 416, 418) is physically proximate to the computer processors (408, 410, 412) and may store computer program instructions that are executed by the computer processors (408, 410, 412), while the computer storage (420, 422, 424) is embodied as non-volatile storage for storing user data, metadata describing the user data, and so on. Referring to the example above in FIG. 1A, for example, the computer processors (408, 410, 412) and computer memory (414, 416, 418) for a particular storage system (402, 404, 406) may reside within one of more of the controllers (110A-110D) while the attached storage devices (171A-171F) may serve as the computer storage (420, 422, 424) within a particular storage system (402, 404, 406).

[0257] In the example depicted in FIG. 4, the depicted storage systems (402, 404, 406) may attach to one or more pods (430, 432) according to some embodiments of the present disclosure. Each of the pods (430, 432) depicted in FIG. 4 can include a dataset (426, 428). For example, a first pod (430) that three storage systems (402, 404, 406) have attached to includes a first dataset (426) while a second pod (432) that two storage systems (404, 406) have attached to includes a second dataset (428). In such an example, when a particular storage system attaches to a pod, the pod's dataset is copied to the particular storage system and then kept up to date as the dataset is modified. Storage systems can be removed from a pod, resulting in the dataset being no longer kept up to date on the removed storage system. In the example depicted in FIG. 4, any storage system which is active for a pod (it is an up-to-date, operating, non-faulted member of a non-faulted pod) can receive and process requests to modify or read the pod's dataset.

[0258] In the example depicted in FIG. 4, each pod (430, 432) may also include a set of managed objects and management operations, as well as a set of access operations to modify or read the dataset (426, 428) that is associated with the particular pod (430, 432). In such an example, the management operations may modify or query managed objects equivalently through any of the storage systems. Likewise, access operations to read or modify the dataset may operate equivalently through any of the storage systems. In such an example, while each storage system stores a separate copy of the dataset as a proper subset of the datasets stored and advertised for use by the storage system, the operations to modify managed objects or the dataset performed and completed through any one storage system are reflected in subsequent management objects to query the pod or subsequent access operations to read the dataset.

[0259] Readers will appreciate that pods may implement more capabilities than just a clustered synchronously replicated dataset. For example, pods can be used to implement tenants, whereby datasets are in some way securely isolated from each other. Pods can also be used to implement virtual arrays or virtual storage systems where each pod is presented as a unique storage entity on a network (e.g., a Storage Area Network, or Internet Protocol network) with separate addresses. In the case of a multi-storage-system pod implementing a virtual storage system, all physical storage systems associated with the pod may present themselves as in some way the same storage system (e.g., as if the multiple physical storage systems were no different than multiple network ports into a single storage system).

[0260] Readers will appreciate that pods may also be units of administration, representing a collection of volumes, file systems, object/analytic stores, snapshots, and other administrative entities, where making administrative changes (e.g., name changes, property changes, managing exports or permissions for some part of the pod's dataset), on any one storage system is automatically reflected to all active storage systems associated with the pod. In addition, pods could also be units of data collection and data analysis, where performance and capacity metrics are presented in ways that aggregate across all active storage systems for the pod, or that call out data collection and analysis separately for each pod, or perhaps presenting each attached storage system's contribution to the incoming content and performance for each a pod.

[0261] One model for pod membership may be defined as a list of storage systems, and a subset of that list where storage systems are considered to be in-sync for the pod. A storage system may be considered to be in-sync for a pod if it is at least within a recovery of having identical idle content for the last written copy of the dataset associated with the pod. Idle content is the content after any in-progress modifications have completed with no processing of new modifications. Sometimes this is referred to as "crash recoverable" consistency. Recovery of a pod carries out the process of reconciling differences in applying concurrent updates to in-sync storage systems in the pod. Recovery can resolve any inconsistencies between storage systems in the completion of concurrent modifications that had been requested to various members of the pod but that were not signaled to any requestor as having completed successfully. Storage systems that are listed as pod members but that are not listed as in-sync for the pod can be described as "detached" from the pod. Storage systems that are listed as pod members, are in-sync for the pod, and are currently available for actively serving data for the pod are "online" for the pod.

[0262] Each storage system member of a pod may have its own copy of the membership, including which storage systems it last knew were in-sync, and which storage systems it last knew comprised the entire set of pod members. To be online for a pod, a storage system must consider itself to be in-sync for the pod and must be communicating with all other storage systems it considers to be in-sync for the pod. If a storage system can't be certain that it is in-sync and communicating with all other storage systems that are in-sync, then it must stop processing new incoming requests for the pod (or must complete them with an error or exception) until it can be certain that it is in-sync and communicating with all other storage systems that are in-sync. A first storage system may conclude that a second paired storage system should be detached, which will allow the first storage system to continue since it is now in-sync with all storage systems now in the list. But, the second storage system must be prevented from concluding, alternatively, that the first storage system should be detached and with the second storage system continuing operation. This would result in a "split brain" condition that can lead to irreconcilable datasets, dataset corruption, or application corruption, among other dangers.

[0263] The situation of needing to determine how to proceed when not communicating with paired storage systems can arise while a storage system is running normally and then notices lost communications, while it is currently recovering from some previous fault, while it is rebooting or

resuming from a temporary power loss or recovered communication outage, while it is switching operations from one set of storage system controller to another set for whatever reason, or during or after any combination of these or other kinds of events. In fact, any time a storage system that is associated with a pod can't communicate with all known non-detached members, the storage system can either wait briefly until communications can be established, go offline and continue waiting, or it can determine through some means that it is safe to detach the non-communicating storage system without risk of incurring a split brain due to the non-communicating storage system concluding the alternative view, and then continue. If a safe detach can happen quickly enough, the storage system can remain online for the pod with little more than a short delay and with no resulting application outages for applications that can issue requests to the remaining online storage systems.

[0264] One example of this situation is when a storage system may know that it is out-of-date. That can happen, for example, when a first storage system is first added to a pod that is already associated with one or more storage systems, or when a first storage system reconnects to another storage system and finds that the other storage system had already marked the first storage system as detached. In this case, this first storage system will simply wait until it connects to some other set of storage systems that are in-sync for the pod.

[0265] This model demands some degree of consideration for how storage systems are added to or removed from pods or from the in-sync pod members list. Since each storage system will have its own copy of the list, and since two independent storage systems can't update their local copy at exactly the same time, and since the local copy is all that is available on a reboot or in various fault scenarios, care must be taken to ensure that transient inconsistencies don't cause problems. For example, if one storage systems is in-sync for a pod and a second storage system is added, then if the second storage system is updated to list both storage systems as in-sync first, then if there is a fault and a restart of both storage systems, the second might startup and wait to connect to the first storage system while the first might be unaware that it should or could wait for the second storage system. If the second storage system then responds to an inability to connect with the first storage system by going through a process to detach it, then it might succeed in completing a process that the first storage system is unaware of, resulting in a split brain. As such, it may be necessary to ensure that storage systems won't disagree inappropriately on whether they might opt to go through a detach process if they aren't communicating.

[0266] One way to ensure that storage systems won't disagree inappropriately on whether they might opt to go through a detach process if they aren't communicating is to ensure that when adding a new storage system to the in-sync member list for a pod, the new storage system first stores that it is a detached member (and perhaps that it is being added as an in-sync member). Then, the existing in-sync storage systems can locally store that the new storage system is an in-sync pod member before the new storage system locally stores that same fact. If there is a set of reboots or network outages prior to the new storage system storing its in-sync status, then the original storage systems may detach the new storage system due to non-communication, but the new storage system will wait. A reverse version of this change might be needed for removing a communicating storage

system from a pod: first the storage system being removed stores that it is no longer in-sync, then the storage systems that will remain store that the storage system being removed is no longer in-sync, then all storage systems delete the storage system being removed from their pod membership lists. Depending on the implementation, an intermediate persisted detached state may not be necessary. Whether or not care is required in local copies of membership lists may depend on the model storage systems use for monitoring each other or for validating their membership. If a consensus model is used for both, or if an external system (or an external distributed or clustered system) is used to store and validate pod membership, then inconsistencies in locally stored membership lists may not matter.

[0267] When communications fail or one or several storage systems in a pod fail, or when a storage system starts up (or fails over to a secondary controller) and can't communicate with paired storage systems for a pod, and it is time for one or more storage systems to decide to detach one or more paired storage systems, some algorithm or mechanism must be employed to decide that it is safe to do so and to follow through on the detach. One means of resolving detaches is use a majority (or quorum) model for membership. With three storage systems, as long as two are communicating, they can agree to detach a third storage system that isn't communicating, but that third storage system cannot by itself choose to detach either of the other two. Confusion can arise when storage system communication is inconsistent. For example, storage system A might be communicating with storage system B but not C, while storage system B might be communicating with both A and C. So, A and B could detach C, or B and C could detach A, but more communication between pod members may be needed to Fig. this out.

[0268] Care needs to be taken in a quorum membership model when adding and removing storage systems. For example, if a fourth storage system is added, then a "majority" of storage systems is at that point three. The transition from three storage systems (with two required for majority) to a pod including a fourth storage system (with three required for majority) may require something similar to the model described previously for carefully adding a storage system to the in-sync list. For example, the fourth storage system might start in an attaching state but not yet attached where it would never instigate a vote over quorum. Once in that state, the original three pod members could each be updated to be aware of the fourth member and the new requirement for a three storage system majority to detach a fourth. Removing a storage system from a pod might similarly move that storage system to a locally stored "detaching" state before updating other pod members. A variant scheme for this is to use a distributed consensus mechanism such as PAXOS or RAFT to implement any membership changes or to process detach requests.

[0269] Another means of managing membership transitions is to use an external system that is outside of the storage systems themselves to handle pod membership. In order to become online for a pod, a storage system must first contact the external pod membership system to verify that it is in-sync for the pod. Any storage system that is online for a pod should then remain in communication with the pod membership system and should wait or go offline if it loses communication. An external pod membership manager could be implemented as a highly available cluster using

various cluster tools, such as Oracle RAC, Linux HA, VERITAS Cluster Server, IBM's HACMP, or others. An external pod membership manager could also use distributed configuration tools such as Etcd or Zookeeper, or a reliable distributed database such as Amazon's DynamoDB.

[0270] In the example depicted in FIG. 4, the depicted storage systems (402, 404, 406) may receive a request to read a portion of the dataset (426, 428) and process the request to read the portion of the dataset locally according to some embodiments of the present disclosure. Readers will appreciate that although requests to modify (e.g., a write operation) the dataset (426, 428) require coordination between the storage systems (402, 404, 406) in a pod, as the dataset (426, 428) should be consistent across all storage systems (402, 404, 406) in a pod, responding to a request to read a portion of the dataset (426, 428) does not require similar coordination between the storage systems (402, 404, 406). As such, a particular storage system that receives a read request may service the read request locally by reading a portion of the dataset (426, 428) that is stored within the storage system's storage devices, with no synchronous communication with other storage systems in the pod. Read requests received by one storage system for a replicated dataset in a replicated cluster are expected to avoid any communication in the vast majority of cases, at least when received by a storage system that is running within a cluster that is also running nominally. Such reads should normally be processed simply by reading from the local copy of a clustered dataset with no further interaction required with other storage systems in the cluster.

[0271] Readers will appreciate that the storage systems may take steps to ensure read consistency such that a read request will return the same result regardless of which storage system processes the read request. For example, the resulting clustered dataset content for any set of updates received by any set of storage systems in the cluster should be consistent across the cluster, at least at any time updates are idle (all previous modifying operations have been indicated as complete and no new update requests have been received and processed in any way). More specifically, the instances of a clustered dataset across a set of storage systems can differ only as a result of updates that have not yet completed. This means, for example, that any two write requests which overlap in their volume block range, or any combination of a write request and an overlapping snapshot, compare-and-write, or virtual block range copy, must yield a consistent result on all copies of the dataset. Two operations should not yield a result as if they happened in one order on one storage system and a different order on another storage system in the replicated cluster.

[0272] Furthermore, read requests can be made time order consistent. For example, if one read request is received on a replicated cluster and completed and that read is then followed by another read request to an overlapping address range which is received by the replicated cluster and where one or both reads in any way overlap in time and volume address range with a modification request received by the replicated cluster (whether any of the reads or the modification are received by the same storage system or a different storage system in the replicated cluster), then if the first read reflects the result of the update then the second read should also reflect the results of that update, rather than possibly returning data that preceded the update. If the first read does not reflect the update, then the second read can either reflect

the update or not. This ensures that between two read requests “time” for a data segment cannot roll backward.

[0273] In the example depicted in FIG. 4, the depicted storage systems (402, 404, 406) may also detect a disruption in data communications with one or more of the other storage systems and determine whether to the particular storage system should remain in the pod. A disruption in data communications with one or more of the other storage systems may occur for a variety of reasons. For example, a disruption in data communications with one or more of the other storage systems may occur because one of the storage systems has failed, because a network interconnect has failed, or for some other reason. An important aspect of synchronous replicated clustering is ensuring that any fault handling doesn’t result in unrecoverable inconsistencies, or any inconsistency in responses. For example, if a network fails between two storage systems, at most one of the storage systems can continue processing newly incoming I/O requests for a pod. And, if one storage system continues processing, the other storage system can’t process any new requests to completion, including read requests.

[0274] In the example depicted in FIG. 4, the depicted storage systems (402, 404, 406) may also determine whether the particular storage system should remain in the pod in response to detecting a disruption in data communications with one or more of the other storage systems. As mentioned above, to be ‘online’ as part of a pod, a storage system must consider itself to be in-sync for the pod and must be communicating with all other storage systems it considers to be in-sync for the pod. If a storage system can’t be certain that it is in-sync and communicating with all other storage systems that are in-sync, then it may stop processing new incoming requests to access the dataset (426, 428). As such, the storage system may determine whether to the particular storage system should remain online as part of the pod, for example, by determining whether it can communicate with all other storage systems it considers to be in-sync for the pod (e.g., via one or more test messages), by determining whether the all other storage systems it considers to be in-sync for the pod also consider the storage system to be attached to the pod, through a combination of both steps where the particular storage system must confirm that it can communicate with all other storage systems it considers to be in-sync for the pod and that all other storage systems it considers to be in-sync for the pod also consider the storage system to be attached to the pod, or through some other mechanism.

[0275] In the example depicted in FIG. 4, the depicted storage systems (402, 404, 406) may also keep the dataset on the particular storage system accessible for management and dataset operations in response to determining that the particular storage system should remain in the pod. The storage system may keep the dataset (426, 428) on the particular storage system accessible for management and dataset operations, for example, by accepting requests to access the version of the dataset (426, 428) that is stored on the storage system and processing such requests, by accepting and processing management operations associated with the dataset (426, 428) that are issued by a host or authorized administrator, by accepting and processing management operations associated with the dataset (426, 428) that are issued by one of the other storage systems, or in some other way.

[0276] In the example depicted in FIG. 4, the depicted storage systems (402, 404, 406) may, however, make the dataset on the particular storage system inaccessible for management and dataset operations in response to determining that the particular storage system should not remain in the pod. The storage system may make the dataset (426, 428) on the particular storage system inaccessible for management and dataset operations, for example, by rejecting requests to access the version of the dataset (426, 428) that is stored on the storage system, by rejecting management operations associated with the dataset (426, 428) that are issued by a host or other authorized administrator, by rejecting management operations associated with the dataset (426, 428) that are issued by one of the other storage systems in the pod, or in some other way.

[0277] In the example depicted in FIG. 4, the depicted storage systems (402, 404, 406) may also detect that the disruption in data communications with one or more of the other storage systems has been repaired and make the dataset on the particular storage system accessible for management and dataset operations. The storage system may detect that the disruption in data communications with one or more of the other storage systems has been repaired, for example, by receiving a message from the one or more of the other storage systems. In response to detecting that the disruption in data communications with one or more of the other storage systems has been repaired, the storage system may make the dataset (426, 428) on the particular storage system accessible for management and dataset operations once the previously detached storage system has been resynchronized with the storage systems that remained attached to the pod.

[0278] In the example depicted in FIG. 4, the depicted storage systems (402, 404, 406) may also go offline from the pod such that the particular storage system no longer allows management and dataset operations. The depicted storage systems (402, 404, 406) may go offline from the pod such that the particular storage system no longer allows management and dataset operations for a variety of reasons. For example, the depicted storage systems (402, 404, 406) may also go offline from the pod due to some fault with the storage system itself, because an update or some other maintenance is occurring on the storage system, due to communications faults, or for many other reasons. In such an example, the depicted storage systems (402, 404, 406) may subsequently update the dataset on the particular storage system to include all updates to the dataset since the particular storage system went offline and go back online with the pod such that the particular storage system allows management and dataset operations, as will be described in greater detail in the resynchronization sections included below.

[0279] In the example depicted in FIG. 4, the depicted storage systems (402, 404, 406) may also identifying a target storage system for asynchronously receiving the dataset, where the target storage system is not one of the plurality of storage systems across which the dataset is synchronously replicated. Such a target storage system may represent, for example, a backup storage system, as some storage system that makes use of the synchronously replicated dataset, and so on. In fact, synchronous replication can be leveraged to distribute copies of a dataset closer to some rack of servers, for better local read performance. One such case is smaller top-of-rack storage systems symmetrically replicated to

larger storage systems that are centrally located in the data center or campus and where those larger storage systems are more carefully managed for reliability or are connected to external networks for asynchronous replication or backup services.

[0280] In the example depicted in FIG. 4, the depicted storage systems (402, 404, 406) may also identify a portion of the dataset that is not being asynchronously replicated to the target storage system by any of the other storage systems and asynchronously replicate, to the target storage system, the portion of the dataset that is not being asynchronously replicated to the target storage system by any of the other storage systems, wherein the two or more storage systems collectively replicate the entire dataset to the target storage system. In such a way, the work associated with asynchronously replicating a particular dataset may be split amongst the members of a pod, such that each storage system in a pod is only responsible for asynchronously replicating a subset of a dataset to the target storage system.

[0281] In the example depicted in FIG. 4, the depicted storage systems (402, 404, 406) may also detach from the pod, such that the particular storage system that detaches from the pod is no longer included in the set of storage systems across which the dataset is synchronously replicated. For example, if storage system (404) in FIG. 4 detached from the pod (430) illustrated in FIG. 4, the pod (430) would only include storage systems (402, 406) as the storage systems across which the dataset (426) that is included in the pod (430) would be synchronously replicated across. In such an example, detaching the storage system from the pod could also include removing the dataset from the particular storage system that detached from the pod. Continuing with the example where the storage system (404) in FIG. 4 detached from the pod (430) illustrated in FIG. 4, the dataset (426) that is included in the pod (430) could be deleted or otherwise removed from the storage system (404).

[0282] Readers will appreciate that there are a number of unique administrative capabilities enabled by the pod model that can further be supported. Also, the pod model itself introduces some issues that can be addressed by an implementation. For example, when a storage system is offline for a pod, but is otherwise running, such as because an interconnect failed and another storage system for the pod won out in mediation, there may still be a desire or need to access the offline pod's dataset on the offline storage system. One solution may be simply to enable the pod in some detached mode and allow the dataset to be accessed. However, that solution can be dangerous and that solution can cause the pod's metadata and data to be much more difficult to reconcile when the storage systems do regain communication. Furthermore, there could still be a separate path for hosts to access the offline storage system as well as the still online storage systems. In that case, a host might issue I/O to both storage systems even though they are no longer being kept in sync, because the host sees target ports reporting volumes with the same identifiers and the host I/O drivers presume it sees additional paths to the same volume. This can result in fairly damaging data corruption as reads and writes issued to both storage systems are no longer consistent even though the host presumes they are. As a variant of this case, in a clustered application, such as a shared storage clustered database, the clustered application running on one host might be reading or writing to one storage system and the same clustered application running on another host

might be reading or writing to the "detached" storage system, yet the two instances of the clustered application are communicating between each other on the presumption that the dataset they each see is entirely consistent for completed writes. Since they aren't consistent, that presumption is violated and the application's dataset (e.g., the database) can quickly end up being corrupted.

[0283] One way to solve both of these problems is to allow for an offline pod, or perhaps a snapshot of an offline pod, to be copied to a new pod with new volumes that have sufficiently new identities that host I/O drivers and clustered applications won't confuse the copied volumes as being the same as the still online volumes on another storage system. Since each pod maintains a complete copy of the dataset, which is crash consistent but perhaps slightly different from the copy of the pod dataset on another storage system, and since each pod has an independent copy of all data and metadata needed to operate on the pod content, it is a straightforward problem to make a virtual copy of some or all volumes or snapshots in the pod to new volumes in a new pod. In a logical extent graph implementation, for example, all that is needed is to define new volumes in a new pod which reference logical extent graphs from the copied pod associated with the pod's volumes or snapshots, and with the logical extent graphs being marked as copy on write. The new volumes should be treated as new volumes, similarly to how volume snapshots copied to a new volume might be implemented. Volumes may have the same administrative name, though within a new pod namespace. But, they should have different underlying identifiers, and differing logical unit identifiers from the original volumes.

[0284] In some cases it may be possible to use virtual network isolation techniques (for example, by creating a virtual LAN in the case of IP networks or a virtual SAN in the case of fiber channel networks) in such a way that isolation of volumes presented to some interfaces can be assured to be inaccessible from host network interfaces or host SCSI initiator ports that might also see the original volumes. In such cases, it may be safe to provide the copies of volumes with the same SCSI or other storage identifiers as the original volumes. This could be used, for example, in cases where the applications expect to see a particular set of storage identifiers in order to function without an undue burden in reconfiguration.

[0285] Some of the techniques described herein could also be used outside of an active fault context to test readiness for handling faults. Readiness testing (sometimes referred to as "fire drills") is commonly required for disaster recovery configurations, where frequent and repeated testing is considered a necessity to ensure that most or all aspects of a disaster recovery plan are correct and account for any recent changes to applications, datasets, or changes in equipment. Readiness testing should be non-disruptive to current production operations, including replication. In many cases the real operations can't actually be invoked on the active configuration, but a good way to get close is to use storage operations to make copies of production datasets, and then perhaps couple that with the use of virtual networking, to create an isolated environment containing all data that is believed necessary for the important applications that must be brought up successfully in cases of disasters. Making such a copy of a synchronously replicated (or even an asynchronously replicated) dataset available within a site (or collection of sites) that is expected to perform a disaster

recovery readiness test procedure and then starting the important applications on that dataset to ensure that it can startup and function is a great tool, since it helps ensure that no important parts of the application datasets were left out in the disaster recovery plan. If necessary, and practical, this could be coupled with virtual isolated networks coupled perhaps with isolated collection of physical or virtual machines, to get as close as possible to a real world disaster recovery takeover scenario. Virtually copying a pod (or set of pods) to another pod as a point-in-time image of the pod datasets immediately creates an isolated dataset that contains all the copied elements and that can then be operated on essentially identically to the originally pods, as well as allowing isolation to a single site (or a few sites) separately from the original pod. Further, these are fast operations and they can be torn down and repeated easily allowing testing to repeated as often as is desired.

[0286] Some enhancements could be made to get further toward perfect disaster recovery testing. For example, in conjunction with isolated networks, SCSI logical unit identities or other types of identities could be copied into the target pod so that the test servers, virtual machines, and applications see the same identities. Further, the administrative environment of the servers could be configured to respond to requests from a particular virtual set of virtual networks to respond to requests and operations on the original pod name so scripts don't require use of test-variants with alternate "test" versions of object names. A further enhancement can be used in cases where the host-side server infrastructure that will take over in the case of a disaster takeover can be used during a test. This includes cases where a disaster recovery data center is completely stocked with alternative server infrastructure that won't generally be used until directed to do so by a disaster. It also includes cases where that infrastructure might be used for non-critical operations (for example, running analytics on production data, or simply supporting application development or other functions which may be important but can be halted if needed for more critical functions). Specifically, host definitions and configurations and the server infrastructure that will use them can be set up as they will be for an actual disaster recovery takeover event and tested as part of disaster recovery takeover testing, with the tested volumes being connected to these host definitions from the virtual pod copy used to provide a snapshot of the dataset. From the standpoint of the storage systems involved, then, these host definitions and configurations used for testing, and the volume-to-host connection configurations used during testing, can be reused when an actual disaster takeover event is triggered, greatly minimizing the configuration differences between the test configuration and the real configuration that will be used in case of a disaster recovery takeover.

[0287] In some cases it may make sense to move volumes out of a first pod and into a new second pod including just those volumes. The pod membership and high availability and recovery characteristics can then be adjusted separately, and administration of the two resulting pod datasets can then be isolated from each other. An operation that can be done in one direction should also be possible in the other direction. At some point, it may make sense to take two pods and merge them into one so that the volumes in each of the original two pods will now track each other for storage system membership and high availability and recovery characteristics and events. Both operations can be accomplished

safely and with reasonably minimal or no disruption to running applications by relying on the characteristics suggested for changing mediation or quorum properties for a pod which were discussed in an earlier section. With mediation, for example, a mediator for a pod can be changed using a sequence consisting of a step where each storage system in a pod is changed to depend on both a first mediator and a second mediator and each is then changed to depend only on the second mediator. If a fault occurs in the middle of the sequence, some storage systems may depend on both the first mediator and the second mediator, but in no case will recovery and fault handling result in some storage systems depending only on the first mediator and other storage systems only depending on the second mediator. Quorum can be handled similarly by temporarily depending on winning against both a first quorum model and a second quorum model in order to proceed to recovery. This may result in a very short time period where availability of the pod in the face of faults depend on additional resources, thus reducing potential availability, but this time period is very short and the reduction in availability is often very little. With mediation, if the change in mediator parameters is nothing more than the change in the key used for mediation and the mediation service used is the same, then the potential reduction in availability is even less, since it now depends only on two calls to the same service versus one call to that service, and rather than separate calls to two separate services.

[0288] Readers will note that changing the quorum model may be quite complex. An additional step may be necessary where storage systems will participate in the second quorum model but won't depend on winning in that second quorum model, which is then followed by the step of also depending on the second quorum model. This may be necessary to account for the fact that if only one system has processed the change to depend on the quorum model, then it will never win quorum since there will never be a majority. With this model in place for changing the high availability parameters (mediation relationship, quorum model, takeover preferences), we can create a safe procedure for these operations to split a pod into two or to join two pods into one. This may require adding one other capability: linking a second pod to a first pod for high availability such that if two pods include compatible high availability parameters the second pod linked to the first pod can depend on the first pod for determining and instigating detach-related processing and operations, offline and in-sync states, and recovery and resynchronization actions.

[0289] To split a pod into two, which is an operation to move some volumes into a newly created pod, a distributed operation may be formed that can be described as: form a second pod into which we will move a set of volumes which were previously in a first pod, copy the high availability parameters from the first pod into the second pod to ensure they are compatible for linking, and link the second pod to the first pod for high availability. This operation may be encoded as messages and should be implemented by each storage system in the pod in such a way that the storage system ensures that the operation happens completely on that storage system or does not happen at all if processing is interrupted by a fault. Once all in-sync storage systems for the two pods have processed this operation, the storage systems can then process a subsequent operation which changes the second pod so that it is no longer linked to the

first pod. As with other changes to high availability characteristics for a pod, this involves first having each in-sync storage system change to rely on both the previous model (that model being that high availability is linked to the first pod) and the new model (that model being its own now independent high availability). In the case of mediation or quorum, this means that storage systems which processed this change will first depend on mediation or quorum being achieved as appropriate for the first pod and will additionally depend on a new separate mediation (for example, a new mediation key) or quorum being achieved for the second pod before the second pod can proceed following a fault that required mediation or testing for quorum. As with the previous description of changing quorum models, an intermediate step may set storage systems to participate in quorum for the second pod before the step where storage systems participate in and depend on quorum for the second pod. Once all in-sync storage systems have processed the change to depend on the new parameters for mediation or quorum for both the first pod and the second pod, the split is complete.

[0290] Joining a second pod into a first pod operates essentially in reverse. First, the second pod must be adjusted to be compatible with the first pod, by having an identical list of storage systems and by having a compatible high availability model. This may involve some set of steps such as those described elsewhere in this paper to add or remove storage systems or to change mediator and quorum models. Depending on implementation, it may be necessary only to reach an identical list of storage systems. Joining proceeds by processing an operation on each in-sync storage system to link the second pod to the first pod for high availability. Each storage system which processes that operation will then depend on the first pod for high availability and then the second pod for high availability. Once all in-sync storage systems for the second pod have processed that operation, the storage systems will then each process a subsequent operation to eliminate the link between the second pod and the first pod, migrate the volumes from the second pod into the first pod, and delete the second pod. Host or application dataset access can be preserved throughout these operations, as long as the implementation allows proper direction of host or application dataset modification or read operations to the volume by identity and as long as the identity is preserved as appropriate to the storage protocol or storage model (for example, as long as logical unit identifiers for volumes and use of target ports for accessing volumes are preserved in the case of SCSI).

[0291] Migrating a volume between pods may present issues. If the pods have an identical set of in-sync membership storage systems, then it may be straightforward: temporarily suspend operations on the volumes being migrated, switch control over operations on those volumes to controlling software and structures for the new pod, and then resume operations. This allows for a seamless migration with continuous uptime for applications apart from the very brief operation suspension, provided network and ports migrate properly between pods. Depending on the implementation, suspending operations may not even be necessary, or may be so internal to the system that the suspension of operations has no impact. Copying volumes between pods with different in-sync membership sets is more of a problem. If the target pod for the copy has a subset of in-sync members from the source pod, this isn't much of a problem:

a member storage system can be dropped safely enough without having to do more work. But, if the target pod adds in-sync member storage systems to the volume over the source pod, then the added storage systems must be synchronized to include the volume's content before they can be used. Until synchronized, this leaves the copied volumes distinctly different from the already synchronized volumes, in that fault handling differs and request handling from the not yet synced member storage systems either won't work or must be forwarded or won't be as fast because reads will have to traverse an interconnect. Also, the internal implementation will have to handle some volumes being in sync and ready for fault handling and others not being in sync.

[0292] There are other problems relating to reliability of the operation in the face of faults. Coordinating a migration of volumes between multi-storage-system pods is a distributed operation. If pods are the unit of fault handling and recovery, and if mediation or quorum or whatever means are used to avoid split-brain situations, then a switch in volumes from one pod with a particular set of state and configurations and relationships for fault handling, recovery, mediation and quorum to another then storage systems in a pod have to be careful about coordinating changes related to that handling for any volumes. Operations can't be atomically distributed between storage systems, but must be staged in some way. Mediation and quorum models essentially provide pods with the tools for implementing distributed transactional atomicity, but this may not extend to inter-pod operations without adding to the implementation.

[0293] Consider even a simple migration of a volume from a first pod to a second pod even for two pods that share the same first and second storage systems. At some point the storage systems will coordinate to define that the volume is now in the second pod and is no longer in the first pod. If there is no inherent mechanism for transactional atomicity across the storage systems for the two pods, then a naive implementation could leave the volume in the first pod on the first storage system and the second pod on the second storage system at the time of a network fault that results in fault handling to detach storage systems from the two pods. If pods separately determine which storage system succeeds in detaching the other, then the result could be that the same storage system detaches the other storage system for both pods, in which case the result of the volume migration recovery should be consistent, or it could result in a different storage system detaching the other for the two pods. If the first storage system detaches the second storage system for the first pod and the second storage system detaches the first storage system for the second pod, then recovery might result in the volume being recovered to the first pod on the first storage system and into the second pod on the second storage system, with the volume then running and exported to hosts and storage applications on both storage systems. If instead the second storage system detaches the first storage system for the first pod and first storage detaches the second storage system for the second pod, then recovery might result in the volume being discarded from the second pod by the first storage system and the volume being discarded from the first pod by the second storage system, resulting in the volume disappearing entirely. If the pods a volume is being migrated between are on differing sets of storage systems, then things can get even more complicated.

[0294] A solution to these problems may be to use an intermediate pod along with the techniques described pre-

viously for splitting and joining pods. This intermediate pod may never be presented as visible managed objects associated with the storage systems. In this model, volumes to be moved from a first pod to a second pod are first split from the first pod into a new intermediate pod using the split operation described previously. The storage system members for the intermediate pod can then be adjusted to match the membership of storage systems by adding or removing storage systems from the pod as necessary. Subsequently, the intermediate pod can be joined with the second pod.

[0295] For further explanation, FIG. 5 sets forth a flow chart illustrating steps that may be performed by storage systems (402, 404, 406) that support a pod according to some embodiments of the present disclosure. Although depicted in less detail, the storage systems (402, 404, 406) depicted in FIG. 5 may be similar to the storage systems described above with reference to FIGS. 1A-1D, FIGS. 2A-2G, FIGS. 3A-3B, FIG. 4, or any combination thereof. In fact, the storage systems (402, 404, 406) depicted in FIG. 5 may include the same, fewer, additional components as the storage systems described above.

[0296] In the example method depicted in FIG. 5, a storage system (402) may attach (508) to a pod. The model for pod membership may include a list of storage systems and a subset of that list where storage systems are presumed to be in-sync for the pod. A storage system is in-sync for a pod if it is at least within a recovery of having identical idle content for the last written copy of the dataset associated with the pod. Idle content is the content after any in-progress modifications have completed with no processing of new modifications. Sometimes this is referred to as “crash recoverable” consistency. Storage systems that are listed as pod members but that are not listed as in-sync for the pod can be described as “detached” from the pod. Storage systems that are listed as pod members, are in-sync for the pod, and are currently available for actively serving data for the pod are “online” for the pod.

[0297] In the example method depicted in FIG. 5, the storage system (402) may attach (508) to a pod, for example, by synchronizing its locally stored version of the dataset (426) along with an up-to-date version of the dataset (426) that is stored on other storage systems (404, 406) in the pod that are online, as the term is described above. In such an example, in order for the storage system (402) to attach (508) to the pod, a pod definition stored locally within each of the storage systems (402, 404, 406) in the pod may need to be updated in order for the storage system (402) to attach (508) to the pod. In such an example, each storage system member of a pod may have its own copy of the membership, including which storage systems it last knew were in-sync, and which storage systems it last knew comprised the entire set of pod members.

[0298] In the example method depicted in FIG. 5, the storage system (402) may also receive (510) a request to read a portion of the dataset (426) and the storage system (402) may process (512) the request to read the portion of the dataset (426) locally. Readers will appreciate that although requests to modify (e.g., a write operation) the dataset (426) require coordination between the storage systems (402, 404, 406) in a pod, as the dataset (426) should be consistent across all storage systems (402, 404, 406) in a pod, responding to a request to read a portion of the dataset (426) does not require similar coordination between the storage systems (402, 404, 406). As such, a particular

storage system (402) that receives a read request may service the read request locally by reading a portion of the dataset (426) that is stored within the storage system's (402) storage devices, with no synchronous communication with other storage systems (404, 406) in the pod. Read requests received by one storage system for a replicated dataset in a replicated cluster are expected to avoid any communication in the vast majority of cases, at least when received by a storage system that is running within a cluster that is also running nominally. Such reads should normally be processed simply by reading from the local copy of a clustered dataset with no further interaction required with other storage systems in the cluster.

[0299] Readers will appreciate that the storage systems may take steps to ensure read consistency such that a read request will return the same result regardless of which storage system processes the read request. For example, the resulting clustered dataset content for any set of updates received by any set of storage systems in the cluster should be consistent across the cluster, at least at any time updates are idle (all previous modifying operations have been indicated as complete and no new update requests have been received and processed in any way). More specifically, the instances of a clustered dataset across a set of storage systems can differ only as a result of updates that have not yet completed. This means, for example, that any two write requests which overlap in their volume block range, or any combination of a write request and an overlapping snapshot, compare-and-write, or virtual block range copy, must yield a consistent result on all copies of the dataset. Two operations cannot yield a result as if they happened in one order on one storage system and a different order on another storage system in the replicated cluster.

[0300] Furthermore, read requests may be time order consistent. For example, if one read request is received on a replicated cluster and completed and that read is then followed by another read request to an overlapping address range which is received by the replicated cluster and where one or both reads in any way overlap in time and volume address range with a modification request received by the replicated cluster (whether any of the reads or the modification are received by the same storage system or a different storage system in the replicated cluster), then if the first read reflects the result of the update then the second read should also reflect the results of that update, rather than possibly returning data that preceded the update. If the first read does not reflect the update, then the second read can either reflect the update or not. This ensures that between two read requests “time” for a data segment cannot roll backward.

[0301] In the example method depicted in FIG. 5, the storage system (402) may also detect (514) a disruption in data communications with one or more of the other storage systems (404, 406). A disruption in data communications with one or more of the other storage systems (404, 406) may occur for a variety of reasons. For example, a disruption in data communications with one or more of the other storage systems (404, 406) may occur because one of the storage systems (402, 404, 406) has failed, because a network interconnect has failed, or for some other reason. An important aspect of synchronous replicated clustering is ensuring that any fault handling doesn't result in unrecoverable inconsistencies, or any inconsistency in responses. For example, if a network fails between two storage systems, at most one of the storage systems can continue

processing newly incoming I/O requests for a pod. And, if one storage system continues processing, the other storage system can't process any new requests to completion, including read requests.

[0302] In the example method depicted in FIG. 5, the storage system (402) may also determine (516) whether to the particular storage system (402) should remain online as part of the pod. As mentioned above, to be 'online' as part of a pod, a storage system must consider itself to be in-sync for the pod and must be communicating with all other storage systems it considers to be in-sync for the pod. If a storage system can't be certain that it is in-sync and communicating with all other storage systems that are in-sync, then it may stop processing new incoming requests to access the dataset (426). As such, the storage system (402) may determine (516) whether to the particular storage system (402) should remain online as part of the pod, for example, by determining whether it can communicate with all other storage systems (404, 406) it considers to be in-sync for the pod (e.g., via one or more test messages), by determining whether the all other storage systems (404, 406) it considers to be in-sync for the pod also consider the storage system (402) to be attached to the pod, through a combination of both steps where the particular storage system (402) must confirm that it can communicate with all other storage systems (404, 406) it considers to be in-sync for the pod and that all other storage systems (404, 406) it considers to be in-sync for the pod also consider the storage system (402) to be attached to the pod, or through some other mechanism.

[0303] In the example method depicted in FIG. 5, the storage system (402) may also, responsive to affirmatively (518) determining that the particular storage system (402) should remain online as part of the pod, keep (522) the dataset (426) on the particular storage system (402) accessible for management and dataset operations. The storage system (402) may keep (522) the dataset (426) on the particular storage system (402) accessible for management and dataset operations, for example, by accepting requests to access the version of the dataset (426) that is stored on the storage system (402) and processing such requests, by accepting and processing management operations associated with the dataset (426) that are issued by a host or authorized administrator, by accepting and processing management operations associated with the dataset (426) that are issued by one of the other storage systems (404, 406) in the pod, or in some other way.

[0304] In the example method depicted in FIG. 5, the storage system (402) may also, responsive to determining that the particular storage system should not (520) remain online as part of the pod, make (524) the dataset (426) on the particular storage system (402) inaccessible for management and dataset operations. The storage system (402) may make (524) the dataset (426) on the particular storage system (402) inaccessible for management and dataset operations, for example, by rejecting requests to access the version of the dataset (426) that is stored on the storage system (402), by rejecting management operations associated with the dataset (426) that are issued by a host or other authorized administrator, by rejecting management operations associated with the dataset (426) that are issued by one of the other storage systems (404, 406) in the pod, or in some other way.

[0305] In the example method depicted in FIG. 5, the storage system (402) may also detect (526) that the disruption in data communications with one or more of the other

storage systems (404, 406) has been repaired. The storage system (402) may detect (526) that the disruption in data communications with one or more of the other storage systems (404, 406) has been repaired, for example, by receiving a message from the one or more of the other storage systems (404, 406). In response to detecting (526) that the disruption in data communications with one or more of the other storage systems (404, 406) has been repaired, the storage system (402) may make (528) the dataset (426) on the particular storage system (402) accessible for management and dataset operations.

[0306] Readers will appreciate that the example depicted in FIG. 5 describes an embodiment in which various actions are depicted as occurring within some order, although no ordering is required. Furthermore, other embodiments may exist where the storage system (402) only carries out a subset of the described actions. For example, the storage system (402) may perform the steps of detecting (514) a disruption in data communications with one or more of the other storage systems (404, 406), determining (516) whether to the particular storage system (402) should remain in the pod, keeping (522) the dataset (426) on the particular storage system (402) accessible for management and dataset operations or making (524) the dataset (426) on the particular storage system (402) inaccessible for management and dataset operations without first receiving (510) a request to read a portion of the dataset (426) and processing (512) the request to read the portion of the dataset (426) locally. Furthermore, the storage system (402) may detect (526) that the disruption in data communications with one or more of the other storage systems (404, 406) has been repaired and make (528) the dataset (426) on the particular storage system (402) accessible for management and dataset operations without first receiving (510) a request to read a portion of the dataset (426) and processing (512) the request to read the portion of the dataset (426) locally. In fact, none of the steps described herein are explicitly required in all embodiments as prerequisites for performing other steps described herein.

[0307] For further explanation, FIG. 6 sets forth a flow chart illustrating steps that may be performed by storage systems (402, 404, 406) that support a pod according to some embodiments of the present disclosure. Although depicted in less detail, the storage systems (402, 404, 406) depicted in FIG. 6 may be similar to the storage systems described above with reference to FIGS. 1A-1D, FIGS. 2A-2G, FIGS. 3A-3B, FIG. 4, or any combination thereof. In fact, the storage systems (402, 404, 406) depicted in FIG. 6 may include the same, fewer, additional components as the storage systems described above.

[0308] In the example method depicted in FIG. 6, two or more of the storage systems (402, 404) may each identify (608) a target storage system (618) for asynchronously receiving the dataset (426). The target storage system (618) for asynchronously receiving the dataset (426) may be embodied, for example, as a backup storage system that is located in a different data center than either of the storage systems (402, 404) that are members of a particular pod, as cloud storage that is provided by a cloud services provider, or in many other ways. Readers will appreciate that the target storage system (618) is not one of the plurality of storage systems (402, 404) across which the dataset (426) is

synchronously replicated, and as such, the target storage system (618) initially does not include an up-to-date local copy of the dataset (426).

[0309] In the example method depicted in FIG. 6, two or more of the storage systems (402, 404) may each also identify (610) a portion of the dataset (426) that is not being asynchronously replicated to the target storage (618) system by any of the other storage systems that are members of a pod that includes the dataset (426). In such an example, the storage systems (402, 404) may each asynchronously replicate (612), to the target storage system (618), the portion of the dataset (426) that is not being asynchronously replicated to the target storage system by any of the other storage systems. Consider an example in which a first storage system (402) is responsible for asynchronously replicating a first portion (e.g., a first half of an address space) of the dataset (426) to the target storage system (618). In such an example, the second storage system (404) would be responsible for asynchronously replicating a second portion (e.g., a second half of an address space) of the dataset (426) to the target storage system (618), such that the two or more storage systems (402, 404) collectively replicate the entire dataset (426) to the target storage system (618).

[0310] Readers will appreciate that through the use of pods, as described above, the replication relationship between two storage systems may be switched from a relationship where data is asynchronously replicated to a relationship where data is synchronously replicated. For example, if storage system A is configured to asynchronously replicate a dataset to storage system B, creating a pod that includes the dataset, storage system A as a member, and storage system B as a member can switch the relationship where data is asynchronously replicated to a relationship where data is synchronously replicated. Likewise, through the use of pods, the replication relationship between two storage systems may be switched from a relationship where data is synchronously replicated to a relationship where data is asynchronously replicated. For example, if a pod is created that includes the dataset, storage system A as a member, and storage system B as a member, by merely unstretching the pod (to remove storage system A as a member or to remove storage system B as a member), a relationship where data is synchronously replicated between the storage systems can immediately be switched to a relationship where data is asynchronously replicated. In such a way, storage systems may switch back-and-forth as needed between asynchronous replication and synchronous replication.

[0311] This switching can be facilitated by the implementation relying on similar techniques for both synchronous and asynchronous replication. For example, if resynchronization for a synchronously replicated dataset relies on the same or a compatible mechanism as is used for asynchronous replication, then switching to asynchronous replication is conceptually identical to dropping the in-sync state and leaving a relationship in a state similar to a “perpetual recovery” mode. Likewise, switching from asynchronous replication to synchronous replication can operate conceptually by “catching up” and becoming in-sync just as is done when completing a resynchronization with the switching system becoming an in-sync pod member.

[0312] Alternatively, or additionally, if both synchronous and asynchronous replication rely on similar or identical

common metadata, or a common model for representing and identifying logical extents or stored block identities, or a common model for representing content-addressable stored blocks, then these aspects of commonality can be leveraged to dramatically reduce the content that may need to be transferred when switching to and from synchronous and asynchronous replication. Further, if a dataset is asynchronously replicated from a storage system A to a storage system B, and system B further asynchronously replicates that data set to a storage system C, then a common metadata model, common logical extent or block identities, or common representation of content-addressable stored blocks, can dramatically reduce the data transfers needed to enable synchronous replication between storage system A and storage system C.

[0313] Readers will further appreciate that through the use of pods, as described above, replication techniques may be used to perform tasks other than replicating data. In fact, because a pod may include a set of managed objects, tasks like migrating a virtual machine may be carried out using pods and the replication techniques described herein. For example, if virtual machine A is executing on storage system A, by creating a pod that includes virtual machine A as a managed object, storage system A as a member, and storage system B as a member, virtual machine A and any associated images and definitions may be migrated to storage system B, at which time the pod could simply be destroyed, membership could be updated, or other actions may be taken as necessary.

[0314] For further explanation, FIG. 7 sets forth a flow chart illustrating an example method of establishing a synchronous replication relationship between two or more storage systems (714, 724, 728) according to some embodiments of the present disclosure. Although depicted in less detail, the storage systems (714, 724, 728) depicted in FIG. 7 may be similar to the storage systems described above with reference to FIGS. 1A-1D, FIGS. 2A-2G, FIGS. 3A-3B, or any combination thereof. In fact, the storage systems (714, 724, 728) depicted in FIG. 7 may include the same, fewer, additional components as the storage systems described above.

[0315] The example method depicted in FIG. 7 includes identifying (702), for a dataset (712), a plurality of storage systems (714, 724, 728) across which the dataset (712) will be synchronously replicated. The dataset (712) depicted in FIG. 7 may be embodied, for example, as the contents of a particular volume, as the contents of a particular shard of a volume, or as any other collection of one or more data elements. The dataset (712) may be synchronized across a plurality of storage systems (714, 724, 728) such that each storage system (714, 724, 728) retains a local copy of the dataset (712). In the examples described herein, such a dataset (712) is synchronously replicated across the storage systems (714, 724, 728) in such a way that the dataset (712) can be accessed through any of the storage systems (714, 724, 728) with performance characteristics such that any one storage system in the cluster doesn't operate substantially more optimally than any other storage system in the cluster, at least as long as the cluster and the particular storage system being accessed are running nominally. In such systems, modifications to the dataset (712) should be made to the copy of the dataset that resides on each storage system (714, 724, 728) in such a way that accessing the dataset (712) on any of the storage systems (714, 724, 728) will

yield consistent results. For example, a write request issued to the dataset must be serviced on all storage systems (714, 724, 728) or serviced on none of the storage systems (714, 724, 728). Likewise, some groups of operations (e.g., two write operations that are directed to same location within the dataset) must be executed in the same order on all storage systems (714, 724, 728) such that the copy of the dataset that resides on each storage system (714, 724, 728) is ultimately identical. Modifications to the dataset (712) need not be made at the exact same time, but some actions (e.g., issuing an acknowledgement that a write request directed to the dataset, enabling read access to a location within the dataset that is targeted by a write request that has not yet been completed on all storage systems) may be delayed until the copy of the dataset (712) on each storage system (714, 724, 728) has been modified.

[0316] In the example method depicted in FIG. 7, identifying (702), for a dataset (712), a plurality of storage systems (714, 724, 728) across which the dataset (712) will be synchronously replicated may be carried out, for example, by examining a pod definition or similar data structure that associates a dataset (712) with one or more storage systems (714, 724, 728) which nominally store that dataset (712). A 'pod', as the term is used here and throughout the remainder of the present application, may be embodied as a management entity that represents a dataset, a set of managed objects and management operations, a set of access operations to modify or read the dataset, and a plurality of storage systems. Such management operations may modify or query managed objects equivalently through any of the storage systems, where access operations to read or modify the dataset operate equivalently through any of the storage systems. Each storage system may store a separate copy of the dataset as a proper subset of the datasets stored and advertised for use by the storage system, where operations to modify managed objects or the dataset performed and completed through any one storage system are reflected in subsequent management objects to query the pod or subsequent access operations to read the dataset. Additional details regarding a 'pod' may be found in previously filed provisional patent application No. 62/518,071, which is incorporated herein by reference. In such an example, the pod definition may include at least an identification of a dataset (712) and a set of storage systems (714, 724, 728) across which the dataset (712) is synchronously replicated. Such a pod may encapsulate some of number of (perhaps optional) properties including symmetric access, flexible addition/removal of replicas, high availability data consistency, uniform user administration across storage systems in relationship to the dataset, managed host access, application clustering, and so on. Storage systems can be added to a pod, resulting in the pod's dataset (712) being copied to that storage system and then kept up to date as the dataset (712) is modified. Storage systems can also be removed from a pod, resulting in the dataset (712) being no longer kept up to date on the removed storage system. In such examples, a pod definition or similar data structure may be updated as storage systems are added to and removed from a particular pod.

[0317] The example method depicted in FIG. 7 also includes configuring (704) one or more data communications links (716, 718, 720) between each of the plurality of storage systems (714, 724, 728) to be used for synchronously replicating the dataset (712). In the example method

depicted in FIG. 6, the storage systems (714, 724, 728) in a pod must communicate with each other both for high bandwidth data transfer, and for cluster, status, and administrative communication. These distinct types of communication could be over the same data communications links (716, 718, 720) or, in an alternative embodiment, these distinct types of communication could be over separate data communications links (716, 718, 720). In a cluster of dual controller storage systems, both controllers in each storage system should have the nominal ability to communicate with both controllers for any paired storage systems (i.e., any other storage system in a pod).

[0318] In a primary/secondary controller design, all cluster communication for active replication may run between primary controllers until a fault occurs. In such systems, some communication may occur between a primary controller and a secondary controller, or between secondary controllers on distinct storage systems, in order to verify that the data communications links between such entities are operational. In other cases, virtual network addresses might be used to limit the configuration needed for of inter-datacenter network links, or to simplify design of the clustered aspect of the storage system. In an active/active controller design, cluster communications might run from all active controllers of one storage system to some or all active controllers in any paired storage systems, or they might be filtered through a common switch, or they might use a virtual network address to simplify configuration, or they might use some combination. In a scale-out design, two or more common network switches may be used such that all scale-out storage controllers within the storage system connect to the network switches in order to handle data traffic. The switches might or might not use techniques to limit the number of exposed network addresses, so that paired storage systems don't need to be configured with the network addresses of all storage controllers.

[0319] In the example method depicted in FIG. 7, configuring (704) one or more data communications links (716, 718, 720) between each of the plurality of storage systems (714, 724, 728) to be used for synchronously replicating the dataset (712) may be carried out, for example, by configuring the storage systems (716, 718, 720) to communicate via defined ports over a data communications network, by configuring the storage systems (716, 718, 720) to communicate over a point-to-point data communications link between two of the storage systems (716, 724, 728), or in a variety of ways. If secure communication is required, some form of key exchange may be needed, or communication could be done or bootstrapped through some service such as SSH (Secure Shell), SSL, or some other service or protocol built around public keys or Diffie-Hellman key exchange or reasonable alternatives. Secure communications could also be mediated through some vendor-provided cloud service tied in some way to customer identities. Alternately, a service configured to run on customer facilities, such as running in a virtual machine or container, could be used to mediate key exchanges necessary for secure communications between replicating storage systems (716, 718, 720). Readers will appreciate that a pod including more than two storage systems may need communication links between most or all of the individual storage systems. In the example depicted in FIG. 6, three data communications links (716, 718, 720) are illustrated, although additional data communications links may exist in other embodiments.

[0320] Readers will appreciate that communication between the storage systems (714, 724, 728) across which the dataset (712) will be synchronously replicated serves some number of purposes. One purpose, for example, is to deliver data from one storage system (714, 724, 728) to another storage system (714, 724, 728) as part of I/O processing. For example, processing a write commonly requires delivering the write content and some description of the write to any paired storage systems for a pod. Another purpose served by data communications between the storage systems (714, 724, 728) may be to communicate configuration changes and analytics data in order to handle creating, extending, deleting or renaming volumes, files, object buckets, and so on. Another purpose served by data communications between the storage systems (714, 724, 728) may be to carry out communication involved in detecting and handling storage system and interconnect faults. This type of communication may be time critical and may need to be prioritized to ensure it doesn't get stuck behind a long network queue delay when a large burst of write traffic is suddenly dumped on the datacenter interconnect.

[0321] Readers will further appreciate that different types of communication may use the same connections, or different connections, and may use the same networks, or different networks, in various combinations. Further, some communications may be encrypted and secured while other communications might not be encrypted. In some cases, the data communications links could be used to forward I/O requests (either directly as the requests themselves or as logical descriptions of the operations the I/O requests represent) from one storage system to another. This could be used, for example, in cases where one storage system has up-to-date and in-sync content for a pod, and another storage system does not currently have up-to-date and in-sync content for the pod. In such cases, as long as the data communications links are running, requests can be forwarded from the storage system that is not up-to-date and in-sync to the storage system that is up-to-date and in-sync.

[0322] The example method depicted in FIG. 7 also includes exchanging (706), between the plurality of storage systems (714, 724, 728), timing information (710, 722, 726) for at least one of the plurality of storage systems (714, 724, 728). In the example method depicted in FIG. 6, timing information (710, 722, 726) for a particular storage system (714, 724, 728) may be embodied, for example, as the value of a clock within the storage system (714, 724, 728). In an alternative embodiment, the timing information (710, 722, 726) for a particular storage system (714, 724, 728) may be embodied as a value which serves as a proxy for a clock value. The value which serves as a proxy for a clock value may be included in a token that is exchanged between the storage systems. Such a value which serves as a proxy for a clock value may be embodied, for example, a sequence number that a particular storage system (714, 724, 728) or storage system controller can internally record as having been sent at a particular time. In such an example, if the token (e.g., the sequence number) is received back, the associated clock value can be found and utilized as the basis for determining whether a valid lease is still in place. In the example method depicted in FIG. 6, exchanging (706) timing information (710, 722, 726) for at least one of the plurality of storage systems (714, 724, 728) between the plurality of storage systems (714, 724, 728) may be carried out, for example, by each storage system (714, 724, 728)

sending timing information to each other storage system (714, 724, 728) in a pod on a periodic basis, on demand, within a predetermined amount of time after a lease is established, within a predetermined amount of time before a lease is set to expire, as part of an attempt to initiate or re-establish a synchronous replication relationship, or in some other way.

[0323] The example method depicted in FIG. 7 also includes establishing (708), in dependence upon the timing information (710, 722, 726) for at least one of the plurality of storage systems (714, 724, 728), a synchronous replication lease, the synchronous replication lease identifying a period of time during which the synchronous replication relationship is valid. In the example method depicted in FIG. 7, a synchronous replication relationship is formed as a set of storage systems (714, 724, 728) that replicate some dataset (712) between these largely independent stores, where each storage systems (714, 724, 728) has its own copy and its own separate internal management of relevant data structures for defining storage objects, for mapping objects to physical storage, for deduplication, for defining the mapping of content to snapshots, and so on. A synchronous replication relationship can be specific to a particular dataset, such that a particular storage system (714, 724, 728) may be associated with more than one synchronous replication relationship, where each synchronous replication relationship is differentiated by the dataset being described and may further consist of a different set of additional member storage systems.

[0324] In the example method depicted in FIG. 7, a synchronous replication lease may be established (708) in dependence upon the timing information (710, 722, 726) for at least one of the plurality of storage systems (714, 724, 728) in a variety of different ways. In one embodiment, the storage systems may establish (708) a synchronous replication lease by utilizing the timing information (710, 722, 726) for each of the plurality of storage systems (714, 724, 728) to coordinate clocks. In such an example, once the clocks are coordinated for each of the storage systems (714, 724, 728), the storage system may establish (708) a synchronous replication lease that extends for a predetermined period of time beyond the coordinated clock values. For example, if the clocks for each storage system (714, 724, 728) are coordinated to be at a value of X, the storage systems (714, 724, 728) may each be configured to establish a synchronous replication lease that is valid until X+2 seconds.

[0325] In an alternative embodiment, the need to coordinate clocks between the storage systems (714, 724, 728) may be avoided while still achieving a timing guarantee. In such an embodiment, a storage controller within each storage system (714, 724, 728) may have a local monotonically increasing clock. A synchronous replication lease may be established (708) between storage controllers (such as a primary controller in one storage system communicating with a primary controller in a paired storage system) by each controller sending its clock value to the other storage controllers along with the last clock value it received from the other storage controller. When a particular controller receives back its clock value from another controller, it adds some agreed upon lease interval to that received clock value and uses that to establish (708) its local synchronous replication lease. In such a way, the synchronous replication lease may be calculated in dependence upon a value of a local clock that was received from another storage system.

[0326] Consider an example in which a storage controller in a first storage system (714) is communicating with a storage controller in a second storage system (724). In such an example, assume that the value of the monotonically increasing clock for the storage controller in the first storage system (714) is 1000 milliseconds. Further assume that the storage controller in the first storage system (714) sends a message to the storage controller in the second storage system (724) indicating that its clock value at the time that the message was generated was 1000 milliseconds. In such an example, assume that 500 milliseconds after the storage controller in the first storage system (714) sent a message to the storage controller in the second storage system (724) indicating that its clock value at the time that the message was generated was 1000 milliseconds, the storage controller in the first storage system (714) receives a message from the storage controller in a second storage system (724) indicating that: 1) the value of the monotonically increasing clock in the storage controller in the second storage system (724) was at a value of 5000 milliseconds when the message was generated, and 2) the last value of the monotonically increasing clock in the storage controller in the first storage system (714) that was received by the second storage system (724) was 1000 milliseconds. In such an example, if the agreed upon lease interval is 2000 milliseconds, the first storage system (714) will establish (708) a synchronous replication lease that is valid until the monotonically increasing clock for the storage controller in the first storage system (714) is at a value of 3000 milliseconds. If the storage controller in the first storage system (714) does not receive a message from the storage controller in the second storage system (724) that includes an updated value of the monotonically increasing clock for the storage controller in the first storage system (714) by the time that the monotonically increasing clock for the storage controller in the first storage system (714) reaches a value of 3000 milliseconds, the first storage system (714) will treat the synchronous replication lease to have expired and may take various actions as described in greater detail below. Readers will appreciate that storage controllers within the remaining storage systems (724, 728) in a pod may react similarly and perform a similar tracking and updating of the synchronous replication lease. Essentially, the receiving controller can be assured that the network and the paired controllers were running somewhere during that time interval, and it can be assured that the paired controller received a message that it sent somewhere during that time interval. Without any coordination in clocks, the receiving controller can't know exactly where in that time interval the network and the paired controller were running, and can't really know if there were queue delays in sending its clock value or in receiving back its clock value.

[0327] In a pod consisting of two storage systems, each with a simple primary controller, where the primary controllers are exchanging clocks as part of their cluster communication, each primary controller can use the activity lease to put a bound on when it won't know for certain that the paired controller was running. At the point it becomes uncertain (when the controller's connection's activity lease has expired), it can start sending messages indicating that it is uncertain and that a properly synchronized connection must be reestablished before activity leases can again be resumed. These messages may be received and responses may not be received, if the network is working in one direction but is not working properly in the other direction.

This may be the first indication by a running paired controller that the connection isn't running normally, because its own activity lease may not yet have expired, due to a different combination of lost messages and queue delays. As a result, if such a message is received, it should also consider its own activity lease to be expired, and it should start sending messages of its own attempting to coordinate synchronizing the connection and resuming of activity leases. Until that happens and a new set of clock exchanges can succeed, neither controller can consider its activity lease to be valid.

[0328] In this model, a controller can wait for lease interval seconds after it started sending reestablish messages, and if it hasn't received a response, it can be assured that either the paired controller is down or the paired controller's own lease for the connection will have expired. To handle minor amounts of clock drift, it may wait slightly longer than the lease interval (i.e., a reestablishment lease). When a controller receives a reestablish message, it could consider the reestablishment lease to be expired immediately, rather than waiting (since it knows that the sending controller's activity lease has expired), but it will often make sense to attempt further messaging before giving up, in case message loss was a temporary condition caused, for example, by a congested network switch.

[0329] In an alternative embodiment, in addition to establishing a synchronous replication lease, a cluster membership lease may also be established upon receipt of a clock value from a paired storage system or upon receipt back of a clock exchanged with a paired storage system. In such an example, each storage system may have its own synchronous replication lease and its own cluster membership lease with every paired storage system. The expiration of a synchronous replication lease with any pair may result in paused processing. Cluster membership, however, cannot be recalculated until the cluster membership lease has expired with all pairs. As such, the duration of the cluster membership lease should be set, based on the message and clock value interactions, to ensure that the cluster membership lease with a pair will not expire until after a pair's synchronous replication link for that link has expired. Readers will appreciate that a cluster membership lease can be established by each storage system in a pod and may be associated with a communication link between any two storage systems that are members of the pod. Furthermore, the cluster membership lease may extend after the expiration of the synchronous replication lease for a duration of time that is at least as long as the time period for expiration of the synchronous replication lease. The cluster membership lease may be extended on receipt of a clock value received from a paired storage system as part of a clock exchange, where the cluster membership lease period from the current clock value may be at least as long as the period established for the last synchronous replication lease extension based on exchanged clock values. In additional embodiments, additional cluster membership information can be exchanged over a connection, including when a session is first negotiated. Readers will appreciate that in embodiments that utilize a cluster membership lease, each storage system (or storage controller) may have its own value for the cluster membership lease. Such a lease should not expire until it can be assured that all synchronous replication leases across all pod members will have expired given that the cluster lease expiration allows establishing new membership such as

through a mediator race and the synchronous replication lease expiration forces processing of new requests to pause. In such an example, the pause must be assured to be in place everywhere before cluster membership actions can be taken.

[0330] Readers will appreciate that although only one of the storage systems (714) is depicted as identifying (702), for a dataset (712), a plurality of storage systems (714, 724, 728) across which the dataset (712) will be synchronously replicated, configuring (704) one or more data communications links (716, 718, 720) between each of the plurality of storage systems (714, 724, 728) to be used for synchronously replicating the dataset (712), exchanging (706), between the plurality of storage systems (714, 724, 728), timing information (710, 722, 726) for at least one of the plurality of storage systems (714, 724, 728), and establishing (708), in dependence upon the timing information (710, 722, 726) for at least one of the plurality of storage systems (714, 724, 728), a synchronous replication lease, the remaining storage systems (724, 728) may also carry out such steps. In fact, all three storage systems (714, 724, 728) may carry out one or more of the steps described above at the same time, as establishing a synchronous replication relationship between two or more storage systems (714, 724, 728) may require collaboration and interaction between two or more storage systems (714, 724, 728).

[0331] For further explanation, FIG. 8 sets forth an example of a hybrid storage system for synchronously replicating datasets across a cloud-based storage system (803) and multiple, physical storage systems (402, 404 . . . 406) implemented similarly to the storage systems described in FIGS. 4-7, and in accordance with some embodiments of the present disclosure.

[0332] In the example depicted in FIG. 8, the cloud-based storage system (803) is created entirely in a cloud computing environment (852) such as, for example, Amazon Web Services ('AWS'), Microsoft Azure, Google Cloud Platform, IBM Cloud, Oracle Cloud, and others. The cloud-based storage system (803) may be used to provide services similar to the services that may be provided by the storage systems described above.

[0333] Further, in this example, the physical storage systems (402, 404 . . . 406) may include some or all of the components described above with reference to FIGS. 1A-7.

[0334] As depicted in the hybrid configuration of FIG. 8, one or more datasets (426, 428) may be synchronously replicated among the physical storage systems (402, 404 . . . 406) and also among the cloud-based storage system (803) implemented within a remote cloud computing environment (852) that is connected across a network (850), such as the Internet. In this example, there are two datasets (426, 428) that correspond to two respective pods (853, 854), where the pods (853, 854) include all features and functionality described above with reference to FIGS. 1A-7.

[0335] However, in other examples, not depicted, a hybrid configuration may synchronously replicate datasets across one or more physical storage systems (402, 404 . . . 406) and also across a virtual machine based implementation of a storage system, such as a virtual machine based implementation of storage systems (306) described above with reference to FIGS. 1A-7. Further, in this example of a hybrid configuration, the virtual machine based implementation of a storage system may be on a same network as the physical storage systems (402, 404 . . . 406), or the virtual machine

based storage system may be at a remote location and connected across a wide area network.

[0336] In the depiction of FIG. 8, one or more datasets (426, 428) are synchronously replicated between the cloud-based storage system (803) and the physical storage systems (402, 404 . . . 406) similarly to the synchronously replicated datasets among only the physical storage systems (402, 404 . . . 406).

[0337] However, in other examples, one or more datasets (426, 428) may be synchronously replicated among the physical storage systems (402, 404 . . . 406), but where the one or more datasets (426, 428) are asynchronously stored at the cloud-based storage system (803).

[0338] In this way, in the event of a failure or in the event of a request to reconstruct or clone one or more pods (853, 854), the recovery, reconstruction, or cloning may be performed using only data stored on the cloud-based storage system (803) as a source storage system such that a target storage system, whether virtual or physical, may be used to resume operation of the original storage systems within the context of any prior replication or other relationship for the dataset.

[0339] Further, in this example, a target storage system may be one or more physical storage systems, such as storage system (306), or where a target storage system may be another cloud-based storage system, such as cloud-based storage system (803).

[0340] For further explanation, FIG. 9 sets forth an example of an additional cloud-based storage system (902) in accordance with some embodiments of the present disclosure. In the example depicted in FIG. 50, the cloud-based storage system (902) is created entirely in a cloud computing environment (900) such as, for example, Amazon Web Services ('AWS'), Microsoft Azure, Google Cloud Platform, IBM Cloud, Oracle Cloud, and others. The cloud-based storage system (902) may be used to provide services similar to the services that may be provided by the storage systems described above. For example, the cloud-based storage system (902) may be used to provide block storage services to users of the cloud-based storage system (902), the cloud-based storage system (902) may be used to provide storage services to users of the cloud-based storage system (902) through the use of solid-state storage, and so on.

[0341] The cloud-based storage system (902) depicted in FIG. 50 may operate in a manner that is somewhat similar to the cloud-based storage system (902) depicted in FIG. 49, as the cloud-based storage system (902) depicted in FIG. 50 includes a storage controller application (906) that is being executed in a cloud computing instance (904). In the example depicted in FIG. 50, however, the cloud computing instance (904) that executes the storage controller application (906) is a cloud computing instance (904) with local storage (908). In such an example, data written to the cloud-based storage system (902) may be stored in both the local storage (908) of the cloud computing instance (904) and also in cloud-based object storage (910) in the same manner that the cloud-based object storage (910) was used above. In some embodiments, for example, the storage controller application (906) may be responsible for writing data to the local storage (908) of the cloud computing instance (904) while a software daemon (912) may be responsible for ensuring that the data is written to the

cloud-based object storage (910) in the same manner that the cloud-based object storage (910) was used above.

[0342] Readers will appreciate that a cloud-based storage system (902) depicted in FIG. 50 may represent a less expensive, less robust version of a cloud-based storage system than was depicted in FIG. 49. In yet alternative embodiments, the cloud-based storage system (902) depicted in FIG. 50 could include additional cloud computing instances with local storage that supported the execution of the storage controller application (906), such that failover can occur if the cloud computing instance (904) that executes the storage controller application (906) fails. Likewise, in other embodiments, the cloud-based storage system (902) depicted in FIG. 50 can include additional cloud computing instances with local storage to expand the amount local storage that is offered by the cloud computing instances in the cloud-based storage system (902).

[0343] Readers will appreciate that many of the failure scenarios described above with reference to FIG. 49 would also apply to cloud-based storage system (902) depicted in FIG. 50. Likewise, the cloud-based storage system (902) depicted in FIG. 50 may be dynamically scaled up and down in a similar manner as described above. The performance of various system-level tasks may also be executed by the cloud-based storage system (902) depicted in FIG. 50 in an intelligent way, as described above.

[0344] Readers will appreciate that, in an effort to increase the resiliency of the cloud-based storage systems described above, various components may be located within different availability zones. For example, a first cloud computing instance that supports the execution of the storage controller application may be located within a first availability zone while a second cloud computing instance that also supports the execution of the storage controller application may be located within a second availability zone. Likewise, the cloud computing instances with local storage may be distributed across multiple availability zones. In fact, in some embodiments, an entire second cloud-based storage system could be created in a different availability zone, where data in the original cloud-based storage system is replicated (synchronously or asynchronously) to the second cloud-based storage system so that if the entire original cloud-based storage system went down, a replacement cloud-based storage system (the second cloud-based storage system) could be brought up in a trivial amount of time.

[0345] Readers will appreciate that the cloud-based storage systems described herein may be used as part of a fleet of storage systems. In fact, the cloud-based storage systems described herein may be paired with on-premises storage systems. In such an example, data stored in the on-premises storage may be replicated (synchronously or asynchronously) to the cloud-based storage system, and vice versa.

[0346] For further explanation, FIG. 10 sets forth a flow chart illustrating an example method of servicing I/O operations in a cloud-based storage system (1004). Although depicted in less detail, the cloud-based storage system (1004) depicted in FIG. 6 may be similar to the cloud-based storage systems described above and may be supported by a cloud computing environment (1002).

[0347] The example method depicted in FIG. 10 includes receiving (1006), by the cloud-based storage system (1004), a request to write data to the cloud-based storage system (1004). The request to write data may be received, for example, from an application executing in the cloud com-

puting environment, by a user of the storage system that is communicatively coupled to the cloud computing environment, and in other ways. In such an example, the request can include the data that is to be written to the cloud-based storage system (1004). In other embodiments, the request to write data to the cloud-based storage system (1004) may occur at boot-time when the cloud-based storage system (1004) is being brought up.

[0348] The example method depicted in FIG. 10 also includes deduplicating (1008) the data. Data deduplication is a data reduction technique for eliminating duplicate copies of repeating data. The cloud-based storage system (1004) may deduplicate (1008) the data, for example, by comparing one or more portions of the data to data that is already stored in the cloud-based storage system (1004), by comparing a fingerprint for one or more portions of the data to fingerprints for data that is already stored in the cloud-based storage system (1004), or in other ways. In such an example, duplicate data may be removed and replaced by a reference to an already existing copy of the data that is already stored in the cloud-based storage system (1004).

[0349] The example method depicted in FIG. 10 also includes compressing (1010) the data. Data compression is a data reduction technique whereby information is encoded using fewer bits than the original representation. The cloud-based storage system (1004) may compress (1010) the data by applying one or more data compression algorithms to the data, which at this point may not include data that data that is already stored in the cloud-based storage system (1004).

[0350] The example method depicted in FIG. 10 also includes encrypting (1012) the data. Data encryption is a technique that involves the conversion of data from a readable format into an encoded format that can only be read or processed after the data has been decrypted. The cloud-based storage system (1004) may encrypt (1012) the data, which at this point may have already been deduplicated and compressed, using an encryption key. Readers will appreciate that although the embodiment depicted in FIG. 10 involves deduplicating (1008) the data, compressing (1010) the data, and encrypting (1012) the data, other embodiments exist in which fewer of these steps are performed and embodiment exist in which the same number of steps or fewer are performed in a different order.

[0351] The example method depicted in FIG. 10 also includes storing (1014), in solid-state storage of the cloud-based storage system (1004), the data. Storing (1014) the data in solid-state storage of the cloud-based storage system (1004) may be carried out, for example, by storing (1016) the data in local storage (e.g., SSDs) of one or more cloud computing instances, as described in more detail above. In such an example, the data may be spread across the local storage of many cloud computing instances, along with parity data, to implement RAID or RAID-like data redundancy.

[0352] The example method depicted in FIG. 10 also includes storing (1018), in object-storage of the cloud-based storage system (1004), the data. Storing (1018) the data in object-storage of the cloud-based storage system can include creating (1020) one or more equal sized objects, wherein each equal sized object includes a distinct chunk of the data, as described in greater detail above.

[0353] The example method depicted in FIG. 10 also includes receiving (1022), by the cloud-based storage system, a request to read data from the cloud-based storage

system (1004). The request to read data from the cloud-based storage system (1004) may be received, for example, from an application executing in the cloud computing environment, by a user of the storage system that is communicatively coupled to the cloud computing environment, and in other ways. The request can include, for example, a logical address of the data that is to be read from the cloud-based storage system (1004).

[0354] The example method depicted in FIG. 10 also includes retrieving (1024), from solid-state storage of the cloud-based storage system (1004), the data. Readers will appreciate that the cloud-based storage system (1004) may retrieve (1024) the data from solid-state storage of the cloud-based storage system (1004), for example, by the storage controller application forwarding the read request to the cloud computing instance that includes the requested data in its local storage. Readers will appreciate that by retrieving (1024) the data from solid-state storage of the cloud-based storage system (1004), the data may be retrieved more rapidly than if the data were read from cloud-based object storage, even though the cloud-based object storage does include a copy of the data.

[0355] For further explanation, FIG. 11 sets forth a flow chart illustrating an additional example method of servicing I/O operations in a cloud-based storage system (1004). The example method depicted in FIG. 11 is similar to the example method depicted in FIG. 10, as the example method depicted in FIG. 11 also includes receiving (1006) a request to write data to the cloud-based storage system (1004), storing (1014) the data in solid-state storage of the cloud-based storage system (1004), and storing (1018) the data in object-storage of the cloud-based storage system (1004).

[0356] The example method depicted in FIG. 11 also includes detecting (1102) that at least some portion of the solid-state storage of the cloud-based storage system has become unavailable. Detecting (1102) that at least some portion of the solid-state storage of the cloud-based storage system has become unavailable may be carried out, for example, by detecting that one or more of the cloud computing instances that includes local storage has become unavailable, as described in greater detail below.

[0357] The example method depicted in FIG. 11 also includes identifying (1104) data that was stored in the portion of the solid-state storage of the cloud-based storage system that has become unavailable. Identifying (1104) data that was stored in the portion of the solid-state storage of the cloud-based storage system that has become unavailable may be carried out, for example, through the use of metadata that maps some identifier of a piece of data (e.g., a sequence number, an address) to the location where the data is stored. Such metadata, or separate metadata, may also map the piece of data to one or more object identifiers that identify objects stored in the object-storage of the cloud-based storage system that contain the piece of data.

[0358] The example method depicted in FIG. 11 also includes retrieving (1106), from object-storage of the cloud-based storage system, the data that was stored in the portion of the solid-state storage of the cloud-based storage system that has become unavailable. Retrieving (1106) the data that was stored in the portion of the solid-state storage of the cloud-based storage system that has become unavailable from object-storage of the cloud-based storage system may be carried out, for example, through the use of metadata described above that maps the data that was stored in the

portion of the solid-state storage of the cloud-based storage system that has become unavailable to one or more objects stored in the object-storage of the cloud-based storage system that contain the piece of data. In such an example, retrieving (1106) the data may be carried out by reading the objects that map to the data from the object-storage of the cloud-based storage system.

[0359] The example method depicted in FIG. 11 also includes storing (1108), in solid-state storage of the cloud-based storage system, the retrieved data. Storing (1108) the retrieved data in solid-state storage of the cloud-based storage system may be carried out, for example, by creating replacement cloud computing instances with local storage and storing the data in the local storage of one or more of the replacement cloud computing instances, as described in greater detail above.

[0360] Readers will appreciate that although the embodiments described above relate to embodiments in which data that was stored in the portion of the solid-state storage of the cloud-based storage system that has become unavailable is essentially brought back into the solid-state storage layer of the cloud-based storage system by retrieving the data from the object-storage layer of the cloud-based storage system, other embodiments are within the scope of the present disclosure. For example, because data may be distributed across the local storage of multiple cloud computing instances using data redundancy techniques such as RAID, in some embodiments the lost data may be brought back into the solid-state storage layer of the cloud-based storage system through a RAID rebuild.

[0361] For further explanation, FIG. 12 sets forth a flow chart illustrating an example method of servicing I/O operations in a cloud-based storage system (1204). Although depicted in less detail, the cloud-based storage system (1204) depicted in FIG. 12 may be similar to the cloud-based storage systems described above and may be supported by a cloud computing environment (1202).

[0362] The example method depicted in FIG. 12 includes receiving (1206), by the cloud-based storage system (1204), a request to write data to the cloud-based storage system (1204). The request to write data may be received, for example, from an application executing in the cloud computing environment, by a user of the storage system that is communicatively coupled to the cloud computing environment, and in other ways. In such an example, the request can include the data that is to be written to the cloud-based storage system (1204). In other embodiments, the request to write data to the cloud-based storage system (1204) may occur at boot-time when the cloud-based storage system (1204) is being brought up.

[0363] The example method depicted in FIG. 12 also includes deduplicating (1208) the data. Data deduplication is a data reduction technique for eliminating duplicate copies of repeating data. The cloud-based storage system (1204) may deduplicate (1208) the data, for example, by comparing one or more portions of the data to data that is already stored in the cloud-based storage system (1204), by comparing a fingerprint for one or more portions of the data to fingerprints for data that is already stored in the cloud-based storage system (1204), or in other ways. In such an example, duplicate data may be removed and replaced by a reference to an already existing copy of the data that is already stored in the cloud-based storage system (1204).

[0364] The example method depicted in FIG. 12 also includes compressing (1210) the data. Data compression is a data reduction technique whereby information is encoded using fewer bits than the original representation. The cloud-based storage system (1204) may compress (1210) the data by applying one or more data compression algorithms to the data, which at this point may not include data that data that is already stored in the cloud-based storage system (1204).

[0365] The example method depicted in FIG. 12 also includes encrypting (1212) the data. Data encryption is a technique that involves the conversion of data from a readable format into an encoded format that can only be read or processed after the data has been decrypted. The cloud-based storage system (1204) may encrypt (1212) the data, which at this point may have already been deduplicated and compressed, using an encryption key. Readers will appreciate that although the embodiment depicted in FIG. 12 involves deduplicating (1208) the data, compressing (1210) the data, and encrypting (1212) the data, other embodiments exist in which fewer of these steps are performed and embodiment exist in which the same number of steps or fewer are performed in a different order.

[0366] The example method depicted in FIG. 12 also includes storing (1214), in block-storage of the cloud-based storage system (1204), the data. Storing (1214) the data in block-storage of the cloud-based storage system (1204) may be carried out, for example, by storing (1216) the data in local storage (e.g., SSDs) of one or more cloud computing instances, as described in more detail above. In such an example, the data spread across local storage of multiple cloud computing instances, along with parity data, to implement RAID or RAID-like data redundancy.

[0367] The example method depicted in FIG. 12 also includes storing (1218), in object-storage of the cloud-based storage system (1204), the data. Storing (1218) the data in object-storage of the cloud-based storage system can include creating (1220) one or more equal sized objects, wherein each equal sized object includes a distinct chunk of the data, as described in greater detail above.

[0368] The example method depicted in FIG. 12 also includes receiving (1222), by the cloud-based storage system, a request to read data from the cloud-based storage system (1204). The request to read data from the cloud-based storage system (1204) may be received, for example, from an application executing in the cloud computing environment, by a user of the storage system that is communicatively coupled to the cloud computing environment, and in other ways. The request can include, for example, a logical address of the data that is to be read from the cloud-based storage system (1204).

[0369] The example method depicted in FIG. 12 also includes retrieving (1224), from block-storage of the cloud-based storage system (1204), the data. Readers will appreciate that the cloud-based storage system (1204) may retrieve (1224) the data from block-storage of the cloud-based storage system (1204), for example, by the storage controller application forwarding the read request to the cloud computing instance that includes the requested data in its local storage. Readers will appreciate that by retrieving (1224) the data from block-storage of the cloud-based storage system (1204), the data may be retrieved more rapidly than if the data were read from cloud-based object storage, even though the cloud-based object storage does include a copy of the data.

[0370] For further explanation, FIG. 13 sets forth a flow chart illustrating an additional example method of servicing I/O operations in a cloud-based storage system (1204). The example method depicted in FIG. 13 is similar to the example method depicted in FIG. 12, as the example method depicted in FIG. 13 also includes receiving (1206) a request to write data to the cloud-based storage system (1204), storing (1214) the data in block-storage of the cloud-based storage system (1204), and storing (1218) the data in object-storage of the cloud-based storage system (1204).

[0371] The example method depicted in FIG. 13 also includes detecting (1302) that at least some portion of the block-storage of the cloud-based storage system has become unavailable. Detecting (1302) that at least some portion of the block-storage of the cloud-based storage system has become unavailable may be carried out, for example, by detecting that one or more of the cloud computing instances that includes local storage has become unavailable, as described in greater detail below.

[0372] The example method depicted in FIG. 13 also includes identifying (1304) data that was stored in the portion of the block-storage of the cloud-based storage system that has become unavailable. Identifying (1304) data that was stored in the portion of the block-storage of the cloud-based storage system that has become unavailable may be carried out, for example, through the use of metadata that maps some identifier of a piece of data (e.g., a sequence number, an address) to the location where the data is stored. Such metadata, or separate metadata, may also map the piece of data to one or more object identifiers that identify objects stored in the object-storage of the cloud-based storage system that contain the piece of data.

[0373] The example method depicted in FIG. 13 also includes retrieving (1306), from object-storage of the cloud-based storage system, the data that was stored in the portion of the block-storage of the cloud-based storage system that has become unavailable. Retrieving (1306) the data that was stored in the portion of the block-storage of the cloud-based storage system that has become unavailable from object-storage of the cloud-based storage system may be carried out, for example, through the use of metadata described above that maps the data that was stored in the portion of the block-storage of the cloud-based storage system that has become unavailable to one or more objects stored in the object-storage of the cloud-based storage system that contain the piece of data. In such an example, retrieving (1306) the data may be carried out by reading the objects that map to the data from the object-storage of the cloud-based storage system.

[0374] The example method depicted in FIG. 13 also includes storing (1308), in block-storage of the cloud-based storage system, the retrieved data. Storing (1308) the retrieved data in block-storage of the cloud-based storage system may be carried out, for example, by creating replacement cloud computing instances with local storage and storing the data in the local storage of one or more of the replacement cloud computing instances, as described in greater detail above.

[0375] Readers will appreciate that although the embodiments described above relate to embodiments in which data that was stored in the portion of the block-storage of the cloud-based storage system that has become unavailable is essentially brought back into the block-storage layer of the cloud-based storage system by retrieving the data from the

object-storage layer of the cloud-based storage system, other embodiments are within the scope of the present disclosure. For example, because data may be distributed across the local storage of multiple cloud computing instances using data redundancy techniques such as RAID, in some embodiments the lost data may be brought back into the block-storage layer of the cloud-based storage system through a RAID rebuild.

[0376] For further explanation, FIG. 14 sets forth a flow chart illustrating an example method for resiliency in a cloud-based storage system (1400) according to some embodiments of the present disclosure.

[0377] In some implementations, a cloud-based storage system (1400), such as the cloud-based storage systems described above with reference to FIGS. 8-13, may be configured to be reliable against data loss based at least in part on using parity information for stored data that is mixed with enough self-describing information to ensure recoverability of the data.

[0378] In some examples, self-describing information may be generated by accumulating information about all segment shards as the segment shards are received into a first tier of cloud storage that may not be sufficiently reliable to satisfy threshold requirements for durability. Further, in some implementations, these self-described segments may be copied from the first tier of cloud storage into a second tier of cloud storage.

[0379] In some implementations, as described above, data resiliency may be based on recovering data lost in a first tier of cloud storage of a cloud-based storage system (1400) by copying the lost data from the second tier of cloud storage of the cloud-based storage system (1400)—where the second tier of cloud storage is generally more reliable than the first tier of cloud storage.

[0380] However, in other implementations, data resiliency may be based on generated recovery information that is stored along with storage data within the first tier of cloud storage, and in this way, data recovery may be based entirely on data and recovery information stored within the first tier of cloud storage. In such an example, a cloud-based storage system (1400) may be configured to provide a single tier of cloud storage without a second tier of cloud storage while maintaining a level of durability that satisfies a specified durability threshold.

[0381] As described above, in the case where the cloud-based storage system (1400) is implemented within an Amazon cloud computing environment, a first tier of cloud storage may include one or more virtual computing instances, such as EBS instances, and where a second tier of cloud storage may be a cloud object store, such as S3.

[0382] In some implementations, a segment may be considered a logical container of data referenced by an address space separate from a physical storage location in which a segment is stored. In one example, a segment may be referenced by a segment number, and a segment may include metadata from which storage data may be recovered.

[0383] Further, in some examples, each segment may be protected, or increase durability, against memory failure or other types of system failures by breaking or dividing a segment into multiple data shards and multiple parity shards. In some implementations, the data shards and the parity shards may be distributed, or striped, across multiple, distinct, data stores, such as across multiple cloud computing instances within a first tier of cloud storage in a cloud-based

storage system (1400). In some examples, a “segment” may refer to a logical container of data and a place within an address space of segments, and a “stripe” may refer to a same set of shards as a segment and may include information on how the shards are distributed among data stores and redundancy information (or parity information).

[0384] In some examples, a cloud-based storage system (1400) may be configured to be reliable against complete failure of one or two or more cloud computing instances based at least in part on using erasure codes, where the cloud-based storage system (1400) may include a first tier of cloud storage that includes storage elements implemented by cloud computing instances and/or storage provided by provisioned solid state devices. In some examples, erasure codes may be based on single or double parity, as with RAID-5 or RAID-6, or against uncorrectable faults, corruptions, or complete failures of individual elements within a storage element by reconstructing content from elsewhere within the same storage tier of the cloud-based storage system (1400) or through the single and double parity protection used for complete failure of a storage element.

[0385] In other words, in some examples, the various storage elements of a first tier of cloud storage of a cloud-based storage system (1400), individually or in combination, may be used to implement multiple, different RAID levels or combinations of RAID levels. In the following examples, a RAID stripe is data that is stored among a set of memory regions mapped across a set of storage elements, where each memory region on a given storage element stores a portion of the RAID stripe and may be referred to as a “strip,” a “stripe element,” or a “shard.”

[0386] Given that a cloud-based storage system (1400) may simultaneously implement various combinations of RAID levels, a “RAID stripe” may refer to all the data that is stored within a given RAID stripe corresponding to a given RAID level. Generally, in the following examples, a strip, stripe element, or shard is one or more consecutive blocks of memory on a single storage element—in other words, an individual strip, stripe element, or shard is a portion of a RAID stripe distributed onto a single storage element among a set of storage elements. In this way, a RAID level may depend on how the RAID stripe is distributed among a set of storage elements.

[0387] In some implementations, with regard to erasure coding in general, erasure coding may be described in terms of $N+R$ schemes, where for every N units of content data, an additional R units of redundancy data is written such that up to R failures may be tolerated while being able to recover all N units of content data. For example, RAID-5 is an example $N+1$ scheme, whereas RAID-6 describes a set of $N+2$ schemes. In other examples, a scheme may be based on Galois Fields, or other types of mathematics that can cover a wide range of N and R combinations.

[0388] Specifically, in some cases, each given storage element within a cloud-based storage system (1400) may, by default, detect faults locally and directly using, as some examples, a checksum or mathematical integrity check for all data that is read or written to the given storage element. However, in cases where a given storage element in a cloud-based storage system (1400) does not perform local fault detection, other techniques may be applied, such various coding techniques that store additional shards beyond a fault tolerance (e.g., three parity shards that are usable to recover up to two faults), or through the use of encoding all

data, including parity or redundancy data, in a manner that can be used to detect corrupted data. In general, in the embodiments described herein, a storage element within a first tier of cloud storage may, by default, perform localized data integrity checks, where data shards and redundancy shards may be distinct from each other. However, in some examples, no such restriction to, or reliance upon, localized integrity checking is presumed.

[0389] It should further be noted that erasure coding schemes may be used within a cloud-based storage system (1400) in a variety of ways. Traditional storage systems, which were originally designed for spinning disk storage devices, typically allocate large RAID-5 and RAID-6 layouts relatively statically as a set of relatively large, such as multi-gigabyte, $N+R$ shards, where each shard is a unit of stored content data or a unit of stored redundancy data. In such schemes, in contrast to a cloud-based storage system (1400), overwrites of previously stored content includes replacing existing content data shards and calculating and replacing existing redundancy data shards using some sequence of steps to ensure that the operations may be done safely in the event that a fault occurs during the sequence of steps. In some cases, these sequence of steps may be computationally intensive; for example, in some cases, the sequence of steps includes reading old content and redundancy data, or reading from most existing content data shards at the same logical offset in a RAID layout in order to calculate new redundancy data. In other examples, the sequence of steps may include multiple persisting steps in order to ensure that an interrupted operation can be recovered.

[0390] By contrast to spinning disks, a storage element within a cloud-based storage system (1400)—such as a cloud computing instance—does not suffer from mechanical failures. Therefore, generally, a storage element in a cloud-based storage system (1400) does not suffer from slowdowns due to randomization of data locations because given that there is no mechanical delay in reading a block, any read unaffected by a concurrent write or a concurrent garbage collection is generally quite fast irrespective of whether previous, subsequent, or concurrent reads are from nearby or farther away locations or logical addresses. In some cases, a penalty for turning relatively sequential reads into relative random reads is so high for disk, that disk based storage systems generally avoid randomizing locations of data so that logical sequential reads by applications stay sequential on disk, which is part of why it is so common for RAID-5 and RAID-6 mappings to be kept relatively static, with the mapping retained when data is overwritten (some file system based models such as ZFS are exceptions to this).

[0391] Further, because a storage element in a cloud-based storage system (1400) does not have mechanical delays inherent in spinning disks, an occasional or even frequent extra read to Fig. out where a block resides does not incur a high performance penalty, consequently, scattering data as it is written (resulting in a large index that can exceed available cache memory that results in the need to read the index in order to locate data) has few downsides. For example, performance impact for a small read performed before a large read is performed will be negligible. Further, bandwidth for servicing a large sequential read request by issuing a set of smaller physical random reads (the result of scattering content on writes and overwrites) is generally only a bit less than the bandwidth for servicing large

sequential read requests by issuing a small number of larger physical reads. Specifically, a cloud-based storage system (1400) may implement an API that supports multiple, parallel accesses to stored data, which provides for additional bandwidth increases due to reading data that is distributed among the storage elements in a cloud-based storage system (1400) such that data may be read in parallel.

[0392] Consequently, because storage elements in a cloud-based storage system (1400) may be provisioned on demand, in some implementations, data received in a cloud-based storage system (1400) may be written into N dynamically allocated shards of a few kilobytes to a few megabytes as content data is written, with R matching redundancy shards allocated and written to match, thereby achieving $N+R$ recoverability. Such a protocol for processing writes avoids replacing existing data in place, thus avoiding the need to modify in-place redundancy data in place through some safe update scheme. Collections of stripes may then be written out based on whatever patterns achieve the best write throughput.

[0393] In this way, in some implementations of a cloud-based storage system (1400), erasure codes may be used to ensure that any completely written data can survive failures within one or more storage elements of a cloud-based storage system (1400).

[0394] In some implementations, consistent recovery may be performed with varying subsets of storage elements among the multiple storage elements of the cloud-based storage system (1400). For example, if a storage controller fails and reboots, or one controller fails and another controller takes over for the failed controller, or when some number of storage elements go offline and then come back online, or when a cloud-based storage system (1400) is stopped through some sequence of events (power failure, power surge, an inadvertent administrative shutdown command, or a bug, among other examples), storage data may require recovery from the available storage elements of the cloud-based storage system (1400). During such a recovery, there may be a second event that results in the storage service provided by the cloud-based storage system (1400) going offline and then later being recovered, and that sequence of interrupted recoveries may repeat itself many times. During each recovery, a different set of storage elements may come online, or some storage elements may come online much slower than other storage elements, resulting in the storage service deciding to recover from the online storage elements, but with a different set of such storage elements each time. Such a sequence of interrupted recovering may present an issue for data that is stored redundantly across storage elements of the cloud-based storage system (1400), such as through writing the same data to multiple such storage elements (RAID-1 or mirroring) or through writing an erasure code (such as RAID-5 or RAID-6) across multiple storage elements intended to achieve reliability in the face of one or more storage element failures.

[0395] As one example of handling interrupted recoveries, consider storage elements utilized in a cloud-based storage system (1400) that is two-drive-failure redundant, such as one that writes $N+2$ segments of data as RAID-6 style stripes formed as some number, say N , units of data (e.g., shards) and two similarly sized shards storing P and Q parity. Consider further that segments are written out as new data flows into the cloud-based storage system (1400), with no

fixed correspondence of the virtual address of stored data to the physical location of that data, rather than being partially rewritten in place to fixed locations as has been typical of traditional RAID storage systems.

[0396] In this example, the content of a cloud-based storage system (1400) may be considered to be committed and recoverable content may be determined after a restart that only had access to the persisted content. However, there may be a problem: the recoverable content depends on the specific set of storage elements that are available and ready at the time recovery starts. If N or N+1 storage elements out of N+2 storage elements were written for a particular N+2 stripe prior to the fault that preceded a recovery, then recoverability depends on which storage elements are available when recovery starts. If N of the storage elements that were written are available on recovery, then the content of that RAID stripe is recoverable. If N-1 of the storage elements that were written are available for recovery, then at least some data from that stripe is not recoverable, yet one or two drives that were not written for some stripes might be available during startup, so there are combinations of one or two storage element failures that can lose some partially written data and other combinations of one or two storage element failures that do not lose the same partially written data.

[0397] Continuing with this example, one solution for this problem is to operate at startup, based on whatever data is recoverable, and to ignore any data that is not recoverable. Under normal operation, an incomplete write of all shards of an N+2 stripe should only happen if some in-progress writes did not complete prior to a cloud-based storage system (1400) or storage controller fault, or if a storage element fails resulting in a persistent partial failure. In this way, a storage element failure may be handled by writing the failed shards to new locations in other storage elements, including newly provisioned storage elements, prior to considering the write complete, or by somehow persistently marking the storage element as having failed so that it will not be considered as a viable source for up-to-date content on the cloud-based storage system (1400) or storage controller startup.

[0398] To conclude with this example, as soon as a write of a stripe is complete (or as soon as any data is N+2 redundant through one or more techniques), the cloud-based storage system (1400) logic may consider the write to be durable and recoverable and therefore the cloud-based storage system (1400) may move on to some new set of writes that depend on that previous write being guaranteed recoverable. For example, a first write might store data that represents the establishment of a snapshot of a volume, and a second write might represent new data written to that volume after the snapshot. The second write is written within the context of structures established by the first write.

[0399] In some embodiments, as an alternative, the snapshot and the write may be concurrent such that the write is included in the snapshot (thus being associated with one version of the durable structures for the volume and its snapshot), or the write could be excluded from the snapshot (thus being associated with another version of the durable structures for the volume and its snapshots), or either the snapshot or the write or both might be backed out as never having happened, which are all consistent and acceptable outcomes if neither the snapshot request nor the write were completed and signaled to a requester as completed.

[0400] Continuing with this example, while the snapshot and the write are being processed, the cloud-based storage system (1400), or a component storage controller or storage element, might fault a first time, and the snapshot—at the time of the first fault—may have been written to N out of N+2 storage elements, while the write might have been written to M out of M+2 elements. In this example, during recovery, the cloud-based storage system (1400) may take action to clean up or move forward data structures so that the cloud-based storage system (1400) is ready to process new writes.

[0401] Further, in this example, if one or two storage elements are not available during recovery, then either the snapshot, or the write, or both, or neither might be recoverable without the two unavailable storage elements. Subsequent first recovery actions may depend on which storage elements are not available, which may involve writing down new data that is protected by some alternate L+2 data protection scheme on some other set of storage elements to finalize a clean structure for the volume and the volume's chain of snapshots and of the data that does or does not fall into one or another of the snapshots.

[0402] In this example, if the writing of the L+2 redundant data is written to only L storage elements of the cloud-based storage system (1400) before another, or second, cloud-based storage system (1400) or storage controller fault, then subsequent second recovery of the first recover actions may also depend on which storage elements are available during the second recovery.

[0403] However, in this example, a different two storage elements may be unavailable during this second recovery, thereby resulting in a different answer for whether the snapshot or the write are recoverable and some independent answer for whether the first recovery action is recoverable. Such a scenario may cause a problem if the first recovery actions are recoverable but the second recovery actions determine a different answer for whether the snapshot or the write is recoverable. Note that determining that written information is recoverable can be calculated from the available stored data, whereas unrecoverable information may simply be unknown to recovery so that recovery sequence of operations may not explicitly determine that it is unrecoverable. As an example, for a 2+2 RAID-6 stripe, say that only two data shards were written prior to a fault that resulted in a later recovery, then, if only two of those two shards are available during recovery, then the data is recoverable. Otherwise, in this example, if only the P and Q shards are available during recovery, then there is no available knowledge of the stripe at all.

[0404] In this example, there may exist the following scenario: the first recovery determines that both the snapshot and the write into the volume are recoverable, but the recovery action must then determine whether the write should be included in the snapshot or excluded from the snapshot. The first recovery actions might then include writing a record that the snapshot exists and has a name and might include metadata changes that include the write content in the snapshot, with a reference to the write content added to a table.

[0405] Continuing with this scenario, if during the second recovery, the write of the L+2 data from the first recovery actions is recovered, as is the snapshot, but the write is not recovered, then there may be metadata associated with the snapshot that includes data which is not itself recoverable

(and may be entirely unknown given the set of storage elements available). Such a result may be inconsistent and may corrupt metadata, depending on how the cloud-based storage system (1400) implementations handles this scenario.

[0406] Further, during a second recovery, actions may be taken to complete recovered actions by writing further data and metadata based on what was recovered, and a fault might occur during that recovery, resulting in a third recovery based on perhaps a different set of available and unavailable storage elements.

[0407] While it may be that the likelihood of some of these scenarios is low because such scenarios may depend on particular combinations of faults during narrow time windows during the writing of redundant data after just enough data has been written for the data to be recoverable with one or two storage elements unavailable during recovery-without more data having been written so the data is guaranteed to be recoverable no matter which two storage elements are subsequently unavailable. In other words, such scenarios may require a sequence of faults and recoveries where each fault happens during one of these narrow time windows, and they require that a different set of one or two storage elements be unavailable during subsequent recoveries. However, while these scenarios may only occur infrequently, or rarely, these scenarios are possible, and these implementations provide strategies to mitigate, or eliminate, such scenario consequences.

[0408] In some implementations, strategies to prevent such scenarios may include using two-phase commit, where data is written out as a set of pending changes, and once all changes are written, a much smaller commit is written out. However, the commit may have the same problem. In other words, if a cloud-based storage system (1400) is expected to recover with two failed storage elements, then at least three copies of the commit must be written, or it must itself be written using an $N+2$ redundancy scheme. As an example, if one or two of three copies are written, then one recovery sequence may be aware of the commit, while a second recovery sequence may fail to be aware of the commit, or vice versa. In this example, if the existence of the commit itself is used as a basis for subsequent steps in recovery, and if a second recovery depends for correctness on seeing the same commit (or lack thereof) to ensure that anything written in subsequent steps is handled correctly, then the same issue applies where inconsistency between a first recovery and a second recovery can lead to corruption.

[0409] Further, the examples herein may implement any $N+R$ scheme, where R is a number of shards representing redundancy data, where any N valid, written, and available shards (whether data or redundancy shards) may be used to recover content. Further, if at least N shards, but fewer than $N+R$ shards, are written prior to a fault, then if at least N of the written shards are available during recovery, then the associated data is recoverable. However, if fewer than N of the written shards are available during recovery, then some of the content may not be recoverable. Further in this example, if there is a second fault during a first recovery that leads to a second recovery, or an eventual third fault leading to a third recovery, and so on, then a different set of available storage elements may, as described above, alter the set of written stripes for which a sufficient number of shards are available.

[0410] In some implementations of a cloud-based storage system (1400), recovery may be based on recording available storage element lists. Such available storage element lists may provide a solution for determining a consistent set of recoverable data across multiple reboots, where the solution may include terminating efforts for a second recovery that detects failed storage elements if that second recovery follows a first recovery with an incompatible, or different, set of detected failed storage elements. Such a recovery protocol may operate in one of two ways: during recovery, before making any other changes that could lead to inconsistencies in subsequent recoveries, an available storage element list is generated that indicates a set of storage elements that are included in the given recovery sequence. The available storage element list may be written to all available storage elements. Until the available storage element list is written to all available storage elements, and until the writes are successful to a sufficient number of those storage elements that a subsequent detection on recovery is guaranteed, then further recovery is prevented from proceeding.

[0411] However, this solution may present a problem: how can a list of available storage elements be recorded in a way that is reliably recoverable on a second, third, or fourth recovery each with an inconsistent set of available storage elements? While maintaining such a list may be difficult, this is simpler information than a set of all erasure coded stripes. One example solution, in a cloud-based storage system (1400) that is redundant against R failures, includes writing the list of available storage elements to at least $2 \times R + 1$ storage elements before proceeding further with recovery. In this example, with an R set to two (2), a subsequent recovery that is missing a different set of two storage elements will see at least three of those lists of available storage elements. Alternately, if the writing of the list of available storage elements had been written to R storage elements or less, a subsequent recovery might not see the list of available storage elements, or might see only one copy of the list. If a subsequent recovery does not see the list of available storage elements at all, then the previous recovery could not have advanced to making any incompatible changes, and a new list of available storage elements can be expressed to a different set of $2 \times R + 1$ storage elements. If a subsequent recovery does not see at least $R + 1$ copies of a list of available storage elements, a prior recovery could not have advanced to the point of making changes and the current list of available storage elements could be written out. Alternately, if a subsequent recovery sees any copy of the prior list of available storage elements, it could use that list of available storage elements.

[0412] Continuing with this solution, regardless of the manner in which the list of available storage elements is made reliable, once a fault during a first recovery has led to analysis for a second recovery, if it is determined that the available storage elements during analysis for the second recovery does not sufficiently overlap with the available storage elements during a first recovery that might have proceeded past the point of writing the list of available storage elements, then recovery is stopped. If a sufficient number of those storage elements do come back online, then the second recovery can proceed after that point, but not before. In other words, in this example, the union of the list of available storage elements from a first recovery and the

list of available storage elements from a second recovery cannot be a set larger than R storage elements.

[0413] In some implementations of a cloud-based storage system (1400), recovery may be based on identities of allowed stripes. In this example, a solution may be more tolerant of different sets of storage elements being available on subsequent recovery from prior interrupted recoveries. For example, a set of allowed commit identities may be defined to be associated with data that is allowed in the cloud-based storage system (1400), and a set of disallowed commit identities may be defined to be associated with data that is not allowed in the cloud-based storage system (1400) for one or more reasons.

[0414] In this example solution, there may be several types of data, including one or more of: erasure coded stripes, each of which is associated with at least one commit identity, recent commits of commit identities, a set of allowed commit identities, a set of disallowed commit identities, or a set of future commit identities.

[0415] Continuing with this example, when writing data into an erasure coded stripe, until that data is written completely, such that it is guaranteed to be recovered by any sufficiently large set of storage elements available during a recovery, a storage controller may not consider the data committed. Such a protocol that delays data as being committed may include waiting until all shards of, say, an N+R stripe are fully written before anything in that N+R stripe can commit. In this case, there may be only one commit identity associated with the stripe, where the commit identity may be stored somewhere in the stripe that should be available during recovery. If parts of a stripe can be made durable and recoverable without making the entire stripe durable and recoverable, as suggested in previous sections, such as by writing sub-regions of shards with matching dual parity, or by mirroring writes to fast memory or by writing partial M+R shards of an N+R stripe (or any of the other techniques described previously), then commit identities may be associated with those partial elements of a stripe, but then the rest of this argument applies to that partial stripe which must still be completely persisted before proceeding to committing that partial stripe.

[0416] Further in this example, before any such written data may be relied upon (before it is considered recoverable, durable content of the cloud-based storage system (1400), the commit identities associated with the written data must be written down to ensure they will end up in the set of allowed commit identities. The set of allowed commit identities may be written into headers for subsequent shards. In other cases, with regard to the storage elements above, the sets of allowed or disallowed commit identities may be stored in durable storage available to the storage elements, such as an object store such as Amazon S3 or some other, quicker, durable memory. Writing the sets of allowed or disallowed commit identities into headers for subsequent shards may depend on waiting for sufficient new data to trigger that new data being persisted into new shards. In some examples, writing commit identities into durable memory may be done quickly, allowing written data to be considered durably recoverable more quickly.

[0417] For data to be considered reliably committed, such that the cloud-based storage system (1400) implementation may continue operating in reliance upon the data having been reliably committed, associated commit identities for the data considered reliably committed must be written to at

least R+1 storage elements, though commit identities could be written to more storage elements than R+1 before proceeding.

[0418] In some implementations, recovery of recently written data may be based upon identifying commit identities for the data being recovered, and upon determining whether those commit identities were written out as committed. In this example, if the identified commit identities are determined to be committed, then the data has been written completely and may be safely considered to have been committed. In other words, if the identified commit identities are determined to be committed, then the data may be recoverable regardless of which R or fewer subsets of an N+R stripe are not available. To ensure that a subsequent recovery will also see the commit identities as committed (possibly using a different subset of available storage elements), recovered commits of commit identities should be written down again, if they had not already been written to a sufficient number of storage elements to ensure they are recoverable.

[0419] In some examples, a situation may arise where a recovery is unable to identify the commit records for a set of commit identities. In one example solution, during a given recovery process, if the given recovery process determines the list of commit identities whose commits the given recovery process did not find, then that list of commit identities may be written into a disallowed list of storage elements, where the disallowed list explicitly removes a commit identity from being considered valid content for the cloud-based storage system (1400), or where the disallowed list stores an indication that the commit identity is invalid. More specifically, a recovery process may generate two lists: (a) the allowed list that represents the set of all commit identities which represent valid content for the cloud-based storage system (1400), and (b) the disallowed list that represents a set of commit identities that specifically do not represent valid content for the cloud-based storage system (1400). Once both the allowed list and the disallowed list have been determined, the allowed list and the disallowed list may be written out as new data, in some cases represented by a set of new commit identities and committed by writing those new commit identities to a sufficient number of storage elements.

[0420] Continuing with this example, one solution to determine the set of commit identities to add to the disallowed list is to determine which commit identities exist in written data but that lack persisted commits of those commit identities found during recovery. Another example solution to determine the set of missing commit identities is to establish, during operation of the cloud-based storage system (1400), a set of allowed future commit identities, and to make the set of allowed future commit identities durable before any of those commit identities can be used for writing new data. This results in three lists: (a) allowed commit identities, (b) disallowed commit identities, and (c) potential future commit identities. All three lists can be written together or separately, where the three lists may be associated with commit identities for the writes that persist the lists, and committed by writing commit records for those commit identities to a sufficient number of storage elements. In some examples, determining commit identities to add into the disallowed list during recovery may include reading the last committed future commit identities list (where the committed future commit identities list may depend upon

finding the last such list for which a commit record could be recovered), determining the subset of the committed future commit identities list for which no commit record was found, and then adding that subset to the list of disallowed commit identities.

[0421] In this example, the lists of allowed, disallowed, and future commit identities can be simplified by making commit identities sequential. During normal operation that does not include faults and corresponding recoveries, the future commit identities list may be described as, or may indicate, a range of numbers from some already used and fully committed number to some future number that has not yet been fully committed. At cloud-based storage system (1400) startup, or storage controller startup, or after a recovery, the start of that range of numbers may be set to some value that must be past any commit identity that might have been missed due an unavailable storage elements. Before the sequence numbers within the range have all been used, a new range may be written, where the process of writing the new range may use and commit at least one commit identity—consequently, the new range should be written out prior to using the last sequence number within the current range. Further, as data is written, committed, and the data commits are added to the allowed list, the beginning of the range may be advanced to a commit identity prior to any still in progress write and commit.

[0422] Continuing with this example, the use of ranges of sequential identifiers may also simplify the allowed and disallowed lists. For example, during recovery, any commit identity not already on the disallowed list that precedes the first number on the future commit identity range may be considered allowed—unless the commit identity was already disallowed such as in a previous incomplete recovery. Further, even if one of those commit identities had never been used, the commit identity could not have been associated with partially written and committed data, which creates opportunities for compacting allowed and disallowed lists into a set of ranges. In this example, the set of commit identities between the start of the future commit identities range and one prior to the first number in the range for which a commit record of a commit identity is found is disallowed, creating one range. Further, the set of commit identities between one after the last number in the future commit identity range for which a commit record of a commit identity is found and the last number in the future commit identity range itself is also disallowed, thereby creating another range. In some cases, other commit identities in the future commit identity range may produce a messier set which may include individual commit identities that are allowed or disallowed, or potentially compressible sub-ranges where all commit identities in a range are allowed or disallowed.

[0423] To complete this example, an additional step or consideration for using ranges may be to track instances of encountering an error while writing out data prior to the data being committed. For example, if an erasure coded partial or complete stripe cannot be written out completely due, for example, to an erase block or storage element failure, then the content may be rewritten to new partial or complete stripes on a new set of storage elements. In this case, the first failed attempt at writing the data should be invalidated, and this invalidation may be done by adding any commit identities associated with the failed writes to the disallowed list. Otherwise, the simple range model described in the previous

paragraph may fail to disallow the incomplete data on a recovery and might add the data to the allowed list.

[0424] In some implementations of a cloud-based storage system (1400), such as the examples described above, the data and recovery models may be implemented without use of multiple addressable storage classes.

[0425] In some implementations, reorganizing or reformatting of data stored in a first tier of cloud storage as it is transferred to a second tier of cloud storage may not require additional commit phases because the data may be, with high probability, already be guaranteed to be on a sufficient number of storage elements before a storage controller instructs it to reorganize or reformat the data during transfer.

[0426] In some implementations, models of mirroring content to multiple storage elements prior to converting it to another storage format, such as a RAID-6 format, or models of writing M+R content prior to transforming it into N+R content may need to associate commit identities with M+R content. In this example, for any other partial committable write of a complete erasure coded stripe to commit, it will have at least one commit identity associated with the content so that it can operate through the commit algorithm. Further, the recording of a commit identity could itself be written as a subsequent partial committable write even to the same erasure coded stripe.

[0427] In some implementations of a cloud-based storage system (1400), the features of a cloud-based storage system (1400) may be applied to various redundancy models. Further, storage elements within the cloud-based storage system (1400) may also have internal redundancy, or storage controllers, or storage controller instances, may redundantly store data that is stored within storage elements to handle localized failures of individual blocks, erase blocks, or chips. Most of the following examples consider storage elements that fail completely, or that do not make themselves available in a timely fashion. However, in some cases, some data in, say, an N+R stripe might be unavailable due to a failed or corrupted read within an otherwise operating and available storage element. Further, internal redundancy may reduce the number of cases where this becomes a problem, to the point that it is statistically implausible, but internal redundancy may also fail, and some storage elements may not deploy enough of it to recover from all non-catastrophic failure modes.

[0428] Continuing with this example, handling such failures may be solved by an implementation that avoids coming online if exactly R storage elements do not boot properly, and the implementation may instead wait to come online until the number of unavailable storage elements drops to R-1 or fewer. Alternately, an implementation might determine the recoverability of all data that might have been in flight at the time of the fault that preceded recovery, and then ensure that none of the data encountered an unrecoverable error before proceeding to making changes that might affect subsequent recoveries. As an alternative, if a latent corruption is encountered in a block for which at least one additional (but not currently available) redundancy shard might have been written, the cloud-based storage system (1400) may pause or fault waiting to determine if a storage element for that redundant shard eventually comes back online.

[0429] Further, note that a cloud-based storage system (1400) may use various schemes for different data. For example, some data may be written as two mirrored copies

that are safe from one failure, other data, such as from a higher priority process, application, or user, might be written in a RAID-5 style N+1 scheme, and other data might be written in a RAID-6 style N+2 scheme, or even using three or four mirrored copies or using N+3 schemes for triple failure redundancy (perhaps for critical but low-volume metadata). Further, different data might be striped or mirrored across different subsets of storage elements, where some subsets may overlap in various ways and other subsets might not. Any statements about the interrelation between complete, recoverable, incomplete, and unrecoverable data should then consider the completion and recoverability model associated with each type of data, so for example if content of a RAID-6 stripe follows and depends on a supposedly completed RAID-5 or two-mirrored-copy write, then the RAID-6 stripe's dual-failure-redundant validity during recovery may depend on the recoverability of single-failure-redundant content, regardless of how the set of storage elements for each written dataset do or do not overlap.

[0430] Further, a cloud-based storage system (1400) may divide up storage elements such that redundancy operates within constrained groups of storage elements (possibly following natural disasters such as enclosures or internal networks and/or system faults). Such constrained groups of storage elements may be called pools, or write groups, or RAID groups, or referred to by other names. In this example, such a constrained group of storage elements is referred to as a write group, where a principle behind a write groups is that any N+R stripe that utilizes any storage element within the write group will only store shards on other storage elements in the same write group. For example, if there are 12 storage elements in write group A and 12 storage elements in write group B, then any pairing of data and redundancy will be constrained within either the 12 storage elements in write group A or the 12 storage elements in write group B. In some cases, write groups for a cloud-based storage system (1400) may be different sizes and not uniform sizes.

[0431] Further, in some examples, as storage is added, write groups may be extended to include more storage elements, or additional write groups may be added to the cloud-based storage system (1400). In some cases, incremental addition of storage elements may cause making a choice between making existing write groups too large or making a new write group that is too small—in such a case, the cloud-based storage system (1400) may split existing write groups and transform existing content to match the new write group layouts. These constraints may limit damage caused by failure of storage elements by limiting the intersections of failed storage elements across all stripes. As an illustrative example: if two storage elements failed in write group A and one storage element failed in write group B, no N+2 stripe could encounter all three failed storage elements while writing out stripes or during recovery because any stripe stored in write group A which might include write group A's two failed storage elements will not include the failed storage element in write group B, and no stripe in write group B which might include write group B's one failed storage element will not include either of the two failed storage elements in write group A.

[0432] To complete this example, in a cloud-based storage system (1400) that implements write groups or some similar constraint in allocating shards of redundant data, previous

discussions concerning numbers of failed storage elements that allow continued operation or a successful recovery should apply those rules to such individual groups of storage elements, rather than to the entire cloud-based storage system (1400).

[0433] Turning now to the flow chart illustrated in FIG. 14, the flow chart includes receiving (1402), for storage within a first tier of cloud storage of a cloud-based storage system (1400), one or more segments of data; generating (1404), for each of one or more shards of data of the one or more segments of data, self-describing information for recoverability of the one or more shards of data; and storing (1406), within a second tier of cloud storage of the cloud-based storage system (1400), both the one or more shards of data and the generated self-describing information for recoverability of the one or more shards of data.

[0434] Receiving (1402), for storage within the first tier of cloud storage of a cloud-based storage system (1400), one or more segments (1455) of data may be carried out as described above with reference to a cloud-based storage system (1400) receiving data for storage. Further, as described above, the first tier of cloud storage of a cloud-based storage system (1400) may include multiple storage elements, where a storage element may be implemented as a cloud computing instance that is instantiated within a cloud computing environment (316), or where a storage element may be implemented as a solid state storage device provisioned within a cloud computing environment (316). Further still, the first tier of cloud storage may implement a block-level storage protocol. In this example, the one or more segment (1455) of data may be included within one or more blocks (1454) of data included within an I/O operation (1452) received (1402) from a computing device, such as a host (1451) computer.

[0435] Generating (1404), for each of the one or more shards (1456) of data of the one or more segments (1455) of data, self-describing information (1458) for recoverability of the one or more shards (1456) of data may be carried out as described above with regard to the multiple techniques for generating self-describing information.

[0436] Storing (1406), within a second tier of cloud storage of the cloud-based storage system (1400), both the one or more shards (1456) of data and the generated (1404) self-describing information (1458) for recoverability of the one or more shards (1456) of data may be carried out as described above. For example, as described above, the cloud-based storage system (1400) may include a first tier of cloud storage implemented by cloud computing instances that provide block-level storage, and include a second tier of cloud storage implemented by an object store—where one or more of the storage controllers of the cloud-based storage system (1400) may transfer or copy the one or more shards (1456) of data and the self-describing information (1458) from block-level storage to object storage. In some examples, each object in the object storage may include one or more shards (1456) of data.

[0437] For further explanation, FIG. 15 sets forth a flow chart illustrating an example method of data resiliency in a cloud-based storage system (1500A) in the context of a multiple storage systems (1500A-1500N) synchronously replicating a dataset (1558) according to some embodiments of the present disclosure.

[0438] As illustrated in FIG. 15, the storage systems (1500A-1500N) synchronously replicating a dataset (1558)

include both hardware-based storage systems (1500B-1500N) and a cloud-based storage system (1500A) implemented within a cloud computing environment (316). For clarity, only one cloud-based storage system (1500A) is depicted, however, in other examples, there may be multiple cloud-based storage systems, or multiple cloud-based storage systems without any hardware-based storage systems.

[0439] In this examples, the cloud-based storage system (1500A) may provide similar services as those described for the cloud-based storage systems above, with reference to FIGS. 8-14. For example, the cloud-based storage system (1500A) may be used to provide block storage services to users based on use of services of a cloud-computing environment, such as cloud computing environments described in FIGS. 8-14, including storage provided by virtual computing instances or storage provided by solid-state storage, and so on.

[0440] In this example, the storage systems (1500A-1500N) depicted in FIG. 15A may be similar to the storage systems described above with reference to FIGS. 1A-1D, FIGS. 2A-2G, FIGS. 3A-3D, or any combination thereof. In fact, the storage systems (1500A-1500N) depicted in FIG. 15 may include the same, fewer, or additional components as the storage systems described above.

[0441] As described above, metadata may be synchronized among storage systems that are synchronously replicating a dataset. Such metadata may be referred to as common metadata, or shared metadata, that is stored by a storage system on behalf of a pod (1554) related to the mapping of segments of content stored within the pod (1554) to virtual address within storage objects within the pod, where information related to those mappings is synchronized between member storage systems for the pod to ensure correct behavior—or better performance—for storage operations related to the pod (1554).

[0442] In some examples, a storage object may implement a volume or a snapshot. The synchronized metadata may include: (a) information to keep volume content mappings synchronized among the storage systems in the pod; (b) tracking data for recovery checkpoints or for in-progress write operations; (c) information related to the delivery of data and mapping information to a remote storage system for asynchronous or periodic replication.

[0443] Information to keep volume content mappings synchronized among the storage systems (1500A-1500N) in the pod (1554) may enable efficient creating of snapshots, which in turn enables that subsequent updates, copies of snapshots, or snapshot removals may be performed efficiently and consistently across the pod member storage systems.

[0444] Tracking data for recovery checkpoints or for in-progress write operations may enable efficient crash recovery and efficient detection of content or volume mappings that may have been partially or completely applied on individual storage systems for a pod (1554), but that may not have been completely applied on other storage systems for the pod (1554).

[0445] Information related to the delivery of data and mapping information to a remote storage system for asynchronous or periodic replication may enable more than one member storage system for a pod (1554) to serve as a source for the replicated pod content with minimal concerns for dealing with mismatches in mapping and differencing metadata used to drive asynchronous or periodic replication.

[0446] Further, synchronized metadata describing mapping of segments to storage objects is not limited to mappings themselves, and may include additional information such as sequence numbers (or some other value for identifying stored data), timestamps, volume/snapshot relationships, checkpoint identities, trees or graphs defining hierarchies, or directed graphs of mapping relationships, among other storage system information.

[0447] As depicted in FIG. 15, multiple storage systems (1500A-1500N) that are synchronously replicating a dataset (1558) may be in communication with each other storage system (1500B-1500N) in an in-sync list for a pod—where storage systems may exchange metadata describing I/O operations to perform and metadata describing updates to be made to respective, local metadata representations of the dataset (1558) stored on individual storage systems.

[0448] The design of modern storage environments can involve the use of different types of components that can interoperate with each other. Frequently, components may be used with out-of-box functionality provided by a storage provider. For example, a cloud storage service provider platform (such as Amazon AWS or Microsoft Azure) offers several storage components and services that can be combined with other components in building a storage system. Readers will appreciate that such components may have inherent performance or capacity constraints that limit an organization's ability to optimize use of the storage system. For example, known methods of creating storage systems may lack the ability to maximize the use of a component without incurring significant expenses or having to use costly workarounds.

[0449] FIG. 16 sets forth a flow chart illustrating an example method of providing block-based storage in accordance with some embodiments of the present disclosure. As depicted in FIG. 16, a virtual storage system 1604 includes a virtual drive layer 1606. The virtual drive layer 1606 can further include a plurality of virtual drives 1610, 1612, 1614, and 1616, with four virtual drives being shown purely as an example. As described above, any of the plurality of virtual drives 1610, 1612, 1614, and 1616, may execute on cloud computing instances (also referred to as virtual storage device servers) with local instance storage. The plurality of virtual drives 1610, 1612, 1614, and 1616 may emulate a storage device, such as a disk drive or SSD. Virtual drive 1610 also includes virtual drive instance storage 1650 and virtual storage system optimization module 1640. Any other virtual drive in virtual drive layer 1606 can include these components as well, with only virtual drive 1610 being depicted with these components for ease of presentation. In one embodiment, virtual drive layer 1606 may include the ability to add, remove, replace, upgrade, or modify any virtual drive component without impact to a client of virtual storage system 1604. For example, virtual drive 1612 may be replaced with another virtual drive 1699 (not shown) without altering a level of service provided to a client of virtual storage system 1604.

[0450] In one embodiment, virtual storage system optimization module 1640 may be a software component embodied by virtual drive servers, a virtual storage system controller, or some combination of the two. The embodiments disclosed herein discuss extending the out-of-box virtual storage system logic provided via the virtual drive servers and/or a virtual storage system controller in order to perform the processes described. More specifically, one or more of the

plurality of virtual drives may execute operations described herein that may collectively be termed virtual drive logic. Virtual storage system optimization module **1640** is shown as part of virtual drive **1610** but in other embodiments, virtual storage system optimization module **1640** may be external to a virtual drive component of virtual storage system **1604**. While FIG. **16** shows virtual storage system optimization module **1640** as part of virtual drive **1610**, some or all of virtual drives **1610-1616** may include components similar to virtual storage system optimization module **1640**. Moreover, the virtual storage system logic that performs the virtual storage system management, optimization, and/or adjustment processes described herein may be performed by a combination of the various virtual storage system optimization modules that are associated with different virtual drives. In other words, logic or process execution may be shared by logic components of various virtual drives.

[0451] FIG. **16** also depicts controller **1602**, which may be a storage system controller similar to storage array controllers **110A-D** described above with respect to FIG. **1A** and configured to carry out various storage tasks including writing data received from client computing devices to virtual storage system **1604**, erasing data from virtual storage system **1604**, retrieving data from virtual storage system **1604** and providing data to client computing devices, monitoring and reporting of disk utilization and performance, performing redundancy operations, such as Redundant Array of Independent Drives ('RAID') or RAID-like data redundancy operations, compressing data, encrypting data, and so forth.

[0452] Virtual drive instance storage **1650** may be embodied as a local instance store associated with virtual drive **1610** that is configured to provide, for example, private storage to a compute instance. Virtual drive instance storage **1650** may be, for example, block-based instance storage, such as an Amazon Elastic Block Store (EBS) storage volume that is attached to the computing instance that is associated with a virtual drive. In one embodiment, virtual drive instance storage **1650** may also operate as a cache for caching data associated with I/O operations that are received by virtual storage system **1604** prior to the data being transmitted to, for example, backend storage layer **1630**. Moreover, the disclosed embodiments include the ability to increase or decrease an amount of local instance storage without impact to a client of virtual storage system **1604**.

[0453] As shown in FIG. **16**, backend storage layer **1630** can include a plurality of storage subsystems **412a-n**. A storage subsystem, as used herein, may be a storage component capable of persistent storage of data in a block-based storage format. In one embodiment, one or more of the plurality of storage subsystems **412a-n** may be cloud-based storage subsystems. In other words, backend storage layer **1630** may, in some embodiments, be composed of some storage subsystems that are cloud-based whereas other storage subsystems may be non-cloud-based as well (e.g., local, on-premises, private cloud, or other cloud block-based storage that is not provided through a public cloud infrastructure). In one embodiment, backend storage layer **1630** may include the ability to add, remove, replace, upgrade, or modify any storage subsystem without impact to a client of virtual storage system **1604**. For example, storage subsystem **412a** may be replaced with another storage subsystem **412x**

(not shown) without altering a level of service provided to a client of virtual storage system **1604**.

[0454] As one example, storage subsystem **412a** may be a block-based storage system, such as Azure Premium SSDs. Azure Premium SSDs are relatively high-performance solid state drives designed to provide persistent storage that can support I/O intensive workloads. In some implementations, Azure Premium SSDs can be provisioned from a cloud-storage services provider (e.g., Microsoft Azure), with a defined pricing model that is fixed by the cloud-storage services provider. Readers will appreciate that a product such as Azure Premium SSDs may offer higher performance relative to another storage solution (such as another block-based storage solution or non-block-based storage solution), with performance definable in terms of, for example, throughput or IOPS that can be handled by an Azure Premium SSD but that is capped based on pricing. In other words, a particular Azure Premium SSD may have the ability to provide higher throughput or IOPS for a higher price.

[0455] Moreover, an Azure Premium SSD may be of different types, v1 or v2. Azure Premium SSD v1 may offer certain maximum levels of capacity or performance, such as a disk size of 32,767 GiB, a maximum possible throughput of 900 MB/s, and a maximum of 20,000 IOPS. Azure Premium SSD v2 may offer higher maximum levels of capacity or performance, such as a disk size of 65,536 GiB, a maximum possible throughput of 1200 MB/s, and a maximum of 80,000 IOPS.

[0456] As another example, storage subsystem **412a** may be a different kind of block-based storage system, such as an Azure Ultra Disk. Azure Ultra disks may be specifically configured for I/O-intensive workloads such as top tier databases (for example, SAP HANA, SQL, or Oracle databases), or other transaction-heavy workloads. Ultra Disks may be billed such that the provisioned size is rounded up to the nearest disk size offer. For example, if a 200 GiB Ultra Disk is provisioned for a certain time period, at the end of that time period the user is billed for 256 GiB, the nearest disk size offer defined for Ultra Disks by Azure.

[0457] As another example, storage subsystem **412a** may be a different kind of block-based storage system, such as an Amazon Elastic Block Store (EBS), which may be provisioned in the form of block storage volumes having defined IOPS, throughput, and capacity, Google Cloud Persistent Disks which do not require provisioning in the form of volumes, or other forms of block-based storage.

[0458] In some embodiments, the performance capabilities of storage subsystems **412a-n** may include other features that can be described in addition to raw performance capabilities such as throughput and IOPS. For example, storage subsystems **412a-n** may provide disk bursting capabilities. Disk bursting capabilities include the ability to provide a higher level of performance (e.g., higher IOPS or throughput) for a limited amount of time. For example, storage subsystems **412a-n** may be configured to provide higher-than-normal performance during startup, or to handle batch jobs that occur at predictable times, or to respond to unpredictable workload spikes. As an example, an Azure Premium SSD may be provisioned to provide limited disk bursting such that for an Azure Premium SSD v1, throughput can be increased from 900 MB/s to 1000 MB/s for a limited time, such as 30 minutes.

[0459] The other capabilities of storage subsystems **412a-n** may include creating snapshots. For example, full snapshots or images may be stored on one or more of storage subsystems **412a-n**. In some embodiments, there may be various options for snapshot storage on storage subsystems **412a-n**. In the case of Azure Premium SSDs or Ultra Disks, these options may include locally redundant storage (LRS), which involves synchronously replicating data among one or more other storage components (such as other disks) within a particular data center of a particular region. Another snapshot storage option can be zone-redundant storage (ZRS), which involves synchronously replicating data among one or more other storage components in other availability zones of the cloud-storage service provider (e.g., Microsoft Azure) in a particular region. Other examples may include geo-redundant storage (GRS)—which involves storing additional copies of data in another paired region associated with the cloud-storage service provider, read-access geo-redundant (RA-GRS) which is similar to GRS, but allows data to be read from two different regions, and object replication for block blob storage—a type of replication usable for block blobs.

[0460] In some embodiments, each storage subsystem of backend storage layer **1630** may be a block-based storage subsystem. In other embodiments, backend storage layer **1630** can include different types of block-based storage systems. As used herein, the term “type” may refer to block-based storage systems having different providers, manufacturers, configurations, and so on. For example, storage subsystem **412a** may be a block-based storage device of a first type, whereas storage subsystem **412b** may be a block-based storage device of a second type. As a more specific example, storage subsystem **412a** may comprise an Azure Premium SSD v1 and storage subsystem **412b** may comprise an Azure Premium SSD v2.

[0461] Readers will appreciate that implementing a storage system using cloud storage service provider components should account for the limitations of these components which may be quite different from hardware components used in a physical storage system. For example, there are block-based storage capabilities implemented within cloud platforms, such as Elastic Block Store, which could be used in place of disk drives. However, these are expensive to use because they are often billed based on I/O traffic as well as capacity, and often do not support concurrent sharing between compute instances, all of which limits their practicality. In some embodiments, one or more subsystems of backend storage layer **1630** may have certain performance specifications that are defined by a cloud storage service provider that provides those storage subsystems. For example, where the storage subsystem in use is an Azure Premium SSD, Microsoft Azure may have defined certain performance limitations that are associated with use of the Azure Premium SSD. As a more specific example, an Azure-based system may involve use of an Azure virtual machine that has certain performance specifications available for a particular cost, such as a certain number of IOPS that can be processed by the Azure Premium SSD using the Azure virtual machine, a certain amount of network bandwidth that is available for each Azure virtual machine, and the like.

[0462] Readers will appreciate that embodiments of the present disclosure may be configured to transcend these provider-defined limitations through the use of virtual drives

as described in greater detail below. For example, the disclosed virtual drives may not have the same performance limitations as those associated with the cloud storage service provider virtual machines. As an example of this, the virtual drives may be able to take advantage of different networking configurations that bypass network bandwidth limits. Moreover, the virtual drives may be configured to process a higher number of IOPS than is possible with a cloud storage service provider’s virtual machine, which may not be configurable or may be configurable for higher performance but at a higher cost.

[0463] As described above, the pricing model associated with Azure Premium SSDs (or any other storage solution provisioned by a cloud-storage services provider, may result in artificial constraints being placed on the performance of virtual storage system **1604**. For example, virtual storage system **1604** may be unable to satisfy increasing performance requirements because, at a certain level, the cost of continuing to obtain more Azure Premium SSDs or more performance from the same set of Azure Premium SSDs can become prohibitively large. There may be additional technical limitations. For example, Azure Premium SSDs may not be able to support encryption at host, thus necessitating that encryption functions be configured at a virtual machine, which may need to be provisioned at additional cost. To bypass these limitations, the embodiments described herein employ virtual drives of virtual drive layer **1606** in various ways so as to unlock additional performance and capabilities (e.g., higher than available for a certain cost, or higher performance up to a maximum for the component) without incurring additional costs.

[0464] The example method depicted in FIG. 16 includes creating **1620** a virtual storage system for storing a dataset, the virtual storage system including a virtual drive layer that includes a plurality of virtual drives for storing at least a portion of the dataset as block data; and a block storage backend layer for storing at least the portion of the dataset as block data, the block storage backend layer including one or more block-based storage devices. Creating **1620** the virtual storage system can include instantiation and interoperation of various different components, each of which are described in greater detail above. Particularly in the case of components provided by cloud storage service providers, creating **1620** virtual storage system **1604** can also include execution of infrastructure-as-code (IaC) templates or commands (e.g., Hashicorp Terraform commands, Amazon CloudFormation commands, Azure CLI commands, PowerShell commands, etc.). As an example, deploying an Azure Premium SSD for use with virtual storage system **1604** can include using commands to create the Azure Premium SSD in an Azure availability zone, selecting the disk SKU, selecting the logical sector size, and so on.

[0465] Creating **1620** the virtual storage system can also include instantiating a plurality of virtual drives (e.g., virtual drives **1610-1616**). For example, a plurality of computing instances (e.g., cloud computing instances such as Amazon EC2) can be instantiated and local instance storage can be attached to each of the plurality of computing instances. Creating **1620** the virtual storage system can also include configuring the plurality of virtual drives to receive I/O operations via a network that includes controller **1602**, and to process I/O operations in conjunction with backend storage layer **1630**.

[0466] In some embodiments, the plurality of virtual drives may have a first I/O performance capacity that is higher than a second I/O performance capacity associated with the block-based storage devices of the block storage backend layer. For example, in some embodiments, virtual drive 1610 (and/or virtual drives 1612, 1614, and 1616) may have the ability to host the persona of an NVRAM device and/or an SSD or other disk or storage type that can provide IOPS, network bandwidth specifications, or storage specifications that exceed those that are specified by a cloud storage service provider. Moreover, virtual storage system optimization module 1640 can configure a particular virtual drive to host multiple storage devices such as multiple SSDs depending on latency requirements, bandwidth requirements, IOPS ratios, and so on.

[0467] The example method depicted in FIG. 16 also includes receiving 1622 an I/O operation with respect to the dataset. For example, controller 1602 may receive an I/O operation from a client computing device (e.g., a host computing device that hosts an application that uses virtual storage system 1604 for data storage). The I/O operation, such as a read operation or write operation may be directed at data stored within one or more of storage subsystems 412a-n of backend storage layer 1630. Virtual storage system 1604 may then receive the I/O operation via controller 1602.

[0468] The example method depicted in FIG. 16 also includes storing 1624, based on the received I/O operation, in the block storage backend layer, and an update to the dataset. As described above, backend storage layer 1630 may entirely or substantially entirely comprise block-based storage, such that storage subsystems 412a-n represent a plurality of block-based storage systems such as Azure Premium SSDs. Storing 1624 an update to the dataset can include storing additional data received as part of the I/O operation. Storing 1624 an update to the dataset can also include storing a snapshot or other data object representing the dataset in response to the I/O operation. Storing 1624 an update to the dataset can also include storing hash values, encryption keys, metadata, deduplicated versions of the dataset, or compressed versions of the dataset in one or more of storage subsystems 412a-n.

[0469] FIG. 17 sets forth a flow chart illustrating an example method of providing block-based storage in accordance with some embodiments of the present disclosure. The method depicted in FIG. 17 is similar to the method depicted in FIG. 16 in that the method depicted in FIG. 17 also includes creating 1620 a virtual storage system for storing a dataset, the virtual storage system including: a virtual drive layer that includes a plurality of virtual drives for storing at least a portion of the dataset as block data; and a block storage backend layer for storing at least the portion of the dataset as block data, the block storage backend layer including one or more block-based storage devices; receiving 1622 an I/O operation with respect to the dataset; and based on the received I/O operation, storing 1624, in the block storage backend layer, an update to the dataset.

[0470] The example method of FIG. 17 differs from that of FIG. 16 in that the example method of FIG. 17 also includes configuring 1702 one or more of the plurality of virtual drives with virtual drive logic for managing processing of I/O operations in the virtual storage system. Configuring 1702 one or more of the plurality of virtual drives with virtual drive logic for managing processing of I/O operations

in the virtual storage system can include performing certain functions using one or more of virtual drives 1610-1616 that may otherwise be performed using controller 1602. For example, components of virtual drives 1610-1616 (such as virtual storage system optimization module 1640) may adjust components of virtual storage system 1604, such as adding, removing, or modifying virtual drives, or adding, removing, or modifying one or more of storage subsystems 412a-n.

[0471] Configuring 1702 one or more of the plurality of virtual drives with virtual drive logic for managing processing of I/O operations in the virtual storage system can include bypassing performance limitations that would ordinarily be associated with use of certain components such as block-based storage systems provided by cloud storage service providers. These performance limitations may include throughout, IOPS, and capacity limitations and, as described above, may be artificially created caps on performance that are associated with cost tiers for using the component. As an example, backend storage layer 1630 may comprise a set of Microsoft Azure Premium SSDs. These SSDs may be provisioned with a first number of IOPS at a certain price. However, virtual drives 1610-1616 may be configured to receive a higher second number of IOPS than the total IOPS limit for that Azure Premium SSD. Virtual drives 1610-1616 may use their attached local instance storage (e.g., virtual drive instance storage 1650) as a cache to be able to process the higher second number of IOPS in conjunction with backend storage layer 1630. The local instance storage may be used to service the IOPS overhead, thereby enabling higher IOPS to be processed by virtual storage system 1604 while still maintaining a cost of using the Azure Premium SSD that is associated with the lower first number of IOPS.

[0472] Configuring 1702 one or more of the plurality of virtual drives with virtual drive logic for managing processing of I/O operations in the virtual storage system can also include bypassing technical limitations associated with block-based storage systems provided by cloud storage service providers. For example, virtual drives 1610-1616 can be used to provide encryption, compression, or deduplication functionality that may or may not be available on-device (or not available with a certain level of performance) at a storage solution such as an Azure premium SSD or Ultra Disk. Moreover, virtual drives 1610-1616 may be configured to adjust storage components to bypass other technical limitations. For example, certain components such as Ultra Disks may not be expandable without deallocating an attached virtual machine or detaching the Ultra Disk itself. To prevent performance issues, virtual drives 1610-1616 can be used to expand storage capacity by adding additional block-based storage to replace an Ultra Disk that needs to be detached (e.g., to be replaced with a larger Ultra Disk) without causing deallocation or detachment. This can be achieved by continuing to receive I/O operations and using local instance storage at a virtual drive as a cache while the backend storage layer is modified or upgraded. Once a replacement is in place, virtual drives 1610-1616 can be configured to redirect I/O to the new component, resulting in a non-performance-disruptive upgrade of the backend storage layer component. Moreover, certain components such as Ultra Disks may not support snapshotting, in which case virtual drives 1610-1616 can create snapshots of data of an Ultra Disk in another backend storage layer 1630 com-

ponent (such as an Azure Premium SSD). This can be done, for example, by copying the Ultra Disk data to local instance storage of a virtual drive, such as virtual drive instance storage **1650**, and then transferring that data to an Azure Premium SSD that can create the snapshot.

[0473] FIG. **18** sets forth a flow chart illustrating an example method of providing block-based storage in accordance with some embodiments of the present disclosure. The method depicted in FIG. **18** is similar to the method depicted in FIG. **16** in that the method depicted in FIG. **18** also includes creating **1620** a virtual storage system for storing a dataset, the virtual storage system including: a virtual drive layer that includes a plurality of virtual drives for storing at least a portion of the dataset as block data; and a block storage backend layer for storing at least the portion of the dataset as block data, the block storage backend layer including one or more block-based storage devices; receiving **1622** an I/O operation with respect to the dataset; and based on the received I/O operation, storing **1624**, in the block storage backend layer, an update to the dataset.

[0474] The example method of FIG. **18** differs from that of FIG. **16** in that the example method of FIG. **18** also includes instantiating **1802** the one or more block-based storage devices as cloud-based solid state drives. Instantiating **1802** the one or more block-based storage devices as cloud-based solid state drives can include execution of infrastructure-as-code (IaC) templates or commands (e.g., Hashicorp Terraform commands, Amazon CloudFormation commands, Azure CLI commands, Powershell commands, etc.). As an example, deploying an Azure Premium SSD for use with virtual storage system **1604** can include using commands to create the Azure Premium SSD in an Azure availability zone, selecting the disk SKU, selecting the logical sector size, and so on. Instantiating **1802** the one or more block-based storage devices can also include configuring other settings such as snapshot policies (e.g., use of local redundant storage, zone redundant storage, etc.).

[0475] FIG. **19** sets forth a flow chart illustrating an example method of providing block-based storage in accordance with some embodiments of the present disclosure. The method depicted in FIG. **19** is similar to the method depicted in FIG. **16** in that the method depicted in FIG. **19** also includes creating **1620** a virtual storage system for storing a dataset, the virtual storage system including: a virtual drive layer that includes a plurality of virtual drives for storing at least a portion of the dataset as block data; and a block storage backend layer for storing at least the portion of the dataset as block data, the block storage backend layer including one or more block-based storage devices; receiving **1622** an I/O operation with respect to the dataset; and based on the received I/O operation, storing **1624**, in the block storage backend layer, an update to the dataset.

[0476] The example method of FIG. **19** differs from that of FIG. **16** in that the example method of FIG. **19** also includes storing **1902** data for a received I/O operation in one or more of the plurality of virtual drives. Storing **1902** data for a received I/O operation in one or more of the plurality of virtual drives can include using virtual drive instance storage **1650** to store data, such as data that is received as part of an I/O operation or data that is to be provided to a client computing device as a result of processing of the I/O operation. In some embodiments, storing **1902** data for a received I/O operation in one or more of the plurality of virtual drives can include caching data at different virtual

drives of virtual drive layer **1606**. Caching the data can include dynamically adjusting the cache amount based on the overall performance of virtual storage system **1604**.

[0477] FIG. **20** sets forth a flow chart illustrating an example method of providing block-based storage in accordance with some embodiments of the present disclosure. The method depicted in FIG. **20** is similar to the method depicted in FIG. **16** in that the method depicted in FIG. **20** also includes creating **1620** a virtual storage system for storing a dataset, the virtual storage system including: a virtual drive layer that includes a plurality of virtual drives for storing at least a portion of the dataset as block data; and a block storage backend layer for storing at least the portion of the dataset as block data, the block storage backend layer including one or more block-based storage devices; receiving **1622** an I/O operation with respect to the dataset; and based on the received I/O operation, storing **1624**, in the block storage backend layer, an update to the dataset.

[0478] The example method of FIG. **20** differs from that of FIG. **16** in that the example method of FIG. **20** also includes transferring **2002**, from the virtual drive layer to the block storage backend layer, the data for the received I/O operation. Transferring **2002**, from the virtual drive layer to the block storage backend layer, the data for the received I/O operation can include copying cached data from the local instance storage of a virtual drive, such as virtual drive instance storage **1650** to one or more of storage subsystems **412a-n** of backend storage layer **1630**.

[0479] Example embodiments are described largely in the context of a fully functional computer system. Readers of skill in the art will recognize, however, that the present disclosure also may be embodied in a computer program product disposed upon computer readable storage media for use with any suitable data processing system. Such computer readable storage media may be any storage medium for machine-readable information, including magnetic media, optical media, or other suitable media. Examples of such media include magnetic disks in hard drives or diskettes, compact disks for optical drives, magnetic tape, and others as will occur to those of skill in the art. Persons skilled in the art will immediately recognize that any computer system having suitable programming means will be capable of executing the steps of the method as embodied in a computer program product. Persons skilled in the art will recognize also that, although some of the example embodiments described in this specification are oriented to software installed and executing on computer hardware, nevertheless, alternative embodiments implemented as firmware or as hardware are well within the scope of the present disclosure.

[0480] Embodiments can include a system, a method, and/or a computer program product. The computer program product may include a computer readable storage medium (or media) having computer readable program instructions thereon for causing a processor to carry out aspects of the present disclosure.

[0481] The computer readable storage medium can be a tangible device that can retain and store instructions for use by an instruction execution device. The computer readable storage medium may be, for example, but is not limited to, an electronic storage device, a magnetic storage device, an optical storage device, an electromagnetic storage device, a semiconductor storage device, or any suitable combination of the foregoing. A non-exhaustive list of more specific examples of the computer readable storage medium includes

the following: a portable computer diskette, a hard disk, a random access memory (RAM), a read-only memory (ROM), an erasable programmable read-only memory (EPROM or Flash memory), a static random access memory (SRAM), a portable compact disc read-only memory (CD-ROM), a digital versatile disk (DVD), a memory stick, a floppy disk, a mechanically encoded device such as punch-cards or raised structures in a groove having instructions recorded thereon, and any suitable combination of the foregoing. A computer readable storage medium, as used herein, is not to be construed as being transitory signals per se, such as radio waves or other freely propagating electromagnetic waves, electromagnetic waves propagating through a waveguide or other transmission media (e.g., light pulses passing through a fiber-optic cable), or electrical signals transmitted through a wire.

[0482] Computer readable program instructions described herein can be downloaded to respective computing/processing devices from a computer readable storage medium or to an external computer or external storage device via a network, for example, the Internet, a local area network, a wide area network and/or a wireless network. The network may comprise copper transmission cables, optical transmission fibers, wireless transmission, routers, firewalls, switches, gateway computers and/or edge servers. A network adapter card or network interface in each computing/processing device receives computer readable program instructions from the network and forwards the computer readable program instructions for storage in a computer readable storage medium within the respective computing/processing device.

[0483] Computer readable program instructions for carrying out operations of the present disclosure may be assembler instructions, instruction-set-architecture (ISA) instructions, machine instructions, machine dependent instructions, microcode, firmware instructions, state-setting data, or either source code or object code written in any combination of one or more programming languages, including an object oriented programming language such as Smalltalk, C++ or the like, and conventional procedural programming languages, such as the “C” programming language or similar programming languages. The computer readable program instructions may execute entirely on the user’s computer, partly on the user’s computer, as a stand-alone software package, partly on the user’s computer and partly on a remote computer or entirely on the remote computer or server. In the latter scenario, the remote computer may be connected to the user’s computer through any type of network, including a local area network (LAN) or a wide area network (WAN), or the connection may be made to an external computer (for example, through the Internet using an Internet Service Provider). In some embodiments, electronic circuitry including, for example, programmable logic circuitry, field-programmable gate arrays (FPGA), or programmable logic arrays (PLA) may execute the computer readable program instructions by utilizing state information of the computer readable program instructions to personalize the electronic circuitry, in order to perform aspects of the present disclosure.

[0484] Aspects of the present disclosure are described herein with reference to flowchart illustrations and/or block diagrams of methods, apparatus (systems), and computer program products according to some embodiments of the disclosure. It will be understood that each block of the

flowchart illustrations and/or block diagrams, and combinations of blocks in the flowchart illustrations and/or block diagrams, can be implemented by computer readable program instructions.

[0485] These computer readable program instructions may be provided to a processor of a general purpose computer, special purpose computer, or other programmable data processing apparatus to produce a machine, such that the instructions, which execute via the processor of the computer or other programmable data processing apparatus, create means for implementing the functions/acts specified in the flowchart and/or block diagram block or blocks. These computer readable program instructions may also be stored in a computer readable storage medium that can direct a computer, a programmable data processing apparatus, and/or other devices to function in a particular manner, such that the computer readable storage medium having instructions stored therein comprises an article of manufacture including instructions which implement aspects of the function/act specified in the flowchart and/or block diagram block or blocks.

[0486] The computer readable program instructions may also be loaded onto a computer, other programmable data processing apparatus, or other device to cause a series of operational steps to be performed on the computer, other programmable apparatus or other device to produce a computer implemented process, such that the instructions which execute on the computer, other programmable apparatus, or other device implement the functions/acts specified in the flowchart and/or block diagram block or blocks.

[0487] The flowchart and block diagrams in the Figures illustrate the architecture, functionality, and operation of possible implementations of systems, methods, and computer program products according to various embodiments of the present disclosure. In this regard, each block in the flowchart or block diagrams may represent a module, segment, or portion of instructions, which comprises one or more executable instructions for implementing the specified logical function(s). In some alternative implementations, the functions noted in the block may occur out of the order noted in the figures. For example, two blocks shown in succession may, in fact, be executed substantially concurrently, or the blocks may sometimes be executed in the reverse order, depending upon the functionality involved. It will also be noted that each block of the block diagrams and/or flowchart illustration, and combinations of blocks in the block diagrams and/or flowchart illustration, can be implemented by special purpose hardware-based systems that perform the specified functions or acts or carry out combinations of special purpose hardware and computer instructions.

What is claimed is:

1. A method of providing block-based storage, the method comprising:

creating a virtual storage system including:

- a virtual drive layer comprising a plurality of virtual drives, wherein at least one virtual drive includes a local storage component for storing data from one or more I/O operations associated with a dataset as block data; and
- a backend storage layer comprising one or more block-based storage devices for storing at least a portion of the dataset as block data;

receiving an I/O operation associated with the dataset, including storing data for the I/O operation in one or more virtual drives of the virtual drive layer; and based on the I/O operation, storing, in the backend storage layer, an update to the dataset as block data, wherein the update is generated at the virtual drive layer.

2. The method of claim 1, wherein the local storage component comprises local instance storage associated with a cloud computing instance hosting the at least one virtual drive.

3. The method of claim 1, wherein storing data for the I/O operation in the one or more virtual drives includes caching the data in the local storage component prior to transferring the data to the backend storage layer.

4. The method of claim 1, wherein storing, in the backend storage layer, the update to the dataset further comprises transferring, from the virtual drive layer to the backend storage layer, the data for the received I/O operation.

5. The method of claim 1, wherein the backend storage layer includes a plurality of block-based storage devices comprising at least two different types of cloud-based solid state drives.

6. The method of claim 5, wherein storing the update to the dataset in the backend storage layer comprises selecting one of the block-based storage devices based on I/O performance capacity or cost constraints.

7. The method of claim 1, further comprising configuring the virtual drive layer to continue processing I/O operations during a modification of the backend storage layer, including replacement or upgrade of a block-based storage device.

8. An apparatus comprising a virtual storage system, the virtual storage system comprising:

a virtual drive layer comprising a plurality of virtual drives, wherein at least one virtual drive includes a local storage component for storing data from one or more I/O operations associated with a dataset as block data; and

a backend storage layer comprising one or more block-based storage devices for storing at least a portion of the dataset as block data,

wherein the apparatus is configured to:

receive an I/O operation associated with the dataset, including storing data for the I/O operation in one or more virtual drives of the virtual drive layer; and based on the I/O operation, store, in the backend storage layer, an update to the dataset as block data, wherein the update is generated at the virtual drive layer.

9. The apparatus of claim 8, wherein the local storage component comprises local instance storage associated with a cloud computing instance hosting the at least one virtual drive.

10. The apparatus of claim 8, wherein storing data for the I/O operation in the one or more virtual drives includes caching the data in the local storage component prior to transferring the data to the backend storage layer.

11. The apparatus of claim 8, wherein storing, in the backend storage layer, the update to the dataset further

comprises transferring, from the virtual drive layer to the backend storage layer, the data for the received I/O operation.

12. The apparatus of claim 8, wherein the backend storage layer includes a plurality of block-based storage devices comprising at least two different types of cloud-based solid state drives.

13. The apparatus of claim 12, wherein storing the update to the dataset in the backend storage layer comprises selecting one of the block-based storage devices based on I/O performance capacity or cost constraints.

14. The apparatus of claim 8, wherein the virtual drive layer is further configured to continue processing I/O operations during a modification of the backend storage layer, including replacement or upgrade of a block-based storage device.

15. A non-transitory computer-readable storage medium storing instructions that, when executed by a processing device, cause the processing device to:

create a virtual storage system including:

a virtual drive layer comprising a plurality of virtual drives, wherein at least one virtual drive includes a local storage component for storing data from one or more I/O operations associated with a dataset as block data; and

a backend storage layer comprising one or more block-based storage devices for storing at least a portion of the dataset as block data;

receive an I/O operation associated with the dataset, including storing data for the I/O operation in one or more virtual drives of the virtual drive layer; and based on the I/O operation, store, in the backend storage layer, an update to the dataset as block data, wherein the update is generated at the virtual drive layer.

16. The non-transitory computer-readable storage medium of claim 15, wherein the local storage component comprises local instance storage associated with a cloud computing instance hosting the virtual drive.

17. The non-transitory computer-readable storage medium of claim 15, wherein storing data for the I/O operation in the one or more virtual drives includes caching the data in the local storage component prior to transferring the data to the backend storage layer.

18. The non-transitory computer-readable storage medium of claim 15, wherein storing, in the backend storage layer, the update to the dataset further comprises transferring, from the virtual drive layer to the backend storage layer, the data for the received I/O operation.

19. The non-transitory computer-readable storage medium of claim 15, wherein the backend storage layer includes a plurality of block-based storage devices comprising at least two different types of cloud-based solid state drives.

20. The non-transitory computer-readable storage medium of claim 19, wherein storing the update to the dataset in the backend storage layer comprises selecting one of the block-based storage devices based on I/O performance capacity or cost constraints.

* * * * *